

```
<h1>오늘의 메뉴</h1>
```

```
<ul> <li class="menu">아메리카노</li>
```

```
</ul>
```

```
.menu { color: #2F6B57; font-size: 18px; }
```

```
.card { display: flex; gap: 12px; }
```

```
@media (max-width: 768px) { .card { display: block; } }
```

3 권 · 레이아웃과 반응형

HTML 과 CSS

HTML 로 글에 이름표를 붙이고, CSS 로 그 모양을 정합니다.

적힌 결과는 전부 진짜 크롬으로 열어서 재어 본 값입니다.

4

단원

20

개념

101

직접 해보기

108

문제

차례

11	배치 display와 position	11
01	문서 흐름	12
02	display	22
03	position 기초	34
04	absolute 와 fixed	44
05	쌓임 순서와 z-index	59

12	Flexbox	91
01	flex 시작하기	92
02	방향과 줄바꿈	108
03	주축 정렬	123
04	교차축 정렬	138
05	늘어나고 줄어들기	155
06	flex로 레이아웃 만들기	172

13	Grid와 반응형	207
01	grid 시작하기	208
02	칸 합치기와 영역	224
03	화면 크기와 미디어쿼리	241
04	반응형 단위와 이미지	255
05	모바일 우선으로 만들기	271

14	종합 연습	311
01	프로필 카드	312
02	음식점 메뉴판	321
03	회원가입 폼	334
04	반응형 랜딩페이지	343

부록 가	연습문제·종합연습 정답	359
------	---------------------	-----

부록 나	심화문제 정답	455
------	----------------	-----

여는 글

시작하기 전에

다 배우고 나면 이런 것을 직접 만듭니다. 지금은 무슨 코드인지 하나도 몰라도 됩니다.

이 책의 표시

블록마다 왼쪽 가장자리 표시가 다릅니다. 표시만 보고 “지금 내가 무엇을 해야 하는지” 알 수 있습니다.

```
console.log("안녕하세요");
```

— 직접 쳐 볼 코드입니다. 파일을 열고 그대로 따라 치세요.

출력 **안녕하세요**

— 실행하면 컴퓨터가 내놓는 값입니다. 내 화면과 같은지 맞춰 보세요.

▢ 직접 해보기 **1**

자기 이름을 출력해 보세요.

— **여러분 차례입니다.** 이 표시가 보이면 손을 움직이세요. 정답은 그 개념 맨 끝에 있습니다.

여기에 코드를 쓰세요

— 연필로 직접 적는 자리입니다.

이런 실수를 합니다

따옴표를 빠뜨림

```
console.log(안녕하세요);
```

따옴표가 없으면 컴퓨터는 ‘안녕하세요’라는 이름의 변수를 찾습니다.

— 여기서 많이 넘어집니다. 미리 보고 피해 가세요.

RUN node 개념01_console_log로_출력하기.js

— 개념마다 맨 위에 있습니다. 이대로 실행하면 됩니다.

시작 전 준비 (처음 한 번만)

자바스크립트 자료와 달리 **설치할 프로그램이 거의 없습니다**. HTML 과 CSS 는 브라우저만 있으면 바로 돌아갑니다.

1. 브라우저

크롬(Chrome) 또는 **엣지(Edge)** 를 쓰세요. 이 자료의 설명은 크롬 기준입니다. 이미 깔려 있을 것입니다.

2. VS Code 설치

코드를 쓰는 편집기입니다. <https://code.visualstudio.com> 에서 받아 설치하세요.

설치 후 VS Code 를 열고 **파일 → 폴더 열기**로 이 **HTMLCSS자료** 폴더를 통째로 여세요. 파일 하나만 여는 것보다 폴더째 여는 편이 훨씬 편합니다.

꼭 하세요 — 화면 나누기 HTML/CSS 는 **코드와 결과를 동시에 봐야** 배워집니다. 화면 왼쪽 절반에 VS Code, 오른쪽 절반에 브라우저를 띄워 두세요. 윈도우에서는 창을 잡고 화면 왼쪽·오른쪽 끝으로 밀면 반씩 채워집니다.

3. 파일을 브라우저로 여는 법

.html 파일을 **더블클릭**하면 브라우저가 열립니다. 그게 전부입니다.

더블클릭했는데 **VS Code** 가 열린다면 — 파일 우클릭 → 연결 프로그램 → 크롬(또는 엣지) → **"항상 이 앱으로 열기"** 체크. VS Code 를 설치하면 자주 이렇게 됩니다.

이것만은 먼저 (제일 많이 하는 실수)

.html 파일은 **저장한다고 화면이 저절로 바뀌지 않습니다**.

VS Code 에서 고친다 → Ctrl + S 로 저장 → 브라우저로 가서 F5 (새로고침)

이 세 단계를 매번 해야 합니다. **"코드는 고쳤는데 화면이 그대로예요"**의 1순위 원인이 **저장을 안 했거나 새로고침을 안 한 것**입니다. 이 자료의  **직접 해보기**는 전부 이 과정을 요구합니다.

파일 읽는 법

개념 파일 (개념01_..., 개념02_...)

브라우저와 VS Code 로 동시에 여세요. 둘을 번갈아 보는 것이 이 자료를 쓰는 방법입니다.

- 브라우저 화면에는 섹션 제목, 짧은 요약, 그리고 실제 결과가 보입니다.
- VS Code 소스에는 그 결과가 어떻게 나왔는지 자세한 설명이 주석으로 들어 있습니다.

파일은 항상 같은 순서로 되어 있습니다.

1. 맨 위 주석 — 이 파일이 무엇을 다루는지, 앞 파일과 어떻게 이어지는지
2. 섹션 N – 제목 · 주제가 바뀌는 지점
3. 점선 상자 — 실제 결과가 그려지는 곳. 소스에서는 `<div class="doc-demo">` 입니다
4.  직접 해보기 · 거의 모든 섹션에 하나씩 있는 짧은 과제. 파란 상자로 화면에도 보입니다. (자주 하는 실수 와 정리 섹션에는 없습니다) 정답은 파일 맨 아래 직접 해보기 정답 주석 블록에 있습니다. 먼저 스스로 해 보세요
5. 자주 하는 실수 · 학생들이 실제로 자주 하는 실수 모음. 빨간 상자(잘못된 예)와 초록 상자(올바른 예)가 나란히 나옵니다
6. — 정리 — · 그 파일의 핵심을 대여섯 줄로 추린 것. 복습할 때 여기만 봐도 됩니다

결과를 적어 둔 주석

주석	뜻	쓰는 곳
<code><!-- 화면: ... --></code>	이 코드가 브라우저에 어떻게 보이는지	HTML 안
<code>/* 화면: ... */</code>	이 규칙이 화면을 어떻게 바꾸는지	<code><style></code> 안

전부 "열어 보면 이렇게 나온다" 는 뜻입니다. 자료에 적힌 결과는 전부 진짜 크롬으로 열어서 재어 본 값입니다. 다르게 나오면 어딘가 잘못 친 것입니다.

파일 맨 아래에 아래는 자료 자동 검사용입니다 라고 적힌 주석 블록이 있습니다. 자료가 스스로 틀리지 않았는지 검사하는 장치입니다. 학습 내용이 아니니 넘어가세요.

연습 파일 (연습문제.html)

TODO가 적힌 빈칸을 채우는 파일입니다. 각 문제는 이렇게 생겼습니다.

```
<!-- —— 문제 1 —— (개념01 섹션2)
```

```
h1 태그로 "우리 동네 빵집" 이라는 제목을 만드세요.
```

```
기대 화면: 큰 굵은 글씨로 "우리 동네 빵집" 이 한 줄 보입니다.
```

```
이렇게 되면 틀린 것: 글씨 크기가 그대로면 h1 이 아니라 p 를 쓴 것입니다.
```

```
-->
```

- 기대 화면과 똑같이 나오면 정답입니다.
- 막히면 연습문제_정답.html 을 보세요. 다만 먼저 10분은 혼자 고민하고 보세요.
- 문제는 기본 → [응용] → [도전] 순서로 어려워집니다. [도전]은 못 풀어도 괜찮습니다.
- 맨 마지막 문제는 [확인] 문항입니다. 일부러 틀리게 써 둔 코드를 주고 "왜 아무 일도 안 일어나는지 찾아서 고치세요"를 시킵니다.

HTML 과 CSS 에는 에러 메시지가 없습니다

이 자료에서 가장 중요한 이야기입니다.

다른 프로그래밍 언어는 틀리면 빨간 글씨로 알려 줍니다. HTML 과 CSS 는 그러지 않습니다. 틀린 부분을 조용히 무시하고, 나머지만 그립니다.

```
태그 이름을 틀림 → 그 태그가 없는 셈 치고 안의 글자만 나옵니다
CSS 속성 이름을 틀림 → 그 한 줄만 버려집니다. 나머지는 멀쩡히 적용됩니다
세미콜론을 빠뜨림 → 그 줄과 다음 줄이 같이 버려지기도 합니다
클래스 이름이 안 맞음 → 아무 일도 일어나지 않습니다
```

그래서 배우는 방법이 다릅니다.

에러를 읽는 게 아니라, 화면이 생각과 다른지 눈으로 알아채야 합니다.

"아무 일도 안 일어난다"가 이 자료에서 가장 자주 만날 증상입니다. 당황하지 말고 아래 순서로 확인하세요.

- 1. 저장했나? (Ctrl + S) → 새로고침했나? (F5)
- 2. 오타는 없나? 특히 클래스 이름, 속성 이름, 콜론과 세미콜론
- 3. F12 를 눌러 개발자도구에서 실제로 어떻게 적용됐는지 본다 (바로 아래)

개발자도구(F12) — 이 자료의 필수 도구

브라우저에서 F12 를 누르면 개발자도구가 열립니다. HTML/CSS 는 이 도구 없이는 배우기 어렵습니다. 10만원부터는 사실상 필수입니다.

하고 싶은 것	방법
이 글자가 어느 태그인지 보기	화면에서 우클릭 → 검사
지금 적용된 CSS 보기	Elements 탭 → 오른쪽 Styles
실제 계산된 값 보기	Elements 탭 → 오른쪽 Computed
박스 모델(여백) 그림 보기	Computed 탭 맨 위의 네모 그림
취소선 그어진 CSS	다른 규칙에 밀려서 안 먹은 것 (08단원 명시도)
휴대폰 크기로 보기	Ctrl + Shift + M (13단원)

F12 를 눌러도 아무 일이 없다면 — 노트북은 Fn + F12 이거나, Ctrl + Shift + I 로도 열립니다. 화면 아무 곳이나 우클릭 → 검사 도 됩니다.

Styles 패널에서는 값을 직접 고쳐 볼 수도 있습니다. 숫자를 클릭하고 위아래 화살표를 누르면 화면이 실시간으로 바뀝니다. 새로고침하면 원래대로 돌아오니 마음껏 만져 보세요.

혼자 공부할 때

1. 브라우저와 VS Code 를 나란히 띄웁니다. 이것부터 하세요.
2. 브라우저에서 결과를 보고, "이건 어느 코드에서 나온 거지?" 를 소스에서 찾습니다.
3. 소스의 주석 설명을 읽습니다. 여기가 진짜 본문입니다.
4.  직접 해보기 는 건너뛰지 마세요. 읽는 것과 직접 고쳐 보는 것은 완전히 다릅니다. 고치고 → 저장하고 → 새로고침하는 습관이 이 자료의 목표 중 하나입니다.
5. 자주 하는 실수 는 빨간 상자와 초록 상자를 비교하면서 읽습니다.
6. 단원이 끝나면 연습문제 를 풀고, 그다음 연습문제_정답 과 비교합니다.
7. 뭔가 이상하면 F12 를 먼저 누르세요. 답은 거의 항상 거기 있습니다.

파일을 망가뜨렸을까 봐 걱정된다면 — 이 폴더를 통째로 한 번 복사해 두고 시작하세요. 마음껏 고칠 수 있어야 빨리 늡니다.

배치 display와 position

OPEN 11_배치_display와_position

저는 블록입니다

저도 블록입니다

```
.gap { margin-right: 40px; }
```

<p>위</p>

01 문서 흐름

02 display

03 position 기초

04 absolute 와 fixed

05 쌓임 순서와 z-index

- 연습문제

문서 흐름

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.



실행하면 나오는 화면 — 문서 흐름

07단원부터 10단원까지 우리는 색, 글자, 상자 크기를 바꿨습니다. 그런데 “이 상자를 저기에 놓아라” 라고 시킨 적은 한 번도 없습니다. 그래도 상자들은 전부 어딘가에 놓여 있었습니다.

브라우저가 알아서 놓았기 때문입니다. 02단원 개념04 에서 “글자와 상자가 흘러가는 것” 을 문서 흐름이라고 불렀습니다. 그 ‘알아서 놓는 기본 규칙’ 의 정식 이름이 문서 흐름(normal flow) 입니다.

규칙은 딱 두 줄입니다.

- 1 위에서 아래로 — 덩어리(블록) 는 한 줄을 통째로 차지하고 세로로 쌓입니다
- 2 왼쪽에서 오른쪽으로 — 글자 같은 것(인라인) 은 옆으로 이어지다가

줄이 꽉 차면 다음 줄로 내려갑니다



11단원에서 배우는 것은 새로운 배치법이 아닙니다. 이미 돌아가고 있는 이 흐름을 어떻게 바꾸는가입니다. 그러니 먼저 흐름이 무엇인지부터 눈으로 봐 둡시다.

이 파일은 배치를 바꾸는 방법을 배우지 않습니다. 바꾸기 전에 지금 이미 무슨 규칙으로 놓이고 있는지를 봅니다. 아래 상자들을 보면서 “왜 이걸 세로로 쌓이고, 저건 옆으로 붙지?” 를 계속 물어 보세요.

아무것도 안 시켜도 배치는 끝나 있습니다

섹션 1

아래 상자 세 개에는 위치를 정하는 CSS 가 하나도 없습니다. 그런데도 순서대로 나란히 놓여 있습니다.

아래 상자 세 개에 준 CSS 는 이것이 전부입니다.

```
.flow-box {
  background: #dbe7fb;      색
  border: 1px solid #2d6cdf; 테두리
  padding: 8px 10px;       안쪽 여백
  margin: 6px 0;           바깥 여백
}
```

전부 10단원에서 배운 ‘모양’ 속성입니다. ‘위치’ 속성은 하나도 없습니다. 그런데 상자는 위에서 아래로 차곡차곡 쌓였고, 가로로는 쓸 수 있는 폭을 꽉 채웠습니다.

누가 그렇게 하라고 시켰을까요? 아무도 안 시켰습니다. 브라우저의 기본 규칙, 즉 문서 흐름이 한 것입니다.

02단원 개념04 에서 “글자가 흘러가는 것” 을 문서 흐름이라고 불렀습니다. 여기서 그 용어를 다시 또렷하게 못 박아 둡니다.

문서 흐름(normal flow)
= 우리가 위치에 대해 아무 지시도 하지 않았을 때
브라우저가 요소를 놓는 기본 순서

“배치를 배운다” 는 말은 사실 “이 기본 순서를 어떻게 어길 것인가를 배운다” 입니다.

글자는 짧은데 상자는 점선 상자의 왼쪽 끝부터 오른쪽 끝까지 꼭 찹니다

첫 번째 상자
두 번째 상자
세 번째 상자

상자의 가로 폭은 글자 길이와 상관없이 없습니다. 글자가 “첫 번째 상자” 여섯 글자뿐인데도 상자는 끝까지 늘어나 있습니다. 이것이 다음 섹션에서 볼 블록의 성질입니다.

직접 해보기 1

위 상자 아래에 <div class="flow-box">네 번째 상자</div> 를 한 줄 더 넣어 보세요. 저장하고 (Ctrl+S) 브라우저에서 새로고침(F5) 하면 상자가 하나 더 쌓입니다.

세로로 쌓이는 것과 옆으로 흐르는 것

섹션 2

같은 흐름 안에서도 요소는 두 부류로 나뉩니다. 한 줄을 통째로 쓰는 것과 글자처럼 옆에 붙는 것입니다.

02단원 개념04 에서 이 두 부류의 이름을 이미 봤습니다.

블록(block) 요소 : `div p h1 ul li section header ...`
한 줄을 통째로 차지합니다. 그래서 다음 요소가 아래로 밀립니다.

인라인(inline) 요소 : `span strong em a img ...`
글자 한 덩어리처럼 취급됩니다. 그래서 옆에 그대로 붙습니다.

그때는 “그렇다더라”로 넘어갔습니다. 여기서는 눈으로 확인합니다.

아래 데모의 노란 조각은 전부 span 입니다. span 은 인라인이라 옆으로 이어지고, 한 줄이 꽉 차면 그다음 줄로 내려갑니다. 줄을 어디서 바꿀지는 브라우저가 창 폭을 보고 그때그때 정합니다. 브라우저 창 폭을 좁혔다 넓혔다 해 보면 줄바꿈 위치가 따라 움직이는 것이 보입니다.

블록  ← 한 줄 전부가 내 것

인라인 [조각][조각][조각][조각]

조각 · [조각] ← 자리가 없으면 다음 줄로

화면 두 상자가 위아래로 놓입니다. 옆으로는 절대 안 붙습니다.

저는 블록입니다
저도 블록입니다

화면 노란 조각들이 한 줄에 옆으로 붙어 가다가 오른쪽 끝에서 다음 줄로 넘어가

두 줄이 됩니다. 브라우저 창을 좁히면 넘어가는 지점이 앞으로 당겨집니다

저는
인라인
이라서
옆으로
계속
이어집니다
그러다가
줄이
모자라면
그때
다음
줄로
내려갑니다
이
문장은
한
줄에
다
들어가지
않을
만큼
길게
만들어
두었습니다

블록과 인라인은 태그마다 이미 정해져 있는 성질입니다. 그 성질을 사람이 바꾸는 방법이 display 이고, 바로 다음 파일에서 배웁니다.

▸ 직접 해보기 2

위의 노란 조각 데모에서 `<span class="flow-word">정말</span>` 을 다섯 개쯤 더 넣어 보세요. 줄이 몇 개로 늘어나는지 세어 보세요.

흐름은 소스에 쓴 순서를 그대로 따릅니다

섹션 3

흐름의 순서는 여러분이 코드를 쓴 순서입니다. 다만 코드에서 줄을 바꾼 것은 화면의 줄바꿈이 아닙니다.

아래 초록 조각 세 개는 코드에서 이렇게 각각 다른 줄에 썼습니다.

```
<span class="order-word" id="n11">첫째</span>
<span class="order-word" id="n12">둘째</span>
<span class="order-word" id="n13">셋째</span>
```

코드로는 세 줄인데 화면에서는 한 줄로 나옵니다. HTML 에서 줄바꿈과 연속된 공백은 전부 '공백 한 칸'으로 합쳐지기 때문입니다.

코드	첫째	둘째	셋째	은 엔터
화면	첫째	둘째	셋째	공백 한 칸씩

그래서 이런 일이 벌어집니다.

"코드에서 엔터를 쳤는데 화면에서 줄이 안 바뀝니다"	→ 정상입니다
"스페이스를 열 번 쳤는데 한 칸만 벌어집니다"	→ 정상입니다

화면에서 진짜로 줄을 바꾸려면 br 태그(02단원)를 쓰거나, 아예 블록 요소로 만들어야 합니다(개념02). 간격을 벌리려면 스페이스가 아니라 margin-padding(10단원)을 씁니다.

화면	초록 조각 세 개가 한 줄로 나란히 놓이고 사이가 한 칸씩만 벌어집니다		
첫째 둘째 셋째			
화면	날말 사이가 한 칸씩만 벌어집니다. 여러 칸이 그대로 보이지 않습니다		
공백을	아주	많이	쳤습니다

▸ 직접 해보기 3

위 "공백을 아주 많이 쳤습니다" 줄의 공백 자리에
 를 하나 넣어 보세요. 그때는 진짜로 줄이 바뀝니다.

이 단원의 지도

섹션 4

11단원에서 배우는 것은 결국 흐름을 바꾸는 손잡이 세 개입니다. 지금 전부 이해할 필요는 없습니다. 어디로 가는지만 봐 두세요.

손잡이 1 — display (개념02)

"이 요소는 한 줄을 통째로 쓰는 놈인가, 글자처럼 옆에 붙는 놈인가"를 바꿉니다. 흐름 안에 그대로 있으면서 성질만 바꿉니다.

손잡이 2 — position (개념03·개념04)

"이 요소를 흐름에서 아예 꺼내서 내가 찍은 좌표에 놓겠다" 입니다. 흐름에서 꺼내면 다른 요소들은 그 요소가 없는 것처럼 자리를 메웁니다.
--

손잡이 3 — z-index (개념05)

손잡이 2 를 쓰면 요소가 겹칠 수 있습니다.
겹쳤을 때 누가 위로 올라올지를 정합니다.

순서가 중요합니다. 아래로 갈수록 흐름을 많이 어깁니다.

흐름 그대로 → display 로 성질만 바꿈 → position 으로 흐름에서 꺼냄

많이 어길수록 나중에 고치기 어려워집니다. 그래서 실무의 원칙은 이렇습니다.

될 수 있으면 흐름에 맡기고, 흐름으로 안 되는 것만 position 을 씁니다.

참고로 여러 개를 가로로 나란히 놓는 진짜 배치 도구는 12단원 flex 와 13단원 grid 입니다. 이 단원은 그 앞의 기초입니다.

화면 회색 상자 안에 점 목록 세 줄이 보이고 각 줄 앞부분이 굵게 보입니다

display – 한 줄 통째로 쓸지, 글자처럼 옆에 붙을지 (개념02)
position – 흐름에서 꺼내 원하는 자리에 놓기 (개념03, 개념04)
z-index – 겹쳤을 때 누가 위로 올라올지 (개념05)

▸ 직접 해보기

4

위 목록에 float — 글 옆에 흘리기 (개념05) 를 한 줄 더 넣어 보세요. 목록이 네 줄이 됩니다.

자주 하는 실수

섹션 5

CSS 도 HTML 과 똑같습니다. 틀려도 에러가 나지 않습니다. 그 줄이 조용히 무시될 뿐입니다. 그래서 “아무 일도 안 일어난다” 가 가장 흔한 증상입니다.

이런 실수를 합니다

스페이스로 간격을 만들려고 함

이런 실수를 합니다

실수 1 — 스페이스를 여러 번 쳐서 간격을 만들려 함

```
<span>왼쪽</span> <span>오른쪽</span>
```

한 칸만 벌어집니다. 연속된 공백은 한 칸으로 합쳐지기 때문입니다. 간격은 10단원에서 배운 margin 이나 padding 으로 만듭니다. 왼쪽 오른쪽

이렇게 하세요

```
.gap { margin-right: 40px; }
```

이런 실수를 합니다

br 을 여러 개 쳐서 세로 간격을 만듦

이런 실수를 합니다

실수 2 — **br** 을 여러 개 써서 세로 간격을 만듦

```
<p>위</p>
<br /><br /><br />
<p>아래</p>
```

화면은 일단 벌어집니다. 그래서 더 나쁩니다. 글자 크기를 바꾸면 간격도 같이 바뀌고, 나중에 margin 으로 고치려 할 때 어디가 진짜 간격인지 못 찾습니다. 세로 간격은 margin 하나로만 만드세요.

이런 실수를 합니다

블록 두 개를 그냥 나란히 쓰면 옆에 붙을 거라 생각

이런 실수를 합니다

실수 3 — **블록** 두 개를 붙여 쓰면 옆에 놓일 거라고 생각함

```
<div>왼쪽 칸</div><div>오른쪽 칸</div>
```

코드에서 붙여 써도 화면에서는 위아래로 쌓입니다. 블록은 한 줄을 통째로 차지한다는 규칙이 먼저이기 때문입니다. 가로로 놓으려면 개념02 의 inline-block 이나 12단원의 flex 가 필요합니다. 왼쪽 칸오른쪽 칸

이런 실수를 합니다

처음부터 **position** 으로 모든 것을 배치

이런 실수를 합니다

실수 4 — 처음부터 좌표로 전부 배치하려 함

```
.a { position: absolute; top: 100px; left: 0; }
.b { position: absolute; top: 160px; left: 0; }
.c { position: absolute; top: 220px; left: 0; }
```

당장은 맞아 보입니다. 그런데 .a 의 글자가 한 줄 늘어나는 순간 세 개가 서로 겹칩니다. 흐름에서 꺼낸 요소는 서로를 밀어내지 않기 때문입니다. 흐름에 맡길 수 있는 것은 끝까지 흐름에 맡기세요.

이런 실수를 합니다

코드의 줄바꿈이 화면 줄바꿈이라고 생각

이런 실수를 합니다

실수 5 — 코드에서 엔터를 쳤으니 화면도 줄이 바뀔 거라 생각함

```
<span>첫째</span>
<span>둘째</span>
```

화면에서는 한 줄로 이어집니다. 엔터는 공백 한 칸이 됩니다. 섹션 3 에서 이미 눈으로 확인했습니다. 진짜 줄바꿈은 `<br />` 이거나 블록 요소입니다.

정리

▸ 직접 해보기 5

정답

1) 섹션 1 의 점선 상자 안, 세 번째 상자 아래에 이렇게 넣습니다.

```
<div class="flow-box">네 번째 상자</div>
```

→ 상자가 네 개로 늘어나고, 아래쪽 내용이 전부 그만큼 밀려 내려갑니다.

흐름 안에 있는 요소는 이렇게 서로를 밀어냅니다.

2) 섹션 2 의 노란 조각 데모 마지막에 이렇게 다섯 줄을 넣습니다.

```
<span class="flow-word">정말</span>
<span class="flow-word">정말</span>
<span class="flow-word">정말</span>
<span class="flow-word">정말</span>
<span class="flow-word">정말</span>
```

→ 조각이 늘어나 두 줄이던 것이 세 줄이 됩니다.

어디서 줄이 바뀌는지는 브라우저가 창 폭을 보고 정합니다.

3) 섹션 3 의 줄을 이렇게 바꿉니다.

```
<div class="order-line" id="spaceLine">공백을 <br />아주 많이 쳤습니다</div>
```

→ “공백을” 다음에서 진짜로 줄이 바뀌어 두 줄이 됩니다.

공백은 합쳐지지만 `br` 은 합쳐지지 않는다는 차이를 확인하세요.

4) 섹션 4 의 목록 마지막 li 아래에 이렇게 넣습니다.

float - 글 옆에 흘리기 (개념05)

→ 목록이 세 줄에서 네 줄이 되고, 회색 상자가 그만큼 높아집니다.

ul 도 li 도 블록이라 세로로 쌓입니다.

이 파일에는 '가르치는 CSS' 가 거의 없습니다.

문서 흐름은 우리가 아무 CSS 도 쓰지 않았을 때 이미 벌어지고 있는 일이라서,
가르칠 속성이 아니라 '보여 줄 사실' 이기 때문입니다.

아래 규칙들은 상자가 어디까지인지 눈에 보이게 만드는 색과 테두리뿐입니다.

섹션 1·2: 블록 요소를 눈에 보이게

```
.flow-box {  
  background: #dbe7fb;  
  border: 1px solid #2d6cdf;  
  padding: 8px 10px;  
  margin: 6px 0;  
}
```

화면 연한 파란 상자가 위에서 아래로 하나씩 쌓이고, 상자 사이가 조금 벌어집니다

인라인 요소를 눈에 보이게

섹션 2

```
.flow-word {  
  background: #ffe08a;  
  border: 1px solid #d9a100;  
  padding: 2px 6px;  
}
```

화면 노란 조각이 한 줄에 옆으로 이어지고, 줄이 꽉 차면 다음 줄로 내려갑니다

소스 순서 데모

섹션 3

```
.order-line {
  background: #f1f1f1;
  border: 1px solid #ccc;
  padding: 10px;
  margin: 6px 0;
}
.order-word {
  background: #d9f0d9;
  border: 1px solid #6aa86a;
  padding: 2px 6px;
}
```

화면 회색 띠 안에 초록 조각 세 개가 한 줄로 나란히 놓입니다

지도

섹션 4

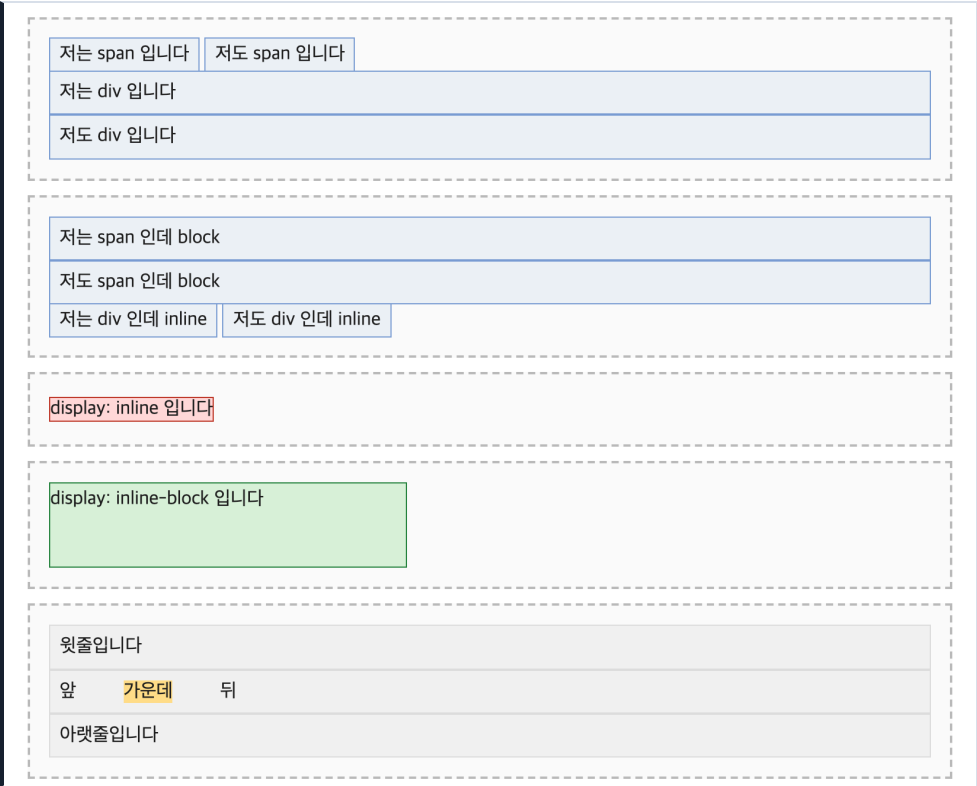
```
.map-list {
  background: #f8f8f8;
  border: 1px solid #ddd;
  padding: 12px 12px 12px 34px;
  margin: 6px 0;
}
```

화면 회색 상자 안에 번호 없는 목록 세 줄이 보입니다

display

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.



실행하면 나오는 화면 — display

02단원 개념04 에서 태그를 두 부류로 나눴습니다.

블록(block) div p h1 ul li section ... 한 줄을 통째로 차지
인라인(inline) span strong em a ... 글자처럼 옆에 붙음

그때는 “태그마다 정해져 있다” 까지만 말하고 끝냈습니다. 그 이야기를 여기서 끝냅니다. 정해져 있는 그 성질을 바꾸는 속성이 display 입니다.

display: block	한 줄을 통째로 차지하는 놈으로 취급하라
display: inline	글자처럼 옆에 붙는 놈으로 취급하라
display: inline-block	옆에는 붙되, 크기는 내가 정한 대로 하라
display: none	아예 없는 셈 처라

이 파일에서 가장 중요한 한 줄은 이것입니다.

inline 요소에는 width, height, 위아래 margin 이 먹지 않습니다.

“먹지 않는다” 는 말은 에러가 난다는 뜻이 아닙니다. CSS 는 그 줄을 암전히 받아 두고, 화면을 그릴 때만 안 씁니다. 그래서 초보자가 가장 오래 헤매는 자리입니다. 섹션 2 와 섹션 3 에서 진짜로 재서 확인합니다.

02단원에서 남겨 둔 블록·인라인 이야기를 여기서 끝냅니다. 특히 섹션 2, 섹션 3 은 초보자가 가장 많이 걸려 넘어지는 자리입니다. “분명히 width 를 줬는데 안 바뀐다” 의 정체가 여기 있습니다.

태그의 기본 성질을 맞추기

섹션 1

태그는 저마다 기본 display 값을 갖고 태어납니다. 그 값을 CSS 로 덮어쓸 수 있습니다.

태그마다 정해진 기본값이 있습니다. 대표적인 것만 적으면 이렇습니다.

```
display: block 인 태그 div p h1~h6 ul ol section header footer
display: inline 인 태그 span strong em a label
display: list-item 인 태그 li (블록처럼 쌓이면서 앞에 점이 붙습니다)
```

아래 첫 상자는 기본값 그대로입니다. span 두 개는 옆으로 붙고, div 두 개는 위아래로 쌓입니다. 예상한 그대로입니다.

두 번째 상자에서는 CSS 로 성질을 맞바꿨습니다.

```
.span-as-block { display: block; } span 을 블록으로
.div-as-inline { display: inline; } div 를 인라인으로
```

결과가 정확히 뒤집힙니다. span 이 위아래로 쌓이고 div 가 옆으로 붙습니다.

여기서 중요한 사실이 하나 나옵니다.

태그의 뜻(의미) 과 화면 모양(display) 은 별개입니다.

div 를 inline 으로 만들었다고 div 가 span 이 되는 것은 아닙니다. 그러니 “옆으로 붙이고 싶으니까 span 을 쓰자” 라고 태그를 고르지 마세요. 태그는 뜻으로 고르고(06단원), 모양은 display 로 맞춥니다.

화면 span 두 개는 한 줄에 나란히 붙고, 그 아래 div 두 개는 각각

한 줄씩 차지하며 위아래로 쌓입니다

```
저는 span 입니다
저도 span 입니다
저는 div 입니다
저도 div 입니다
```

화면 위의 span 두 개가 위아래로 쌓이고, 아래 div 두 개가 한 줄에 나란히 붙습니다.

첫 상자와 정확히 반대 모양입니다

저는 span 인데 block
저도 span 인데 block
저는 div 인데 inline
저도 div 인데 inline

▹ 직접 해보기

1

<style> 의 .div-as-inline 을 display: block; 으로 바꿔 보세요. 아래 div 두 개가 다시 위아래로 쌓입니다. 확인했으면 inline 으로 되돌려 놓으세요.

inline 에는 width·height 가 안 먹습니다

섹션 2

아래 두 조각에는 똑같이 width: 300px; height: 70px; 를 줬습니다. 그런데 하나만 커집니다.

10단원에서 배운 width 와 height 를 인라인 요소에 줘 봅니다.

```
.size-inline { display: inline; width: 300px; height: 70px; }  
.size-iblock { display: inline-block; width: 300px; height: 70px; }
```

두 줄의 차이는 display 하나뿐입니다. 그런데 결과는 완전히 다릅니다.

inline 빨간 조각. 글자 길이만큼만 칠해집니다. 300px 이 안 됩니다.
inline-block 초록 상자. 진짜로 302px x 72px 이 됩니다.
(테두리 1px 이 양쪽에 붙어서 300+2, 70+2 입니다 - 10단원 박스 모델)

왜 이럴까요? 인라인 요소는 브라우저가 '글자 덩어리' 로 취급하기 때문입니다. 글자에 "너 가로 300px 이 돼라" 라고 하는 것은 말이 안 됩니다. 글자는 글자 수만큼만 자리를 씁니다. 그래서 그 지시를 그냥 안 씁니다.

여기서 가장 중요한 것은 다음입니다.

에러가 나지 않습니다. 경고도 없습니다. 아무 일도 안 일어납니다.

개발자 도구(F12) 로 보면 width: 300px 이라는 줄은 멀쩡히 살아 있습니다. 살아 있는데 화면 계산에는 안 쓰입니다. 이것이 CSS 를 배울 때 가장 낯선 부분입니다.

화면 글자 길이만큼만 연분홍으로 칠해진 한 줄짜리 조각입니다.

가로 300px, 세로 70px 은 어디에도 없습니다

display: inline 입니다

화면 가로로 넓고 세로로도 두꺼운 연초록 상자가 생깁니다.

자를 대고 재면 302 × 72 픽셀입니다

display: inline-block 입니다

같은 이유로 인라인 요소에는 height 도 안 먹습니다. 인라인 조각의 세로 크기는 글자 크기와 line-height(09단원) 가 정합니다.

▹ 직접 해보기 2

.size-inline 의 width 를 900px 로 바꿔 보세요. 그래도 아무 일도 안 일어납니다. 값이 문제가 아니라 display 가 문제라는 것을 확인하는 것이 목적입니다.

inline 에는 위아래 margin 도 안 먹습니다

섹션 3

margin: 30px 40px 을 똑같이 줬는데, 인라인에서는 좌우 40px 만 먹고 위아래 30px 은 무시됩니다.

10단원에서 margin 을 배웠습니다. margin: 30px 40px 은 이런 뜻이었습니다.

위 30px 오른쪽 40px 아래 30px 왼쪽 40px

이 값을 인라인 조각에 그대로 줍니다.

```
.mg-inline { display: inline;      margin: 30px 40px; }
.mg-iblock { display: inline-block; margin: 30px 40px; }
```

두 데모 모두 회색 띠 세 개를 위아래로 붙여 놓고, 가운데 띠 안에만 노란 조각을 넣었습니다. 노란 조각의 좌우로는 "앞", "뒤" 라는 글자가 있습니다.

첫 번째 데모(inline) 를 보세요.

좌우 "앞" 과 "뒤" 가 노란 조각에서 멀리 떨어졌습니다 → 40px 이 먹었습니다
위아래 회색 띠 세 개는 여전히 딱 붙어 있습니다 → 30px 은 안 먹었습니다

두 번째 데모(inline-block) 를 보세요.

가운데 회색 띠만 위아래로 두꺼워졌습니다 → 30px 이 먹었습니다

즉 인라인에서 무시되는 것은 margin 전체가 아니라 위아래 방향뿐입니다. padding 도 비슷합니다.

인라인에 위아래 padding 을 주면 배경색은 위아래로 번지지만 주변 줄을 밀어내지는 못해서, 윗줄 글자와 겹쳐 보이는 일이 생깁니다.

기억할 한 줄은 이것입니다.

세로 방향으로 무언가를 하고 싶으면 그 요소는 인라인이면 안 됩니다.

화면 노란 조각 좌우로만 넓게 벌어지고, 회색 띠 세 개는 그대로 딱 붙어 있습니다

윗줄입니다
앞가운데뒤
아랫줄입니다

화면 가운데 회색 띠가 위아래로 두꺼워져서 아랫줄이 아래로 밀려납니다

윗줄입니다
앞가운데뒤
아랫줄입니다

재미있는 것은, 개발자 도구로 첫 번째 노란 조각을 찍어 보면 margin-top 이 30px 이라고 그대로 적혀 있다는 점입니다. 값은 들어가 있는데 화면 계산에서만 빠집니다. 이 파일 맨 아래 자동 검사에서도 그 사실을 그대로 확인합니다.

▣ 직접 해보기 3

.mg-inline 의 margin 을 80px 40px 으로 바꿔 보세요. 위아래는 여전히 꿈쩍도 안 합니다. 확인했으면 30px 40px 으로 되돌려 놓으세요.

inline-block 이 문제를 풀니다

섹션 4

inline-block 은 바깥에서는 인라인, 안에서는 블록입니다. 옆으로는 붙으면서 크기는 내 마음대로 정합니다.

지금까지 나온 세 값을 한자리에 놓고 비교합니다.

| | 옆으로 붙나? | width·height 가 먹나? | 위아래 margin 이 먹나? |
|--------------|---------|--------------------|------------------|
| block | X | 0 | 0 |
| inline | 0 | X | X |
| inline-block | 0 | 0 | 0 |

inline-block 은 두 좋은 점만 골라 가진 값입니다. 그래서 “상자 여러 개를 가로로 나란히” 를 만들 때 오랫동안 이것을 썼습니다.

```
.card { display: inline-block; width: 150px; height: 80px; }
```

아래 카드 세 장이 그렇게 만든 것입니다. 각 카드의 콘텐츠 영역은 진짜로 150px 이고, 여기에 안쪽 여백 8px 씩과 테두리 2px 씩을 더해 화면에서는 170px 을 차지합니다(10단원 박스 모델). 그러면서 셋이 한 줄에 나란히 놓입니다.

다만 inline-block 에는 성가신 점이 하나 있습니다. 카드와 카드 사이에 아무도 안 준 틈이 6px 쯤 생깁니다.

```
<div class="card">카드 1</div>
<div class="card">카드 2</div>
    L 이 줄바꿈이 '공백 한 칸' 이 되어 카드 사이에 깎니다
    (개념01 섹션3 에서 본 그 규칙입니다)
```

인라인이니까 글자 취급을 받고, 그래서 단어 사이 공백이 생기는 것입니다. 이 틈을 없애는 방법은 여러 가지지만 전부 조금씩 지저분합니다. 그래서 요즘은 이런 가로 배치를 12단원의 flex 로 합니다. 지금은 inline-block 이 무엇인지, 왜 틈이 생기는지만 알면 충분합니다.

화면 가로 170px 짜리 카드 세 장이 한 줄에 나란히 놓이고,

카드 사이에 6px 쯤 되는 얇은 틈이 보입니다

```
카드 1
카드 2
카드 3
```

카드 안의 글자가 가운데로 온 것은 09단원에서 배운 text-align: center 때문입니다. display 가 한 일이 아닙니다. 두 가지를 섞어서 기억하지 마세요.

➤ 직접 해보기 4

.card 의 display 를 block 으로 바꿔 보세요. 카드 세 장이 위아래로 쌓입니다. 확인했으면 inline-block 으로 되돌려 놓으세요.

display: none 과 visibility: hidden

섹션 5

둘 다 눈에서 사라집니다. 차이는 딱 하나, 자리를 차지하느냐입니다.

무언가를 안 보이게 하는 방법이 두 가지 있습니다.

| | |
|--------------------|-----------------------|
| display: none | 없는 셈 칩니다. 자리도 안 남습니다. |
| visibility: hidden | 투명해집니다. 자리는 그대로 남습니다. |

아래 두 데모는 완전히 같은 구조입니다. B 줄을 숨기는 방법만 다릅니다.

| | | |
|---------|--------------|----------------------------|
| 첫 번째 데모 | A / (없음) / C | → C 가 위로 올라와 A 바로 밑에 붙습니다 |
| 두 번째 데모 | A / (빈칸) / C | → A 와 C 사이에 빈 줄만큼 자리가 남습니다 |

어느 쪽을 쓰면 될까요?

메뉴를 접었다 폼다 하듯 정말로 없애고 싶으면 → `display: none`
빈칸은 유지한 채 잠깐만 안 보이게 하고 싶으면 → `visibility: hidden`

실무에서는 `display: none` 을 훨씬 많이 씁니다. `visibility: hidden` 은 “자리가 흔들리면 안 되는” 자리에서만 씁니다.

참고로 `display: none` 인 요소는 화면에서 사라질 뿐 HTML 에서 지워진 것은 아닙니다. 코드에는 그대로 있고, 개발자 도구로도 보입니다.

화면 A 와 C 두 줄만 보이고, 두 줄이 서로 바로 붙어 있습니다.

B 가 있던 자리는 흔적도 없습니다

A 줄입니다
B 줄입니다 (`display: none`)
C 줄입니다

화면 A 와 C 사이에 한 줄만큼 빈자리가 뿔 뚫려 있습니다.

테두리도 글자도 안 보이는데 자리는 그대로입니다

A 줄입니다
B 줄입니다 (`visibility: hidden`)
C 줄입니다

`visibility` 는 `display` 와 아주 헷갈리는 이름이지만 전혀 다른 속성입니다. `display: hidden` 이나 `visibility: none` 같은 값은 없습니다. 그렇게 쓰면 그 줄은 조용히 무시됩니다.

▫ 직접 해보기 5

.none-box 를 `visibility: hidden;` 으로 바꿔 보세요. 두 데모가 똑같은 모양이 됩니다. 확인했으면 `display: none;` 으로 되돌리세요.

자주 하는 실수

섹션 6

여기 있는 것들은 전부 에러가 나지 않습니다. “왜 안 되지?” 하면서 오래 헤매게 되는 종류입니다.

이런 실수를 합니다
인라인에 `width` 주기

이런 실수를 합니다

실수 1 — 인라인 요소에 width 를 줌

```
<span class="btn">보내기</span>

.btn { width: 200px; background: #2d6cdf; }
```

아무 일도 안 일어납니다. 글자 길이만큼만 파랗게 칠해집니다. 버튼처럼 폭이 있는 것을 만들려면 display: inline-block 을 먼저 붙여야 합니다. 이 단원에서 가장 자주 나는 실수입니다.

이렇게 하세요

```
.btn {
display: inline-block;
width: 200px;
background: #2d6cdf;
}
```

이런 실수를 합니다

인라인에 위아래 margin 주기

이런 실수를 합니다

실수 2 — 인라인 요소에 위아래 margin 을 줌

```
.tag { margin-top: 40px; } /* .tag 는 span 입니다 */
```

위아래 간격이 하나도 안 벌어집니다. 그런데 좌우 margin 은 잘 먹습니다. 그래서 “margin 은 되는데 왜 위만 안 되지?” 하고 더 헛갈립니다. 세로로 밀고 싶으면 display 를 inline-block 이나 block 으로 바꾸세요.

이런 실수를 합니다

display: none 인데 자리는 남을 거라고 생각

이런 실수를 합니다

실수 3 — display: none 이면 빈칸은 남을 거라고 생각함

```
.msg { display: none; }
```

자리까지 통째로 사라집니다. 아래에 있던 요소들이 전부 위로 올라옵니다. 그래서 “숨겼더니 아래 내용이 갑자기 들쭉였다” 는 일이 생깁니다. 자리를 남기고 싶으면 visibility: hidden 을 쓰세요.

이런 실수를 합니다

inline-block 사이 틈을 이상하게 여김

이런 실수를 합니다

실수 4 — inline-block 사이의 틈을 **margin** 탓으로 오해함

```
.card { display: inline-block; margin: 0; }
```

margin: 0 을 줬는데도 카드 사이에 6px 쯤 틈이 남습니다. 그 틈은 margin 이 아니라 코드에서 카드 사이에 친 줄바꿈(공백)입니다. 인라인이라 글자 취급을 받아서 단어 사이 공백이 낀 것입니다. margin 을 아무리 0 으로 만들어도 사라지지 않습니다.

이런 실수를 합니다

값 이름을 잘못 씀

이런 실수를 합니다

실수 5 — 값 이름을 잘못 씀

```
.box { display: inlineblock; }  
.box { display: hidden; }
```

둘 다 그 줄 전체가 조용히 무시됩니다. 요소는 원래 성질 그대로 남습니다. inline-block 은 가운데 붙임표가 있어야 하고, 숨기는 값은 display: none 입니다. hidden 은 visibility 의 값입니다.

정리

▢ 직접 해보기 6

정답

1) style 안의 이 규칙을

```
.div-as-inline { display: inline; }
```

이렇게 바꿉니다.

```
.div-as-inline { display: block; }
```

→ “저는 div 인데 inline” 두 개가 한 줄에 붙어 있다가 위아래로 쌍입니다.

클래스 이름은 그대로여도 화면은 display 값을 따라갑니다.
이름이 아니라 값이 화면을 정한다는 점을 확인하세요.

2) .size-inline 의 width 를 900px 로 바꿉니다.

```
.size-inline { display: inline; width: 900px; height: 70px; }
```

→ 화면은 조금도 안 바뀝니다. 글자 길이만큼 그대로입니다.

값을 키운다고 되는 문제가 아니라는 것이 요점입니다.

3) .mg-inline 의 margin 을 바꿉니다.

```
.mg-inline { display: inline; margin: 80px 40px; }
```

→ 좌우는 그대로 40px 이고, 위아래는 여전히 0 인 것처럼 보입니다.

회색 띠 세 개가 계속 딱 붙어 있습니다.

4) .card 의 display 를 바꿉니다.

```
.card { display: block; width: 150px; height: 80px; }
```

→ 카드 세 장이 한 줄에서 세 줄로 바뀝니다.

폭 150px 은 그대로 지켜집니다. 블록에서도 width 는 잘 먹으니까요.
카드 사이 6px 틈도 사라집니다. 이제 글자가 아니니 공백이 안 깎입니다.

5) .none-box 를 바꿉니다.

```
.none-box { visibility: hidden; }
```

→ 첫 번째 데모에도 빈 줄이 생겨서 두 데모가 똑같아집니다.

두 값의 차이가 '자리를 남기느냐' 하나뿐이라는 것을 확인하는 것이 목적입니다.

태그의 기본 성질과 그것을 맞바꾸기

섹션 1

```
.tag-mark {
  background: #eef2f7;
  border: 1px solid #7a9bd4;
  padding: 4px 8px;
}
.span-as-block {
  display: block;
}
```

화면 span 인데도 한 줄을 통째로 차지해서 위아래로 쌓입니다

```
.div-as-inline {
  display: inline;
}
```

화면 div 인데도 글자처럼 옆으로 나란히 붙습니다

inline 에 width·height 를 줘 보기

섹션 2

```
.size-inline {
  display: inline;
  width: 300px;
  height: 70px;
  background: #ffd8d8;
  border: 1px solid #c0392b;
}
```

화면 아무 일도 안 일어납니다. 글자 길이만큼만 칠해집니다

```
.size-iblock {
  display: inline-block;
  width: 300px;
  height: 70px;
  background: #d8f0d8;
  border: 1px solid #27853f;
}
```

화면 가로 300px, 세로 70px 짜리 초록 상자가 생깁니다

inline 에 margin 을 줘 보기

섹션 3

```
.mg-row {
  background: #f4f4f4;
  border: 1px solid #ddd;
  padding: 4px 8px;
}
```

화면 회색 띠 세 개가 서로 딱 붙어 쌓입니다 (margin 을 안 줬으니까요)

```
.mg-inline {
  display: inline;
  margin: 30px 40px;
  background: #ffe08a;
}
```


화면 좌우로만 40px 벌어집니다. 위아래 30px 는 화면에 나타나지 않습니다

```
.mg-iblock {
  display: inline-block;
  margin: 30px 40px;
  background: #ffe08a;
}
```

화면 좌우 40px 에 더해 위아래 30px 도 벌어져서 가운데 줄이 훨씬 두꺼워집니다

inline-block 카드

섹션 4

```
.card {
  display: inline-block;
  width: 150px;
  height: 80px;
  background: #eef2f7;
  border: 2px solid #2d6cdf;
  padding: 8px;
  text-align: center;
}
```

화면 같은 크기 카드 세 장이 한 줄에 나란히 놓입니다

숨기는 두 가지 방법

섹션 5

```
.vis-row {
  background: #eef2f7;
  border: 1px solid #7a9bd4;
  padding: 6px 10px;
  margin: 4px 0;
}
.none-box {
  display: none;
}
```

화면 그 줄이 통째로 사라지고, 아래 줄이 위로 올라와 자리를 메웁니다

```
.hidden-box {
  visibility: hidden;
}
```

화면 그 줄이 안 보이는데 빈자리는 그대로 남습니다

position 기초

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.



실행하면 나오는 화면 — position 기초

개념02 의 display 는 흐름 안에서 성질만 바꿨습니다. “몇 픽셀 옮겨라” 라고 시킬 수는 없었습니다. 그 일을 하는 속성이 position 입니다. 값은 다섯 개입니다.

| | | |
|----------|-----------------------------|--------|
| static | 기본값. 흐름이 놓아 주는 자리에 그대로 있습니다 | |
| relative | 원래 자리를 남겨 둔 채 겹모습만 옮깁니다 | ← 이 파일 |
| absolute | 흐름에서 빠져나와 기준 상자 안에 놓입니다 | ← 개념04 |
| fixed | 흐름에서 빠져나와 화면(창) 에 붙습니다 | ← 개념04 |
| sticky | 스크롤하다가 정해진 위치에서 붙습니다 | ← 개념04 |

옮길 거리는 top / right / bottom / left 네 속성으로 지정합니다.

top: 20px 위에서 20px 만큼 떨어진 곳으로
left: 30px 왼쪽에서 30px 만큼 떨어진 곳으로

여기서 이 파일의 핵심 문장이 나옵니다.

top / right / bottom / left 는 position 이 static 이 아닐 때만 먹습니다.

기본값이 static 이니까, 아무것도 안 한 요소에 top 을 주면 아무 일도 안 일어납니다. 에러도 안 납니다. 그냥 버려집니다. 이 규칙 하나 때문에 초보자는 몇 시간씩 헤맬니다. 섹션 2 에서 재서 확인합니다.

이 파일에서는 position 의 기본값 static 과 첫 번째 값 relative 두 개만 다룹니다. "top 을 줬는데 왜 안 움직이지?" 의 답이 섹션 2 에 있습니다.

모든 요소는 이미 position 을 갖고 있습니다

섹션 1

position 을 안 썼다고 없는 것이 아닙니다. 안 쓰면 static 입니다. 그게 기본값입니다.

CSS 속성에는 거의 다 '기본값' 이 있습니다. 아무것도 안 쓰면 그 기본값이 조용히 적용되고 있는 것입니다.

position 의 기본값 = static
static 의 뜻 = "흐름이 놓아 주는 자리에 가만히 있다"

즉 지금까지 우리가 본 모든 상자는 전부 position: static 이었습니다. 개념01 에서 본 문서 흐름은 static 인 요소들이 만든 결과였습니다.

아래 데모에는 상자가 두 개 있습니다.

첫 번째 아무것도 안 씀 → position 은 static
두 번째 position: relative; 만 씀 → position 은 relative

그런데 화면에서는 두 상자가 완전히 똑같아 보입니다. 왜 그럴까요?

relative 는 "움길 준비" 만 시킬 뿐, 얼마나 움길지는 안 정하기 때문입니다.
얼마나 움길지는 top / right / bottom / left 로 따로 말해 줘야 합니다.

그래서 position: relative 한 줄만 쓰는 것은 실수가 아닙니다. 오히려 개념04 에서 아주 자주 쓰는 관용구가 됩니다. 그때 다시 만납니다.

화면 두 상자가 똑같이 생겼고, 왼쪽 끝도 위아래 간격도 같습니다.

relative 를 쓴 티가 하나도 안 납니다

저는 아무것도 안 썼습니다
저는 position: relative 만 썼습니다

모양은 같아도 브라우저가 아는 값은 다릅니다. 위 상자는 static, 아래 상자는 relative 입니다. 이 파일 맨 아래 자동 검사에서 그 값을 그대로 확인합니다.

▹ 직접 해보기

1

<style> 의 .pos-relative 에 left: 50px; 한 줄을 넣어 보세요. 아래 상자만 오른쪽으로 밀려납니다. 확인했으면 그 줄을 지우세요.

static 에는 top·left 가 안 먹습니다

섹션 2

아래 분홍 상자에는 top: 40px; left: 60px; 이 분명히 적혀 있습니다. 그런데 조금도 안 움직입니다.

분홍 상자에 준 CSS 는 이것입니다.

```
.st-try {  
  position: static; ← 기본값을 일부러 다시 적었습니다  
  top: 40px;  
  left: 60px;  
  background: #ffd8d8;  
}
```

background 는 잘 먹었습니다. 상자가 분홍색으로 바뀌었으니까요. 그런데 top 과 left 는 아무 일도 안 했습니다.

브라우저 입장에서 보면 당연합니다.

```
static = "흐름이 정해 준 자리에 있으라"  
top    = "위에서 이만큼 떨어진 자리로 가라"
```

두 말이 서로 모순입니다. 그래서 브라우저는 top 쪽을 무시합니다.

개념02 에서 본 것과 똑같은 일이 벌어집니다.

```
인라인에 준 width → 값은 300px 로 남아 있는데 화면 계산에서만 안 씀  
static 에 준 top → 값은 40px 로 남아 있는데 화면 계산에서만 안 씀
```

개발자 도구(F12) 로 이 분홍 상자를 찍어 보면 top: 40px 이 멀쩡히 적혀 있습니다. 지워지지도 않았고 빨간 줄이 그어져 있지도 않습니다. 그냥 안 쓰일 뿐입니다. 이 파일 맨 아래 자동 검사에서도 40px 이라는 값을 그대로 확인합니다.

결과는 늘 같습니다. 아무 일도 안 일어나고 에러도 안 납니다.

고치는 법은 하나뿐입니다. position 을 static 이 아닌 값으로 바꾸는 것입니다.

화면 아래 상자가 분홍색이 된 것 말고는 달라진 것이 없습니다.

두 상자의 왼쪽 끝이 정확히 같은 자리에 있습니다

기준 상자입니다
저는 top: 40px; left: 60px; 을 받았습니다

기억할 한 줄입니다. top-right-bottom-left 를 쓰기 전에는 반드시 position 을 먼저 바꿔야 합니다.

▹ 직접 해보기 2

.st-try 의 position: static; 을 position: relative; 로 바꿔 보세요. 상자가 오른쪽 아래로 훌쩍 움직입니다. 확인했으면 static 으로 되돌려 놓으세요.

relative 는 원래 자리를 남겨 둡니다

섹션 3

relative 로 옮긴 요소는 겉모습만 이동합니다. 흐름이 잡아 둔 원래 자리는 그대로 비어 있습니다.

상자 세 개를 한 줄에 놓고, 가운데 상자에만 이렇게 줬습니다.

```
.rel-move { position: relative; top: 14px; left: 30px; }
```

여기서 top: 14px 은 “화면 맨 위에서 14px” 이 아닙니다. relative 의 기준은 자기 자신의 원래 자리입니다.

top: 14px 원래 있던 자리보다 14px 아래로
left: 30px 원래 있던 자리보다 30px 오른쪽으로

진짜 요점은 그다음입니다.



옮기기 전



옮긴 뒤

2 의 원래 자리는 비어 있고
3 은 여전히 제자리입니다

상자 3 이 왼쪽으로 당겨와 빈자리를 메우지 않습니다. 브라우저는 상자 2 가 여전히 원래 자리에 있다고 생각하기 때문입니다. 그림만 살짝 옮겨 그렸을 뿐입니다.

그래서 relative 의 성질을 이렇게 외우지 말고 이렇게 이해하세요.

자리는 그대로 예약해 둔 채, 보이는 그림만 이동한다

아래 두 데모를 위아래로 비교하면 상자 1 과 3 의 위치가 완전히 같습니다. 옮긴 줄과 안 옮긴 줄의 전체 높이도 같습니다. 자리가 그대로 남아 있다는 증거입니다.

화면 파란 상자 세 개가 한 줄로 나란히 놓입니다

1
2
3

화면 가운데 상자만 노란색으로 오른쪽 아래로 비스듬히 내려가 있습니다.

1 번과 3 번은 위 데모와 똑같은 자리에 그대로 있고, 2 번이 원래 있던 자리는 빈칸으로 남아 있습니다. 옮겨진 2 번이 3 번의 왼쪽 끝을 조금 덮고 있는 것도 보입니다

1
2
3

웁긴 상자가 옆 상자를 밀어내지 않기 때문에 많이 옮기면 다른 요소를 그냥 덮어 버립니다. 그래서 `relative` 로는 몇 px 정도의 미세 조정만 하는 것이 보통입니다.

▸ 직접 해보기 3

`.rel-move` 의 `left` 를 140px 으로 바꿔 보세요. 지금은 3 번의 왼쪽 끝만 살짝 덮고 있는데, 그때는 3 번을 거의 다 덮어 버립니다. 확인했으면 30px 으로 되돌려 놓으세요.

relative 는 어디에 쓰나

섹션 4

쓰임새는 크게 둘입니다. 몇 px 미세 조정과, 다음 파일에서 쓸 기준점 만들기입니다.

쓰임새 1 · 몇 px 만 살짝 올리거나 내리기

```
.badge-up { position: relative; top: -7px; }
```

`top` 에 음수를 주면 위로 올라갑니다. 아래 문장에서 빨간 NEW 조각이 글줄보다 살짝 떠 보이는 것이 그것입니다.

여기서 개념02 섹션3 을 떠올려 보세요. 이 NEW 조각은 `span`, 곧 인라인입니다. 그래서 `margin-top: -7px` 로는 같은 일을 할 수 없습니다. 인라인에는 위아래 `margin` 이 안 먹으므로 조각이 1px 도 안 움직입니다. 진짜로 바뀌서 재 보면 아무 데도 안 옮긴 자리 그대로입니다.

반대로 `relative` 의 `top` 은 `display` 가 무엇이든 먹습니다. 인라인 조각을 세로로 몇 px 만 옮기고 싶을 때 `relative` 를 쓰는 이유가 이것입니다.

쓰임새 2 · 기준점 만들기 (개념04 예고)

```
.stage { position: relative; }
```

top 도 left 도 안 썼으니 화면은 하나도 안 바뀝니다. 그런데 이 한 줄이 아주 중요한 일을 합니다.

"내 안에 있는 absolute 요소들아, 나를 기준으로 삼아라"

라고 표시를 해 두는 일입니다. 지금은 아무 일도 안 하는 파란 상자로만 보이겠지만, 개념04 에서 이 상자 안에 배지를 붙일 때 이 한 줄이 전부를 좌우합니다.

아래 파란 상자를 눈에 담아 두세요. 다음 파일에서 다시 나옵니다.

화면 빨간 NEW 조각이 글줄 기준선보다 조금 위로 떠 있습니다

오늘의 추천 메뉴 NEW 크림 파스타

화면 연한 파란 배경의 사각형 상자 하나가 보일 뿐입니다.

relative 를 넣기 전과 모양이 똑같습니다

이 상자에는 position: relative 한 줄만 들어 있습니다.

▹ 직접 해보기 4

.badge-up 의 top 을 7px 로 바꿔 보세요(음수 부호를 떼는 것입니다). NEW 조각이 반대로 아래로 내려갑니다.

자주 하는 실수

섹션 5

position 을 배우고 나면 실수의 모양이 늘 비슷합니다. 코드는 멀쩡한데 화면이 안 바뀝니다.

이런 실수를 합니다

static 요소에 top 주기

이런 실수를 합니다

실수 1 — position 을 안 바꾸고 top 만 줌

```
.box { top: 40px; left: 60px; }
```

아무 일도 안 일어납니다. position 을 안 썼으니 기본값 static 이고, static 에는 top-left 가 안 먹습니다. 이 단원에서 가장 많이 나는 실수입니다.

이렇게 하세요

```
.box { position: relative; top: 40px; left: 60px; }
```

이런 실수를 합니다

원래 자리가 사라질 거라고 생각

이런 실수를 합니다

실수 2 — relative 로 옮기면 원래 자리도 따라올 거라고 생각함

```
.item2 { position: relative; left: 200px; }
```

옮긴 요소만 이동하고 원래 자리는 빈칸으로 남습니다. 그래서 “가운데가 뺨 뚫렸어요” 라는 말이 나옵니다. 고장이 아니라 원래 그렇습니다. 자리까지 없애고 싶으면 relative 가 아니라 absolute 를 써야 합니다 (개념04).

이런 실수를 합니다

단위를 빠뜨림

이런 실수를 합니다

실수 3 — 단위를 빠뜨림

```
.box { position: relative; top: 40; }
```

그 줄만 조용히 무시됩니다. position 은 잘 먹었는데 top 만 안 먹은 상태라 더 헛갈립니다. 0 을 뺀 모든 숫자에는 px 같은 단위가 필요합니다(09단원).

이런 실수를 합니다

많이 옮겨서 남을 덮음

이런 실수를 합니다

실수 4 — relative 로 크게 옮겨서 옆 요소를 덮음

```
.menu { position: relative; left: 300px; }
```

옆 요소는 비켜 주지 않습니다. 옮긴 요소가 그 위를 덮어 버려서 아래 글자가 가려집니다. 클릭도 안 됩니다. 큰 이동에는 relative 대신 absolute 를 쓰거나, 아예 12단원의 flex 로 배치를 다시 짜야 합니다.

이런 실수를 합니다

relative 만 쓰고 안 먹는다고 생각

이런 실수를 합니다

실수 5 — relative 만 썼는데 안 움직인다고 당황함

```
.stage { position: relative; }
```

이건 실수가 아닙니다. relative 는 옮길 준비만 시킵니다. 얼마나 옮길지는 top·left 로 따로 말해 줘야 합니다. 그리고 일부러 안 움직이게 두는 이 한 줄이 개념04 의 가장 중요한 관용구가 됩니다.

정리

▢ 직접 해보기

5

정답

1) style 안의 규칙을 이렇게 바꿉니다.

```
.pos-relative { position: relative; left: 50px; }
```

→ 아래 상자만 오른쪽으로 50px 밀려납니다. 위 상자는 그대로입니다.

relative 는 "준비" 이고 left 가 "실제 이동" 이라는 것을 확인하세요.

2) .st-try 의 첫 줄을 이렇게 바꿉니다.

```
.st-try { position: relative; top: 40px; left: 60px; ... }
```

→ 분홍 상자가 오른쪽으로 60px, 아래로 40px 움직입니다.

top·left 를 고친 것이 아니라 position 한 단어를 고쳤을 뿐인데 갑자기 두 줄이 살아납니다. 이 파일의 핵심입니다.

3) .rel-move 의 left 를 이렇게 바꿉니다.

```
.rel-move { position: relative; top: 14px; left: 140px; }
```

→ 2 번 상자가 3 번 상자를 거의 다 덮어 버립니다.

3 번은 여전히 안 비켜 줍니다. 흐름은 2 번이 원래 자리에 있다고 믿으니까요.

4) .badge-up 의 top 을 이렇게 바꿉니다.

```
.badge-up { position: relative; top: 7px; ... }
```

→ NEW 조각이 글줄보다 아래로 처집니다.

음수는 위로, 양수는 아래로. top 은 "위에서 떨어진 거리" 라서
값이 커질수록 아래로 내려간다고 이해하면 헷갈리지 않습니다.

섹션 1:2: 기본값 static 확인용 상자

```
.pos-box {  
  background: #eef2f7;  
  border: 1px solid #7a9bd4;  
  padding: 8px 10px;  
  margin: 6px 0;  
}  
.pos-relative {  
  position: relative;  
}
```

화면 아무 변화도 없습니다. top·left 를 안 줬기 때문입니다

```
.st-try {  
  position: static;  
  top: 40px;  
  left: 60px;  
  background: #ffd8d8;  
  border-color: #c0392b;  
}
```

화면 색만 바뀌고 자리는 그대로입니다. top·left 가 통째로 버려집니다

relative 로 옮기기

섹션 3

```
.rel-box {  
  display: inline-block;  
  width: 110px;  
  height: 46px;  
  background: #dbe7fb;  
  border: 1px solid #2d6cdf;  
  text-align: center;  
  line-height: 46px;  
}  
.rel-move {  
  position: relative;  
  top: 14px;  
  left: 30px;  
  background: #ffe08a;
```

```
border-color: #d9a100;
}
```

화면 노란 상자만 오른쪽으로 30px, 아래로 14px 옮겨집니다.

옆의 상자 1, 3 은 조금도 안 움직입니다

미세 조정과 기준점

섹션 4

```
.badge-up {
  position: relative;
  top: -7px;
  background: #c0392b;
  color: #fff;
  font-size: 12px;
  padding: 2px 6px;
}
```

화면 빨간 NEW 조각이 글줄보다 7px 위로 살짝 떠 보입니다

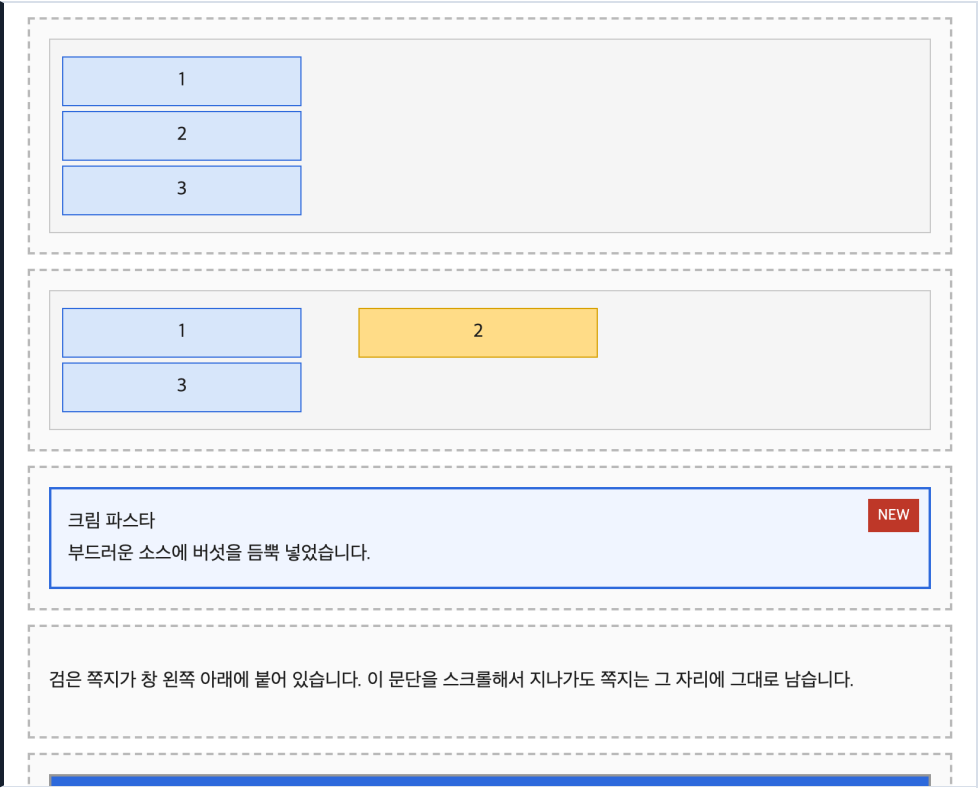
```
.stage {
  position: relative;
  height: 90px;
  background: #f4f8ff;
  border: 2px solid #2d6cdf;
  padding: 10px;
}
```

화면 지금은 그냥 파란 테두리 상자입니다. 다음 파일에서 기준으로 쓰입니다

absolute 와 fixed

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.



실행하면 나오는 화면 — absolute 와 fixed

개념03 의 relative 는 예외가 발랐습니다. 원래 자리를 남겨 두고 그림만 살짝 옮겼습니다.

이 파일의 absolute 와 fixed 는 그렇지 않습니다. 흐름에서 아예 빠져나옵니다.

흐름에서 빠진다는 = 다른 요소들이 그 요소를 없는 것으로 치고 자리를 다시 잡는다

그래서 이렇게 됩니다.

| | | |
|------|-----------|-------------------------------|
| 흐름 안 | [1][2][3] | 2 를 옮기면 자리가 빈 채로 남음(relative) |
| 흐름 밖 | [1][3] 2 | 2 가 없는 셈이 되어 3 이 앞으로 당겨짐 |

그다음 질문은 “그럼 어디를 기준으로 놓을 것인가” 입니다. 값마다 다릅니다.

| | |
|----------|-----------------------------------|
| absolute | position 이 static 이 아닌 가장 가까운 조상 |
| fixed | 브라우저 창 (화면) |
| sticky | 평소엔 원래 자리, 스크롤이 정해진 지점에 닿으면 그때 붙음 |

absolute 의 기준 설명이 길고 어렵게 들릴 텐데, 실제로는 관용구 하나입니다.

붙이고 싶은 부모에 position: relative; ← 기준이 되겠다는 표시
 붙일 자식에 position: absolute; ← 그 기준 안에서 좌표 지정

이 두 줄이 짝입니다. 부모 쪽 한 줄을 빠뜨리면 자식이 화면 구석으로 날아갑니다. 섹션 6 에서 진짜로 날아간 모습을 보게 됩니다.

이 파일에는 스크롤해야만 보이는 데모가 두 개 있습니다(섹션 3, 섹션 4). 화면 왼쪽 아래 구석에 붙어 있는 검은 쪽지도 이 파일이 만든 것입니다. 마우스 휠을 굴리면서 무엇이 따라오고 무엇이 안 따라오는지 보세요.

absolute 는 흐름에서 빠집니다

섹션 1

아래 두 무대는 완전히 같습니다. 아래쪽만 2 번 상자에 position: absolute 를 줬습니다.

두 무대에 준 CSS 는 같습니다.

```
.fl-stage { position: relative; background: #f7f7f7; padding: 10px; }
.fl-box   { width: 200px; height: 40px; margin: 4px 0; }
```

아래 무대에서만 2 번 상자에 한 덩어리를 더 줬습니다.

```
.abs-out { position: absolute; top: 10px; left: 260px; }
```

결과를 두 가지로 나눠 보세요.

움긴 상자 · 2 번이 무대 오른쪽 위로 날아갔습니다. 여기까지는 예상대로입니다.

남은 상자 · 3 번이 위로 올라와 2 번이 있던 자리를 메웠습니다.

그리고 회색 무대 자체가 한 칸만큼 낮아졌습니다.

두 번째가 relative 와의 결정적 차이입니다.

relative 자리를 남겨 둠 → 무대 높이가 그대로
 absolute 자리를 없앴 → 무대 높이가 줄어듦

무대 높이를 우리가 정해 준 적이 없다는 점이 중요합니다. 안에 든 상자 세 개가 무대 높이를 만들고 있었는데, 그중 하나가 흐름에서 빠지자 무대가 두 개짜리 높이로 줄어든 것입니다.

이 성질 때문에 absolute 를 쓸 때는 항상 이 질문을 해야 합니다.

"이 요소가 빠지면 남은 것들이 어떻게 다시 배치되는가"

회색 무대에 붙어 있는 position: relative 한 줄은 개념03 섹션4 에서 예고한 그 관용구입니다. 다음 섹션에서 그 줄이 무슨 일을 하는지 봅니다.

화면 파란 상자 세 개가 회색 무대 안에 위아래로 쌓입니다

1
2
3

화면 노란 2 번 상자가 무대 오른쪽 위에 따로 떠 있고,

1 번과 3 번이 붙어 있으며, 회색 무대가 위 무대보다 눈에 띄게 낮아졌습니다

1
2
3

▸ 직접 해보기 1

.abs-out 의 position 을 relative 로 바꿔 보세요. 노란 상자는 비슷한 곳으로 가는데 회색 무대 높이가 다시 커집니다. 확인했으면 되돌려 놓으세요.

기준이 되는 조상

섹션 2

absolute 의 기준은 position 이 static 이 아닌 가장 가까운 조상입니다.

절차는 이렇습니다. 브라우저가 absolute 요소를 만나면

1 부모를 봅니다. position 이 static 이 아니면 → 그 부모가 기준

2 static 이면 그 위로. 또 static 이면 또 위로.

3 끝까지 못 찾으면 → 페이지 전체(맨 처음 화면) 가 기준

그래서 실무에서는 이렇게 씁니다.

```
.card { position: relative; } ← 기준이 되겠다고 표시만 해 둬  
.badge { position: absolute; top: 8px; right: 8px; }
```

.card 에는 top 도 left 도 없습니다. 그래서 카드는 조금도 안 움직입니다. 그런데 이 한 줄이 있느냐 없느냐로 배지의 운명이 갈립니다.

기준이 정해지면 좌표는 그 기준 상자의 안쪽(테두리 안) 을 기준으로 잡니다.

top: 8px 기준 상자의 위쪽 안쪽 면에서 8px 아래
right: 8px 기준 상자의 오른쪽 안쪽 면에서 8px 왼쪽

left 대신 right 를 쓴 것에 주목하세요. 오른쪽에 붙이고 싶으면 right 를 쓰는 것이 훨씬 편합니다. left 로 하려면 카드 폭을 알아야 하는데, 카드 폭은 화면 크기에 따라 바뀝니다.

top/right/bottom/left 를 짝지어 쓰는 법도 여기서 정리해 둡니다.

top + left 왼쪽 위 구석 기준
top + right 오른쪽 위 구석 기준
bottom + left 왼쪽 아래 구석 기준
bottom + right 오른쪽 아래 구석 기준

화면 파란 테두리 카드의 오른쪽 위 구석에 빨간 NEW 조각이 붙어 있습니다.

카드 안쪽 면에서 위로도 오른쪽으로도 8px 씩 떨어져 있습니다

NEW
크림 파스타
부드러운 소스에 버섯을 듬뿍 넣었습니다.

.card-ok 에서 position: relative; 한 줄만 지우면 NEW 조각이 카드를 떠나 페이지 맨 위 구석으로 날아갑니다. 섹션 6 실수 1 에 그 모습을 진짜로 띄워 놓았습니다.

◁ 직접 해보기 2

.badge 의 right: 8px; 를 left: 8px; 로 바꿔 보세요. NEW 조각이 카드 왼쪽 위 구석으로 옮겨 갑니다. 확인했으면 되돌려 놓으세요.

fixed

섹션 3

fixed 의 기준은 페이지가 아니라 브라우저 창입니다. 그래서 스크롤해도 안 움직입니다.

이 "브라우저 창" 을 정확히는 뷰포트(viewport) 라고 부릅니다. 페이지가 실제로 그려지는 영역이라는 뜻입니다. 13단원에서 화면 크기에 따라 모양을 바꿀 때 이 이름으로 다시 나옵니다.

이 파일에는 fixed 요소가 딱 하나 있습니다.

```
.fixed-note {
  position: fixed;
  left: 12px;
  bottom: 12px;
  width: 150px;
}
```

지금 브라우저 창 왼쪽 아래 구석을 보세요. 검은 쪽지가 하나 붙어 있습니다. 페이지를 위아래로 스크롤해도 그 쪽지만 제자리에 남습니다. 다른 글자들은 전부 위로 흘러가는데 쪽지만 안 흘러갑니다.

absolute 와 뭐가 다를까요? 기준이 다릅니다.

absolute 기준 = 조상 상자 → 페이지의 일부라서 페이지와 함께 스크롤됩니다
fixed 기준 = 브라우저 창 → 페이지와 상관없이 창에 붙어 있습니다

그래서 fixed 는 조상에 relative 를 걸든 말든 상관하지 않습니다. 아무리 깊이 들어 있어도 기준은 언제나 창입니다.

실무에서 fixed 를 쓰는 곳은 정해져 있습니다.

항상 떠 있는 맨 위 메뉴 막대
오른쪽 아래에 붙어 있는 '맨 위로' 버튼
화면 전체를 덮는 팝업

조심할 점도 있습니다. fixed 요소는 흐름 밖에 있어서 아래 내용을 그냥 덮습니다. 그래서 화면이 좁은 휴대폰에서는 글자를 가리기 쉽습니다. 크게 만들지 마세요.

화면 이 상자에는 글자만 있습니다. 검은 쪽지는 상자 안이 아니라

브라우저 창 왼쪽 아래 구석에 따로 떠 있습니다

검은 쪽지가 창 왼쪽 아래에 붙어 있습니다. 이 문단을 스크롤해서 지나가도 쪽지는 그 자리에 그대로 남습니다.

검은 쪽지는 이 문단 근처의 코드에서 만든 것이 아닙니다. <body> 맨 아래에 한 줄로 넣어 두었습니다. 코드를 어디에 쓰든 fixed 는 창을 기준으로 삼기 때문에 상관없습니다.

▹ 직접 해보기 3

.fixed-note 의 bottom: 12px; 를 top: 12px; 로 바꿔 보세요. 쪽지가 창 왼쪽 위로 올라갑니다. 스크롤해도 여전히 안 따라 내려옵니다. 확인했으면 되돌려 놓으세요.

sticky

섹션 4

sticky 는 평소에는 흐름 안에 암전히 있다가, 정해진 지점에 닿는 순간 그 자리에 붙습니다.

sticky 는 relative 와 fixed 를 반씩 섞은 값입니다.

스크롤이 닿기 전 relative 처럼 흐름 안 원래 자리에 있습니다
스크롤이 닿은 뒤 fixed 처럼 그 자리에 붙어 버팁니다

붙는 지점은 top / bottom 으로 정합니다.


```
.sticky-head { position: sticky; top: 0; }
```

“스크롤 영역 위쪽에서 0px 지점에 닿으면 그때부터 붙어라” 라는 뜻입니다.

아래 흰 상자는 높이가 180px 이고 overflow: auto 가 걸려 있습니다(10단원). 안에 든 내용이 더 길어서 상자 안에서 따로 스크롤이 됩니다. 마우스를 상자 위에 올리고 휠을 굴려 보세요. 파란 줄만 상자 맨 위에 붙은 채 남고 숫자 줄들은 그 밑으로 지나갑니다.

여기서 꼭 알아야 할 것 하나.

top 또는 bottom 을 안 주면 sticky 는 아무 일도 안 합니다.

position: sticky 만 써 놓고 “안 붙어요” 하는 경우가 대부분 이것입니다. 어디에 붙을지를 안 말해 줬으니 붙을 수가 없습니다.

실무에서는 페이지 전체에 걸쳐 맨 위 메뉴 막대를 만들 때 많이 씁니다. fixed 와 달리 처음에는 자리를 차지하고 있다가 나중에 붙기 때문에, 페이지가 갑자기 들쭉이는 일이 없습니다.

화면 높이 180px 짜리 흰 상자 안에 파란 줄 하나와 숫자 줄들이 있습니다.

상자 안에서 스크롤하면 파란 줄만 맨 위에 붙어 남습니다

저는 sticky 입니다. 위에 붙습니다.

1번 줄입니다
2번 줄입니다
3번 줄입니다
4번 줄입니다
5번 줄입니다
6번 줄입니다
7번 줄입니다
8번 줄입니다
9번 줄입니다
10번 줄입니다
11번 줄입니다
12번 줄입니다

▹ 직접 해보기 4

.sticky-head 의 top: 0; 을 지워 보세요. 파란 줄이 더 이상 안 붙고 숫자 줄들과 함께 위로 흘러갑니다. 확인했으면 되돌려 놓으세요.

다섯 값을 한 표로

섹션 5

여기까지 나온 position 값 다섯 개를 한자리에 놓고 비교합니다. 이 표 하나면 이 단원의 절반이 정리됩니다.

표를 읽는 순서를 알려 드립니다.

먼저 '흐름에서 빠지나' 칸을 세로로 훑으세요.

빠지는 것은 absolute 와 fixed 둘뿐입니다.

이 둘만 다른 요소들의 배치를 혼듭니다.

그다음 '기준' 칸을 보세요. 이것이 값을 고르는 실제 기준입니다.

| | |
|---------------------|------------------------------|
| 부모 상자 안 어딘가에 붙이고 싶다 | → absolute (부모에 relative 필수) |
|---------------------|------------------------------|

| | |
|------------|---------|
| 창에 고정하고 싶다 | → fixed |
|------------|---------|

| | |
|----------------|----------|
| 스크롤하다 붙게 하고 싶다 | → sticky |
|----------------|----------|

| | |
|-----------------|------------|
| 몇 px 만 살짝 밀고 싶다 | → relative |
|-----------------|------------|

표에 안 적은 것도 하나 있습니다. absolute 와 fixed 는 흐름에서 빠지기 때문에, width 를 안 주면 블록 요소여도 폭이 글자 길이만큼으로 줄어듭니다. “absolute 로 바꿨더니 상자가 갑자기 훌쩍해졌다” 의 원인입니다.

| | |
|----|---------------------------------|
| 화면 | 여섯 줄짜리 표가 보이고 맨 윗줄만 연한 파란 배경입니다 |
|----|---------------------------------|

값

무엇을 기준으로 놓나
흐름에서 빠지나
top·left 가 먹나
스크롤하면

static

기준 없음 (흐름이 준 자리)
아니요
안 먹습니다
같이 흘러갑니다

relative

자기 자신의 원래 자리
아니요 (자리 남음)
먹습니다
같이 흘러갑니다

absolute

static 이 아닌 가장 가까운 조상
네 (자리 없어짐)
먹습니다
같이 흘러갑니다

fixed

브라우저 창
네 (자리 없어짐)
먹습니다
안 움직입니다

sticky

평소엔 원래 자리, 닿으면 스크롤 영역
아니요 (자리 남음)
붙을 지점을 정합니다
어느 지점부터 붙습니다

.fixed-note 의 position 을 absolute 로 바꿔 보세요. 처음에는 아무것도 안 바뀐 것처럼 보이는데, 스크롤을 내리면 쪽지가 페이지와 함께 위로 흘러가 사라집니다. 표의 '스크롤하면' 칸과 맞는지 확인하세요. 확인했으면 fixed 로 되돌려 놓으세요.

자주 하는 실수

섹션 6

absolute 는 잘못 쓰면 요소가 엉뚱한 곳으로 사라집니다. 어디로 갔는지 찾는 방법까지 같이 익혀 두세요.

이런 실수를 합니다

부모에 relative 를 안 줌

이런 실수를 합니다

실수 1 — 부모에 relative 를 안 주고 absolute 를 씀

```
.card { border: 2px solid #999; } /* relative 가 없습니다 */
.badge { position: absolute; top: 6px; left: 6px; }
```

아래 붉은 카드 안에 배지를 넣었는데 카드 안에 안 보입니다. 기준이 될 조상을 못 찾아서 페이지 맨 위 왼쪽 구석으로 날아갔습니다. 지금 페이지를 맨 위까지 올려서 왼쪽 구석을 보세요. 거기 붙어 있습니다. 길 잃은 배지 이 카드 안에 배지가 있어야 하는데 비어 있습니다.

이렇게 하세요

```
.card { position: relative; border: 2px solid #999; }
.badge { position: absolute; top: 6px; left: 6px; }
```

이런 실수를 합니다

흐름에서 빠진 것을 모름

이런 실수를 합니다

실수 2 — absolute 로 바꿨더니 아래 내용이 위로 올라옴

```
.item2 { position: absolute; top: 0; left: 300px; }
```

굵긴 요소만 신경 쓰다가 남은 요소들이 다 같이 올라오는 것을 놓칩니다. 흐름에서 빠지면 그 자리가 통째로 없어지기 때문입니다. 자리를 남기고 싶다면 absolute 가 아니라 relative 를 써야 합니다.

이런 실수를 합니다

width 를 안 줌

이런 실수를 합니다

실수 3 — absolute 로 바꿨더니 상자가 훌쩍해짐

```
.panel { position: absolute; top: 0; left: 0; }
```

div 는 블록이라 원래 가로로 꽉 채웠는데, absolute 가 되는 순간 글자 길이만큼으로 줄어듭니다. 흐름에서 빠지면 “부모 폭을 꽉 채운다” 는 규칙도 같이 사라지기 때문입니다. 폭이 필요하다면 width 를 직접 적어 주세요.

이런 실수를 합니다

fixed 의 기준을 착각

이런 실수를 합니다

실수 4 — fixed 가 부모 상자 기준일 거라고 생각함

```
.box { position: relative; }
.inner { position: fixed; top: 0; left: 0; }
```

.inner 는 .box 왼쪽 위가 아니라 브라우저 창 왼쪽 위 구석에 붙습니다. fixed 는 조상을 아예 쳐다보지 않습니다. 부모 안에 붙이고 싶으면 absolute 를 쓰세요.

이런 실수를 합니다

sticky 에 top 을 안 줌

이런 실수를 합니다

실수 5 — sticky 를 썼는데 안 붙음

```
.head { position: sticky; }
```

아무 일도 안 일어납니다. 어디에 붙을지를 안 말해 줬기 때문입니다. top: 0; 처럼 붙을 지점을 반드시 같이 적어야 합니다. 부모에 overflow: hidden 이 걸려 있어도 안 붙으니 그것도 확인하세요.

정리

▹ 직접 해보기 6

정답

1) .abs-out 의 첫 줄을 이렇게 바꿉니다.

```
.abs-out { position: relative; top: 10px; left: 260px; ... }
```

→ 노란 상자는 여전히 오른쪽으로 가지만, 회색 무대가 다시 세 칸 높이로 커집니다.

relative 는 원래 자리를 남겨 두니까요. 무대 높이가 두 값의 차이를 그대로 보여 줍니다.

2) .badge 의 right 줄을 이렇게 바꿉니다.

```
.badge { position: absolute; top: 8px; left: 8px; ... }
```

→ NEW 조각이 카드 오른쪽 위에서 왼쪽 위로 옮겨 가고 “크림 파스타” 글자를 덮습니다.

흐름에서 빠진 요소라 글자가 비켜 주지 않는다는 것도 같이 확인하세요.

3) .fixed-note 의 bottom 줄을 이렇게 바꿉니다.

```
.fixed-note { position: fixed; left: 12px; top: 12px; ... }
```

→ 검은 쪽지가 창 왼쪽 위 구석으로 올라갑니다.

스크롤을 아무리 내려도 창 왼쪽 위에 그대로 있습니다.

4) .sticky-head 에서 top: 0; 줄을 지웁니다.

```
.sticky-head { position: sticky; background: #2d6cdf; ... }
```

→ 파란 줄이 더 이상 안 붙고 숫자 줄들과 같이 위로 사라집니다.

sticky 에는 붙을 지점이 반드시 필요하다는 뜻입니다.

5) .fixed-note 의 첫 줄을 이렇게 바꿉니다.

```
.fixed-note { position: absolute; left: 12px; bottom: 12px; ... }
```

→ 스크롤을 안 한 상태에서는 쪽지가 있던 자리 그대로입니다. 달라진 것이 없어 보입니다.

그런데 스크롤을 내리면 쪽지가 페이지와 함께 위로 흘러가 사라집니다.
기준이 창에서 페이지로 바뀐 것입니다.

자리가 안 변한 이유는 섹션2 의 3번 규칙 때문입니다.
조상 중에 position 이 걸린 것이 하나도 없어서 페이지 전체가 기준이 되었는데,
그 기준 상자의 아래쪽이 바로 '맨 처음 화면의 아래쪽' 이라서
bottom: 12px 이 처음에는 fixed 일 때와 같은 자리를 가리킵니다.
달라지는 것은 스크롤할 때부터입니다.

흐름에서 빠진다는 것

섹션 1

```
.fl-stage {  
  position: relative;
```

```

background: #f7f7f7;
    border: 1px solid #ccc;
    padding: 10px;
}
.fl-box {
    width: 200px;
    height: 40px;
    background: #dbe7fb;
    border: 1px solid #2d6cdf;
    margin: 4px 0;
    line-height: 40px;
    text-align: center;
}
.abs-out {
    position: absolute;
    top: 10px;
    left: 260px;
    background: #ffe08a;
    border-color: #d9a100;
}

```

화면 노란 상자가 회색 무대의 오른쪽 위로 날아가고,

남은 상자들이 위로 올라와 빈자리를 메웁니다

기준이 되는 조상

섹션 2

```

.card-ok {
    position: relative;
    background: #f4f8ff;
    border: 2px solid #2d6cdf;
    padding: 14px;
}

```

화면 이 한 줄 덕분에 안쪽 배지가 카드 오른쪽 위 구석에 붙습니다

```
.card-txt {
  margin: 0;
}
.badge {
  position: absolute;
  top: 8px;
  right: 8px;
  background: #c0392b;
  color: #fff;
  font-size: 13px;
  padding: 3px 8px;
}
```

화면 빨간 NEW 조각이 카드 오른쪽 위 구석에서 8px 씩 떨어져 붙습니다

섹션 3·4: 스크롤 안내

```
.scroll-hint {
  background: #fff3a0;
  border: 2px solid #d9a100;
  padding: 10px 14px;
  margin: 14px 0;
  font-size: 21px;
  font-weight: bold;
  text-align: center;
}
```

화면 노란 바탕에 굵은 큰 글씨로 안내가 한 줄 보입니다

fixed

섹션 3

```
.fixed-note {
  position: fixed;
  left: 12px;
  bottom: 12px;
  width: 150px;
  background: #2b2b2b;
  color: #fff;
  font-size: 12px;
  line-height: 1.5;
  padding: 8px 10px;
}
```

화면 검은 쪽지가 브라우저 창 왼쪽 아래 구석에 붙어 있고,

페이지를 아무리 스크롤해도 그 자리에 그대로 있습니다

sticky

섹션 4

```
.sticky-scroll {
  height: 180px;
  overflow: auto;
  background: #fff;
  border: 2px solid #999;
}
.sticky-head {
  position: sticky;
  top: 0;
  background: #2d6cdf;
  color: #fff;
  padding: 8px 12px;
  font-weight: bold;
}
```

화면 상자 안에서 스크롤하면 이 파란 줄만 상자 맨 위에 붙어 남습니다

```
.sticky-item {
  padding: 6px 12px;
  border-bottom: 1px solid #eee;
}
```

비교 표

섹션 5

border-collapse 는 표 전용 속성이라 이 자료에서 따로 가르치지 않습니다. 칸과 칸 사이의 이중 선을 없애 주는 줄이라고만 알아 두세요.

```
.cmp {
  border-collapse: collapse;
  background: #fff;
}
.cmp th,
.cmp td {
  border: 1px solid #bbb;
  padding: 6px 10px;
  font-size: 14px;
  text-align: left;
}
.cmp th {
```

```
background: #eef2f7;
}
```

화면 머리줄 하나 + 값 다섯 줄, 모두 여섯 줄짜리 비교 표가 보입니다.

머리줄만 연한 파란 배경입니다

기준을 못 찾은 **absolute**

섹션 6

```
.badge-lost {
  position: absolute;
  top: 6px;
  left: 6px;
  background: #c0392b;
  color: #fff;
  font-size: 13px;
  padding: 3px 8px;
}
```

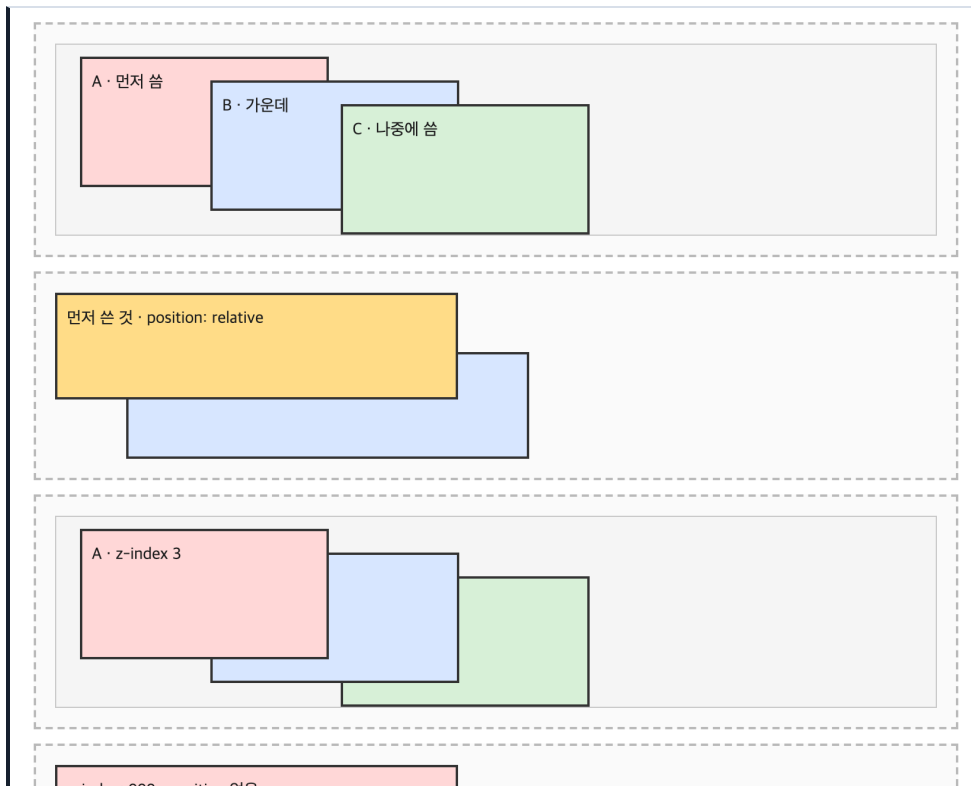
화면 이 배지는 카드 안이 아니라 페이지 맨 위 왼쪽 구석에 가 있습니다

```
.card-bad {
  background: #fff6f6;
  border: 2px solid #e0b4b4;
  padding: 14px;
}
```

쌓임 순서와 z-index

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.



실행하면 나오는 화면 — 쌓임 순서와 z-index

개념04 에서 요소를 흐름 밖으로 꺼내는 법을 배웠습니다. 흐름 밖으로 꺼내면 요소들이 서로 밀어내지 않으니 겹칠 수 있습니다.

겹치면 새로운 질문이 생깁니다. 누가 위에 보이는가?

화면은 평평해 보이지만 브라우저는 종이를 여러 장 겹쳐 놓듯 그립니다. 이 앞뒤 순서를 쌓임 순서라고 부릅니다.

화면 앞쪽 (사용자 쪽)
↑ [3] z-index 가 큰 것
| [2] position 이 걸린 것
| [1] 그냥 흐름 안에 있는 것
화면 뒤쪽

기본 규칙은 두 줄입니다.

- 1 position 이 걸린 요소가, 안 걸린 요소보다 위에 그려집니다
- 2 둘 다 같은 조건이면 HTML 에서 나중에 쓴 것이 위에 그려집니다

이 순서를 사람이 바꾸는 속성이 z-index 입니다. 다만 z-index 에는 함정이 두 개 있습니다.

함정 1 position 이 static 이면 z-index 를 아무리 크게 줘도 무시됩니다
함정 2 숫자는 형제끼리만 비교됩니다. 999 가 2 에게 지는 일이 생깁니다

마지막에는 float 를 짧게 소개합니다. 예전에 배치용으로 쓰던 속성인데 지금은 쓰임이 하나만 남았습니다.

이 파일의 데모는 전부 상자가 서로 겹쳐 있습니다. 어느 상자가 위에 보이는지, 그리고 그 이유가 무엇인지만 보면 됩니다. 마지막 섹션에서는 float 를 짧게 만나고 12단원으로 넘어갑니다.

나중에 쓴 것이 위에 그려집니다

섹션 1

아래 세 상자에는 z-index 가 하나도 없습니다. 그런데도 앞뒤 순서가 정해져 있습니다.

세 상자에 준 CSS 에서 다른 것은 위치와 색뿐입니다.

```
.z-a { top: 10px; left: 20px; background: #ffd8d8; } 분홍
.z-b { top: 30px; left: 130px; background: #d8e8ff; } 하늘
.z-c { top: 50px; left: 240px; background: #d8f0d8; } 초록
```

HTML 에는 A, B, C 순서로 썼습니다. 결과를 보세요.

초록(C) 이 하늘(B) 위에, 하늘(B) 이 분홍(A) 위에 있습니다.

즉 나중에 쓴 것이 위에 옵니다. 브라우저가 코드를 위에서 아래로 읽으면서 차례대로 그리기 때문입니다. 나중에 그린 것이 앞의 것을 덮는 것은 당연합니다.

HTML 순서 A → B → C
그리는 순서 A 를 먼저, 그 위에 B, 그 위에 C

그래서 z-index 를 배우기 전에 먼저 해 볼 일이 하나 있습니다.

겹치는 순서를 바꾸고 싶다면, HTML 에서 두 줄의 순서를 바꾸면 끝입니다.

이걸로 되는 경우가 생각보다 많습니다. z-index 는 순서를 못 바꾸는 상황에서만 꺼내는 도구입니다.

화면 왼쪽부터 분홍, 하늘, 초록 상자가 계단처럼 겹쳐 있고

오른쪽 상자가 왼쪽 상자를 덮고 있습니다. 즉 초록이 맨 위, 분홍이 맨 아래입니다

A · 먼저 씀
B · 가운데
C · 나중에 씀

▣ 직접 해보기 1

위 HTML 에서 boxA 줄을 잘라 boxC 줄 아래로 옮겨 보세요. 분홍 상자가 맨 위로 올라옵니다.
확인했으면 되돌려 놓으세요.

position 이 걸린 쪽이 위입니다

섹션 2

순서보다 더 센 규칙이 하나 있습니다. position 이 걸린 요소는 안 걸린 요소보다 무조건 위입니다.

아래 데모에서는 노란 상자를 먼저 쓰고 하늘색 상자를 나중에 썼습니다. 섹션 1 의 규칙대로라면 나중에 쓴 하늘색이 위에야 합니다.

```
<div class="ord-box ord-pos">먼저 쓴 것</div>    position: relative
<div class="ord-box ord-plain">나중에 쓴 것</div> position 없음(static)
```

그런데 결과는 반대입니다. 먼저 쓴 노란 상자가 위에 보입니다. 노란 상자에만 position: relative 가 걸려 있기 때문입니다.

브라우저가 그리는 순서를 크게 보면 이렇습니다.

```
1단계 배경과 테두리
2단계 흐름 안의 보통 요소(static)
3단계 position 이 걸린 요소      ← 여기가 더 앞
4단계 z-index 가 붙은 요소들 사이의 다툼
```

3단계가 2단계보다 나중이니 position 이 걸린 쪽이 항상 위에 옵니다. HTML 순서는 같은 단계 안에서만 따집니다.

두 상자를 겹치게 만든 방법도 잠깐 보세요.

```
.ord-plain { margin-top: -40px; margin-left: 60px; }
```

10단원에서 배운 margin 에 음수를 준 것입니다. 음수 margin 은 요소를 반대 방향으로 끌어당겨 앞 요소와 겹치게 만듭니다. 여기서는 겹침을 만들려고 일부러 쓴 것이고, 실무에서 자주 쓰지는 않습니다.

화면 노란 상자가 하늘색 상자를 덮고 있습니다.

하늘색 상자의 왼쪽 위 모서리가 노란 상자에 가려 안 보입니다

먼저 쓴 것 · position: relative
나중에 쓴 것 · position 없음

그래서 "왜 이 상자가 자꾸 위로 올라오지?" 싶을 때는 그 상자에 position 이 걸려 있는지부터 보세요. z-index 를 만지기 전에 확인할 일입니다.

▣ 직접 해보기 2

.ord-pos 에서 position: relative; 를 지워 보세요. 이제 순서 규칙만 남아서 하늘색 상자가 위로 올라옵니다. 확인했으면 되돌려 놓으세요.

z-index 로 순서 정하기

섹션 3

z-index 는 숫자가 큰 쪽이 위입니다. 단, position 이 static 이면 안 먹습니다.

첫 번째 데모는 섹션 1 과 완전히 같은 상자 세 개인데 z-index 만 더 켜었습니다.

```
.z2-a { z-index: 3; }   분홍  
.z2-b { z-index: 2; }   하늘  
.z2-c { z-index: 1; }   초록
```

결과가 섹션 1 과 정확히 뒤집힙니다. 이제 분홍이 맨 위입니다. HTML 순서는 그대로인데 숫자가 순서를 이겼습니다.

숫자 자체에는 뜻이 없습니다. 서로 비교만 합니다.

1, 2, 3 과 10, 20, 30 은 결과가 같습니다
음수도 됩니다. z-index: -1 은 아무것도 안 준 형제보다 뒤로 갑니다

두 번째 데모가 이 파일에서 제일 중요합니다.

| | |
|---------------------------------|-------------|
| .zs-static { z-index: 999; } | position 없음 |
| .zs-rel { position: relative; } | z-index 없음 |

999 대 (없음) 이면 999 가 이겨야 할 것 같습니다. 그런데 집니다. z-index 는 position 이 static 이 아닌 요소에만 먹기 때문입니다. static 인 요소에게 z-index 는 그냥 없는 말입니다.

개발자 도구로 분홍 상자를 찍어 보면 z-index 는 여전히 999 라고 적혀 있습니다. 개념02 에서 인라인 요소의 width 가 300px 로 남아 있던 것과 똑같습니다. 값은 멀쩡히 있는데 화면을 그릴 때만 안 쓰입니다.

이 파일 맨 아래 자동 검사에서도 999 라는 값을 그대로 확인합니다.

이 규칙 때문에 이런 상황이 자주 생깁니다.

"z-index 를 9999 로 올렸는데도 안 올라와요"
→ 십중팔구 그 요소에 position 이 안 걸려 있습니다.

z-index 를 쓸 때는 항상 position 과 짝으로 씁니다.

```
.modal { position: relative; z-index: 10; }
```

화면 이번에는 왼쪽 분홍 상자가 맨 위에 있습니다.

섹션 1 과 정확히 반대로, 왼쪽 상자가 오른쪽 상자를 덮습니다

```
A · z-index 3
B · z-index 2
C · z-index 1
```

화면 아래 하늘색 상자가 위 분홍 상자를 덮습니다.

999 라는 숫자가 아무 일도 못 했습니다

```
z-index: 999 · position 없음
z-index 없음 · position: relative
```

👉 직접 해보기 3

.zs-static 에 position: relative; 를 한 줄 넣어 보세요. 그 순간 분홍 상자가 위로 올라옵니다. 999 가 그제야 일을 합니다. 확인했으면 그 줄을 지우세요.

숫자는 형제끼리만 비교됩니다

섹션 4

z-index: 999 가 z-index: 2 에게 지는 일이 있습니다. 비교는 같은 부모 밑에서만 일어나기 때문입니다.

구조는 이렇습니다.

```
부모 A (position: relative; z-index: 1)
└─ 자식 (position: absolute; z-index: 999)
부모 B (position: relative; z-index: 2)
```

999 짜리 자식이 2 짜리 부모 B 에게 집니다. 왜 그럴까요?

z-index 를 가진 요소는 자기 안에 작은 세계를 하나 만듭니다. 그 세계 안의 숫자는 그 세계 안에서만 뜻이 있습니다. 이 작은 세계를 쌓임 맥락이라고 부릅니다.

부모 A 의 세계 · 안에서는 999 가 최고입니다

바깥 세계 · 에서는 부모 A(1) 와 부모 B(2) 만 비교됩니다

바깥에서 보면 A 가 통째로 B 보다 뒤입니다. A 안에서 999 든 100만이든, A 라는 덩어리 자체가 뒤에 있으니 소용이 없습니다.

비유하자면 이렇습니다.
1반에서 1등을 해도, 1반 전체가 2반 뒤에 서 있으면 2반 꼴찌보다 뒤입니다.

지금 단계에서는 이름과 이 현상만 알면 충분합니다.

숫자를 아무리 키워도 안 올라오면, 그 요소의 부모 쪽을 보세요.

쌓임 맥락을 만드는 조건은 여러 가지가 있지만 지금 아는 것은 하나면 됩니다. position 이 걸린 요소에 z-index 를 숫자로 주면 쌓임 맥락이 생깁니다.

화면 파란 테두리의 부모 B 가 분홍 자식 상자의 오른쪽 부분을 덮고 있습니다.

자식의 999 가 부모 B 의 2 를 이기지 못합니다

부모 A · z-index 1
자식 · z-index 999

부모 B · z-index 2

▫ 직접 해보기

4

.ctx-a 의 z-index: 1; 을 z-index: 3; 으로 바꿔 보세요. 이번엔 부모 A 쪽 세계가 통째로 앞으로 나와서 분홍 자식이 부모 B 를 덮습니다. 확인했으면 되돌려 놓으세요.

float 짧게

섹션 5

float 는 요소를 한쪽으로 붙이고 나머지 글자가 그 옆으로 흐르게 하는 속성입니다.

float 의 원래 용도는 신문이나 잡지의 편집입니다.

사진을 왼쪽에 붙이고 기사 글이 사진 옆을 감싸며 흐르게 한다

아래 데모가 정확히 그것입니다.

```
.ft-thumb { float: left; width: 120px; height: 90px; margin: 0 14px 8px 0; }
```

파란 네모가 왼쪽에 붙고, 글자가 그 오른쪽과 아래를 감쌉니다. 이것은 개념02 의 inline-block 으로는 안 됩니다. inline-block 은 글자를 감싸게 하지 못하고 한 줄만 나란히 놓습니다.

그런데 float 에는 성가신 성질이 있습니다. float 한 요소는 부모의 높이 계산에서 빠집니다. 그래서 안에 든 것이 float 뿐이면 부모 높이가 0 이 되어 무너집니다. 위 데모에서 부모에 overflow: hidden 을 준 것이 그 대책입니다(10단원).

예전 이야기를 조금 해야 합니다.

flex 와 grid 가 없던 시절에는 화면을 좌우로 나눌 방법이 float 뿐이었습니다. 그래서 사이드바, 메뉴, 카드 배열을 전부 float 로 만들었습니다. 높이가 무너지고 다음 요소가 밀려 올라오는 사고가 늘 따라다녔습니다.

지금은 그럴 필요가 없습니다.

| | |
|---------------|---------------------|
| 여러 개를 가로로 나란히 | → 12단원 flex |
| 바둑판처럼 행과 열로 | → 13단원 grid |
| 글 옆에 이미지를 흘리기 | → float ← 이것만 남았습니다 |

정리하면 이렇습니다. float 는 배치 도구가 아니라 글 흘리기 도구입니다. 그 용도로만 쓰세요.

화면 왼쪽에 파란 네모가 붙고 처음 네 줄은 네모 오른쪽에서 시작합니다.

네모가 끝나는 다섯째 줄부터는 글자가 상자 왼쪽 끝에서 시작합니다

사진 자리

크림 파스타는 우유와 생크림을 함께 써야 맛이 부드럽습니다.
버섯은 미리 센 불에 볶아 물기를 날려 두면 소스가 묽어지지 않습니다.
면은 봉지에 적힌 시간보다 1분 짧게 삶고, 소스에 넣어 마저 익힙니다.
마지막에 치즈를 갈아 올리면 완성입니다. 소금은 면을 삶는 물에 넉넉히 넣고, 소스에는 거의 넣지 않는 편이 간을 맞추기 쉽습니다. 접시를 미리 데워 두면 식탁에 올렸을 때 소스가 굳지 않습니다. 여기까지 읽는 동안 글줄이 왼쪽 네모를 피해 흐르다가, 네모가 끝나는 곳부터 왼쪽 끝까지 넓어졌습니다. 그것이 float 가 하는 일입니다.

▣ 직접 해보기 5

.ft-thumb 의 float 를 right 로 바꿔 보세요. 네모가 오른쪽으로 가고 글이 왼쪽으로 흐릅니다. 확인했으면 left 로 되돌려 놓으세요.

자주 하는 실수

섹션 6

겹침 문제는 화면만 보서는 원인을 못 찾습니다. 아래 다섯 가지가 원인의 거의 전부입니다.

이런 실수를 합니다

static 에 z-index

이런 실수를 합니다

실수 1 — position 없이 z-index 만 줌

```
.top-layer { z-index: 9999; }
```

아무 일도 안 일어납니다. z-index 는 position 이 static 이 아닌 요소에만 먹습니다. 숫자를 더 키우는 것은 소용이 없습니다. position: relative; 한 줄을 같이 넣으세요.

이렇게 하세요

```
.top-layer { position: relative; z-index: 9999; }
```

이런 실수를 합니다

숫자 키우기로 해결하려 함

이런 실수를 합니다

실수 2 — 안 올라와서 숫자만 계속 키움

```
.popup { position: absolute; z-index: 99999999; }
```

부모가 이미 낮은 z-index 로 자기 세계를 만들어 뒀다면 안에서 숫자를 아무리 키워도 소용이 없습니다. 고칠 곳은 그 요소가 아니라 부모입니다. 부모의 z-index 를 올리거나, 그 요소를 부모 밖으로 꺼내세요.

이런 실수를 합니다

단위 붙이기

이런 실수를 합니다

실수 3 — z-index 에 단위를 붙임

```
.card { position: relative; z-index: 10px; }
```

그 줄이 통째로 무시됩니다. z-index 는 길이가 아니라 순서를 나타내는 그냥 정수입니다. 10 이라고만 쓰세요. px 를 붙이는 습관 때문에 자주 나는 실수입니다.

이런 실수를 합니다

float 로 배치

이런 실수를 합니다

실수 4 — float 로 화면을 좌우로 나눔

```
.left { float: left; width: 300px; }
.right { float: left; width: 500px; }
```

모양은 나옵니다. 그런데 부모의 높이가 0 이 되어 다음 내용이 위로 겹쳐 올라옵니다. 옛날에는 이걸 고치는 요령이 여러 개 있었지만, 지금은 그냥 12단원 flex 를 쓰면 됩니다.

이런 실수를 합니다

배경이 투명해서 비쳐 보임

이런 실수를 합니다

실수 5 — 배경색을 안 줘서 아래 글자가 비쳐 보임

```
.overlay { position: absolute; top: 0; left: 0; }
```

요소의 기본 배경은 투명입니다. 위에 제대로 올라가 있는데도 아래 글자가 그대로 비쳐서 글자끼리 겹쳐 읽을 수 없게 됩니다. "z-index 가 안 먹네" 가 아니라 배경색이 없는 것입니다. background 를 주면 해결됩니다(07단원).

정리

▹ 직접 해보기 6

정답

1) 섹션 1 의 세 줄 순서를 이렇게 바꿉니다.

```
<div class="z-box z-b" id="boxB">B · 가운데</div>
<div class="z-box z-c" id="boxC">C · 나중에 씬</div>
<div class="z-box z-a" id="boxA">A · 먼저 씬</div>
```

→ 분홍 상자가 맨 위로 올라옵니다. 위치도 색도 안 바꿨는데 순서만으로 앞뒤가 바뀝니다.

CSS 를 한 글자도 안 고치고 겹침 순서를 바꾼 것입니다.

2) style 안의 .ord-pos 에서 position 줄을 지웁니다.

```
.ord-pos { background: #ffe08a; }
```

→ 이제 둘 다 static 이라 순서 규칙만 남습니다.

나중에 쓴 하늘색 상자가 노란 상자를 덮습니다. 섹션 1 과 같은 상태가 됩니다.

3) .zs-static 에 한 줄을 넣습니다.

```
.zs-static { position: relative; z-index: 999; ... }
```

→ 분홍 상자가 하늘색 상자 위로 올라옵니다.

999 라는 값은 처음부터 있었는데 position 한 줄이 생기자 그제야 일을 합니다.
"값이 있는데 안 먹는다" 의 전형입니다.

4) .ctx-a 의 z-index 를 바꿉니다.

```
.ctx-a { position: relative; z-index: 3; ... }
```

→ 부모 A 의 세계가 통째로 부모 B 보다 앞으로 나옵니다.

그 안에 있는 분홍 자식도 같이 앞으로 나와 부모 B 를 덮습니다.
바꾼 것은 자식이 아니라 부모라는 점이 요점입니다.

5) .ft-thumb 의 float 를 바꿉니다.

```
.ft-thumb { float: right; margin: 0 0 8px 14px; }
```

→ 파란 네모가 오른쪽에 붙고 글이 왼쪽에서 흐릅니다.

margin 방향도 같이 바뀌 주면 글자와의 간격이 자연스러워집니다.

섹션 1:3: 겹치는 상자를 놓을 무대

```
.z-stage {  
  position: relative;  
  height: 160px;  
  background: #f7f7f7;  
  border: 1px solid #ccc;  
}  
.z-box {  
  position: absolute;  
  width: 190px;  
  height: 90px;  
  padding: 8px;  
  border: 2px solid #333;  
  font-size: 14px;  
}  
.z-a {  
  top: 10px;  
  left: 20px;
```

```

}

.z-b {
  top: 30px;
  left: 130px;
  background: #d8e8ff;
}

.z-c {
  top: 50px;
  left: 240px;
  background: #d8f0d8;
}

```

화면 분홍, 하늘, 초록 상자가 계단처럼 조금씩 겹칩니다

position 이 걸린 쪽이 위

섹션 2

```

.ord-box {
  width: 320px;
  height: 70px;
  padding: 8px;
  border: 2px solid #333;
  font-size: 14px;
}

.ord-pos {
  position: relative;
  background: #ffe08a;
}

```

화면 먼저 썼는데도 뒤 상자 위로 올라옵니다

```

.ord-plain {
  margin-top: -40px;
  margin-left: 60px;
  background: #d8e8ff;
}

```

화면 나중에 썼는데도 앞 상자에 가려집니다

```
.z2-a {
  z-index: 3;
}
.z2-b {
  z-index: 2;
}
.z2-c {
  z-index: 1;
}
```

화면 숫자가 큰 분홍 상자가 맨 위로 올라와 섹션 1 과 순서가 뒤집힙니다

```
.zs-static {
  z-index: 999;
  width: 320px;
  height: 70px;
  padding: 8px;
  border: 2px solid #333;
  background: #ffd8d8;
  font-size: 14px;
}
```

화면 999 를 줬는데도 아래 상자에 가려집니다. static 이라서 무시됩니다

```
.zs-rel {
  position: relative;
  margin-top: -40px;
  margin-left: 60px;
  width: 320px;
  height: 70px;
  padding: 8px;
  border: 2px solid #333;
  background: #d8e8ff;
  font-size: 14px;
}
```

```
.ctx-a {
  position: relative;
  z-index: 1;
  width: 340px;
```

```
padding: 8px;
background: #ffe08a;
border: 2px solid #333;
font-size: 14px;
}
.ctx-child {
position: absolute;
z-index: 999;
top: 44px;
left: 150px;
width: 300px;
height: 70px;
padding: 8px;
background: #ffd8d8;
border: 2px solid #c0392b;
font-size: 14px;
}
```

화면 999 인데도 아래 파란 상자에 가려집니다

```
.ctx-b {
position: relative;
z-index: 2;
margin-top: -26px;
margin-left: 200px;
width: 340px;
height: 90px;
padding: 8px;
background: #d8e8ff;
border: 2px solid #2d6cdf;
font-size: 14px;
}
```

float

섹션 5

```
.ft-wrap {
overflow: hidden;
background: #fafafa;
border: 1px solid #ddd;
padding: 10px;
}
.ft-thumb {
float: left;
width: 120px;
height: 90px;
```

```
background: #dbe7fb;
border: 1px solid #2d6cdf;
text-align: center;
line-height: 90px;
font-size: 14px;
}
```

화면 파란 네모가 왼쪽에 붙고 글자가 그 오른쪽으로 흘러 감쌉니다

```
.ft-text {
margin: 0;
}
```


배치 (display와 position)

푸는 법

1. 이 파일을 VS Code 로 열고, 동시에 브라우저로도 엽니다.
 2. 위쪽 style 안에서 “문제 N” 이라고 적힌 자리를 찾습니다.
 3. TODO 자리에 코드를 씁니다.
 4. 저장(Ctrl+S) 하고 브라우저에서 새로고침(F5) 합니다.
 5. 각 문제의 “기대 화면” 과 같아졌는지 눈으로 확인합니다.
- 1~10 은 기본, 11~12 는 응용, 13~14 는 도전, 15 는 확인 문제입니다.
- 답이 안 나오면 개념 파일의 해당 섹션을 다시 보세요. 문제마다 적어 두었습니다.

저는 첫 번째 조각입니다

저는 두 번째 조각입니다

크기를 가질 조각

옆에 붙어 있는 글자

A 줄입니다

B 줄입니다

C 줄입니다

A 줄입니다

B 줄입니다

C 줄입니다

웁길 상자

실행하면 나오는 화면 — 배치 (display와 position)

이 파일 위쪽 <style> 안에 문제 1부터 15까지의 빈자리가 순서대로 들어 있습니다. 그 자리에 코드를 쓰고 저장한 뒤 새로고침하세요. 아래 상자들의 모양이 문제의 “기대 화면” 과 같아지면 맞은 것입니다.

문제 1 (개념02 섹션 1)

아래 두 조각은 span 이라 한 줄에 나란히 붙어 있습니다. 첫 번째 조각(#q1) 만 한 줄을 통째로 차지하게 만드세요.

기대 출력 첫 조각이 상자 왼쪽 끝부터 오른쪽 끝까지 늘어나고,
두 번째 조각은 그 아래 다음 줄로 내려갑니다.
이렇게 되면 틀린 것: 두 조각이 계속 한 줄에 붙어 있으면 값이 잘못된 것입니다.

저는 첫 번째 조각입니다
저는 두 번째 조각입니다

문제 2 (개념02 섹션 2·4)

아래 조각(#q2) 은 span 이라서 width 와 height 가 안 먹습니다. 먹게 만든 다음 가로 180px, 세로 60px 짜리 상자로 만드세요. 세 줄이 필요합니다.

기대 출력 글자 길이와 상관없이 가로로 넓고 세로로 두꺼운 상자가 됩니다.
오른쪽 글자는 그 옆에 그대로 붙어 있습니다.
이렇게 되면 틀린 것: 크기가 안 변하면 display 를 안 바꾼 것입니다.

크기를 가질 조각
옆에 붙어 있는 글자

문제 3 (개념02 섹션 5)

가운데 줄(#q3) 을 자리까지 통째로 없애 보세요.

기대 출력 A 줄과 C 줄만 남고, 두 줄이 서로 바로 붙습니다.
이렇게 되면 틀린 것: 글자만 사라지고 빈 줄이 남으면 다른 값을 쓴 것입니다.

A 줄입니다
B 줄입니다
C 줄입니다

문제 4 (개념02 섹션 5)

이번에는 가운데 줄(#q4) 을 자리는 남긴 채 안 보이게 하세요.

기대 출력 A 줄과 C 줄 사이에 한 줄만큼 빈자리가 뿔 뚫려 있습니다.
이렇게 되면 틀린 것: 빈자리 없이 두 줄이 붙으면 문제 3 과 같은 답을 쓴 것입니다.

A 줄입니다
B 줄입니다
C 줄입니다

문제 5 (개념03 섹션 2·3)

회색 무대 안의 상자(#q5) 를 원래 자리보다 오른쪽으로 40px, 아래로 20px 옮기세요. 세 줄이 필요합니다.

기대 출력 파란 상자가 무대 왼쪽 위 구석에서 오른쪽 아래로 비스듬히 내려갑니다.
이렇게 되면 틀린 것: 안 움직이면 position 줄을 안 쓴 것입니다.

옮길 상자

문제 6 (개념03 섹션 3·4)

나란히 놓인 상자 두 개 중 두 번째(#q6) 만 원래 자리보다 12px 위로 올리세요. 두 줄이 필요합니다. 위로 올리려면 top 에 음수를 씁니다.

기대 출력 두 번째 상자만 살짝 떠 보이고, 첫 번째 상자는 그대로입니다.
두 번째 상자가 원래 있던 자리는 그대로 비어 있습니다.
이렇게 되면 틀린 것: 두 상자가 같이 올라가면 잘못된 선택자에 쓴 것입니다.

그대로 있을 상자
위로 올릴 상자

문제 7 (개념04 섹션 2)

카드(#q7Card) 의 오른쪽 위 구석에 빨간 배지(#q7Badge) 를 붙이세요. 카드 안쪽 면에서 위로 8px, 오른쪽으로 8px 떨어뜨립니다. 카드에 한 줄, 배지에 세 줄이 필요합니다.

기대 출력 빨간 NEW 조각이 카드 오른쪽 위 구석에 붙습니다.
"오늘의 메뉴" 글자는 카드 왼쪽 위에 그대로 있습니다.
이렇게 되면 틀린 것: 배지가 페이지 맨 위 구석으로 날아가면
카드 쪽 한 줄을 빠뜨린 것입니다.

NEW

오늘의 메뉴

크림 파스타와 마늘빵

문제 8 (개념04 섹션 1·2)

무대(#q8Stage) 는 이미 기준이 되어 있습니다. 상자(#q8Box) 를 무대의 오른쪽 아래 구석에서 10px 씩 떨어진 곳에 놓으세요. 세 줄이 필요합니다. 오른쪽·아래를 기준으로 잡는 속성을 쓰세요.

기대 출력 파란 상자가 회색 무대의 오른쪽 아래 구석에 붙습니다.
무대 왼쪽 위에 있던 글자가 위로 올라와 자리를 메웁니다.
이렇게 되면 틀린 것: 상자가 왼쪽 위에 그대로 있으면 position 을 안 바꾼 것입니다.

구석으로 갈 상자
이 글자는 무대 안에 그대로 남습니다.

문제 9 (개념05 섹션 2·3)

분홍 상자(#q9Back) 에는 이미 position: relative 와 z-index: 2 가 걸려 있습니다. 그래서 나중에 쓴 초록 상자(#q9Front) 가 뒤로 밀려 있습니다. 초록 상자가 위로 올라오게 만드세요. 두 줄이 필요합니다.

기대 출력 초록 상자가 분홍 상자를 덮습니다.
이렇게 되면 틀린 것: z-index 만 크게 주면 아무 일도 안 일어납니다.
z-index 는 position 이 static 이 아닐 때만 먹습니다.

뒤에 있는 상자
앞으로 올 상자

문제 10 (개념05 섹션 5)

파란 네모(#q10Thumb) 를 왼쪽에 붙이고 글이 그 옆으로 흐르게 하세요. 한 줄이면 됩니다.

기대 출력 파란 네모가 왼쪽에 붙고, 처음 네 줄이 네모 오른쪽에서 시작합니다.
네모가 끝난 다섯째 줄부터는 글자가 상자 왼쪽 끝에서 시작합니다.
이렇게 되면 틀린 것: 네모 아래로 글이 통째로 내려가면 아직 안 바뀐 것입니다.

그림

이 글은 왼쪽 네모를 피해 흐르게 될 문장입니다. 문장이 충분히 길어야 네모 옆을 지나 아래까지 이어지는 모습을 볼 수 있습니다. 신문 기사에서 사진 옆에 글이 붙어 있는 모양을 떠올리면 됩니다. 이 방식은 지금도 글 옆에 그림을 놓을 때 쓰입니다. 화면을 좌우로 나누는 일에는 쓰지 않습니다. 그림과 글이 서로 다른 상자에 들어 있는데도 한 덩어리처럼 보이는 것이 이 방식의 좋은 점입니다. 이쯤 오면 글줄이 네모 아래로 내려와 상자 왼쪽 끝에서 시작하는 것이 보입니다.

문제 11 [응용] (개념02 섹션 4)

응용 · 카드 세 장을 한 줄에 나란히 놓고, 세 장 전체를 무대 가운데로 모으세요.

(1) 각 카드(.q11-card)의 width는 150px, height는 70px입니다. 세 줄이 필요합니다.

(안쪽 여백과 테두리가 더해지므로 화면에서 재면 170px 이 나옵니다)

(2) 그다음 카드 세 장을 통째로 가로 가운데로 모으세요. 부모(#q11Row)에 한 줄만 씁니다.

힌트: (2)는 11단원 속성이 아닙니다. inline-block은 이름 그대로 '인라인' 성질을

갖고 있어서, 부모가 글자를 정렬하듯이 다룰 수 있습니다.
09단원 개념04에서 배운 그 속성을 부모에 써 보세요.

기대 출력 같은 크기 카드 세 장이 한 줄에 나란히 놓이고,
세 장 뭉치가 점선 상자의 가로 가운데에 옵니다.
카드 사이에는 6px 쯤 되는 얇은 틈이 보입니다(개념02 섹션4 참고).
이렇게 되면 틀린 것: 카드가 위아래로 쌓이면 display를 안 바꾼 것입니다.
카드는 나란한데 왼쪽에 붙어 있으면 (2)를 안 한 것입니다.
margin: 0 auto로는 안 됩니다. 그건 블록 상자를 옮기는 방법입니다.

카드 1

카드 2

카드 3

문제 12 [응용] (개념04 섹션 4)

응용 · 아래 흰 상자는 안에서 따로 스크롤됩니다.

파란 머리줄(#q12Head)이 스크롤해도 상자 맨 위에 붙어 있게 만드세요. 두 줄이 필요합니다.

기대 출력 상자 안에서 스크롤하면 파란 줄만 맨 위에 남고
숫자 줄들이 그 밑으로 지나갑니다.
이렇게 되면 틀린 것: 파란 줄이 같이 위로 사라지면 붙을 지점을 안 정해 준
것입니다.

머리줄입니다
1번 줄입니다
2번 줄입니다
3번 줄입니다
4번 줄입니다
5번 줄입니다
6번 줄입니다
7번 줄입니다
8번 줄입니다
9번 줄입니다
10번 줄입니다

문제 13 [도전] (개념04 섹션 2 · 개념05 섹션 3)

도전 · 노란 덮개(#q13Cover) 가 무대를 덮고 있습니다.

빨간 라벨(#q13Label) 을 무대 오른쪽 아래 구석에서 20px 씩 떨어뜨려 놓고, 덮개보다 위에 보이게
만드세요. 네 줄이 필요합니다.

기대 출력 빨간 라벨이 무대 오른쪽 아래 구석에 붙고
노란 덮개 위에 또렷하게 올라와 보입니다.
이렇게 되면 틀린 것: 라벨이 노란 덮개에 가려 안 보이면 z-index 가
빠졌거나
position 이 static 이어서 z-index 가 무시된 것입니다.

추천
노란 덮개

문제 14 [도전] (개념02 섹션 4 · 개념04 섹션 2)

도전 · 메뉴 칸 세 개를 한 줄에 나란히 놓고(width 150px, height 60px),

가운데 칸(#q14Item2) 의 오른쪽 위 구석에 배지(#q14New) 를 붙이세요. 배지는 칸 안쪽 면에서 위로
4px, 오른쪽으로 4px 떨어집니다.

쓸 곳이 세 군데입니다.

.q14-item 한 줄에 나란히 놓고 크기 정하기 (세 줄)
 #q14Item2 배지의 기준이 되겠다는 표시 (한 줄)
 #q14New 오른쪽 위 구석에 붙이기 (세 줄)

기대 출력 메뉴 칸 세 개가 한 줄에 놓이고,
 가운데 칸 오른쪽 위 구석에만 작은 빨간 배지가 붙습니다.
 이렇게 되면 틀린 것: 배지가 페이지 맨 위 구석으로 날아가면
 가운데 칸에 기준 표시를 안 한 것입니다.

국수

파스타
 NEW

덮밥

문제 15 [확인] (개념03 섹션 2 · 개념05 섹션 3)

확인 · 아래 노란 상자(#q15)의 CSS에는 top, left, z-index가 모두 적혀 있습니다.

그런데 셋 다 아무 일도 하지 않습니다. 왜 그런지 찾아서 한 줄만 더하세요.

실마리: 이 세 속성은 공통 조건이 하나 있습니다. 개념03 섹션 2와

개념05 섹션 3에서 같은 말을 두 번 했습니다.

기대 출력 노란 상자가 분홍 상자 위로 올라오고,
 동시에 오른쪽으로 20px, 아래로 20px 더 움직입니다.
 이렇게 되면 틀린 것: 숫자만 크게 바꾸면 여전히 아무 일도 안 일어납니다.

분홍 상자
 노란 상자

정리

여기부터 "문제 1"이 나오기 전까지는 문제용 꺾대기입니다.
 색과 크기만 정해 둔 것이니 고치지 마세요.

```
.q-stage {
  position: relative;
  height: 120px;
  background: #f7f7f7;
  border: 1px solid #ccc;
  padding: 10px;
}
.q-box {
  background: #dbe7fb;
  border: 1px solid #2d6cdf;
  padding: 8px 10px;
}
.q-tile {
  display: inline-block;
  width: 160px;
  height: 50px;
  background: #dbe7fb;
  border: 1px solid #2d6cdf;
  text-align: center;
  line-height: 50px;
}
.q-slab {
  width: 260px;
  height: 60px;
  background: #dbe7fb;
  border: 1px solid #2d6cdf;
  text-align: center;
  line-height: 60px;
}
.q-row {
  background: #f4f4f4;
  border: 1px solid #ddd;
  padding: 4px 8px;
}
.q-card {
  background: #f4f8ff;
```



```

    border: 2px solid #2d6cdf;
    padding: 14px;
}
.q-badge {
    background: #c0392b;
    color: #fff;
    font-size: 13px;
    padding: 3px 8px;
}
.q-txt {
    margin: 0;
}
.q-float {
    overflow: hidden;
    background: #fafafa;
    border: 1px solid #ddd;
    padding: 10px;
}
.q-thumb {
    width: 110px;
    height: 80px;
    margin: 0 14px 8px 0;
    background: #dbe7fb;
    border: 1px solid #2d6cdf;
    text-align: center;
    line-height: 80px;
}
.q-scroll {
    height: 160px;
    overflow: auto;
    background: #fff;
    border: 2px solid #999;
}
.q-item {
    padding: 6px 12px;
    border-bottom: 1px solid #eee;
}

```

문제 9 와 문제 15 에서 '뒤에 있는 상자' 로 쓰는 규칙입니다

```

#q9Back,
#q15Back {
    position: relative;
    z-index: 2;
    background: #ffd8d8;
}

```

문제 13 에서 라벨을 덮어 버리는 덮개입니다

```
#q13Cover {
  position: absolute;
  top: 10px;
  right: 10px;
  width: 300px;
  height: 110px;
  background: #ffe08a;
  border: 1px solid #d9a100;
  text-align: center;
  line-height: 110px;
}
```

여기부터 문제입니다. TODO 자리에 코드를 쓰세요.

문제 1 (개념02 섹션 1)

```
#q1 {
```

여기에 코드를 쓰세요

```
}
```

문제 2 (개념02 섹션 2·4)

```
#q2 {
```

여기에 코드를 쓰세요

```
}
```

문제 3 (개념02 섹션 5)

```
#q3 {
```

여기에 코드를 쓰세요

```
}
```

문제 4 (개념02 섹션 5)

```
#q4 {
```

여기에 코드를 쓰세요

```
}
```

문제 5 (개념03 섹션 2·3)

```
#q5 {
```

여기에 코드를 쓰세요

```
}
```

문제 6 (개념03 섹션 3·4)

```
#q6 {
```

여기에 코드를 쓰세요

```
}
```

문제 7 (개념04 섹션 2)

```
#q7Card {
```

여기에 코드를 쓰세요

```
}
```

```
#q7Badge {
```

여기에 코드를 쓰세요

```
}
```

문제 8 (개념04 섹션 1.2)

```
#q8Box {
```

여기에 코드를 쓰세요

```
}
```

문제 9 (개념05 섹션 2.3)

```
#q9Front {
  margin-top: -30px;
  margin-left: 50px;
  background: #d8f0d8;
```

여기에 코드를 쓰세요

```
}
```

문제 10 (개념05 섹션 5)

```
#q10Thumb {
```

여기에 코드를 쓰세요

```
}
```

문제 11 (개념02 섹션 4)

```
.q11-card {  
  background: #eef2f7;  
  border: 2px solid #2d6cdf;  
  padding: 8px;  
  text-align: center;
```

여기에 코드를 쓰세요

```
}
```

문제 12 (개념04 섹션 4)

```
#q12Head {  
  background: #2d6cdf;  
  color: #fff;  
  padding: 8px 12px;  
  font-weight: bold;
```

여기에 코드를 쓰세요

```
}
```

문제 13 (개념04 섹션 2 · 개념05 섹션 3)

```
#q13Label {
  background: #c0392b;
  color: #fff;
  font-size: 13px;
  padding: 4px 10px;
```

여기에 코드를 쓰세요

```
}
```

문제 14 (개념02 섹션 4 · 개념04 섹션 2)

```
.q14-item {
  background: #eef2f7;
  border: 1px solid #7a9bd4;
  text-align: center;
  line-height: 60px;
```

여기에 코드를 쓰세요

```
}
#q14Item2 {
```

여기에 코드를 쓰세요

```
}
#q14New {
  background: #c0392b;
  color: #fff;
  font-size: 12px;
  line-height: 1.4;
  padding: 2px 6px;
```

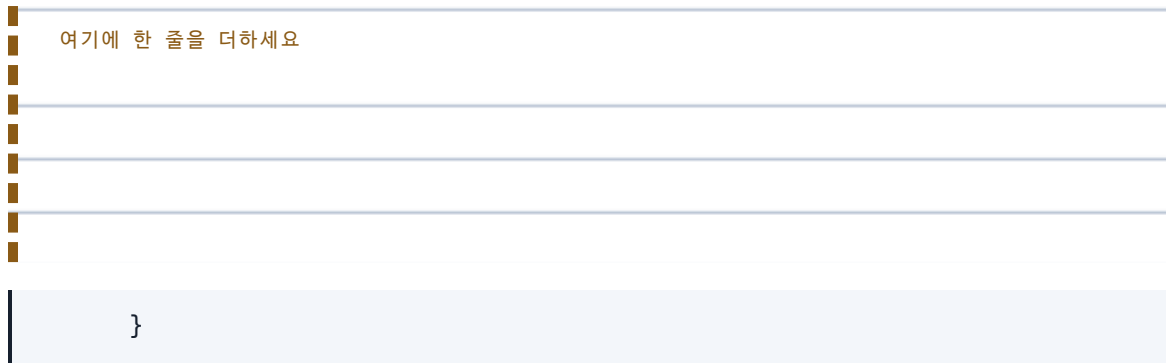
여기에 코드를 쓰세요

```
}
```

문제 15 (개념03 섹션 2 · 개념05 섹션 3)

아래 규칙은 일부러 잘못 써 두었습니다. 한 줄만 더하면 전부 살아납니다.

```
#q15 {
  margin-top: -30px;
  margin-left: 50px;
  background: #ffe08a;
  top: 20px;
  left: 20px;
  z-index: 3;
```

11단원 마무리 배치 display와 position

자주 넘어지는 자리

- 1 inline 요소에 width 를 줘도 안 먹는 것

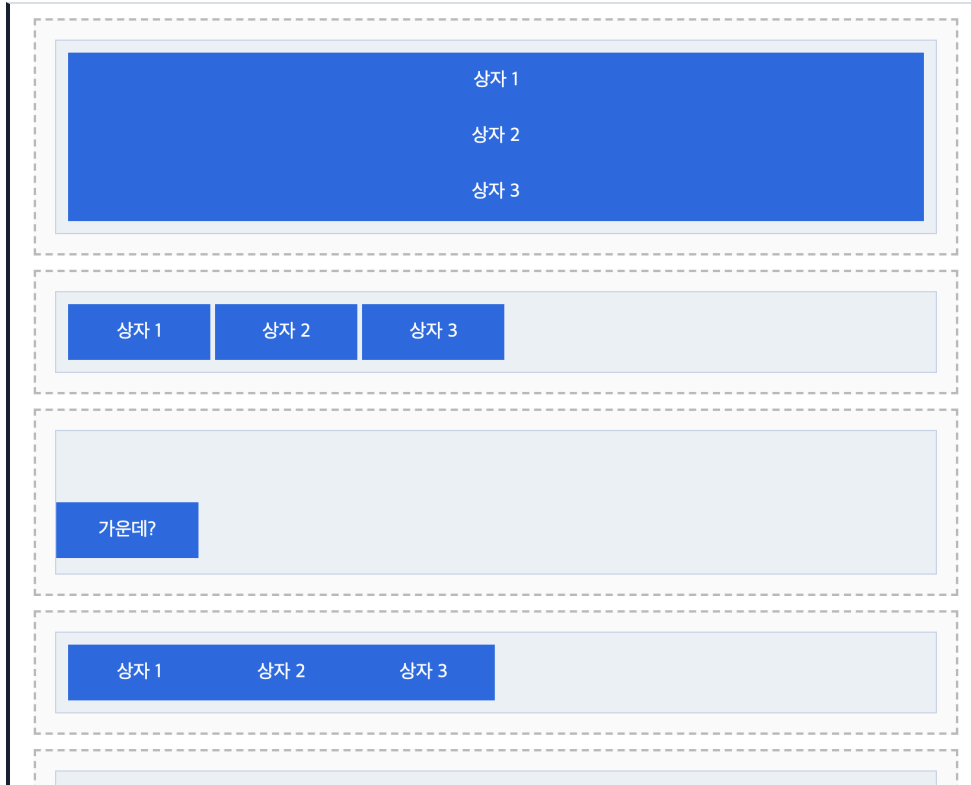
Flexbox

OPEN 12_Flexbox

```
.typo-wrap { display: flexbox; }  
.stack-wrap {  
background: #eef2f7;  
border: 1px solid #c9d6ea;
```

- 01 flex 시작하기
- 02 방향과 줄바꿈
- 03 주축 정렬
- 04 교차축 정렬
- 05 늘어나고 줄어들기
- 06 flex로 레이아웃 만들기
- 연습문제

flex 시작하기



실행하면 나오는 화면 — flex 시작하기

12단원 · 개념 01 — flex 시작하기

실행: 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.

11단원까지 배운 것으로 화면을 만들어 보면 이상하게 어려운 일이 하나 있습니다. 바로 “상자 몇 개를 가로로 나란히 놓고, 가운데에 두는 것” 입니다.

div 는 블록 요소라서 가로 폭을 전부 차지하고 아래로 쌓입니다(11단원). 나란히 놓으려면 display: inline-block 이나 float 를 써야 했는데, 둘 다 원래 그 일을 하라고 만든 도구가 아니었습니다. 그래서 이상한 틈이 생기거나, 부모의 높이가 0 이 되어 버리거나 했습니다.

세로 가운데 정렬은 더 까다롭습니다. position: absolute 와 top 을 써도 “상자의 위쪽 모서리”가 가운데에 올 뿐, 상자 자체가 가운데로 오지는 않습니다. 섹션 1 에서 직접 해 보겠습니다.

flex(플렉스) 는 바로 이 문제를 풀려고 나중에 만들어진 배치 방법입니다. “한 줄에 늘어놓고 남는 공간을 어떻게 나눌지” 를 정하는 도구입니다.

이 파일에서 배울 것은 딱 세 가지입니다.

- 1 display: flex 를 어디에 쓰는가
- 2 컨테이너와 아이템이라는 두 역할
- 3 주축과 교차축

이 세 가지를 확실히 해 두면 개념02 부터는 속성 이름만 외우면 됩니다.

이 파일은 flex 가 왜 필요한지와 누구에게 속성을 붙이는지 두 가지만 다룹니다. 아래 상자들을 위에서 아래로 순서대로 보세요. 앞쪽 상자는 일부러 불편하게 만들어 둔 것이고, 뒤쪽 상자가 flex 로 고친 결과입니다.

flex 없이 나란히 놓아 보기

섹션 1

먼저 11단원까지 배운 방법만으로 상자를 가로로 놓아 보겠습니다. 생각보다 잘 안 됩니다. 이 불편함을 느끼는 것이 이 섹션의 목적입니다.

시도 0 · 그냥 div 세 개를 놓으면 어떻게 되나

```
<div class="stack-wrap"> <div class="stack-box">상자 1</div> <div class="stack-box">
상자 2</div> <div class="stack-box">상자 3</div>
</div>
```

div 는 블록(block) 요소입니다. 블록 요소는 두 가지 성질이 있습니다(11단원).

- 1 가로 폭을 부모만큼 전부 차지한다
- 2 그래서 다음 요소는 아래로 밀려 내려간다

그 결과가 아래 첫 번째 상자입니다. 세 개가 세로로 쌓입니다. 재어 보면 파란 상자 하나의 폭이 802px 입니다. 부모 안쪽 폭을 통째로 먹은 것입니다.

화면 파란 상자 세 개가 세로로 쌓입니다. 상자 하나의 폭은 802px 입니다

상자 1
상자 2
상자 3

시도 1 · display: inline-block 으로 나란히 놓기

```
.inline-box { display: inline-block; width: 120px; }
```

inline-block 은 “글자처럼 옆으로 놓이되 폭과 높이는 가질 수 있는 상태”입니다(11단원). 이걸 쓰면 확실히 가로로 놓이기는 합니다.

그런데 상자 사이에 6px 짜리 틈이 생깁니다. margin 을 준 적이 없는데도 그렇습니다.

이유는 이렇습니다. inline-block 은 “글자처럼” 놓입니다. 그런데 우리가 HTML 을 쓸 때 태그와 태그 사이에 줄바꿈과 들여쓰기를 넣습니다. 그 줄바꿈이 브라우저에게는 띄어쓰기 한 칸으로 읽힙니다. 즉 상자과 상자 사이에 공백 문자가 들어 있는 것입니다.

없애려면 태그를 한 줄에 붙여 쓰거나 부모의 글자 크기를 0 으로 만들어야 합니다. 둘 다 “코드 모양을 망가뜨려서 화면을 고치는” 방법입니다. 좋은 방법이 아닙니다.

화면 상자 세 개가 가로로 놓입니다. 첫 상자과 둘째 상자 사이가 6px 벌어져 있습니다

상자 1
상자 2
상자 3

시도 2 · 세로 가운데에 놓아 보기

```
.pos-wrap { height: 120px; position: relative; }  
.pos-box { position: absolute; top: 50%; }
```

높이 120px 짜리 회색 칸 안에서 상자를 세로 가운데에 두고 싶습니다. 11단원에서 배운 도구 중에 자리를 직접 정하는 것은 position: absolute 와 top 뿐입니다.

11단원의 예제는 top 에 px 만 썼습니다. 여기서는 % 를 써 봅시다. 10단원에서 width 에 쓰던 그 % 이고, 기준이 부모라는 점도 같습니다. top: 50% 는 “부모 높이의 절반 지점에 상자의 위쪽 모서리를 둔다” 는 뜻입니다. 상자의 가운데가 아니라 위쪽 모서리 기준입니다.

부모 안쪽 높이 120 의 절반 = 60 → 상자 위 모서리가 60 지점에 옵니다
상자 높이는 47 → 진짜 가운데라면 위 모서리가 36.5 지점

그래서 23.5px 만큼 아래로 내려가 있습니다. 눈으로도 아래쪽에 치우쳐 보입니다. 이걸 정확히 맞추려면 상자 높이를 미리 알아서 빼 주거나 아직 배우지 않은 변환(transform) 속성이 필요합니다.

화면 상자가 회색 칸의 가운데보다 아래로 치우쳐 있습니다

가운데?

정리하면, 지금까지의 도구는 원래 다른 일을 하라고 만든 것을 빌려 쓰는 방식이었습니다. 그래서 예상 못 한 틈이 생기고 계산이 필요했습니다.

▢ 직접 해보기 1

시도 2 의 .pos-box 에서 top: 50% 를 top: 0 으로 바꿔 보세요. 상자가 맨 위로 붙습니다. 확인했으면 되돌려 놓으세요.

display: flex 한 줄

섹션 2

같은 일을 flex 로 하면 부모에 한 줄만 쓰면 끝납니다.

섹션 1 의 시도 1 과 HTML 은 완전히 같습니다. CSS 만 이렇게 바꿨습니다.

```
.side-wrap {  
  display: flex;      ← 이 한 줄이 전부입니다  
}
```

display 는 11단원 개념02 에서 배운 그 속성입니다. 거기서는 block, inline, inline-block, none 을 봤습니다. flex 는 그 목록에 하나 더 있는 값입니다. 새로운 속성이 아닙니다. 다만 앞의 네 값이 “나 자신이 어떻게 놓일까” 를 정했다면, flex 는 “내 자식들을 어떻게 늘어놓을까” 를 정한다는 점이 다릅니다.

중요한 것은 이 한 줄을 어디에 썼느냐입니다. 상자(자식)가 아니라 상자를 감싼 부모에 썼습니다.

결과를 보면 상자 사이의 6px 틈도 사라졌습니다. flex 는 자식을 “글자처럼” 다루지 않기 때문에 코드의 줄바꿈이 공백이 되지 않습니다. 시도 1 에서 골치였던 문제가 저절로 없어진 것입니다.

화면 상자 세 개가 틈 없이 딱 붙어 가로로 나란히 놓입니다

상자 1
상자 2
상자 3

세로 가운데 정렬도 마찬가지입니다.

```
.center-wrap {
  height: 120px;
  display: flex;
  justify-content: center; ← 가로 가운데
  align-items: center;      ← 세로 가운데
}
```

뒤의 두 줄은 개념03 과 개념04 에서 제대로 배웁니다. 지금은 “부모에 세 줄 쓰면 계산 없이 정중앙에 온다”는 것만 보세요.

섹션 1 의 시도 2 에서는 23.5px 이 어긋났습니다. 여기서는 상자 높이를 몰라도 정확히 가운데에 옵니다. 브라우저가 대신 재 주기 때문입니다.

화면 상자가 회색 칸의 가로세로 정중앙에 놓입니다

가운데

display: flex 는 부모에게 주는 명령입니다. “너의 자식들을 한 줄로 늘어놓아라” 라는 뜻이지, 자기 자신을 어떻게 하라는 뜻이 아닙니다.

▸ 직접 해보기 2

.side-wrap 의 display: flex 줄 앞뒤에 /* 와 */ 를 붙여 주석으로 막아 보세요. 상자가 다시 세로로 쌓입니다. 확인했으면 주석을 푸세요.

컨테이너와 아이тем

섹션 3

flex 를 쓰는 순간 요소들은 두 가지 역할로 나뉩니다. 이 단원에서 가장 먼저 확실히 해 둘 것입니다.

용어를 여기서 정의합니다. 앞으로 계속 이 두 단어만 씁니다.

컨테이너(container) display: flex 를 쓴 그 요소. 부모입니다.

아이템(item) 컨테이너의 바로 아래 자식들. 전부 자동으로 아이템이 됩니다.

그림으로 보면 이렇습니다.

```
<div class="role-wrap">      ← 컨테이너 (display: flex 를 여기에 씀)
  <div class="role-item">    ← 아이템 1
    <span class="role-inner"> ← 아이템이 아닙니다 (손자)
  </div>
  <div class="role-item">    ← 아이템 2
  </div>
</div>
```


중요한 규칙이 하나 있습니다.

아이템이 되는 것은 "바로 아래 자식" 뿐입니다. 손자는 아이템이 아닙니다.

아래 데모에서 흰 상자 두 개는 아이템이라 가로로 나란히 놓입니다. 그런데 흰 상자 안의 노란 조각들은 손자라서 아이템이 아닙니다. 그래서 노란 조각은 원래대로 위아래로 쌓입니다.

흰 상자의 폭도 눈여겨보세요. 재어 보면 71px 입니다. div 는 블록 요소라 원래 부모 안쪽 폭(800px)을 전부 차지해야 하는데 그러지 않았습니니다. 아이템이 되면 폭이 "내용만큼" 으로 바뀝니다.

"부모에 flex 를 걸었는데 왜 안쪽은 그대로지?" 하는 질문의 답이 이것입니다. flex 는 한 단계만 내려갑니다.

화면 흰 상자 두 개는 가로로 나란히,

그 안의 노란 조각은 위아래로 쌓인 채 그대로입니다

손자 A

손자 B

손자 C

손자 D

아이템이 되면 그 요소의 성질이 바뀝니다. 위 데모의 흰 상자는 원래 블록 요소라 폭을 전부 차지해야 하는데, 아이템이 되면서 내용만큼만 차지하게 되었습니다. 11단원에서 배운 블록·인라인 구분이 아이템에게는 그대로 통하지 않는다고 생각하세요.

▸ 직접 해보기 3

.role-wrap 안에 <div class="role-item">아이템 3</div> 를 하나 더 넣어 보세요. 아무 설정도 안 했는데 세 번째 상자가 저절로 옆에 붙습니다.

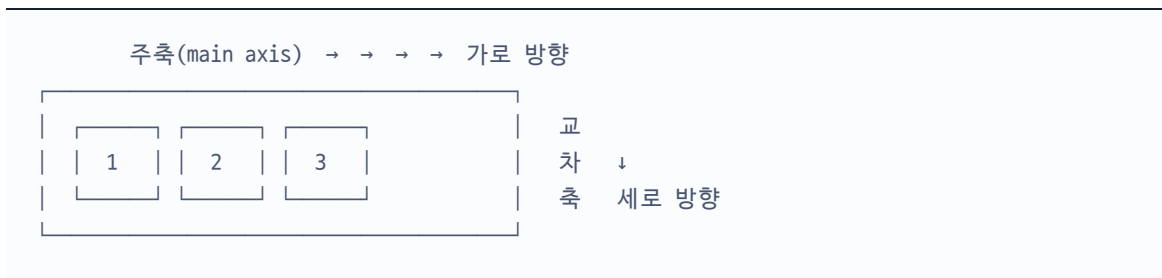
주축과 교차축 섹션 4

flex 의 모든 정렬 속성은 축(axis) 을 기준으로 말합니다. 축 두 개의 이름을 여기서 외워 두면 나머지가 전부 쉬워집니다.

flex 컨테이너에는 방향이 두 개 있습니다.

주축(main axis) 아이템이 줄지어 놓이는 방향
교차축(cross axis) 주축과 직각인 방향

기본값(flex-direction: row)일 때 그림은 이렇습니니다.



“주축은 아이템이 줄 서는 방향” 이라고 기억하세요. 기본값에서는 왼쪽에서 오른쪽이 주축이고, 위에서 아래가 교차축입니다.

아래 데모의 회색 칸은 안쪽 높이를 120px 로 정해 두었습니다. 파란 상자의 글자는 한 줄(47px)뿐인데 상자 높이가 120px 로 늘어나 있습니다. 이것이 교차축 쪽의 기본 동작입니다. 남으면 늘려서 채웁니다. 개념04 에서 이 동작의 이름(stretch)을 배웁니다.

화면 상자 세 개가 왼쪽부터 차례로 놓이고,

글자는 한 줄인데 상자 높이는 120px 까지 늘어나 있습니다



축 이름이 중요한 이유는 속성 이름이 축을 가리키기 때문입니다. justify-content 는 주축을, align-items 는 교차축을 다룹니다. “가로 정렬 / 세로 정렬” 이라고 외우면 개념02 에서 방향을 바꾸는 순간 전부 틀립니다.

▹ 직접 해보기 4
 .axis-wrap 의 height: 120px 을 height: 200px 으로 바꿔 보세요. 파란 상자의 높이도 같이 커집니다. 교차축 방향으로 늘어나고 있다는 증거입니다.

속성은 누구에게 붙이나

섹션 5

이 단원에서 가장 많이 나는 사고는 속성을 엉뚱한 쪽에 쓰는 것입니다. 지금 표로 한 번 보고, 헛갈릴 때마다 여기로 돌아오세요.

flex 속성은 두 무리로 나뉩니다.

컨테이너에 쓰는 것 · 부모에게 씁니다

| | | |
|-----------------|---------|--------|
| display: flex | | |
| flex-direction | 방향 | (개념02) |
| flex-wrap | 줄바꿈 | (개념02) |
| justify-content | 주축 정렬 | (개념03) |
| gap | 사이 간격 | (개념03) |
| align-items | 교차축 정렬 | (개념04) |
| align-content | 여러 줄 정렬 | (개념04) |

아이템에 쓰는 것 · 자식에게 씁니다

| | | |
|-------------|-------------|--------|
| flex-grow | 남는 공간 나눠 갖기 | (개념05) |
| flex-shrink | 모자랄 때 줄어들기 | (개념05) |
| flex-basis | 기본 크기 | (개념05) |
| flex | 위 세 개의 단축 | (개념05) |
| align-self | 나만 다르게 정렬 | (개념04) |
| order | 순서 바꾸기 | (개념05) |

구분하는 요령이 있습니다.

| |
|-----------------------------|
| "여럿을 한꺼번에 어떻게 늘어놓을까" → 컨테이너 |
| "나 하나가 어떻게 될까" → 아이템 |

justify-content 는 아이템 전체를 어디에 모을지 정하는 것이니 컨테이너입니다. align-self 는 나 혼자만 다르게 놓겠다는 것이니 아이템입니다.

아래 두 데모는 같은 justify-content: center 를 서로 다른 쪽에 쓴 것입니다.

| | |
|--------------|----------------------------------|
| 화면 | 상자 두 개가 회색 칸 한가운데로 모입니다 |
| 상자 1
상자 2 | |
| 화면 | 상자가 왼쪽에 그대로 붙어 있습니다. 아무 변화가 없습니다 |
| 상자 1
상자 2 | |

아래쪽 데모에서 에러는 한 줄도 나지 않습니다. CSS 는 뜻이 통하지 않는 선언을 조용히 무시합니다. "분명히 썼는데 왜 안 되지?" 의 대부분은 이렇게 자리를 잘못 잡은 경우입니다.

.miss-item 의 justify-content: center 를 잘라내어 .miss-wrap 안으로 옮겨 보세요. 그제서야 빨간 상자가 가운데로 모입니다. 확인했으면 되돌려 놓으세요.

자주 하는 실수

섹션 6

다시 강조합니다. CSS 는 틀려도 에러가 나지 않습니다. 화면이 안 바뀌는 것이 유일한 신호입니다.

이런 실수를 합니다

display: flex 를 아이টে에 씀

이런 실수를 합니다

실수 1 — 나란히 놓고 싶은 상자 자신에게 display: flex 를 씀

```
.side-box { display: flex; } /* 상자 자신에게 */
```

상자들은 여전히 세로로 쌓입니다. display: flex 는 자기 자식을 늘어놓는 명령이라서, 상자 자신에게 쓰면 그 상자 안쪽만 손댔 뿐 옆 상자와는 아무 상관이 없습니다. 나란히 놓고 싶은 상자들의 부모에 써야 합니다. 엉뚱한 변화가 하나 생기기는 합니다. 상자 안 글자가 가운데에서 왼쪽으로 옮겨 갑니다. 상자가 컨테이너가 되면서 text-align: center 가 글자에 더 이상 통하지 않기 때문입니다. 바라던 것과 전혀 다른 곳이 바뀌니 더 헛갈립니다.

이렇게 쓰세요

```
.side-wrap { display: flex; } /* 상자들을 감싼 부모에게 */
```

이런 실수를 합니다

justify-content 를 아이টে에 씀

이런 실수를 합니다

실수 2 — 아이টে에 justify-content 를 씀

```
.miss-item { justify-content: center; }
```

섹션 5 아래쪽 데모가 바로 이것입니다. 상자는 꿈쩍도 하지 않습니다. justify-content 는 컨테이너가 아이টে들을 어디에 모을지 정하는 속성이라, 아이টে가 자기한테 써 봐야 지시할 대상이 없습니다.

이런 실수를 합니다

값 이름을 flexbox 로 씀

이런 실수를 합니다

실수 3 — display: flexbox 라고 씀

```
.typo-wrap { display: flexbox; }
```

기술의 별명이 “플렉스박스” 라서 값 이름도 그럴 거라고 생각하기 쉽습니다. 그런 값은 없습니다. 값 하나가 틀리면 그 한 줄만 통째로 버려집니다. 아래 상자가 세로로 쌓여 있는 것이 그 결과입니다. 값은 flex 입니다. 상자 1 상자 2

이런 실수를 합니다

손자까지 정렬될 거라 기대

이런 실수를 합니다

실수 4 — 손자도 나란히 놓일 거라고 생각함

```
.grand-wrap { display: flex; } /* 손자에게는 안 갑니다 */
```

컨테이너의 바로 아래 자식만 아이템이 됩니다. 아래에서 흰 상자는 나란히 놓이지만, 그 안의 노란 조각은 그대로 세로로 쌓여 있습니다. 안쪽도 나란히 놓고 싶다면 흰 상자에도 따로 display: flex 를 써야 합니다. 손자 A 손자 B 손자 C 손자 D

이런 실수를 합니다

축 이름을 가로세로로 외움

이런 실수를 합니다

실수 5 — 주축을 그냥 “가로” 라고 외움

```
주축 = 가로, 교차축 = 세로 /* 기본값일 때만 맞습니다 */
```

지금은 맞는 말이지만 개념02 에서 flex-direction: column 을 쓰는 순간 주축이 세로가 됩니다. 그때 justify-content 가 갑자기 위아래를 움직여서 “고장 났나?” 싶어집니다. 주축은 아이템이 줄 서는 방향이라고 외우세요.

정리

▢ 직접 해보기

6

정답

1) .pos-box 의 top 을 이렇게 바꿉니다.

```
.pos-box { position: absolute; top: 0; }
```

→ 상자가 회색 칸 맨 위에 딱 붙습니다.

top 이 "상자의 위쪽 모서리 위치" 라는 뜻을 확인할 수 있습니다.
되돌릴 때는 top: 50% 로 다시 씁니다.

2) .side-wrap 을 이렇게 바꿉니다.

```
.side-wrap {  
  background: #eef2f7;  
  border: 1px solid #c9d6ea;  
  padding: 10px;  
  /* display: flex; */  
}
```

→ 상자 세 개가 다시 세로로 쌓입니다. 폭은 width: 120px 을 준 그대로 120px 이고

위에서 아래로 한 줄씩 내려갑니다.
display: flex 한 줄이 하던 일이 그만큼 컸다는 뜻입니다.

3) .role-wrap 안 마지막에 이렇게 한 줄 넣습니다.

```
<div class="role-item">아이템 3</div>
```

→ 흰 상자가 세 개가 되어 셋 다 가로로 나란히 놓입니다.

컨테이너에 자식을 넣기만 하면 자동으로 아이템이 됩니다.

4) .axis-wrap 의 높이를 바꿉니다.

```
.axis-wrap { height: 200px; }
```

→ 파란 상자의 높이도 120px 에서 200px 로 같이 커집니다.

교차축 방향(세로)으로는 남은 자리를 알아서 채우기 때문입니다.
반면 주축 방향인 가로 폭은 100px 그대로입니다.

5) 두 곳을 이렇게 고칩니다.

```
.miss-wrap { display: flex; justify-content: center; }  
.miss-item { /* justify-content 를 지웁니다 */ }
```

→ 빨간 상자 두 개가 회색 칸 가운데로 모입니다.

속성은 그대로인데 붙인 자리만 바꿨을 뿐입니다.

flex 없이 나란히 놓아 보기

섹션 1

```
.stack-wrap {
  background: #eef2f7;
  border: 1px solid #c9d6ea;
  padding: 10px;
}
.stack-box {
  background: #2d6cdf;
  color: #fff;
  padding: 10px 14px;
  text-align: center;
}
```

화면 파란 상자 세 개가 위에서 아래로 한 줄씩 쌓입니다

```
.inline-wrap {
  background: #eef2f7;
  border: 1px solid #c9d6ea;
  padding: 10px;
}
.inline-box {
  display: inline-block;
  background: #2d6cdf;
  color: #fff;
  padding: 10px 14px;
  width: 120px;
  box-sizing: border-box;
  text-align: center;
}
```

화면 상자 세 개가 가로로 놓이지만 상자 사이에 정체 모를 틈이 생깁니다

```
.pos-wrap {
  background: #eef2f7;
  border: 1px solid #c9d6ea;
  height: 120px;
  position: relative;
```

```
.pos-box {
  background: #2d6cdf;
  color: #fff;
  padding: 10px 14px;
  width: 120px;
  box-sizing: border-box;
  text-align: center;
  position: absolute;
  top: 50%;
}
```

화면 상자가 가운데보다 아래로 치우칩니다 (진짜 가운데가 아닙니다)

display: flex 한 줄

섹션 2

```
.side-wrap {
  background: #eef2f7;
  border: 1px solid #c9d6ea;
  padding: 10px;
  display: flex;
}
```

화면 아래 파란 상자 세 개가 가로로 딱 붙어 나란히 놓입니다

```
.side-box {
  background: #2d6cdf;
  color: #fff;
  padding: 10px 14px;
  width: 120px;
  box-sizing: border-box;
  text-align: center;
}

.center-wrap {
  background: #eef2f7;
  border: 1px solid #c9d6ea;
  height: 120px;
  display: flex;
  justify-content: center;
  align-items: center;
}
```

화면 상자 한 개가 가로세로 모두 진짜 정중앙에 놓입니다


```
.center-box {
  background: #2d6cdf;
  color: #fff;
  padding: 10px 14px;
  width: 120px;
  box-sizing: border-box;
  text-align: center;
}
```

컨테이너와 아이템

섹션 3

```
.role-wrap {
  background: #eef2f7;
  border: 2px solid #2d6cdf;
  padding: 10px;
  display: flex;
}
.role-item {
  background: #fff;
  border: 1px solid #c9d6ea;
  padding: 8px;
}
.role-inner {
  background: #ffe9b8;
  border: 1px solid #e0bb62;
  padding: 4px 6px;
  font-size: 13px;
}
```

화면 파란 테두리 안에 흰 상자 두 개가 나란히,

흰 상자 안의 노란 조각은 여전히 위아래로 쌓입니다

주축과 교차축

섹션 4

```
.axis-wrap {
  background: #eef2f7;
  border: 1px solid #c9d6ea;
  padding: 10px;
  height: 120px;
  display: flex;
}
.axis-item {
```

```

color: #fff;
padding: 10px 14px;
width: 100px;
box-sizing: border-box;
text-align: center;
}

```

화면 상자 세 개가 왼쪽부터 차례로 놓이고, 세로로는 상자가 통째로 늘어납니다

속성을 어디에 붙이나

섹션 5

```

.where-wrap {
background: #eef2f7;
border: 1px solid #c9d6ea;
padding: 10px;
display: flex;
justify-content: center;
}

```

화면 상자 두 개가 컨테이너 한가운데로 모입니다

```

.where-item {
background: #2d6cdf;
color: #fff;
padding: 10px 14px;
width: 140px;
box-sizing: border-box;
text-align: center;
}

.miss-wrap {
background: #eef2f7;
border: 1px solid #c9d6ea;
padding: 10px;
display: flex;
}

.miss-item {
background: #b9432f;
color: #fff;
padding: 10px 14px;
width: 140px;
box-sizing: border-box;
text-align: center;
justify-content: center;
}

```

화면 아무 일도 일어나지 않습니다. 상자는 그냥 왼쪽에 붙어 있습니다

자주 하는 실수

섹션 6

```
.typo-wrap {
  background: #eef2f7;
  border: 1px solid #c9d6ea;
  padding: 10px;
  display: flexbox;
}
```

화면 flexbox 라는 값은 없어서 통째로 무시됩니다. 상자가 세로로 쌓입니다

```
.typo-box {
  background: #b9432f;
  color: #fff;
  padding: 10px 14px;
  text-align: center;
}

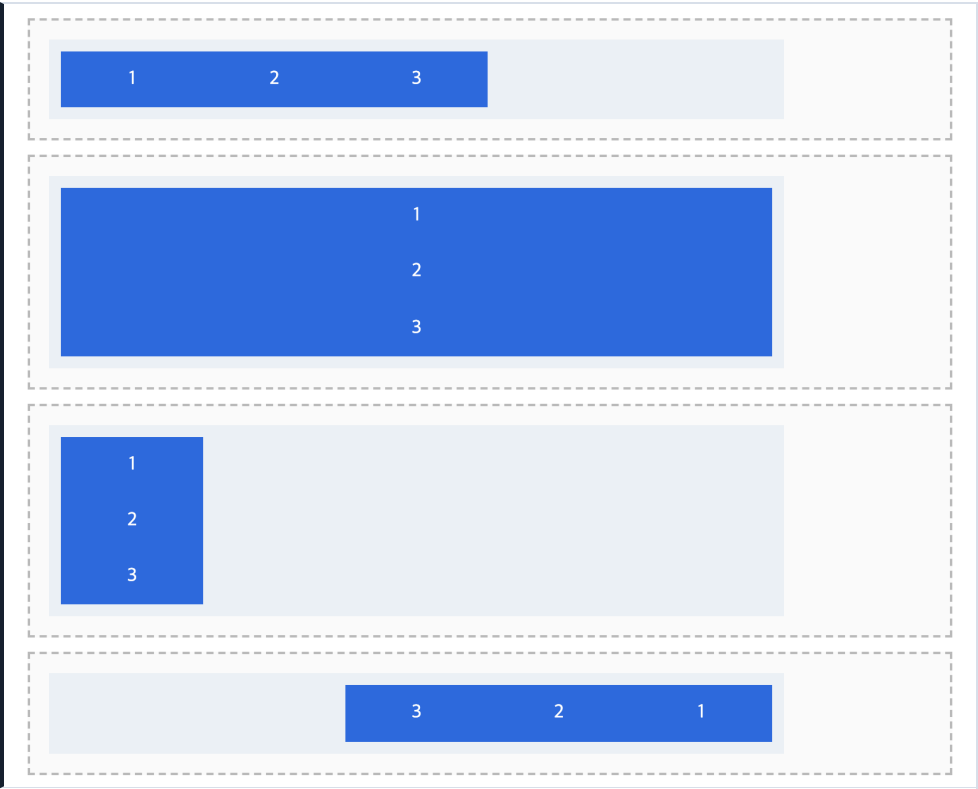
.grand-wrap {
  background: #eef2f7;
  border: 1px solid #c9d6ea;
  padding: 10px;
  display: flex;
}

.grand-item {
  background: #fff;
  border: 1px solid #c9d6ea;
  padding: 8px;
  width: 250px;
  box-sizing: border-box;
}

.grand-inner {
  background: #ffe9b8;
  border: 1px solid #e0bb62;
  padding: 4px 6px;
  font-size: 13px;
}
```

화면 흰 상자는 나란히 놓이지만 그 안의 노란 조각은 세로로 쌓인 채입니다

방향과 줄바꿈



실행하면 나오는 화면 — 방향과 줄바꿈

== _____

12단원 · 개념 02 — 방향과 줄바꿈

— _____

실행: 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.

== _____

개념01 에서 컨테이너·아이템과 주축·교차축을 배웠습니다. 이 파일에서는 그 축의 방향을 직접 바꿔 봅니다.
여기서 꼭 챙길 것은 이것 하나입니다.

방향을 바꾸면 주축과 교차축이 통째로 바뀝니다.

“주축은 가로” 라고 외운 사람은 여기서 반드시 헤맬니다. 주축은 가로가 아니라 아이템이 줄 서는 방향입니다. 줄 서는 방향을 세로로 돌리면 주축도 같이 세로로 돌아갑니다.

두 번째로 배울 것은 줄바꿈입니다. flex 는 기본적으로 줄을 바꾸지 않습니다. 아이템이 아무리 많아도 한 줄에 옥여넣습니다. 그래서 아이템이 찌그러집니다. 이것도 직접 보겠습니다.

이 파일의 모든 데모는 ‘무대’라고 이름 붙인 열은 파란 칸 안에서 벌어집니다. 무대의 안쪽 폭은 전부 정확히 600px 입니다. 숫자가 맞는지 직접 세어 볼 수 있게 하려고 폭을 못 박아 두었습니다.

이 파일은 flex-direction 과 flex-wrap 두 속성만 다릅니다. 둘 다 컨테이너에 쓰는 속성입니다. 아래 열은 파란 칸(무대)의 안쪽 폭은 모두 600px 이니, 상자 폭을 더해 보면서 읽으세요.

이 파일에 나오는 “왼쪽 좌표” 는 브라우저 창을 1280px 로 놓고 잰 값입니다. 본문이 가운데 정렬이라 창 폭이 다르면 숫자가 통째로 좌우로 밀립니다. 숫자 자체를 외우지 말고, 상자들 사이의 차이(간격·나뉜 몫)를 보세요. 그 차이는 창 폭과 상관없이 늘 같습니다.

기본 방향 row

섹션 1

display: flex 만 썼을 때 이미 적용되어 있는 값이 flex-direction: row 입니다.

row 는 “행”, 즉 가로 한 줄이라는 뜻입니다.

```
.row-stage {
  display: flex;
  flex-direction: row; ← 안 써도 이 값입니다
}
```

아이템은 왼쪽에서 오른쪽으로 차례대로 놓입니다. 무대 안쪽 폭이 600px, 상자가 120px 씩 세 개이니 360px 을 쓰고 240px 이 남습니다. 남은 240px 은 오른쪽에 그대로 빈 칸으로 남습니다.

재어 보면 상자의 왼쪽 좌표는 이렇습니다.

| | |
|------------|------------------|
| 상자 1 → 238 | (무대 안쪽이 시작되는 자리) |
| 상자 2 → 358 | (238 + 120) |
| 상자 3 → 478 | (358 + 120) |

딱 붙어 있습니다. 사이를 띄우는 방법은 개념03 에서 배웁니다.

화면

파란 상자 1 2 3 이 왼쪽부터 붙어 놓이고 오른쪽 240px 이 빈 칸으로 남습니다

1
2
3

flex-direction 은 컨테이너에 쓰는 속성입니다. 상자 하나하나에 쓰면 아무 일도 일어나지 않습니다.

▫ 직접 해보기 1

.row-stage 안에 <div class="tile">4</div> 를 하나 더 넣어 보세요. 120px 짜리가 네 개가 되어 480px 을 쓰고 120px 이 남습니다.

column

섹션 2

이 섹션이 이 파일에서 가장 중요합니다. 방향을 바꾸면 주축과 교차축이 서로 자리를 바꿉니다.

column 은 "열", 즉 세로 한 줄이라는 뜻입니다.

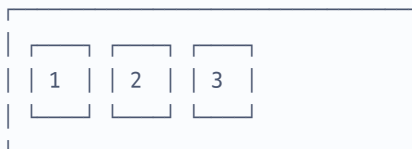
```
.col-stage {  
  display: flex;  
  flex-direction: column;  
}
```

그림으로 비교하면 이렇습니다.

— flex-direction: row

주축 → → → 가로

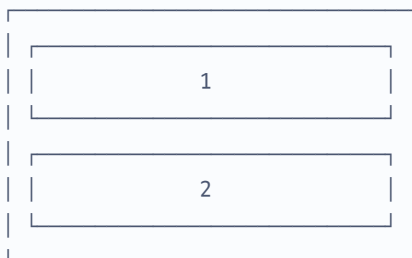
교차축 ↓ 세로



— flex-direction: column

주축 ↓ 세로

교차축 → 가로



아래 데모에서 상자의 폭이 600px 로 넓어진 것을 보세요. 폭을 넓히라고 시킨 적이 없습니다. 교차축이 가로로 바뀌었기 때문입니다.

개념01 에서 “교차축 방향으로는 남으면 늘려서 채운다” 고 했습니다. row 일 때는 교차축이 세로였으니 높이가 늘었고, column 일 때는 교차축이 가로이니 폭이 늘어납니다. 같은 규칙입니다.

화면 상자 세 개가 위에서 아래로 쌓이고, 상자 폭이 600px 로 넓어집니다

1
2
3

“그럼 flex 를 안 쓴 것과 똑같은 것 아닌가요?” 라는 질문이 나옵니다. 보이는 모양만 비슷할 뿐 안쪽은 완전히 다릅니다.

폭을 직접 정해 주면 차이가 드러납니다.

```
.col-fixed-stage .tile { width: 120px; }
```

상자가 왼쪽에 붙어 세로로 쌓입니다. 이때 왼쪽이 교차축의 시작점입니다. 개념04 에서 align-items 로 이 상자들을 오른쪽이나 가운데로 옮겨 보게 됩니다. flex 를 안 썼다면 그런 것을 할 수 없습니다.

화면 폭 120px 상자 세 개가 왼쪽에 붙은 채 위에서 아래로 쌓입니다

1
2
3

지금 당장 외울 문장은 이것입니다. row 면 주축이 가로, column 이면 주축이 세로. 개념03·04 의 정렬 속성이 전부 이 문장 위에 얹힙니다.

◀ 직접 해보기 2

<style> 에 .col-fixed-stage .tile { width: 300px; } 규칙을 새로 넣어 보세요. 세로로 쌓인 채 폭만 넓어집니다. 주축(세로)은 그대로이고 교차축(가로) 크기만 바뀐 것입니다.

reverse

섹션 3

뒤에 -reverse 를 붙이면 줄을 채우기 시작하는 쪽이 반대가 됩니다.

값은 모두 네 가지입니다.

| | |
|----------------|----------------|
| row | 왼쪽 → 오른쪽 (기본값) |
| row-reverse | 오른쪽 → 왼쪽 |
| column | 위 → 아래 |
| column-reverse | 아래 → 위 |

row-reverse 를 쓰면 주축의 시작점이 오른쪽으로 옮겨 갑니다. HTML 에는 1, 2, 3 순서로 썼는데 화면에는 3, 2, 1 로 보입니다.

재어 보면 상자 1 의 왼쪽 좌표가 718 입니다. 무대 안쪽이 238 에서 시작해 600px 이니 오른쪽 끝은 838, 거기서 상자 폭 120 을 뺀 자리가 718 입니다. 오른쪽 끝에 딱 붙은 것입니다.

화면 오른쪽 끝부터 1, 2, 3 순으로 채워져 눈에는 3 2 1 로 보입니다

1
2
3

column-reverse 는 아래에서 위로 채웁니다. 무대 높이를 200px 로 정해 두었기 때문에 아래쪽에 몰려 있는 것이 잘 보입니다.

화면 상자들이 무대 아래쪽에 몰려 있고, 위에서부터 3 2 1 순으로 보입니다

1
2
3

-reverse 는 보이는 순서만 뒤집습니다. HTML 에 적힌 순서는 그대로입니다. 그래서 눈으로 읽는 순서와 코드 순서가 달라집니다. 화면을 보지 않고 코드 순서대로 읽어 주는 프로그램을 쓰는 사람에게는 여전히 1, 2, 3 으로 읽힙니다. 꼭 필요할 때만 쓰세요.

▹ 직접 해보기

3

.colrev-stage 의 값을 flex-direction: column 으로 바꿔 보세요. 상자들이 무대 위쪽으로 올라가고 순서도 1, 2, 3 이 됩니다.

nowrap 이 기본값

섹션 4

아이템이 많아져도 flex 는 줄을 바꾸지 않습니다. 대신 아이템을 억지로 줄여서 한 줄에 밀어 넣습니다.

무대 안쪽 폭은 600px 입니다. 여기에 폭 150px 짜리 상자를 다섯 개 넣었습니다.

$150 \times 5 = 750$ → 600 보다 150px 더 큼니다

상식적으로는 넘쳐야 합니다. 그런데 화면을 보면 딱 맞게 들어가 있습니다. flex 아이탬은 자리가 모자라면 자기 폭을 줄이기 때문입니다.

150px 짜리 다섯 개가 150px 을 나눠서 30px 씩 양보 → 각각 120px

재어 보면 정말 120px 입니다. width: 150px 이라고 분명히 썼는데도 그렇습니다. 이 성질에는 flex-shrink 라는 이름이 있고 개념05 에서 자세히 다룹니다. 여기서는 “안 시켰는데 줄어든다” 는 사실만 확인하면 됩니다.

화면 상자 다섯 개가 한 줄에 들어가 있고, 폭은 120px 씩으로 줄어 있습니다

150px
150px
150px
150px
150px

글자가 든 상자라면 더 심해집니다. 이번에는 폭 200px 짜리 상자 네 개를 넣었습니다.

200 × 4 = 800 → 600 보다 200px 더 큼니다
네 개가 50px 씩 양보 → 각각 150px 로 줄어듭니다

폭이 줄면 글자가 겹쳐서 두 줄이 되고, 그만큼 상자 키가 커집니다. 재어 보면 상자 키가 47px 에서 74px 로 커져 있습니다. 디자인이 무너지는 전형적인 모습입니다.

화면 빨간 상자 네 개가 200px 에서 150px 로 줄고,

오늘의 추천 메뉴
신선한 아침 빵
따뜻한 수프 한 그릇
달콤한 후식 모음

◦ 직접 해보기 4

.nowrap-stage 안의 상자를 두 개 지워 세 개만 남겨 보세요. 150 × 3 = 450 이라 600px 안에 여유가 생기고, 상자 폭이 150px 그대로 나옵니다.

flex-wrap: wrap

섹션 5

컨테이너에 flex-wrap: wrap 한 줄을 더하면 자리가 모자랄 때 아랫줄로 내려갑니다.

```
.wrap-stage {
  display: flex;
  flex-wrap: wrap;
}
```

같은 상자 다섯 개가 이번에는 이렇게 배치됩니다.

$600 \div 150 = 4 \rightarrow$ 한 줄에 네 개까지 들어갑니다 다섯 번째는 자리가 없으니 아랫줄로 내려갑니다

중요한 것은 폭입니다. 이제 상자가 찌그러지지 않고 150px 그대로입니다. 줄일 필요가 없어졌기 때문입니다.

값은 세 가지입니다.

nowrap 줄을 바꾸지 않습니다 (기본값) wrap 모자라면 아랫줄로 내려갑니다 wrap-reverse 모자라면
윗줄로 올라갑니다

화면 윗줄에 네 개, 아랫줄에 한 개가 놓이고 폭은 모두 150px 그대로입니다

```
150px
150px
150px
150px
150px
```

wrap-reverse 는 줄이 쌓이는 방향을 뒤집습니다. 넘친 다섯 번째가 아래가 아니라 위로 올라갑니다. 즉
교차축의 시작점이 아래쪽으로 옮겨 간 것입니다.

화면 다섯 번째 상자가 윗줄에 혼자 놓이고 나머지 네 개가 아랫줄에 놓입니다

```
150px
150px
150px
150px
150px
```

화면 폭이 좁아졌을 때 알아서 접히게 만드는 것이 flex-wrap: wrap 입니다. 13단원에서 배울 반응형
화면의 절반은 이 한 줄로 해결됩니다.

▹ 직접 해보기

5

.wrap-stage 의 값을 잠깐 flex-wrap: nowrap 으로 바꿔 보세요. 다섯 개가 한 줄로 돌아오면서 폭이
다시 120px 로 줄어듭니다. 확인했으면 wrap 으로 되돌리세요.

flex-flow 단축

flex-direction 과 flex-wrap 을 한 줄로 합쳐 쓰는 방법입니다.

아래 두 코드는 완전히 같은 뜻입니다.

길게 쓴 것 · [단축으로 쓴 것]

```
flex-direction: row;          flex-flow: row wrap;
flex-wrap: wrap;
```

순서는 방향이 먼저, 줄바꿈이 나중입니다. 10단원에서 배운 background 단축과 같은 방식입니다.

한쪽만 써도 됩니다.

```
flex-flow: column;      → 방향만 column, 줄바꿈은 기본값 nowrap
flex-flow: wrap;        → 줄바꿈만 wrap, 방향은 기본값 row
```

다만 초보 단계에서는 두 줄로 나눠 쓰는 편을 권합니다. 남이 읽을 때 무엇을 정한 것인지 바로 보이기 때문입니다. 단축은 남의 코드에서 만났을 때 알아보기만 하면 충분합니다.

화면 섹션 5 의 wrap 데모와 똑같이 윗줄 네 개, 아랫줄 한 개입니다

```
150px
150px
150px
150px
150px
```

⇒ 직접 해보기 6
.flow-stage 의 값을 flex-flow: row-reverse wrap 으로 바꿔 보세요. 윗줄이 오른쪽부터 채워지는 것을 확인할 수 있습니다.

자주 하는 실수

아래 다섯 가지 모두 에러가 나지 않습니다. 화면이 생각과 다를 뿐입니다.

이런 실수를 합니다
display: flex 없이 flex-direction 만 씀

이런 실수를 합니다

실수 1 — display: flex 를 빼먹고 flex-direction 만 씀

```
.noflex-stage { flex-direction: row; } /* display 가 없습니다 */
```

아무 일도 일어나지 않습니다. flex-direction 은 flex 컨테이너에게만 뜻이 있는 속성입니다. 컨테이너가 아니면 브라우저는 이 줄을 조용히 넘깁니다. 아래가 그 결과입니다. 1 2

이렇게 쓰세요

```
.noflex-stage { display: flex; flex-direction: row; }
```

이런 실수를 합니다

column 인데 가로 정렬을 justify-content 로 함

이런 실수를 합니다

실수 2 — column 에서 좌우로 옮기려고 justify-content 를 씀

```
.swap-stage { flex-direction: column; justify-content: center; }
```

상자가 좌우 가운데로 갈 거라 기대하지만 위아래 가운데로 갑니다. column 에서는 주축이 세로이고, justify-content 는 주축을 다루기 때문입니다. 좌우로 옮기려면 개념04 에서 배울 align-items 를 써야 합니다. 1 2

이런 실수를 합니다

flex-wrap 을 안 줘서 찌그러짐

이런 실수를 합니다

실수 3 — flex-wrap 을 안 줘서 상자가 찌그러짐

```
.nowrap-stage { display: flex; } /* flex-wrap 이 없습니다 */
```

width: 150px 이라고 분명히 썼는데 화면에서는 120px 입니다. “내가 준 폭이 왜 안 먹지?” 하고 width 만 계속 들여다보게 되는데, 범인은 빠진 flex-wrap 한 줄입니다. 섹션 4 와 섹션 5 를 나란히 비교해 보세요.

이런 실수를 합니다

아이템에 flex-direction 을 씀

이런 실수를 합니다

실수 4 — 상자 하나하나에 flex-direction 을 씀

```
.tile { flex-direction: column; } /* 아이템에 썼습니다 */
```

아무 변화가 없습니다. 방향은 컨테이너가 아이TEM들을 어느 쪽으로 줄 세울지 정하는 것입니다. 아이TEM 자신은 자기가 어디에 놓일지 정하지 못합니다. 개념01 섹션 5 의 표를 다시 보세요.

이런 실수를 합니다

값 이름을 vertical / horizontal 로 씀

이런 실수를 합니다

실수 5 — 값을 vertical, horizontal 이라고 씀

```
.col-stage { flex-direction: vertical; } /* 그런 값은 없습니다 */
```

그 한 줄이 통째로 버려지고 기본값 row 로 남습니다. 세로로 세우고 싶었는데 가로로 그대로 있는 것이 유일한 신호입니다. 값은 row, column, row-reverse, column-reverse 네 개뿐입니다.

정리

⇒ 직접 해보기

7

정답

1) .row-stage 안 마지막에 한 줄 넣습니다.

```
<div class="tile">4</div>
```

→ 상자가 네 개가 되어 480px 을 쓰고 오른쪽에 120px 이 남습니다.

네 번째 상자의 왼쪽 좌표는 598 입니다. (478 + 120)

2) style 안에 이 규칙을 새로 넣습니다.

```
.col-fixed-stage .tile { width: 300px; }
```

→ 상자 폭이 120px 에서 300px 로 넓어집니다. 여전히 세로로 쌓이고

왼쪽에 붙어 있습니다. 주축(세로)은 건드리지 않았기 때문입니다.

3) .colrev-stage 를 이렇게 고칩니다.

```
.colrev-stage { flex-direction: column; height: 200px; }
```

→ 상자들이 무대 위쪽으로 올라가고, 보이는 순서도 1, 2, 3 이 됩니다.

아래쪽 59px 이 빈 칸으로 남습니다. (200 에서 47 짜리 세 개를 뺀 값)

4) .nowrap-stage 안의 card 를 두 개 지워 세 개만 남깁니다.

```
<div class="stage nowrap-stage"> <div class="card">150px</div> <div class="card">150px</div> <div class="card">150px</div> </div>
```

→ $150 \times 3 = 450$ 이라 600px 안에 여유가 생깁니다.

상자 폭이 150px 그대로 나오고 오른쪽에 150px 이 남습니다.
줄일 필요가 없으면 줄이지 않는다는 것을 확인하는 문제입니다.

5) .wrap-stage 를 이렇게 고칩니다.

```
.wrap-stage { display: flex; flex-wrap: nowrap; }
```

→ 다섯 개가 한 줄로 돌아오고 폭이 120px 로 줄어듭니다.

섹션 4 와 똑같은 모양입니다. 확인했으면 wrap 으로 되돌리세요.

6) .flow-stage 를 이렇게 고칩니다.

```
.flow-stage { display: flex; flex-flow: row-reverse wrap; }
```

→ 윗줄이 오른쪽 끝부터 채워집니다. 화면에는 윗줄이 4 3 2 1 로,

아랫줄에 다섯 번째가 오른쪽에 붙어 보입니다.

이 파일에서 공통으로 쓰는 '무대'

안쪽 폭이 정확히 600px 입니다. 계산을 눈으로 확인하려고 폭을 못 박아 두었습니다.

(width 600 은 안쪽 폭, padding 10 은 그 바깥이므로 겉 폭은 620px 입니다)

```
.stage {
  width: 600px;
  padding: 10px;
  background: #eef2f7;
}
```

화면 열린 파란 바탕의 가로로 긴 칸이 생깁니다

```
.tile {
  background: #2d6cdf;
  color: #fff;
  padding: 10px 0;
  width: 120px;
  box-sizing: border-box;
  text-align: center;
}
```

화면 폭 120px 짜리 진한 파란 상자가 됩니다

row

섹션 1

```
.row-stage {
  display: flex;
  flex-direction: row;
}
```

화면 상자 세 개가 왼쪽부터 오른쪽으로 붙어 놓입니다

column

섹션 2

```
.col-stage {
  display: flex;
  flex-direction: column;
}
```

화면 상자 세 개가 위에서 아래로 쌓이고, 가로로는 600px 까지 늘어납니다

```
.col-stage .tile {
  width: auto;
}
```

위 규칙은 이 데모에서만 .tile 의 width: 120px 을 없던 일로 되돌리는 줄입니다.

auto 는 "폭을 내가 안 정한다" 는 뜻이라, 넓히라고 시킨 것이 아닙니다.
폭을 안 정했을 때 무슨 일이 벌어지는지를 보려고 지운 것입니다.

(폭을 직접 정하면 어떻게 되는지는 바로 아래 col-fixed-stage 데모에 있습니다)

```
.col-fixed-stage {  
  display: flex;  
  flex-direction: column;  
}
```

화면 상자 세 개가 위에서 아래로 쌓이고, 폭 120px 인 채 왼쪽에 붙습니다

reverse

섹션 3

```
.rowrev-stage {  
  display: flex;  
  flex-direction: row-reverse;  
}
```

화면 1, 2, 3 이 오른쪽 끝부터 채워져 화면에는 3 2 1 순서로 보입니다

```
.colrev-stage {  
  display: flex;  
  flex-direction: column-reverse;  
  height: 200px;  
}
```

화면 1, 2, 3 이 아래쪽 끝부터 채워져 화면에는 위에서부터 3 2 1 로 보입니다

nowrap (기본값)

섹션 4

```
.nowrap-stage {  
  display: flex;  
  flex-wrap: nowrap;  
}
```

화면 상자 다섯 개가 한 줄에 억지로 들어가느라 폭이 150 에서 120 으로
찌그러집니다

```
.nowrap-stage .card,  
.wrap-stage .card,  
.wrapprev-stage .card,  
.flow-stage .card {
```



```
color: #fff;
padding: 10px 0;
width: 150px;
box-sizing: border-box;
text-align: center;
}

.squeeze-stage {
display: flex;
flex-wrap: nowrap;
}
```

화면 글자가 든 상자가 눌리면서 글자가 여러 줄로 접히고 상자 키가 커집니다

```
.squeeze-stage .card {
background: #b9432f;
color: #fff;
padding: 10px 6px;
width: 200px;
box-sizing: border-box;
text-align: center;
}
```

wrap

섹션 5

```
.wrap-stage {
display: flex;
flex-wrap: wrap;
}
```

화면 한 줄에 네 개가 들어가고 다섯 번째가 아랫줄로 내려갑니다

```
.wraprev-stage {
display: flex;
flex-wrap: wrap-reverse;
}
```

화면 아랫줄이 될 다섯 번째 상자가 위쪽 줄에 놓입니다

```
.flow-stage {
  display: flex;
  flex-flow: row wrap;
}
```

화면 섹션 5 의 wrap 데모와 완전히 같은 모양이 됩니다

자주 하는 실수

```
.swap-stage {
  display: flex;
  flex-direction: column;
  justify-content: center;
  height: 160px;
}
```

화면 세로로 쌓인 상자들이 위아래 가운데로 모입니다 (좌우가 아닙니다)

```
.swap-stage .tile {
  width: 200px;
}

.noflex-stage {
  flex-direction: row;
}
```

화면 display: flex 가 없어서 이 줄은 통째로 무시됩니다. 상자가 세로로 쌓입니다

```
.noflex-stage .tile {
  width: 200px;
}
```

주축 정렬



실행하면 나오는 화면 — 주축 정렬

12단원 · 개념 03 — 주축 정렬

실행: 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.

개념02 에서 아이템이 줄 서는 방향을 정했습니다. 그런데 줄을 세우고 나면 거의 항상 공간이 남습니다.

무대 안쪽 폭 600px, 상자 100px 짜리 세 개 → 300px 이 남습니다

이 남은 300px 을 어디에 둘 것인가를 정하는 속성이 justify-content 입니다. 왼쪽에 몰아 둘 수도, 오른쪽에 둘 수도, 상자 사이에 나눠 끼울 수도 있습니다.

이름이 어렵게 느껴지지만 뜻은 단순합니다.

| | |
|---------|------------|
| justify | 줄을 맞춘다 |
| content | 내용물(아이템들)을 |

한 가지만 기억하세요.

justify-content 는 언제나 '주축' 을 다룹니다.

지금은 기본값이 row 라서 주축이 가로입니다. 그래서 좌우로 움직입니다. 개념02 에서 본 것처럼 flex-direction 을 column 으로 바꾸면 같은 속성이 위아래를 움직입니다. 속성이 바뀐 것이 아니라 축이 돌아간 것입니다.

이 파일 뒤쪽에서는 gap 도 배웁니다. 상자 사이만 딱 띄우고 싶을 때 쓰는, 요즘 방식의 간단한 속성입니다.

이 파일의 무대(열은 파란 칸)는 모두 안쪽 폭 600px, 상자는 모두 폭 100px 입니다. 그래서 남은 공간은 항상 300px 입니다. 이 300px 이 어디로 가지만 보면 여섯 가지 값이 한눈에 구분됩니다.

이 파일에 나오는 “왼쪽 좌표” 는 브라우저 창을 1280px 로 놓고 켄 값입니다. 본문이 가운데 정렬이라 창 폭이 다르면 숫자가 통째로 좌우로 밀립니다. 숫자 자체를 외우지 말고, 상자들 사이의 차이(간격·나뉜 몫)를 보세요. 그 차이는 창 폭과 상관없이 늘 같습니다.

남는 공간이라는 개념

섹션 1

정렬을 “상자를 옮기는 일”로 생각하면 값 여섯 개가 잘 외워지지 않습니다. 남는 공간을 어디에 끼우는가로 생각하면 전부 한 줄로 설명됩니다.

무대와 상자의 관계를 그림으로 보면 이렇습니다.



기본값은 flex-start 입니다. 남는 공간을 전부 오른쪽 끝에 몰아 두는 값입니다. 그래서 상자가 왼쪽에 붙어 보입니다.

재어 보면 상자의 왼쪽 좌표는 238, 338, 438 입니다. 238 은 무대 안쪽이 시작되는 자리이고, 이후 100px 씩 더해집니다.

화면 상자 셋이 왼쪽에 딱 붙고 오른쪽에 300px 이 빈 칸으로 남습니다

1
2
3

남는 공간이 없으면 justify-content 는 아무 일도 하지 않습니다. 상자가 무대를 꽉 채우고 있을 때 값을 바꿔도 화면이 안 변하는 이유가 이것입니다.

▸ 직접 해보기 1

.stage 의 width 를 잠깐 300px 으로 바꿔 보세요. 남는 공간이 0 이 되어 상자 셋이 무대를 정확히 꽉 채웁니다. 확인했으면 600px 으로 되돌리세요.

여섯 가지 값

섹션 2

아래 여섯 줄은 HTML 이 완전히 같고 값 하나만 다릅니다. 위에서 아래로 훑으면서 남는 300px 이 어디로 갔는지 찾아보세요.

여섯 가지 값을 한 문장씩으로 정리하면 이렇습니다.

| | | |
|---------------|---------------------|----------------|
| flex-start | 남는 공간을 전부 뒤(오른쪽)에 | → 상자가 앞에 붙습니다 |
| center | 남는 공간을 앞뒤로 절반씩 | → 상자가 가운데에 옵니다 |
| flex-end | 남는 공간을 전부 앞(왼쪽)에 | → 상자가 뒤에 붙습니다 |
| space-between | 남는 공간을 상자 '사이'에만 나눴 | → 양 끝에 딱 붙습니다 |
| space-around | 상자마다 좌우로 같은 여백을 줌 | → 가장자리가 사이의 절반 |
| space-evenly | 모든 틈을 똑같이 나눔 | → 가장자리도 사이도 같음 |

실제 좌표는 이렇습니다. 무대 안쪽은 238 에서 시작해 838 에서 끝납니다.

| | |
|---------------|-----------------|
| flex-start | 238 / 338 / 438 |
| center | 388 / 488 / 588 |
| flex-end | 538 / 638 / 738 |
| space-between | 238 / 488 / 738 |
| space-around | 288 / 488 / 688 |
| space-evenly | 313 / 488 / 663 |

가운데 상자는 네 값에서 모두 488 입니다. 좌우 대칭이니 당연한 결과입니다. 차이는 첫 상자와 마지막 상자에서만 납니다.

화면 여섯 줄의 상자 위치가 모두 다릅니다.

위에서부터 왼쪽 · 가운데 · 오른쪽 · 양끝 · 조금씩 · 고르게 입니다

`justify-content: flex-start` (기본값)

1
2
3

`justify-content: center`

1
2
3

`justify-content: flex-end`

1
2
3

`justify-content: space-between`

1
2
3

`justify-content: space-around`

1
2
3

`justify-content: space-evenly`

1
2
3

`flex-start`, `flex-end` 는 “왼쪽·오른쪽”이 아니라 주축의 시작·끝이라는 뜻입니다. 그래서 `flex-direction: row-reverse` 이면 `flex-start` 가 오른쪽이 됩니다. 이름에 `left`, `right` 가 들어 있지 않은 이유가 이것입니다.

▹ 직접 해보기 2

.jc-end 의 값을 justify-content: flex-start 로 바꿔 보세요. 세 번째 줄이 첫 번째 줄과 똑같아집니다.

space 세 가지를 숫자로

섹션 3

눈으로는 셋이 비슷해 보입니다. 가장자리 여백을 재면 확실히 같습니다.

남는 공간은 300px 입니다. 상자가 셋이면 틈은 네 군데입니다.

틈1 · 상자1 [틈2] 상자2 [틈3] 상자3 [틈4]

왼쪽 끝

오른쪽 끝

세 값이 이 네 군데를 어떻게 채우는지가 전부입니다.

| | |
|---------------|--|
| space-between | 틈1 = 0, 틈2 = 150, 틈3 = 150, 틈4 = 0
→ 사이에만 300 을 2 등분 |
| space-around | 틈1 = 50, 틈2 = 100, 틈3 = 100, 틈4 = 50
→ 상자마다 좌우 50 씩 배정. 상자 사이에서 50 + 50 이 겹쳐 100 |
| space-evenly | 틈1 = 75, 틈2 = 75, 틈3 = 75, 틈4 = 75
→ 네 군데를 그냥 4 등분 |

around 의 가장자리가 사이의 절반인 이유는 “상자마다 좌우로” 나눠 주기 때문입니다. 가운데 틈에서는 왼쪽 상자의 오른쪽 뒀과 오른쪽 상자의 왼쪽 뒀이 더해집니다.

실제로 재어 보면 첫 상자 왼쪽 좌표가 이렇게 나옵니다.

| | | |
|---------------|-----|------------|
| space-between | 238 | (238 + 0) |
| space-around | 288 | (238 + 50) |
| space-evenly | 313 | (238 + 75) |

“고르게 띄우고 싶다” 면 대개 space-evenly 가 원하던 그림입니다.

화면 아래로 갈수록 첫 상자가 오른쪽으로 조금씩 밀려납니다 (238 → 288 → 313)

space-between → 가장자리 0 / 사이 150

1
2
3

space-around → 가장자리 50 / 사이 100

1
2
3

space-evenly → 가장자리 75 / 사이 75

1
2
3

▫ 직접 해보기

3

위 세 무대에서 상자를 하나씩 지워 두 개만 남겨 보세요. 남은 공간이 400px 이 되면서 세 값의 간격이 전부 달라집니다. (between 은 400, around 은 가장자리 100·사이 200, evenly 는 전부 133.3 이 됩니다)

gap

섹션 4

justify-content 는 남은 공간을 나눠 주는 속성이라 “정확히 30px 만 띄우기” 같은 일은 하지 못합니다. 그건 gap 의 일입니다.

```
.gap-stage {  
  display: flex;
```

gap: 30px; ← 아이템 사이를 30px 씩 띄웁니다

```
}
```

gap 도 컨테이너에 쓰는 속성입니다. 상자 사이에만 들어가고 무대의 가장자리에는 붙지 않는다는 점이 핵심입니다.

총 폭 = $100 \times 3 + 30 \times 2 = 360$ → 남은 공간 240 은 오른쪽에 그대로

재어 보면 상자 왼쪽 좌표는 238, 368, 498 입니다. 130px 씩 늘어납니다. (상자 폭 100 + 간격 30)

화면 상자 사이가 30px 씩 벌어지고 전체는 왼쪽에 붙어 있습니다

1
2
3

gap 과 justify-content 는 같이 쓸 수 있습니다. 싸우지 않습니다. gap 은 “적어도 이만큼은 띄워라” 이고, justify-content 는 “그리고도 남으면 어떻게 할래” 입니다.

아래는 gap: 30px 과 space-between 을 같이 쓴 것입니다. 남은 공간이 240px 이나 되니 사이가 150px 까지 벌어져서 gap 30px 은 결과에 아무 영향을 주지 못했습니다.

화면 상자가 양 끝에 붙고 사이가 150px 로 벌어집니다

1
2
3

줄이 여러 개일 때도 gap 하나면 됩니다. 가로 간격과 세로 간격에 같이 적용되기 때문입니다.

```
.gap-many-stage { gap: 20px; flex-wrap: wrap; }
.gap-many-stage .tile { width: 135px; }
```

한 줄 = $135 \times 4 + 20 \times 3 = 600$ → 딱 네 개가 들어갑니다
다섯 번째부터는 아랫줄로 내려가고, 줄 사이도 20px 벌어집니다

화면 위줄에 네 개, 아랫줄에 두 개가 놓이고 모든 틈이 20px 입니다

1
2
3
4
5
6

▹ 직접 해보기 4

.gap-stage 의 gap 을 60px 으로 바꿔 보세요. 총 폭이 420px 이 되고 세 번째 상자의 왼쪽 좌표가 558 로 옮겨 갑니다.

gap 이 margin 보다 편한 이유

섹션 5

gap 이 생기기 전에는 아이템에 margin 을 줘서 간격을 만들었습니다. 지금도 되기는 하는데, 끝에 쓸데없는 여백이 하나 더 붙습니다.

아이템마다 margin-right: 30px 을 주면 상자 사이는 30px 로 잘 벌어집니다. 그런데 마지막 상자의 오른쪽에도 30px 이 붙습니다. 아무도 원하지 않은 여백입니다.

평소에는 눈에 안 띄다가, 가운데 정렬을 하는 순간 드러납니다.

상자가 차지한 폭 = $(100 + 30) \times 3 = 390$
 무대 안쪽 폭 600 이므로 남는 공간 = 210, 좌우로 105 씩

첫 상자 왼쪽 = $238 + 105 = 343$
 마지막 상자 오른쪽 = $343 + 390 - 30 = 703$
 눈에 보이는 뭉치의 가운데 = $(343 + 703) \div 2 = 523$
 무대의 진짜 가운데 = $238 + 300 = 538$

15px 왼쪽으로 치우칩니다. 마지막 여백 30px 의 절반입니다. “가운데 정렬했는데 미묘하게 왼쪽인데?” 하는 문제의 정체가 이것입니다.

화면 상자 뭉치가 무대 가운데보다 15px 왼쪽으로 치우쳐 있습니다

1
2
3

고치려면 마지막 상자의 오른쪽 여백만 지워야 합니다. 08단원에서 배운 :nth-child 를 쓰면 이렇게 됩니다.

```
.margin-fix-stage .tile:nth-child(3) { margin-right: 0; }
```

이제 가운데가 맞습니다. 그런데 문제가 있습니다. 상자를 네 개로 늘리면 nth-child(3) 이 아니라 nth-child(4) 를 지워야 합니다. 상자 개수가 바뀔 때마다 CSS 를 따라 고쳐야 하는 것입니다.

화면 상자 뭉치가 무대 가운데에 정확히 옵니다

1
2
3

gap 은 애초에 “사이”에만 들어가므로 마지막에 여백이 붙지 않습니다. 그래서 예외 처리가 필요 없습니다.

```
.gap-center-stage { gap: 30px; justify-content: center; }
```

상자가 차지한 폭 = $100 \times 3 + 30 \times 2 = 360$
 남는 공간 240 을 좌우 120 씩 → 첫 상자 왼쪽 = 358
 뭉치의 가운데 = $358 + 180 = 538$ = 무대의 진짜 가운데

화면 상자 뭉치가 무대 가운데에 정확히 오고, 상자 사이는 30px 입니다

1
2
3

정리하면 간격은 gap, 위치는 justify-content 입니다. margin 은 “이 상자 하나만 유독 떨어뜨리고 싶다” 처럼 예외를 만들 때만 쓰세요.

▹ 직접 해보기 5

.margin-fix-stage 안에 상자를 하나 더 넣어 네 개로 만들어 보세요. :nth-child(3) 규칙이 엉뚱한 상자를 가리키게 되어 가운데가 다시 어긋납니다. gap 쪽 무대는 상자를 늘려도 멀쩡합니다.

자주 하는 실수

섹션 6

전부 에러 없이 조용히 넘어가는 실수들입니다.

이런 실수를 합니다

아이템에 justify-content

이런 실수를 합니다

실수 1 — 상자 자신에게 justify-content 를 씀

```
.tile { justify-content: center; } /* 아이템에 썼습니다 */
```

상자는 꿈쩍도 하지 않습니다. justify-content 는 컨테이너가 자기 아이템들을 어디에 모을지 정하는 속성입니다. “가운데로 가고 싶다” 는 상자의 소원은 CSS 에서 표현할 수 없습니다. 1 2

이렇게 쓰세요

```
.jc-item-stage { justify-content: center; } /* 컨테이너에 */
```

이런 실수를 합니다

글자 정렬과 헷갈림

이런 실수를 합니다

실수 2 — 상자 안의 글자까지 가운데로 갈 거라고 생각함

```
.jc-text-stage { justify-content: center; }
```

상자는 가운데로 모이지만 상자 안 글자는 왼쪽에 그대로 있습니다. justify-content 가 옮기는 것은 아이템이지 글자가 아닙니다. 글자를 가운데로 보내려면 09단원에서 배운 text-align: center 를 상자에 써야 합니다. 글자 위치 글자 위치

이런 실수를 합니다

값 이름을 지어냄

이런 실수를 합니다

실수 3 — 값 이름을 left, right 로 지어냄

```
.jc-typo-stage { justify-content: right-end; }
```

그런 값은 없어서 그 한 줄이 통째로 버려집니다. 기본값 flex-start 로 남아 상자가 왼쪽에 붙습니다. 오른쪽으로 보내는 값의 이름은 flex-end 입니다. 1 2

이런 실수를 합니다

남는 공간이 없는데 정렬을 기대

이런 실수를 합니다

실수 4 — 남는 공간이 없는데 값만 계속 바꿔 봄

```
/* 아이템이 무대를 꽉 채운 상태에서 값만 바꾸는 경우 */
```

여섯 가지 값을 다 넣어 봐도 화면이 하나도 안 바뀝니다. 나눠 줄 공간이 0 이기 때문입니다. 이럴 때는 정렬 값이 아니라 아이템 폭이나 무대 폭을 봐야 합니다. 개념05 에서 배울 flex: 1 이 붙어 있어도 같은 일이 벌어집니다.

이런 실수를 합니다

margin 으로 간격을 만들고 가운데가 안 맞는다고 함

이런 실수를 합니다

실수 5 — margin 으로 간격을 만들고 가운데가 안 맞는다고 함

```
.tile { margin-right: 30px; } /* 마지막 상자에도 붙습니다 */
```

섹션 5 의 첫 무대가 이것입니다. 15px 이 어긋납니다. 정렬은 분명히 center 인데 눈으로는 왼쪽으로 치우쳐 보이니 정렬 값을 계속 바꿔 보면서 시간을 씁니다. 범인은 마지막 상자 오른쪽의 30px 입니다. gap 으로 바꾸면 사라집니다.

정리

▹ 직접 해보기

6

정답

1) .stage 의 폭을 바꿉니다.

```
.stage { width: 300px; padding: 10px; background: #eef2f7; display: flex; }
```

→ $100 \times 3 = 300$ 이라 남는 공간이 0 이 됩니다.

이 파일의 모든 무대가 좁아지고, 여섯 가지 값이 전부 같은 모양이 됩니다.
"남는 공간이 없으면 justify-content 는 할 일이 없다" 를 눈으로 보는 문제입니다.
확인했으면 width: 600px 으로 되돌리세요.

2) .jc-end 를 이렇게 고칩니다.

```
.jc-end { justify-content: flex-start; }
```

→ 세 번째 줄의 상자가 오른쪽에서 왼쪽으로 옮겨 가

첫 번째 줄과 완전히 같은 모양이 됩니다. 첫 상자 좌표가 538 에서 238 로 바뀝니다.

3) 세 무대에서 마지막 상자를 지웁니다.

```
<div class="stage jc-between"> <div class="tile">1</div> <div class="tile">2</div>  
</div>
```

→ 상자가 쓰는 폭이 200 이 되어 남는 공간이 400 으로 늘어납니다.

between 은 사이 하나에 400 을 몰아 주고,
around 은 가장자리 100 · 사이 200,
evenly 는 틈 세 군데를 133.3 씩 나눕니다.
같은 값이라도 아이템 개수가 바뀌면 결과가 달라진다는 점을 확인하세요.

4) .gap-stage 의 gap 을 바꿉니다.

```
.gap-stage { gap: 60px; }
```

→ 총 폭이 $100 \times 3 + 60 \times 2 = 420$ 이 됩니다.

상자 왼쪽 좌표가 238 / 398 / 558 로 벌어집니다.

5) .margin-fix-stage 안에 상자를 하나 더 넣습니다.

```
<div class="stage margin-fix-  
stage"> <div class="tile">1</div> <div class="tile">2</div> <div class="tile">3</div>  
  <div class="tile">4</div>  
</div>
```

→ :nth-child(3) 은 이제 마지막이 아닌 세 번째 상자를 가리킵니다.

세 번째와 네 번째 사이만 붙어 버리고, 마지막 상자 오른쪽에는 여백이 남아 가운데가 다시 15px 어긋납니다.
같은 일을 gap 쪽 무대에서 해 보면 상자를 몇 개로 늘려도 가운데가 맞습니다.

개념02 와 같은 '무대'. 안쪽 폭이 정확히 600px 입니다

```
.stage {  
  width: 600px;  
  padding: 10px;  
  background: #eef2f7;  
  display: flex;  
}
```

화면 열린 파란 칸이 생기고 그 안의 상자들이 가로로 나란히 놓입니다

```
.tile {  
  background: #2d6cdf;  
  color: #fff;  
  padding: 10px 0;  
  width: 100px;  
  box-sizing: border-box;  
  text-align: center;  
}
```

화면 폭 100px 짜리 진한 파란 상자가 됩니다

```
.stage-label {  
  font-size: 13px;  
  color: #555;  
  margin: 14px 0 4px;  
}
```

화면 각 무대 위에 작은 회색 글씨로 어떤 값인지 적힙니다

여섯 가지 값

섹션 2

```
.jc-start {  
  justify-content: flex-start;  
}
```

| | |
|----|--|
| 화면 | 상자 셋이 왼쪽에 붙고 오른쪽에 300px 이 남습니다 |
| | <pre>.jc-center { justify-content: center; }</pre> |
| 화면 | 상자 셋이 가운데로 모이고 좌우에 150px 씩 남습니다 |
| | <pre>.jc-end { justify-content: flex-end; }</pre> |
| 화면 | 상자 셋이 오른쪽에 붙고 왼쪽에 300px 이 남습니다 |
| | <pre>.jc-between { justify-content: space-between; }</pre> |
| 화면 | 양 끝에 딱 붙고 상자 사이만 150px 씩 벌어집니다 |
| | <pre>.jc-around { justify-content: space-around; }</pre> |
| 화면 | 가장자리 50px, 상자 사이 100px 로 벌어집니다 |
| | <pre>.jc-evenly { justify-content: space-evenly; }</pre> |
| 화면 | 가장자리와 상자 사이가 모두 75px 로 똑같이 벌어집니다 |

gap

섹션 4

	<pre>.gap-stage { gap: 30px; }</pre>
화면	상자 사이가 30px 씩 벌어지고 전체가 왼쪽에 붙어 있습니다
	<pre>.gap-between-stage { gap: 30px; justify-content: space-between; }</pre>

화면 양 끝에 붙고 사이가 150px 로 벌어집니다. gap 30 은 최소 간격 역할만 합니다

```
.gap-many-stage {
  gap: 20px;
  flex-wrap: wrap;
}
```

화면 한 줄에 네 개가 들어가고 줄과 줄 사이도 20px 벌어집니다

```
.gap-many-stage .tile {
  width: 135px;
}
```

gap 과 margin 비교

섹션 5

```
.margin-stage {
  justify-content: center;
}
```

화면 상자 뭉치가 가운데보다 15px 왼쪽으로 치우칩니다

```
.margin-stage .tile {
  margin-right: 30px;
}

.margin-fix-stage {
  justify-content: center;
}
```

화면 세 번째 상자의 오른쪽 여백만 지워서 겨우 가운데가 맞습니다

```
.margin-fix-stage .tile {
  margin-right: 30px;
}

.margin-fix-stage .tile:nth-child(3) {
  margin-right: 0;
}

.gap-center-stage {
  gap: 30px;
  justify-content: center;
}
```

화면 손덜 것 없이 정확히 가운데에 옵니다

자주 하는 실수

섹션 6

```
.jc-item-stage {
  background: #f7eeee;
}
```

화면 아이템에 justify-content 를 써도 상자는 왼쪽에 그대로 있습니다

```
.jc-item-stage .tile {
  background: #b9432f;
  justify-content: center;
}

.jc-text-stage {
  background: #f7eeee;
  justify-content: center;
}
```

화면 상자는 가운데로 모이지만 상자 안 글자는 왼쪽에 붙어 있습니다

```
.jc-text-stage .tile {
  background: #b9432f;
  width: 200px;
  text-align: left;
  padding: 10px 8px;
}

.jc-typo-stage {
  background: #f7eeee;
  justify-content: right-end;
}
```

화면 그런 값이 없어서 통째로 무시되고 상자가 왼쪽에 붙습니다

```
.jc-typo-stage .tile {
  background: #b9432f;
}
```

교차축 정렬



실행하면 나오는 화면 — 교차축 정렬

12단원 · 개념 04 — 교차축 정렬

실행: 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.

개념03 에서 주축(아이템이 줄 서는 방향)을 다루는 justify-content 를 배웠습니다. 이 파일은 나머지 한 축, 교차축을 다룹니다.

주축	아이템이 줄 서는 방향	→ justify-content
교차축	그와 직각인 방향	→ align-items

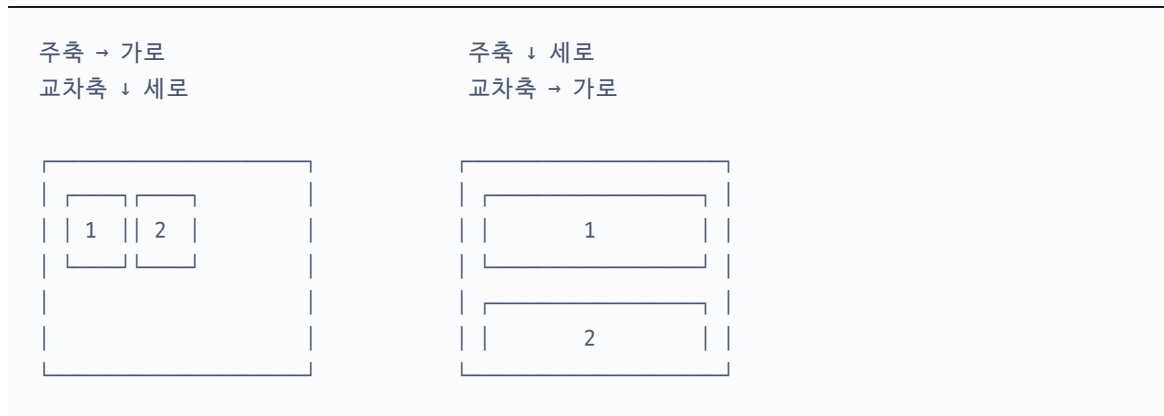
이 파일에서 가장 조심할 것은 이것입니다.

`justify-content` 와 `align-items` 는 "가로 정렬 / 세로 정렬" 이 아닙니다.

지금은 기본값이 row 라서 마침 그렇게 보일 뿐입니다. `flex-direction: column` 으로 바꾸면 둘의 역할이 통째로 맞바뀝니다. 섹션 4 에서 같은 코드로 결과가 뒤집히는 것을 직접 보게 됩니다.

그림으로 두 축을 다시 확인하세요.

row · [column]



이 파일의 무대는 안쪽 폭 600px, 안쪽 높이 160px 입니다. 교차축이 움직이는 것을 보려면 세로로도 남는 공간이 있어야 하기 때문입니다.

— —

이 파일의 무대는 안쪽 폭 600px · 안쪽 높이 160px 입니다. 상자 키는 글자 한 줄 기준으로 47px 이니, 세로로 113px 이 남습니다. 이 113px 이 어디로 가는지 보면 `align-items` 가 한눈에 이해됩니다.

이 파일에 나오는 “왼쪽 좌표” 는 브라우저 창을 1280px 로 놓고 잰 값입니다. 본문이 가운데 정렬이라 창 폭이 다르면 숫자가 통째로 좌우로 밀립니다. 숫자 자체를 외우지 말고, 상자들 사이의 차이(간격·나뉜 몫)를 보세요. 그 차이는 창 폭과 상관없이 늘 같습니다.

기본값 stretch

섹션 1

`align-items` 를 한 번도 안 썼는데도 이미 값이 하나 들어가 있습니다. `stretch`, 즉 교차축 방향으로 늘려서 채우기입니다.

개념01 섹션 4 에서 “글자는 한 줄인데 상자 키가 무대만큼 커진다” 는 것을 봤습니다. 그 동작의 이름이 `stretch` 입니다.

```
.stage { display: flex; height: 160px; }
/* align-items 를 안 썼습니다 */
```

상자에 height 를 준 적이 없으니 브라우저가 알아서 무대 높이만큼 늘립니다. 재어 보면 상자 키가 160px 입니다. 글자 한 줄만 있는데도 그렇습니다.

이 기본값 때문에 생기는 아주 흔한 오해가 있습니다.

"카드 높이를 안 맞췄는데 왜 다 똑같지?" → stretch 때문입니다
"align-items: center 를 썼더니 키가 줄었네?" → stretch 를 꺾기 때문입니다

늘리기를 그만두게 하려면 다른 값을 주면 됩니다. 섹션 2 에서 봅니다.

화면 상자 세 개의 키가 무대 안쪽 높이와 같은 160px 로 늘어나 있습니다

1
2
3

단, stretch 는 "높이를 정하지 않은 아이템" 만 늘립니다. 아이템에 height 를 직접 주면 그 값이 이깁니다.

```
.fixed-h-stage .tile { height: 60px; }
```

아래 무대에서 상자 키는 60px 그대로입니다. 그리고 남은 100px 은 상자 아래쪽에 그대로 남습니다. 즉 높이를 정한 순간 stretch 는 할 일이 없어지고, 상자는 교차축 시작점(위)에 붙습니다.

화면 상자 키가 60px 이고 무대 위쪽에 붙어 있습니다. 아래 100px 이 빕니다

1
2
3

◁ 직접 해보기

1

.fixed-h-stage .tile 의 height: 60px 을 지워 보세요. 상자 키가 다시 160px 로 늘어납니다.
확인했으면 되돌려 놓으세요.

네 가지 값

섹션 2

stretch 를 끄면 상자는 내용만큼의 키(47px) 로 돌아가고, 남은 113px 을 어디에 둘지 고를 수 있게 됩니다.

값을 한 줄씩 정리하면 이렇습니다.

stretch	(기본값) 교차축 방향으로 늘려 채웁니다
flex-start	교차축의 시작점(row 에서는 위)에 붙입니다
center	교차축 가운데에 둡니다
flex-end	교차축의 끝점(row 에서는 아래)에 붙입니다
baseline	글자가 앉은 선을 기준으로 맞춥니다

justify-content 와 이름이 비슷해서 헷갈리기 쉽지만 다른 점이 두 가지 있습니다.

- 1 기본값이 다릅니다. justify-content → flex-start / align-items → stretch
- 2 space-between 같은 값이 없습니다. 아이템이 한 줄에 하나뿐이니 나눌 것이 없습니다.

재어 보면 상자의 위쪽 좌표가 무대 안쪽 위에서부터 이만큼 떨어져 있습니다.

flex-start	0	
center	56.5	((160 - 47) ÷ 2)
flex-end	113	(160 - 47)

화면 상자가 위 · 가운데 · 아래로 내려가고, 셋 다 키는 47px 입니다

align-items: flex-start

1
2
3

align-items: center

1
2
3

align-items: flex-end

1
2
3

baseline 은 글자 크기가 서로 다를 때만 쓸모가 있습니다. 상자의 모서리가 아니라 글자가 앉아 있는 선을 맞춥니다.

아래에서 가운데 상자만 글자 크기가 30px 입니다. 상자 키는 제각각인데 글자의 아랫선은 한 줄로 이어집니다. 제목과 설명을 한 줄에 나란히 놓을 때 유용합니다.

화면 상자 키가 서로 다른데 글자의 아랫선이 한 줄로 맞춰져 있습니다

작은 글자
큰 글자
작은 글자

▣ 직접 해보기 2

.ai-center 의 값을 align-items: stretch 로 바꿔 보세요. 두 번째 줄의 상자가 다시 160px 로 길어져
섹션 1 의 첫 무대와 같아집니다.

align-self

섹션 3

align-items 는 컨테이너에 쓰는 속성이라 아이템 전부에 적용됩니다. 하나만 예외로 두고 싶을 때 쓰는
것이 align-self 입니다.

이 단원에서 아이템에 쓰는 정렬 속성은 align-self 하나뿐입니다.

```
.self-stage { align-items: center; }    ← 컨테이너: 전부 가운데  
.self-top    { align-self: flex-start; } ← 아이템: 나만 위로  
.self-bottom { align-self: flex-end; }   ← 아이템: 나만 아래로  
.self-stretch { align-self: stretch; } ← 아이템: 나만 늘어나기
```

값의 종류는 align-items 와 같습니다. 붙이는 자리만 다릅니다.

주축 쪽에는 이런 속성이 없습니다. justify-self 는 flex 에 없습니다. 주축은 아이тем들이 한 줄로 이어져
있어서 하나만 따로 옮길 수가 없기 때문입니다. (하나만 밀어내는 방법은 개념06 섹션 1 에서 margin 으로
해결합니다)

화면 첫 상자는 가운데, 둘째는 위, 셋째는 아래,

넷째는 위아래로 꼭 찬 160px 입니다

기본
위로
아래로
늘어나기

align-self 는 아이тем에 쓰는 속성입니다. 개념01 섹션 5 의 표에서 아이тем 쪽에 적혀 있던 그 속성입니다.
컨테이너에 쓰면 아무 일도 일어나지 않습니다.

▣ 직접 해보기 3

selfTop 상자의 클래스를 self-top 에서 self-bottom 으로 바꿔 보세요. 두 번째 상자가 위에서 아래로
내려옵니다.

column 이면 뒤바뀝니다

섹션 4

이 단원 최대의 함정입니다. 속성은 그대로인데 결과가 뒤집힙니다. 아래 네 무대는 딱 한 글자, row 와 column 만 다릅니다.

같은 justify-content: flex-end 를 두 방향에서 써 봅니다.

```
.swap-row { flex-direction: row;    justify-content: flex-end; }
.swap-col { flex-direction: column; justify-content: flex-end; }
```

row 에서는 주축이 가로이므로 상자가 오른쪽 끝으로 갑니다. column 에서는 주축이 세로이므로 상자가 아래쪽 끝으로 갑니다.

같은 속성, 같은 값인데 움직이는 방향이 완전히 다릅니다. justify-content 가 “가로 정렬” 이 아니라는 증거입니다.

화면 위 무대는 상자가 오른쪽 끝, 아래 무대는 상자가 아래쪽 끝에 붙습니다

```
flex-direction: row + justify-content: flex-end

1
2

flex-direction: column + justify-content: flex-end

1
2
```

이번에는 align-items: flex-end 를 두 방향에서 써 봅니다.

```
.swapai-row { flex-direction: row;    align-items: flex-end; }
.swapai-col { flex-direction: column; align-items: flex-end; }
```

row 에서는 교차축이 세로이므로 상자가 아래쪽으로 갑니다. column 에서는 교차축이 가로이므로 상자가 오른쪽으로 갑니다.

위의 justify-content 결과와 정확히 반대입니다. 두 표를 나란히 놓으면 이렇습니다.

	justify-content: flex-end	align-items: flex-end
row (기본)	오른쪽	아래쪽
column	아래쪽	오른쪽

외울 것은 표가 아니라 한 문장입니다.

justify-content 는 주축, align-items 는 교차축.
주축은 flex-direction 이 정한다.

화면 위 무대는 상자가 아래쪽 끝, 아래 무대는 상자가 오른쪽 끝에 붙습니다

```
flex-direction: row + align-items: flex-end

1
2

flex-direction: column + align-items: flex-end

1
2
```

column 으로 바꾼 순간 화면이 엉뚱해졌다면, 두 속성의 이름을 서로 바꿔 써 보세요. 대부분 그것으로 해결됩니다.

▣ 직접 해보기 4

.swap-col 의 justify-content: flex-end 를 align-items: flex-end 로 바꿔 보세요. 아래로 가 있던 상자가 오른쪽으로 옮겨 갑니다.

정중앙 정렬 관용구

섹션 5

실무에서 가장 많이 쓰는 세 줄입니다. 통째로 외워도 되는 유일한 것입니다.

```
.dead-center-stage {
display: flex;
```

```
justify-content: center; /* 주축 가운데 */ align-items: center; /* 교차축 가운데 */
```

```
}
```

축이 두 개뿐이니 둘 다 가운데면 무조건 정중앙입니다. 그래서 이 조합은 flex-direction 이 무엇이든 똑같이 동작합니다. 방향을 바꿔도 결과가 그대로인 유일한 경우입니다.

재어 보면 이렇습니다. 무대 안쪽은 가로 238~838, 세로로 160px 입니다.

상자 폭 200 → 가로로 남는 400 을 반씩 → 왼쪽 좌표 238 + 200 = 438 상자 키 47 → 세로로 남는 113 을 반씩 → 위에서 56.5 만큼 내려온 자리

개념01 섹션 1 에서 position 으로는 24px 가까이 어긋났던 그 일이 여기서는 계산 없이 정확히 맞습니다.

화면 상자가 무대의 가로세로 한가운데에 정확히 놓입니다

정중앙

화면 방향을 column 으로 바꿔도 상자 위치가 똑같습니다

정중앙

아이템이 여러 개여도 됩니다. 여러 개면 뭉치 전체가 가운데에 옵니다. 아이템 사이를 띄우고 싶으면 개념03에서 배운 gap 을 더하세요.

▹ 직접 해보기

5

.dead-center-stage 안에 상자를 하나 더 넣고 컨테이너에 gap: 20px 을 더해 보세요. 상자 두 개가 20px 간격으로 붙은 채 뭉치째 가운데에 옵니다.

여러 줄일 때 align-content

섹션 6

align-items 는 한 줄 안에서 아이템을 움직입니다. 줄 자체를 움직이는 것은 align-content 입니다.

조건이 하나 있습니다. 컨테이너에 flex-wrap: wrap 이 있어야 합니다. 기본값인 nowrap 이면 줄이 하나로 고정되어 align-content 가 통째로 무시됩니다. (wrap 만 있으면 줄이 실제로는 한 줄뿐이어도 align-content 는 그 한 줄을 움직입니다)

```
.lines-stage { height: 240px; flex-wrap: wrap; }
.lines-stage .tile { width: 150px; }
```

$600 \div 150 = 4 \rightarrow$ 한 줄에 네 개, 상자가 여섯 개이니 두 줄

align-content 의 기본값도 stretch 입니다. 줄이 두 개이고 무대 안쪽 높이가 240 이니 한 줄에 120px 씩 배정됩니다. 그래서 아무것도 안 쓰면 상자 키가 120px 로 늘어납니다.

값은 justify-content 와 거의 같습니다.

stretch (기본) / flex-start / center / flex-end
space-between / space-around / space-evenly

화면 첫 무대만 상자 키가 120px 이고 나머지 셋은 47px 입니다.

두 줄 뭉치가 위 · 가운데 · 위아래 끝으로 배치됩니다

아무것도 안 씌 (기본값 stretch) → 줄마다 120px 씩

1
2
3
4
5
6

`align-content: flex-start`

1
2
3
4
5
6

`align-content: center`

1
2
3
4
5
6

`align-content: space-between`

1
2
3
4
5
6

세 속성의 관계를 한 줄로 정리하면 이렇습니다. `justify-content` 는 줄 안에서 가로로, `align-items` 는 줄 안에서 세로로, `align-content` 는 줄 뭉치를 세로로 움직입니다. (기본값 row 기준입니다)

⇒ 직접 해보기 6

`.lines-stage` 의 `flex-wrap: wrap` 을 `nowrap` 으로 바꿔 보세요. 네 무대가 전부 똑같아집니다. `align-content` 를 준 세 무대도 아무 변화가 없어집니다.

자주 하는 실수

섹션 7

다섯 가지 모두 화면이 안 바뀌는 것이 유일한 신호입니다.

이런 실수를 합니다

column 에서 두 속성을 반대로 씀

이런 실수를 합니다

실수 1 — **column** 인데 좌우 정렬을 **justify-content** 로 함

```
.box { flex-direction: column; justify-content: center; }
```

좌우 가운데로 갈 줄 알았는데 위아래 가운데로 갑니다. **column** 에서 주축은 세로입니다. 좌우로 옮기려면 **align-items: center** 를 써야 합니다. 섹션 4 의 네 무대를 다시 보세요. 이 단원에서 가장 많이 나는 실수입니다.

이런 실수를 합니다

컨테이너에 높이가 없음

이런 실수를 합니다

실수 2 — 높이가 없는 컨테이너에서 **align-items: center** 를 씀

```
.noheight-stage { display: flex; align-items: center; }
```

아무 변화가 없습니다. 높이를 정하지 않으면 컨테이너 키는 가장 큰 아이টে에 딱 맞춰집니다. 남은 세로 공간이 0 이니 가운데로 보낼 자리 자체가 없습니다. **align-items** 가 안 먹는다면 먼저 컨테이너 높이를 확인하세요. 1 2

이런 실수를 합니다

align-self 를 컨테이너에 씀

이런 실수를 합니다

실수 3 — **align-self** 를 컨테이너에 씀

```
.selfmiss-stage { align-self: flex-end; } /* 컨테이너에 썼습니다 */
```

아이템들은 그대로 있습니다. **align-self** 는 아이템이 자기 자리를 스스로 정하는 속성이라 컨테이너가 쓰면 아무 뜻이 없습니다. 전부 옮기려면 **align-items**, 하나만 옮기려면 **align-self** 입니다. 1 2

이런 실수를 합니다

flex-wrap 없이 align-content 를 씀

이런 실수를 합니다

실수 4 — flex-wrap 없이 align-content 를 씀

```
.oneline-stage { align-content: center; } /* wrap 이 없습니다 */
```

아무 일도 일어나지 않습니다. align-content 는 여러 줄을 다루는 속성이라 flex-wrap: wrap 이 없으면 통째로 무시됩니다. 아래 무대에는 wrap 이 없어서 상자가 여전히 160px 로 늘어나 있습니다. 한 줄 안에서 위아래로 옮기고 싶었다면 필요한 것은 align-items 입니다. 1 2

이런 실수를 합니다

stretch 인데 height 를 줘 놓고 안 늘어난다고 함

이런 실수를 합니다

실수 5 — 카드 높이를 맞추려는데 아이템에 height 가 있음

```
.card { height: 60px; } /* stretch 가 이 값을 못 이깁니다 */
```

“카드 키를 자동으로 맞추고 싶은데 안 된다” 는 대부분 이 경우입니다. stretch 는 높이를 정하지 않은 아이템만 늘립니다. 아이템에 적어 둔 height 를 지우면 그때부터 저절로 맞춰집니다. 섹션 1 의 두 번째 무대가 그 예입니다.

정리 —————

=====

▫ 직접 해보기

7

정답

=====

1) .fixed-h-stage .tile 에서 height 를 지웁니다.

```
.fixed-h-stage .tile { }
```

→ 상자 키가 60px 에서 160px 로 늘어납니다.

높이를 정하지 않으면 stretch 가 일을 한다는 것을 확인하는 문제입니다.
되돌릴 때는 height: 60px 를 다시 씁니다.

2) .ai-center 를 이렇게 고칩니다.

```
.ai-center { align-items: stretch; }
```

→ 두 번째 무대의 상자가 47px 에서 160px 로 길어집니다.

가운데 정렬이 사라진 것이 아니라, 위아래를 꽉 채워서 가운데라는 말 자체가 필요 없어진 것입니다.

3) 두 번째 상자의 클래스를 바꿉니다.

```
<div class="tile self-bottom" id="selfTop">위로</div>
```

→ 상자가 무대 위쪽에서 아래쪽으로 내려옵니다.

세 번째 상자와 같은 높이에 놓입니다.

4) .swap-col 을 이렇게 고칩니다.

```
.swap-col { flex-direction: column; align-items: flex-end; }
```

→ 아래쪽에 몰려 있던 상자 두 개가 위쪽으로 올라가고 오른쪽 끝에 붙습니다.

column 에서 교차축은 가로이기 때문입니다.
상자의 왼쪽 좌표가 238 에서 718 로 바뀝니다. (838 에서 상자 폭 120 을 뺀 자리)

5) HTML 과 CSS 를 이렇게 고칩니다.

```
.dead-center-stage {
  justify-content: center;
  align-items: center;
  gap: 20px;
}

<div class="stage dead-center-stage"> <div class="tile">정중앙
</div> <div class="tile">정중앙</div>
</div>
```

→ 상자 두 개가 20px 간격으로 붙은 뭉치가 되어 가운데에 옵니다.

뭉치 폭은 $200 \times 2 + 20 = 420$ 이므로 첫 상자 왼쪽 좌표는 328 입니다.

6) .lines-stage 의 줄바꿈을 끕니다.

```
.lines-stage { height: 240px; flex-wrap: nowrap; }
```

→ 여섯 개가 한 줄에 억지로 들어가면서 상자 폭이 150px 에서 100px 로 줄어듭니다.

(150 × 6 = 900 이 600 안에 들어가느라 각각 50px 씩 양보한 결과입니다)
 동시에 align-content 를 준 세 무대가 첫 무대와 똑같아집니다.
 wrap 이 없으면 align-content 가 통째로 무시되기 때문입니다.
 상자 키는 align-items 의 기본값 stretch 때문에 네 무대 모두 240px 이 됩니다.
 확인했으면 flex-wrap: wrap 으로 되돌리세요.

무대: 안쪽 폭 600px, 안쪽 높이 160px

교차축을 보려면 높이가 있어야 합니다. 그래서 이 파일의 무대에는 높이가 있습니다.

```
.stage {
  width: 600px;
  height: 160px;
  padding: 10px;
  background: #eef2f7;
  display: flex;
}
```

화면 열린 파란 칸이 세로로도 넉넉해집니다

```
.tile {
  background: #2d6cdf;
  color: #fff;
  padding: 10px 0;
  width: 120px;
  box-sizing: border-box;
  text-align: center;
}
```

화면 폭 120px 짜리 진한 파란 상자가 됩니다

```
.stage-label {
  font-size: 13px;
  color: #555;
  margin: 14px 0 4px;
}
```

화면 각 무대 위에 작은 회색 글씨로 어떤 값인지 적힙니다

stretch

섹션 1

```
.fixed-h-stage .tile {
  height: 60px;
}
```

화면 높이를 직접 정한 상자는 stretch 가 있어도 60px 그대로입니다

네 가지 값

섹션 2

```
.ai-start {
  align-items: flex-start;
}
```

화면 상자가 무대 위쪽에 붙고 키는 47px 로 줄어듭니다

```
.ai-center {
  align-items: center;
}
```

화면 상자가 무대 위아래 가운데에 옵니다

```
.ai-end {
  align-items: flex-end;
}
```

화면 상자가 무대 아래쪽에 붙습니다

```
.ai-baseline {
  align-items: baseline;
}
```

화면 글자 크기가 달라도 글자가 앉은 선이 한 줄로 맞춰집니다

```
.ai-baseline .small {
  font-size: 13px;
}
.ai-baseline .big {
  font-size: 30px;
}
```

```
.self-stage {
  align-items: center;
}
```

화면 상자 셋은 가운데, self 를 준 하나만 다른 자리에 놓입니다

```
.self-top {
  align-self: flex-start;
}
.self-bottom {
  align-self: flex-end;
}
.self-stretch {
  align-self: stretch;
}
```

row 와 column 에서 뒤바뀜

```
.swap-row {
  flex-direction: row;
  justify-content: flex-end;
}
```

화면 상자가 오른쪽 끝으로 갑니다 (주축이 가로)

```
.swap-col {
  flex-direction: column;
  justify-content: flex-end;
}
```

화면 같은 코드인데 상자가 아래쪽 끝으로 갑니다 (주축이 세로)

```
.swapai-row {
  flex-direction: row;
  align-items: flex-end;
}
```

화면 상자가 아래쪽 끝으로 갑니다 (교차축이 세로)


```
.swapai-col {
  flex-direction: column;
  align-items: flex-end;
}
```

화면 같은 코드인데 상자가 오른쪽 끝으로 갑니다 (교차축이 가로)

정중앙 관용구

섹션 5

```
.dead-center-stage {
  justify-content: center;
  align-items: center;
}
```

화면 상자가 무대의 가로세로 정확히 한가운데에 놓입니다

```
.dead-center-stage .tile {
  width: 200px;
}

.dead-center-col {
  flex-direction: column;
  justify-content: center;
  align-items: center;
}
```

화면 방향을 바꿔도 두 줄이 그대로라 여전히 정중앙입니다

```
.dead-center-col .tile {
  width: 200px;
}
```

align-content

섹션 6

```
.lines-stage {
  height: 240px;
  flex-wrap: wrap;
}

.lines-stage .tile {
  width: 150px;
}
```

화면 한 줄에 네 개가 들어가고 여섯 개이므로 두 줄이 됩니다

```
.ac-start {
  align-content: flex-start;
}
```

화면 두 줄이 무대 위쪽에 붙고 아래가 비어 있습니다

```
.ac-center {
  align-content: center;
}
```

화면 두 줄이 무대 위아래 가운데로 모입니다

```
.ac-between {
  align-content: space-between;
}
```

화면 첫 줄은 맨 위, 둘째 줄은 맨 아래에 붙습니다

자주 하는 실수

섹션 7

```
.bad-stage {
  background: #f7eeee;
}
.bad-stage .tile {
  background: #b9432f;
}
.noheight-stage {
  height: auto;
  align-items: center;
}
```

화면 높이가 없어서 남는 세로 공간이 0 입니다. 아무 변화가 없습니다

```
.selfmiss-stage {
  align-items: center;
}
.online-stage {
  align-content: center;
}
```

화면 줄이 하나뿐이라 align-content 는 아무 일도 하지 않습니다

늘어나고 줄어들기



실행하면 나오는 화면 — 늘어나고 줄어들기

==

12단원 · 개념 05 — 늘어나고 줄어들기

—

실행: 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.

==

지금까지는 아이템 크기를 그대로 둔 채 '위치'만 옮겼습니다. 이 파일에서는 아이템의 '크기'를 flex 에게 맡깁니다.

여기서 배우는 속성은 전부 아이템에 쓰는 것입니다. 개념01 섹션 5 의 표에서 아래쪽에 있던 것들입니다.

flex-grow	남는 공간이 있을 때 얼마나 더 가져갈까
flex-shrink	공간이 모자랄 때 얼마나 양보할까
flex-basis	나눠 갖기를 시작하기 전의 기본 크기
flex	위 셋을 한 줄로 (가장 많이 씁니다)
order	보이는 순서

이 셋의 관계는 한 문장으로 정리됩니다.

먼저 basis 만큼 자리를 잡고, 남으면 grow 비율로 더 갖고,
모자라면 shrink 비율로 양보한다.

이 파일의 무대는 안쪽 폭이 600px 입니다. 숫자가 딱 떨어지도록 골라 두었으니 직접 더하고 나눠 보면서 읽으세요.

이 파일의 속성은 전부 아이টে에 씁니다. 컨테이너가 아닙니다. 무대 안쪽 폭은 600px 이고, 기본 상자 폭은 100px 입니다. 상자 셋이면 300px 을 쓰고 300px 이 남는다는 것만 기억하고 읽으세요.

이 파일에 나오는 "왼쪽 좌표" 는 브라우저 창을 1280px 로 놓고 잰 값입니다. 본문이 가운데 정렬이라 창 폭이 다르면 숫자가 통째로 좌우로 밀립니다. 숫자 자체를 외우지 말고, 상자들 사이의 차이(간격·나눠 몫)를 보세요. 그 차이는 창 폭과 상관없이 늘 같습니다.

flex-grow

섹션 1

flex-grow 는 남는 공간을 몇 몫 가져갈지 적는 숫자입니다. 기본값은 0 이고, 0 은 "한 몫도 안 받겠다" 는 뜻입니다.

먼저 아무것도 안 준 상태입니다. 기본값이 0 이라 아무도 안 가져갑니다. 상자는 100px 그대로이고 남는 300px 은 오른쪽에 그냥 남습니다.

화면 상자 셋이 100px 씩, 오른쪽에 300px 이 빈 칸으로 남습니다

100px
100px
100px

이번에는 셋 다 1 을 줍니다.

```
.grow-all .tile { flex-grow: 1; }
```

몫을 세는 방법은 이렇습니다.

몫의 합 = $1 + 1 + 1 = 3$
 한 몫의 크기 = 남은 공간 $300 \div 3 = 100$
 각 상자의 최종 폭 = 원래 $100 +$ 받은 $100 = 200$

재어 보면 정말 200px 씩입니다. 왼쪽 좌표는 238 / 438 / 638 입니다. 중요한 것은 "200px 로 만들어라" 라고 시킨 적이 없다는 점입니다. 무대가 넓어지면 상자도 알아서 넓어집니다.

화면 상자 셋이 200px 씩으로 넓어져 무대를 빈틈없이 채웁니다

1
1
1

숫자를 다르게 주면 비율이 달라집니다.

첫 상자 flex-grow: 2, 나머지 1, 1

몫의 합 = $2 + 1 + 1 = 4$
 한 몫 = $300 \div 4 = 75$
 첫 상자 = $100 + 75 \times 2 = 250$
 둘째 상자 = $100 + 75 = 175$
 셋째 상자 = $100 + 75 = 175$

여기서 자주 오해하는 것이 있습니다. 2 는 "두 배 넓게" 가 아니라 "남는 공간에서 두 몫" 이라는 뜻입니다. 250 은 175 의 두 배가 아닙니다. 원래 폭 100 이 그대로 깔려 있기 때문입니다. 폭 자체를 2:1:1 로 만들고 싶다면 섹션 3 의 flex-basis: 0 이 필요합니다.

화면 첫 상자만 250px 로 넓고 나머지 둘은 175px 씩입니다

grow 2
grow 1
grow 1

한쪽만 늘리는 것도 자주 씁니다.

첫 상자 flex-grow: 1, 나머지 0

몫의 합 = 1 → 첫 상자가 300 을 통째로 가져갑니다
 첫 상자 = $100 + 300 = 400$, 나머지는 100 그대로

개념06 에서 만들 "로고는 왼쪽, 메뉴는 오른쪽" 네비게이션 바가 바로 이 원리입니다.

화면 첫 상자만 400px 로 늘어나고 나머지 둘은 100px 그대로입니다

```
grow 1  
grow 0  
grow 0
```

▫ 직접 해보기 1

.grow2 의 값을 flex-grow: 4 로 바꿔 보세요. 몫의 합이 6 이 되어 한 몫은 50 이 되고, 첫 상자는 300px, 나머지는 150px 이 됩니다.

flex-shrink

섹션 2

flex-shrink 는 grow 의 반대입니다. 기본값이 1 이라, 아무것도 안 써도 아이탬은 이미 줄어들 준비가 되어 있습니다.

개념02 섹션 4 에서 "150px 짜리 다섯 개가 120px 로 찌그러진다" 를 봤습니다. 그 범인이 flex-shrink 의 기본값 1 입니다.

이번에는 300px 짜리 셋을 600px 무대에 넣었습니다.

```
필요한 폭 = 300 × 3 = 900  
모자란 폭 = 900 - 600 = 300  
셋이 같은 비율로 양보 → 각각 100 씩 → 최종 200px
```

width: 300px 이라고 분명히 썼는데 화면에서는 200px 입니다. CSS 에서 width 가 지켜지지 않는 흔치 않은 경우입니다.

화면 상자 셋이 200px 씩으로 줄어 무대를 꽉 채웁니다

```
300px  
300px  
300px
```

가운데 상자만 양보를 거절하게 해 봅시다.

```
.shrink0 { flex-shrink: 0; }
```

양보하는 양은 "shrink 값 × 원래 폭" 에 비례해 나눕니다.

첫 상자 $1 \times 300 = 300$
 둘째 상자 $0 \times 300 = 0$ ← 안 줄어듭니다
 셋째 상자 $1 \times 300 = 300$
 합계 600

첫 상자가 줄일 양 $= 300 \times (300 \div 600) = 150 \rightarrow 300 - 150 = 150$
 셋째 상자도 마찬가지로 $\rightarrow 150$
 둘째 상자는 300 그대로
 $150 + 300 + 150 = 600$ ✓

화면 가운데 상자만 300px 로 넓고 양옆은 150px 씩으로 줄었습니다

```
shrink 1
shrink 0
shrink 1
```

셋 다 flex-shrink: 0 을 주면 어떻게 될까요. 아무도 양보하지 않으니 무대 밖으로 빠져나갑니다.

$250 \times 3 = 750 \rightarrow 600$ 짜리 무대에서 150px 이 넘칩니다

아래 무대에서 빨간 상자가 열린 파란 칸 오른쪽 밖으로 나와 있는 것이 보입니다. 진짜 웹페이지에서 이런 일이 생기면 화면 아래에 가로 스크롤바가 생깁니다. “왜 옆으로 스크롤이 되지?” 의 흔한 원인 중 하나입니다.

1 flex-shrink 를 원래대로 두어 줄어들게 한다

2 개념02 에서 배운 flex-wrap: wrap 으로 아랫줄에 내려보낸다

화면 세 번째 빨간 상자가 열린 파란 칸 오른쪽 밖으로 150px 빠져나와 있습니다

```
250px
250px
250px
```

⇒ 직접 해보기

2

.overflow-stage 에 flex-wrap: wrap 을 한 줄 더해 보세요. 세 번째 상자가 아랫줄로 내려가면서 빠져나온 부분이 사라집니다.

flex-basis

섹션 3

flex-basis 는 계산을 시작할 때의 크기입니다. 기본값은 auto 이고, auto 는 “width 를 보라” 는 뜻입니다.

지금까지 “원래 폭 100” 이라고 했던 그 값이 사실 flex-basis 였습니다.

```
flex-basis: auto → width 값을 쓴다 (width 도 없으면 내용 크기)
flex-basis: 200px → width 가 뭐든 200px 로 시작한다
flex-basis: 0 → 아예 0 에서 시작한다
```

아래 무대의 상자에는 width: 100px 과 flex-basis: 200px 이 둘 다 있습니다. 주축이 가로일 때는 flex-basis 가 이깁니다. 그래서 200px 이 됩니다.

```
200 × 3 = 600 → 무대를 정확히 채웁니다
```

화면 상자 셋이 200px 씩으로 무대를 정확히 채웁니다

basis 200
basis 200
basis 200

basis 가 auto 인지 0 인지에 따라 결과가 크게 달라집니다. 글자 길이가 서로 다른 상자 셋으로 비교해 봅니다.

[flex-grow: 1, flex-basis: auto]

```
먼저 각자 글자 길이만큼 자리를 잡습니다. 폭이 제각각입니다.
그다음 남는 공간만 똑같이 3 등분해서 더합니다.
→ 원래 길었던 상자가 계속 더 넓습니다.
→ 재어 보면 123 / 194 / 283 입니다. 셋을 더하면 600 입니다.
```

— flex-grow: 1, flex-basis: 0

```
셋 다 0 에서 시작합니다. 글자 길이를 아예 안 봅니다.
그래서 600 을 그냥 3 등분한 200px 씩이 됩니다.
→ 글자 길이와 상관없이 똑같은 폭입니다.
```

“카드 폭을 똑같이 맞추고 싶다” 면 basis 를 0 으로 두어야 합니다. 이것이 다음 섹션의 flex: 1 이 하는 일입니다.

화면 위 무대는 상자 폭이 123 / 194 / 283 으로 제각각이고,

아래 무대는 셋 다 정확히 200px 로 같습니다

flex-grow: 1 + flex-basis: auto → 폭이 제각각

짧다

조금 더 긴 글자

아주 아주 많이 긴 글자입니다

flex-grow: 1 + flex-basis: 0 → 셋 다 200px

짧다

조금 더 긴 글자

아주 아주 많이 긴 글자입니다

flex-basis 는 주축 방향의 크기를 정합니다. flex-direction: column 이면 폭이 아니라 높이가 됩니다. 여기서도 "가로"가 아니라 "주축"이라는 말이 그대로 쓰입니다.

⇒ 직접 해보기 3

.basis-zero-stage .word 의 flex-basis: 0 을 flex-basis: auto 로 바꿔 보세요. 아래 무대가 위 무대와 똑같아집니다.

flex 단축

섹션 4

실무에서는 셋을 따로 쓰지 않고 flex 한 줄로 씁니다. 그중에서도 flex: 1 이 압도적으로 많이 쓰입니다.

순서는 grow, shrink, basis 입니다.

```
flex: 1 1 0;
    | | ^ flex-basis
    | ^ flex-shrink
    ^ flex-grow
```

그런데 숫자 하나만 쓰는 경우가 대부분입니다. 숫자 하나만 쓰면 나머지가 이렇게 채워집니다.

```
flex: 1      →  flex-grow: 1; flex-shrink: 1; flex-basis: 0%;
```

↳ auto 가 아니라 0 입니다

이 마지막 줄이 핵심입니다. flex: 1 은 basis 를 0 으로 바꿔 버립니다. 그래서 flex: 1 을 붙인 아이тем들은 글자 길이나 width 와 상관없이 폭이 같아집니다.

말로 풀면 이렇습니다.

"내 원래 크기는 잊고, 남는 공간을 똑같이 나눠 갖자"

자주 쓰는 값 네 가지만 알면 충분합니다.

flex: 1	1 1 0%	남는 공간을 똑같이 나눠 갖기 (가장 많이 씁니다)
flex: 2	2 1 0%	남보다 두 몫
flex: auto	1 1 auto	내 내용 크기는 지키면서 남는 공간도 나눠 갖기
flex: none	0 0 auto	늘지도 줄지도 않기 (개념06 의 로고에 씁니다)

화면

상자 셋이 정확히 200px 씩 똑같이 나눠 가집니다

```
flex: 1
flex: 1
flex: 1
```

비율을 주면 섹션 1 과 달리 깔끔하게 나뉩니다. basis 가 0 이기 때문입니다.

```
flex: 2 / flex: 1 / flex: 1

몫의 합 = 4, 나눌 공간 = 600 (basis 가 0 이라 전부 남는 공간입니다)
한 몫 = 150
첫 상자 = 300, 나머지 = 150 씩
```

섹션 1 에서 같은 2 : 1 : 1 이 250 : 175 : 175 였던 것과 비교해 보세요. 이번에는 정확히 두 배입니다. basis 를 0 으로 밀어 버린 덕분입니다.

화면	첫 상자가 300px, 나머지 둘이 150px 씩입니다. 정확히 두 배입니다
----	--

flex: 2

flex: 1

flex: 1

하나만 고정하고 나머지를 늘리는 조합도 아주 많이 씁니다.

```
flex: none → 이 상자는 내 크기 그대로 (100px)
flex: 1 → 남은 것 다 가져가기

첫 상자 100 고정, 남는 500 을 뒤의 둘이 250 씩
```

화면	첫 상자는 100px 그대로, 뒤의 둘이 250px 씩으로 넓어집니다
----	--

none

flex: 1

flex: 1

▢ 직접 해보기 4

.flex-ratio2 를 flex: 3 으로 바꿔 보세요. 몫의 합이 5 가 되어 한 몫이 120 이 되고, 상자 폭이 360 / 120 / 120 이 됩니다.

order

섹션 5

order 는 HTML 을 고치지 않고 화면에 보이는 순서만 바꿉니다. 기본값은 0 이고, 숫자가 작을수록 앞에 옵니다.

아래 HTML 은 1번, 2번, 3번 순서 그대로입니다. CSS 만 이렇게 짰습니다.

```
.order-3 { order: 3; } ← HTML 1번 상자
.order-1 { order: 1; } ← HTML 2번 상자
.order-2 { order: 2; } ← HTML 3번 상자
```

브라우저는 order 값이 작은 것부터 늘어놓습니다. 그래서 화면에는 2번, 3번, 1번 순서로 보입니다.

음수도 됩니다. 기본값이 0 이므로 order: -1 을 준 아이템 하나만 아무것도 안 건드린 다른 아이템들보다 앞으로 나옵니다. 아래 두 번째 무대가 그 예입니다.

화면 왼쪽부터 "HTML 2번", "HTML 3번", "HTML 1번" 순서로 보입니다

HTML 1번
HTML 2번
HTML 3번

화면 왼쪽부터 "HTML 3번", "HTML 1번", "HTML 2번" 순서로 보입니다

HTML 1번
HTML 2번
HTML 3번

order 는 보이는 순서만 바꿉니다. 개념02 의 -reverse 와 같은 주의사항이 그대로 적용됩니다. 코드 순서대로 읽어 주는 프로그램에는 여전히 1번, 2번, 3번으로 읽힙니다. 순서 자체가 중요하다면 HTML 을 고치세요.

▢ 직접 해보기 5

두 번째 무대의 order-first 클래스를 가운데 상자(HTML 2번)로 옮겨 보세요. "HTML 2번", "HTML 1번", "HTML 3번" 순서가 됩니다.

이 섹션의 속성들은 붙이는 자리를 특히 많이 틀립니다. 전부 아이템에 쓰는 속성이라는 것을 기억하세요.

이런 실수를 합니다

flex-grow 를 컨테이너에 씀

이런 실수를 합니다

실수 1 — flex-grow 를 컨테이너에 씀

```
.grow-on-container { display: flex; flex-grow: 1; }
```

아이템은 하나도 안 늘어집니다. flex-grow 는 아이템이 자기 몫을 주장하는 속성입니다. 컨테이너에 쓰면 “이 컨테이너가 자기 부모 안에서 늘어나겠다” 는 뜻이 되는데, 부모가 flex 컨테이너가 아니면 그마저도 무시됩니다. 100px 100px

이렇게 쓰세요

```
.grow-on-container .tile { flex-grow: 1; } /* 아이템에 */
```

이런 실수를 합니다

flex: 1 이 **width** 를 덮어씀

이런 실수를 합니다

실수 2 — flex: 1 을 써 놓고 **width** 가 안 먹는다고 함

```
.width-lost-stage .tile { width: 300px; flex: 1; }
```

width: 300px 이 분명히 있는데 화면에서는 200px 씩입니다. flex: 1 안에 flex-basis: 0% 가 숨어 있어서 width 를 덮어쓰기 때문입니다. 폭을 지키고 싶다면 flex: none 이나 flex: 0 0 300px 을 쓰세요. width 300 width 300 width 300

이런 실수를 합니다

grow 를 백분율로 생각함

이런 실수를 합니다

실수 3 — flex-grow 값을 퍼센트라고 생각함

```
.percent-stage .tile { flex-grow: 50; } /* 50% 라는 뜻이 아닙니다 */
```

50 이든 1 이든 셋 다 같은 값이면 결과는 똑같습니다. flex-grow 는 절대적인 크기가 아니라 다른 아이템과 비교한 몫의 수이기 때문입니다. 아래 무대는 값이 50 인데도 결과가 flex-grow: 1 일 때와 같습니다. grow 50 grow 50 grow 50

이런 실수를 합니다

width 를 줬는데 작아진다

이런 실수를 합니다

실수 4 — width 를 줬는데 상자가 작아진다고 함

```
.card { width: 300px; } /* flex-shrink 기본값 1 이 줄입니다 */
```

아이템의 width 는 공간이 넉넉할 때만 지켜집니다. 모자라면 flex-shrink 의 기본값 1 때문에 알아서 줄어듭니다. 섹션 2 의 첫 무대가 그 예입니다. 폭을 꼭 지켜야 하면 flex-shrink: 0 을 주거나 flex-wrap: wrap 으로 아랫줄에 내려보내세요.

이런 실수를 합니다

order 로 순서를 바꾸고 HTML 을 안 고침

이런 실수를 합니다

실수 5 — order 로 내용 순서를 바꿈

```
.important { order: -1; } /* 중요한 글을 맨 앞으로 */
```

화면에서는 맨 앞으로 오지만 HTML 순서는 그대로입니다. 코드 순서대로 읽는 프로그램에는 여전히 원래 자리에서 읽히고, 키보드 탭 이동 순서도 화면과 어긋납니다. order 는 화면 폭에 따라 잠깐 자리를 바꾸는 용도이지 내용의 순서를 정하는 도구가 아닙니다.

정리

▢ 직접 해보기 6

정답

1) .grow2 를 이렇게 고칩니다.

```
.grow2 { flex-grow: 4; }
```

→ 몫의 합이 $4 + 1 + 1 = 6$, 한 몫은 $300 \div 6 = 50$ 입니다.

첫 상자 = $100 + 50 \times 4 = 300$, 나머지 = $100 + 50 = 150$.
화면에서 첫 상자가 250px 에서 300px 로 더 넓어집니다.

2) .overflow-stage 에 한 줄 더합니다.

```
.overflow-stage { flex-wrap: wrap; }
```

→ $250 \times 2 = 500$ 까지만 한 줄에 들어가고 세 번째 상자가 아랫줄로 내려갑니다.

무대 밖으로 빠져나온 부분이 사라집니다.
flex-shrink: 0 은 그대로인데도 넘침이 해결됩니다.

3) .basis-zero-stage .word 를 이렇게 고칩니다.

```
.basis-zero-stage .word { flex-grow: 1; flex-basis: auto; }
```

→ 아래 무대가 위 무대와 똑같아집니다. 글자가 긴 상자가 더 넓어집니다.

basis 하나가 결과를 완전히 바꾼다는 것을 확인하는 문제입니다.

4) .flex-ratio2 를 이렇게 고칩니다.

```
.flex-ratio2 { flex: 3; }
```

→ 몫의 합이 $3 + 1 + 1 = 5$, 한 몫은 $600 \div 5 = 120$ 입니다.

상자 폭이 $360 / 120 / 120$ 이 됩니다.
basis 가 0 이라 600 을 통째로 나눈다는 점을 다시 확인하세요.

5) order-first 클래스를 가운데 상자로 옮깁니다.

```
<div class="tile" id="orderMinusA">HTML 1번</div>  
<div class="tile order-first" id="orderMinusB">HTML 2번</div>  
<div class="tile" id="orderMinusC">HTML 3번</div>
```

→ 화면 순서가 "HTML 2번", "HTML 1번", "HTML 3번" 이 됩니다.

order 가 -1 인 것 하나만 앞으로 나오고,
나머지는 기본값 0 끼리 HTML 순서를 지킵니다.

무대: 안쪽 폭 600px

```
.stage {
  width: 600px;
  padding: 10px;
  background: #eef2f7;
  display: flex;
}
```

화면 열린 파란 칸 안에 상자들이 가로로 나란히 놓입니다

```
.tile {
  background: #2d6cdf;
  color: #fff;
  padding: 10px 0;
  width: 100px;
  box-sizing: border-box;
  text-align: center;
  font-size: 14px;
}
```

화면 폭 100px 짜리 진한 파란 상자가 됩니다

```
.stage-label {
  font-size: 13px;
  color: #555;
  margin: 14px 0 4px;
}
```

화면 각 무대 위에 작은 회색 글씨로 어떤 값인지 적힙니다

flex-grow

섹션 1

```
.grow-all .tile {
  flex-grow: 1;
}
```

화면 남은 300px 을 셋이 100px 씩 나눠 가져 상자가 200px 이 됩니다

```

.grow2 {
  flex-grow: 2;
}
.grow1 {
  flex-grow: 1;
}
.grow0 {
  flex-grow: 0;
}

```

flex-shrink

섹션 2

```

.shrink-stage .tile {
  width: 300px;
}

```

화면 300px 짜리 셋이 600px 무대에 억지로 들어가 200px 씩으로 줄어듭니다

```

.shrink0 {
  flex-shrink: 0;
}
.shrink2 {
  flex-shrink: 2;
}

.overflow-stage .tile {
  width: 250px;
  flex-shrink: 0;
  background: #b9432f;
}

```

화면 줄어들기를 막았더니 상자들이 무대 밖으로 150px 빠져나옵니다

flex-basis

섹션 3

```

.basis-stage .tile {
  flex-basis: 200px;
}

```

화면 width 100px 을 무시하고 200px 씩 자리를 잡습니다


```
.word {
  background: #2d6cdf;
  color: #fff;
  padding: 10px 8px;
  box-sizing: border-box;
  text-align: center;
  font-size: 14px;
}
```

화면 글자 길이에 따라 폭이 달라지는 파란 상자가 됩니다

```
.basis-auto-stage .word {
  flex-grow: 1;
  flex-basis: auto;
}
```

화면 글자가 긴 상자가 더 넓은 채로 남습니다 (폭이 제각각)

```
.basis-zero-stage .word {
  flex-grow: 1;
  flex-basis: 0;
}
```

화면 글자 길이와 상관없이 셋 다 정확히 200px 이 됩니다

flex 단축

섹션 4

```
.flex1-stage .tile {
  flex: 1;
}
```

화면 세 상자가 정확히 200px 씩 똑같이 나눠 가집니다

```
.flex-ratio2 {
  flex: 2;
}
.flex-ratio1 {
  flex: 1;
}
.flex-none-item {
  flex: none;
}
```

```
.order-stage .tile {
  width: 160px;
}
.order-first {
  order: -1;
}
```

화면 이 상자만 맨 앞으로 옵니다

```
.order-3 {
  order: 3;
}
.order-1 {
  order: 1;
}
.order-2 {
  order: 2;
}
```

자주 하는 실수

```
.bad-stage {
  background: #f7eeee;
}
.bad-stage .tile {
  background: #b9432f;
}
.grow-on-container {
  flex-grow: 1;
}
```

화면 컨테이너에 걸어 봐야 아이템은 그대로입니다

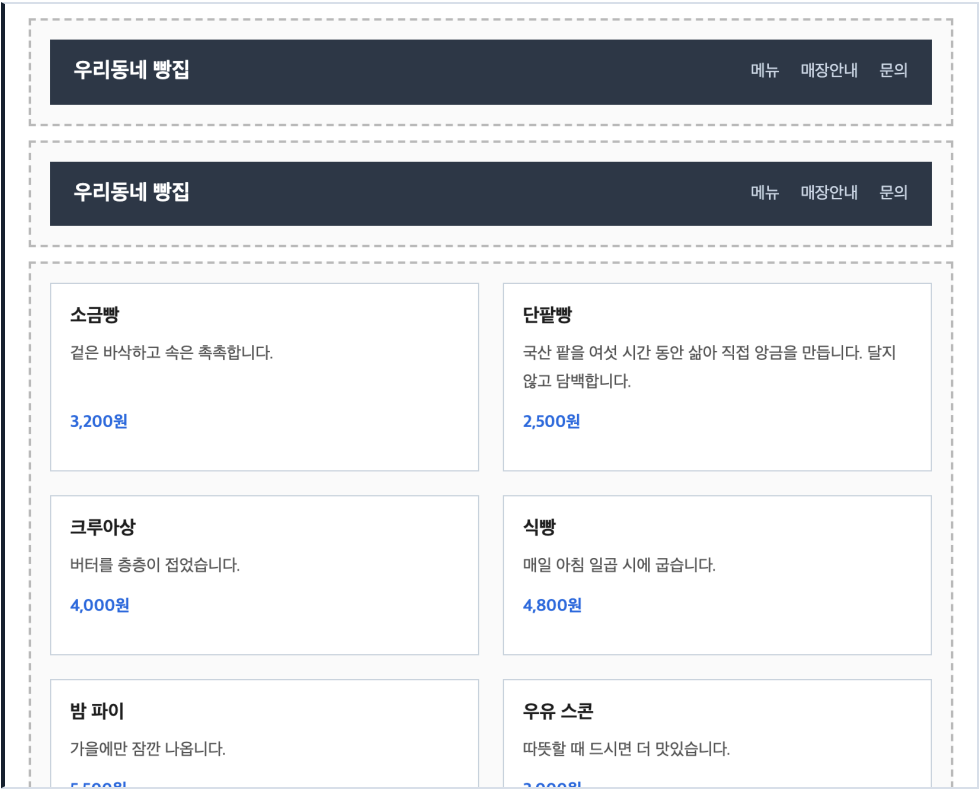
```
.width-lost-stage .tile {
  width: 300px;
  flex: 1;
}
```

화면 width 300px 을 썼는데 200px 씩으로 똑같이 나뉩니다

```
.percent-stage .tile {  
  flex-grow: 50;  
}
```

화면 50 이라고 썼지만 셋이 같은 값이라 결국 똑같이 나눠 가집니다

flex로 레이아웃 만들기



실행하면 나오는 화면 — flex로 레이아웃 만들기

12단원 · 개념 06 — flex 로 레이아웃 만들기

실행: 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.

개념01 부터 개념05 까지 배운 것을 이제 실제 화면 조각으로 조립합니다. 새로 배우는 속성은 하나도 없습니다. 조합만 익히면 됩니다.

만들 것은 네 가지입니다. 웹페이지 대부분이 이 네 조각의 반복입니다.

- ① 상단 내비게이션 바 로고는 왼쪽, 메뉴는 오른쪽
- ② 카드 목록 같은 높이로 나란히, 좁아지면 줄바꿈
- ③ 가운데 정렬 박스 글과 버튼을 화면 한가운데에
- ④ 본문 + 사이드바 2단 한쪽은 고정 폭, 한쪽은 남은 만큼

각 조각마다 “어떤 속성을 왜 골랐는지” 를 먼저 적어 두었습니다. 값을 외우지 말고 고르는 순서를 익히세요. 순서는 늘 이렇습니다.

1 무엇을 나란히 놓을지 정하고 그 부모에 display: flex

2 방향이 가로가 아니면 flex-direction

3 주축에서 어디에 둘지 justify-content

4 교차축에서 어디에 둘지 align-items

5 사이를 띄우려면 gap

6 크기를 나눠 가지려면 아이템에 flex

이 파일의 데모는 무대 폭을 따로 정하지 않았습니다. 자료 본문 폭인 824px 을 그대로 씁니다. 실제 페이지에 가깝게 보려는 것입니다.

새 속성은 없습니다. 개념01~05 의 조합만 나옵니다. 데모가 그려지는 칸의 안쪽 폭은 824px 입니다. 브라우저 창을 좁혔다 넓혔다 하면서 각 조각이 어떻게 반응하는지도 함께 보세요.

이 파일에 나오는 “왼쪽 좌표” 는 브라우저 창을 1280px 로 놓고 잰 값입니다. 본문이 가운데 정렬이라 창 폭이 다르면 숫자가 통째로 좌우로 밀립니다. 숫자 자체를 외우지 말고, 상자들 사이의 차이(간격·나뉜 몫)를 보세요. 그 차이는 창 폭과 상관없이 늘 같습니다.

상단 내비게이션 바

섹션 1

로고는 왼쪽 끝, 메뉴는 오른쪽 끝. 웹페이지에서 가장 많이 만드는 조각입니다.

구조는 두 덩어리입니다. 덩어리가 둘뿐이라는 점이 중요합니다.

```
<header class="topbar">      ← 컨테이너
  <p class="brand">           ← 아이템 1 (왼쪽)
  <nav class="menu">         ← 아이템 2 (오른쪽) 이면서 그 자체로 또 컨테이너
    <a> <a> <a>              ← nav 의 아이템들
  </header>
```

아이템이 둘이고 하나는 맨 앞, 하나는 맨 뒤에 놓고 싶습니다. 개념03 에서 “남는 공간을 사이에만 나눠 준다” 는 값을 배웠습니다.

```
.topbar {
  display: flex;
  justify-content: space-between; ← 둘을 양 끝으로
  align-items: center;           ← 글자 높이가 달라도 위아래 가운데
}
```

align-items: center 를 넣는 이유는 이렇습니다. 가게 이름은 18px, 메뉴는 14px 이라 글자 키가 다릅니다. 기본값 stretch 로 두면 둘 다 띠 높이만큼 늘어나 글자가 위쪽에 붙습니다. center 를 주면 크기가 달라도 가운데 선에 맞습니다.

nav 자신도 컨테이너로 만들어 링크 셋을 가로로 놓고 gap 으로 띄웠습니다. 컨테이너 안에 컨테이너가 들어가는 것이 flex 레이아웃의 기본 모양입니다.

재어 보면 가게 이름의 왼쪽 좌표가 248, 메뉴 덩어리의 왼쪽 좌표가 884 입니다. 띠의 안쪽은 248 에서 1032 까지이고 메뉴 폭이 148 이니, $1032 - 148 = 884$. 메뉴가 오른쪽 끝에 정확히 붙어 있다는 뜻입니다.

화면 짙은 남색 띠 왼쪽 끝에 가게 이름, 오른쪽 끝에 메뉴 세 개가 놓입니다

우리동네 빵집

메뉴
매장안내
문의

같은 결과를 만드는 다른 방법이 하나 더 있습니다. margin 을 auto 로 주는 것입니다.

```
.topbar2 { display: flex; align-items: center; }
.topbar2 .menu { margin-left: auto; }
```

flex 아이템의 margin 을 auto 로 주면 “남는 공간을 이쪽에 전부 몰아 준다” 는 뜻이 됩니다. 메뉴의 왼쪽에 남는 공간을 다 몰아 주니 메뉴가 오른쪽 끝까지 밀려납니다.

결과는 space-between 과 똑같습니다. 그런데 이 방법이 더 나을 때가 있습니다.

아이템이 셋 이상일 때
예를 들어 [로고] [검색창] [메뉴] 에서 메뉴만 오른쪽에 붙이고 싶다면
space-between 은 셋을 고르게 벌려 놓아서 원하는 그림이 안 나옵니다.
이때 메뉴에만 margin-left: auto 를 주면 딱 맞습니다.

개념04 에서 “주축에는 justify-self 가 없다” 고 했습니다. 그 자리를 대신하는 것이 이 margin: auto 입니다.

화면 위 데모와 똑같이 보입니다. 메뉴의 왼쪽 좌표도 같습니다

우리동네 빵집

메뉴
매장안내
문의

⇒ 직접 해보기

1

.topbar 의 justify-content: space-between 을 잠깐 지워 보세요. 메뉴가 가게 이름 바로 옆으로 붙습니다. 확인했으면 되돌리세요.

카드 목록

섹션 2

같은 높이로 나란히, 자리가 모자라면 아랫줄로. 상품 목록·게시글 목록에 늘 쓰는 모양입니다.

```
.card-list {
  display: flex;
```

flex-wrap: wrap; ← 좁아지면 아랫줄로 (개념02) gap: 20px; ← 가로세로 간격 한 번에 (개념03)

```
}
.card {
```

flex: 1 1 240px; ← 240 에서 시작해 남는 만큼 함께 넓어짐 (개념05)

```
}
```

flex: 1 1 240px 을 읽는 법입니다.

flex-grow 1 남는 공간을 똑같이 나눠 갖는다 flex-shrink 1 모자라면 함께 줄어든다 flex-basis 240px
계산은 240 에서 시작한다

한 줄에 몇 개가 들어가는지는 basis 로 정해집니다.

카드 3 개 → $240 \times 3 + 20 \times 2 = 760 \leq 824$ 들어갑니다 카드 4 개 → $240 \times 4 + 20 \times 3 = 1020 > 824$ 안 들어갑니다

그래서 한 줄에 세 개씩 놓이고, 남는 폭을 셋이 나눠 가져 카드 폭이 261px 이 됩니다. ($(824 - 40) \div 3$)

높이는 어떨까요. 카드마다 글 길이가 다른데 화면에서는 키가 똑같습니다. 아무것도 안 했는데 그렇습니다. align-items 의 기본값이 stretch 이기 때문입니다(개념04). “카드 높이 맞추기” 는 flex 에서는 그냥 공짜입니다.

재어 보면 첫 줄 카드 셋은 모두 182px, 둘째 줄 카드 셋은 모두 135px 입니다. 키를 맞추는 범위는 “한 줄 안” 입니다. 줄이 다르면 높이가 달라도 됩니다.

카드 안쪽도 보세요. 가격 줄이 카드 맨 아래에 딱 붙어 있습니다.

```
.card      { display: flex; flex-direction: column; }  
.card-price { margin-top: auto; }
```

카드를 세로 컨테이너로 만들고, 가격 줄의 위쪽 margin 을 auto 로 줬습니다. 섹션 1 에서 본 것과 같은 방법인데 방향만 세로입니다. 남은 세로 공간이 전부 가격 줄 위로 몰려서 가격이 바닥에 붙습니다.

화면 카드가 한 줄에 세 개씩(261px) 두 줄로 놓입니다.

글 길이가 달라도 첫 줄은 모두 182px, 둘째 줄은 모두 135px 로 키가 같고, 가격 줄은 모든 카드에서 바닥에 붙어 있습니다

소금빵
겉은 바삭하고 속은 촉촉합니다.
3,200원

단팥빵

국산 팥을 여섯 시간 동안 삶아 직접 앙금을 만듭니다. 달지 않고 담백합니다.
2,500원

크루아상
버터를 층층이 접었습니다.
4,000원

식빵
매일 아침 일곱 시에 굽습니다.
4,800원

밤 파이
가을에만 잠깐 나옵니다.
5,500원

우유 스콘
따뜻할 때 드시면 더 맛있습니다.
3,000원

브라우저 창을 좁혀 보세요. 카드가 두 개씩, 한 개씩으로 저절로 접힙니다. flex-wrap: wrap 과 flex-basis 만으로 화면 폭에 반응하는 목록이 됩니다. 13단원에서 이 이야기를 이어 갑니다.

▣ 직접 해보기 2

.card 의 flex 를 flex: 1 1 380px 으로 바꿔 보세요. $380 \times 3 + 40 = 1180$ 이라 셋은 못 들어가고 한 줄에 두 개씩 놓입니다.

가운데 정렬 박스

섹션 3

페이지 맨 위에 크게 들어가는 안내 영역입니다. 정중앙 관용구 두 줄이면 끝납니다.

```
.hero {
  display: flex;
```

flex-direction: column; ← 글·글·버튼을 세로로 쌓기 justify-content: center; ← 주축(세로) 가운데
align-items: center; ← 교차축(가로) 가운데

```
  height: 200px;
}
```

여기서 개념04 섹션 4 의 함정을 실제로 만납니다. 방향이 column 이므로

justify-content 가 위아래를 담당하고 align-items 가 좌우를 담당합니다

방향을 row 로 바꾸면 둘의 역할이 그대로 맞바뀝니다. 다만 둘 다 center 이므로 결과는 똑같이 정중앙입니다. 이 조합만은 방향을 신경 쓰지 않아도 되는 이유가 그것입니다.

align-items: center 를 뺐다면 어떻게 될까요. 기본값 stretch 가 살아나서 글과 버튼이 가로로 822px 까지 늘어납니다. (회색 칸의 겉 폭은 824px 이지만 테두리 1px 씩을 빼면 안쪽은 822px 입니다) 버튼이 화면 폭만큼 길어지는 그 현상의 원인이 이것입니다.

화면 회색 칸 한가운데에 제목 · 설명 · 파란 버튼이 세로로 모여 있습니다

```
오늘 구운 빵
매일 아침 일곱 시, 그날 팔 만큼만 굽습니다.
메뉴 보러 가기
```

버튼처럼 보이는 것은 <p> 에 배경색과 안쪽 여백을 준 것입니다. 진짜 <button> 은 05단원에서 배웠으니 그것을 써도 됩니다. 여기서는 flex 로 가운데 두는 것이 주제이니 모양만 흉내 냈습니다.

▣ 직접 해보기 3

.hero 에서 align-items: center 를 잠깐 지워 보세요. 제목·설명·버튼이 전부 가로로 쭉 늘어납니다. 확인했으면 되돌리세요.

사이드바는 폭 고정, 본문은 남는 만큼. 11단원에서 float 는 배치 도구가 아니라 글 흘리기 도구라고 배웠습니다. 그럼 이 2단 배치는 무엇으로 하느냐 — flex 로 하면 이렇습니다.

```
.two-col { display: flex; align-items: flex-start; gap: 20px; }
```

.main-col { flex: 1; } ← 남는 폭을 전부 .side-col { flex: 0 0 220px; } ← 늘지도 줄지도 않는 220px flex: 0 0 220px 을 읽는 법입니다.

flex-grow 0 남아도 안 늘어난다 flex-shrink 0 모자라도 안 줄어든다 flex-basis 220px 항상 220px 계산은 이렇습니다.

전체 824 - 사이드바 220 - 간격 20 = 584 → 본문이 584px

본문은 flex: 1 이므로 화면이 넓어지든 좁아지든 알아서 남는 만큼을 차지합니다. 폭을 직접 적을 필요가 없습니다. 이것이 float 로 2단을 만들 때와 가장 다른 점입니다.

align-items: flex-start 를 준 이유가 있습니다. 기본값 stretch 로 두면 사이드바가 본문 높이만큼 늘어나 빈 상자가 길쭉해집니다. 내용만큼만 차지하게 하려고 위쪽 정렬로 바꿨습니다. 반대로 두 칸의 배경색을 끝까지 맞추고 싶다면 stretch 가 편합니다.

사이드바를 왼쪽에 두고 싶다면 HTML 순서를 바꾸는 것이 가장 좋습니다. 급하면 개념05 의 order 로도 되지만, 읽는 순서가 어긋난다는 점을 기억하세요.

화면 왼쪽 본문이 584px, 오른쪽 사이드바가 220px 로 나란히 놓입니다.

사이드바 높이는 내용만큼(113px)만 차지합니다

오늘의 이야기

빵집 뒤편에는 작은 마당이 있습니다. 아침에 반죽을 올려 두면 햇빛을 받아 천천히 부풀어 오릅니다. 그 시간이 빵 맛의 절반을 만든다고 생각합니다. 그래서 아무리 바빠도 이 시간을 줄이지 않습니다.

영업 시간

평일 07:00 ~ 20:00 주말 08:00 ~ 18:00

▸ 직접 해보기

4

.two-col 의 align-items: flex-start 를 잠깐 지워 보세요. 기본값 stretch 가 살아나 사이드바가 본문과 같은 높이로 길어집니다.

네 조각을 한 페이지로

섹션 5

실제 페이지는 flex 컨테이너 여러 개가 겹겹이 들어 있는 모양입니다. 지금까지 만든 조각을 그대로 포개 봅니다.

구조를 그림으로 보면 이렇습니다. 컨테이너가 네 개 들어 있습니다.

```
<div class="mini-page">
  <header class="topbar">          ← 컨테이너 1 (가로, 양 끝)  섹션1 것 그대로
  <div class="mini-body">          ← 컨테이너 2 (가로, 본문 + 사이드바)
    <div class="mini-main">
      <section class="hero">        ← 컨테이너 3 (세로, 정중앙)  섹션3 것 그대로
      <div class="mini-cards">      ← 컨테이너 4 (가로, 줄바꿈)
    <aside class="mini-side">
```

.topbar 와 .hero 는 앞 섹션에서 만든 클래스를 그대로 다시 쓴 것입니다. 새로 쓴 것은 mini- 로 시작하는 클래스들뿐입니다.

중요한 것은 각 컨테이너가 자기 자식만 신경 쓴다는 점입니다. 개념01 섹션 3 에서 “flex 는 한 단계만 내려간다” 고 한 것이 여기서 편해집니다. 바깥 컨테이너를 고쳐도 안쪽 배치는 그대로 있습니다.

폭 계산은 이렇습니다.

바깥 안쪽 폭 = $824 - \text{테두리 } 2 - \text{안쪽 여백 } 32 = 790$
 사이드바 $180 + \text{간격 } 16$ 을 빼면 본문은 594
 카드는 flex: 1 1 150px 이므로 594 안에 세 개씩
 $(150 \times 3 + 12 \times 2 = 474 \leq 594, \text{ 네 개면 } 150 \times 4 + 12 \times 3 = 636 > 594)$
 카드가 여섯 개이니 세 개씩 두 줄이 되고, 한 카드의 폭은
 $(594 - 24) \div 3 = 190\text{px}$ 입니다

화면 위에 남색 띠, 그 아래 왼쪽에 안내 영역과 카드 여섯 개(190px 씩 두 줄),

오른쪽에 폭 180px 사이드바가 있는 작은 페이지가 보입니다

우리동네 빵집

메뉴
문의

오늘 구운 빵
그날 팔 만큼만 굽습니다.

소금빵 3,200원
단팥빵 2,500원
식빵 4,800원
스콘 3,000원
밤 파이 5,500원
크루아상 4,000원

영업 시간
평일 07:00 ~ 20:00

새로 배운 것은 하나도 없습니다. `display: flex · flex-direction · justify-content · align-items · flex-wrap · gap · flex` 일곱 개로 페이지 한 장이 만들어집니다.

▹ 직접 해보기 5

.mini-side 의 `flex: 0 0 180px` 을 `flex: 0 0 340px` 으로 바꿔 보세요. 사이드바가 넓어진 만큼 본문이 좁아지고, 카드가 한 줄에 세 개에서 두 개로 줄어듭니다.

자주 하는 실수

섹션 6

레이아웃을 조립할 때는 “어느 컨테이너에 쓸 것인가” 를 틀리는 실수가 가장 많습니다.

이런 실수를 합니다

메뉴를 오른쪽으로 보내려고 `nav` 에 `justify-content` 를 씬

이런 실수를 합니다

실수 1 — 메뉴를 오른쪽에 붙이려고 `nav` 에 `justify-content` 를 씬

```
.menu { justify-content: flex-end; } /* nav 안쪽만 움직입니다 */
```

메뉴 덩어리는 꿈쩍도 하지 않습니다. `.menu` 는 링크 세 개의 컨테이너이지, 자기 자신을 옮기는 주체가 아닙니다. `nav` 안에는 남는 공간이 없으니 링크들도 그대로입니다. 움직여야 할 것은 `nav` 의 부모인 `header` 입니다. 우리동네 빵집 메뉴 문의

이렇게 쓰세요

```
.bad-bar { justify-content: space-between; } /* 부모에 */
```

이런 실수를 합니다

`flex-wrap` 을 빠뜨림

이런 실수를 합니다

실수 2 — 카드 목록에 flex-wrap 을 안 씀

```
.bad-cards { display: flex; gap: 20px; } /* flex-wrap 이 없습니다 */
```

카드가 아랫줄로 내려가지 않고 한 줄에 전부 옥여넣어집니다. 여섯 개가 들어가느라 카드 폭이 121px 까지 줄고 글자가 잘게 접힙니다. 카드가 이상하게 좁아졌다면 flex-wrap: wrap 을 빠뜨린 것입니다. 소금빵 3,200원 단팥빵 2,500원 크루아상 4,000원 식빵 4,800원 밤 파이 5,500원 우유 스콘 3,000원

이런 실수를 합니다

카드 높이를 height 로 맞춤

이런 실수를 합니다

실수 3 — 카드 높이를 맞추려고 height 를 직접 씀

```
.card { height: 120px; } /* 글이 길면 넘칩니다 */
```

지금만 맞아 보여도 글이 한 줄만 길어지면 글자가 카드 밖으로 빠져나옵니다. 키를 맞추는 일은 이미 flex 가 하고 있습니다. align-items 의 기본값 stretch 가 한 줄 안의 카드를 자동으로 가장 큰 카드에 맞춥니다. height 는 지우세요.

이런 실수를 합니다

사이드바가 줄어듦

이런 실수를 합니다

실수 4 — 사이드바 폭이 안 지켜짐

```
.side-col { flex: 0 1 220px; } /* shrink 가 1 입니다 */
```

220px 이라고 썼는데 화면에서는 164px 입니다. 가운데 숫자가 flex-shrink 이고, 1 이면 모자랄 때 줄어듭니다. 아래에서는 본문이 flex: 1 1 900px 으로 넓은 자리를 요구해 전체가 824px 을 넘겼고, 그래서 사이드바까지 폭을 양보했습니다. 폭을 꼭 지켜야 하면 가운데를 0 으로 두세요. flex: 0 0 220px 입니다. 본문 본문이 넓은 자리를 요구하면 전체가 컨테이너를 넘칩니다. 그러면 shrink 가 1 인 아이템부터 폭을 양보하기 시작합니다. 사이드바 220px 이라고 썼습니다

이런 실수를 합니다

gap 대신 margin 을 씀

이런 실수를 합니다

실수 5 — 카드 사이를 margin 으로 띄움

```
.card { margin-right: 20px; } /* 줄 끝에도 붙습니다 */
```

줄의 마지막 카드 오른쪽에도 20px 이 붙어서 오른쪽 끝이 다른 요소와 안 맞습니다. 게다가 아랫줄과의 세로 간격은 따로 margin-bottom 으로 또 줘야 하고, 맨 아랫줄에는 그것도 지워야 합니다. gap 한 줄이면 가로세로가 한 번에 해결되고 예외도 없습니다.

정리

▸ 직접 해보기 6

정답

1) .topbar 에서 한 줄을 지웁니다.

```
.topbar {  
  display: flex;  
  align-items: center;  
  background: #2d3a4a;  
  padding: 12px 20px;  
}
```

→ 기본값 flex-start 로 돌아가 메뉴가 가게 이름 바로 옆에 붙습니다.

메뉴의 왼쪽 좌표가 884 에서 362 로 옮겨 갑니다. (248 + 가게 이름 폭 114)
확인했으면 justify-content: space-between 을 다시 넣으세요.

2) .card 의 flex 를 바꿉니다.

```
.card { flex: 1 1 380px; }
```

→ $380 \times 3 + 20 \times 2 = 1180$ 이라 한 줄에 셋은 못 들어갑니다.

한 줄에 두 개씩 세 줄이 되고, 카드 폭은 $(824 - 20) \div 2 = 402px$ 이 됩니다.

3) .hero 에서 한 줄을 지웁니다.

```
.hero {
  display: flex;
  flex-direction: column;
  justify-content: center;
  height: 200px;
}
```

→ 기본값 stretch 가 살아나 제목·설명·버튼이 가로로 822px 까지 늘어납니다.

파란 버튼이 화면 폭만큼 길어지는 것이 가장 눈에 띕니다.
확인했으면 align-items: center 를 다시 넣으세요.

4) .two-col 에서 한 줄을 지웁니다.

```
.two-col { display: flex; gap: 20px; }
```

→ 사이드바 높이가 내용만큼(113px)이 아니라 본문과 같은 높이로 늘어납니다.

배경색이 아래까지 채워지는 것으로 확인할 수 있습니다.
두 칸의 배경을 맞추고 싶을 때는 오히려 이쪽이 편합니다.

5) .mini-side 의 flex 를 바꿉니다.

```
.mini-side { flex: 0 0 340px; }
```

→ 사이드바가 340px 이 되면서 본문이 594 에서 434 로 좁아집니다.

카드 세 개는 $150 \times 3 + 12 \times 2 = 474$ 라서 434 안에 못 들어갑니다.
그래서 한 줄에 두 개씩 세 줄이 되고, 카드 폭은
 $(434 - 12) \div 2 = 211\text{px}$ 이 됩니다.

① 상단 내비게이션 바

```
.topbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  background: #2d3a4a;
  padding: 12px 20px;
}
```

화면 짙은 남색 띠가 생기고, 왼쪽 끝에 가게 이름 · 오른쪽 끝에 메뉴가 놓입니다

```
.brand {
  margin: 0;
  color: #fff;
  font-size: 18px;
  font-weight: bold;
}
.menu {
  display: flex;
  gap: 18px;
}
```

화면 메뉴 글자 세 개가 18px 간격으로 가로로 놓입니다

```
.menu a {
  color: #cfd9e6;
  font-size: 14px;
  text-decoration: none;
}
.menu a:hover {
  color: #fff;
}
```

화면 메뉴에 마우스를 올리면 글자가 흰색으로 밝아집니다

```
.topbar2 {
  display: flex;
  align-items: center;
  background: #2d3a4a;
  padding: 12px 20px;
}
```

화면 위와 똑같이 보이지만 space-between 대신 margin 으로 밀어낸 것입니다

```
.topbar2 .menu {
  margin-left: auto;
}
```

화면 남은 공간을 메뉴 왼쪽에 전부 몰아 주어 메뉴가 오른쪽 끝으로 갑니다

② 카드 목록

```
.card-list {
  display: flex;
  flex-wrap: wrap;
  gap: 20px;
}
```

화면 카드가 한 줄에 세 개씩 놓이고 가로세로 20px 씩 벌어집니다

```
.card {
  flex: 1 1 240px;
  background: #fff;
  border: 1px solid #ccd5e0;
  padding: 14px 16px;
  box-sizing: border-box;
  display: flex;
  flex-direction: column;
}
```

화면 흰 카드가 되고, 카드 안은 세로 방향 컨테이너가 됩니다

```
.card-title {
  margin: 0 0 6px;
  font-size: 16px;
  font-weight: bold;
}
.card-text {
  margin: 0;
  font-size: 14px;
  color: #555;
}
.card-price {
  margin-top: auto;
  padding-top: 10px;
  font-size: 14px;
  color: #2d6cdf;
  font-weight: bold;
}
```

화면 글이 짧은 길든 가격 줄이 카드 맨 아래에 딱 붙습니다

③ 가운데 정렬 박스

```
.hero {  
  display: flex;  
  flex-direction: column;  
  justify-content: center;  
  align-items: center;  
  height: 200px;  
  background: #eef2f7;  
  border: 1px solid #c9d6ea;  
}
```

화면 높이 200px 칸 한가운데에 글 두 줄과 버튼이 세로로 모입니다

```
.hero-title {  
  margin: 0 0 6px;  
  font-size: 22px;  
  font-weight: bold;  
}  
.hero-text {  
  margin: 0 0 14px;  
  font-size: 14px;  
  color: #555;  
}  
.hero-button {  
  background: #2d6cdf;  
  color: #fff;  
  padding: 8px 22px;  
  font-size: 14px;  
  border: none;  
}
```

④ 본문 + 사이드바 2단

```
.two-col {  
  display: flex;  
  align-items: flex-start;  
  gap: 20px;  
}
```

화면 왼쪽 넓은 본문과 오른쪽 좁은 사이드바가 20px 간격으로 나란히 놓입니다

```
.main-col {
  flex: 1;
  background: #fff;
  border: 1px solid #ccd5e0;
  padding: 14px 16px;
  box-sizing: border-box;
}
```

화면 남은 폭을 전부 차지해 584px 이 됩니다

```
.side-col {
  flex: 0 0 220px;
  background: #f4f7fb;
  border: 1px solid #ccd5e0;
  padding: 14px 16px;
  box-sizing: border-box;
}
```

화면 폭 220px 에서 늘지도 줄지도 않습니다

```
.col-title {
  margin: 0 0 8px;
  font-size: 16px;
  font-weight: bold;
}
.col-text {
  margin: 0;
  font-size: 14px;
  color: #555;
}
```

⑤ 네 조각을 한 페이지로

```
.mini-page {
  border: 1px solid #ccd5e0;
}
.mini-body {
  display: flex;
  align-items: flex-start;
  gap: 16px;
  padding: 16px;
}
```

화면 내비게이션 바 아래에 본문과 사이드바가 나란히 놓입니다

```

.mini-main {
  flex: 1;
}
.mini-side {
  flex: 0 0 180px;
  background: #f4f7fb;
  border: 1px solid #ccd5e0;
  padding: 10px 12px;
  box-sizing: border-box;
  font-size: 13px;
}
.mini-cards {
  display: flex;
  flex-wrap: wrap;
  gap: 12px;
  margin-top: 12px;
}
.mini-card {
  flex: 1 1 150px;
  background: #fff;
  border: 1px solid #ccd5e0;
  padding: 10px 12px;
  box-sizing: border-box;
  font-size: 13px;
}

```

자주 하는 실수

섹션 6

```

.bad-bar {
  display: flex;
  background: #6d3a35;
  padding: 12px 20px;
}

```

화면 메뉴가 오른쪽으로 가지 않고 가게 이름 바로 옆에 붙어 있습니다

```

.bad-bar .menu {
  justify-content: flex-end;
}
.bad-cards {
  display: flex;
  gap: 20px;
}

```

화면 flex-wrap 이 없어서 카드 여섯 개가 한 줄에 찌그러져 들어갑니다

```

.bad-cards .card {
  background: #fdf4f3;
  border-color: #e0b4b4;
}
.bad-side {
  display: flex;
  align-items: flex-start;
  gap: 20px;
}
.bad-side .main-col {
  flex: 1 1 900px;
}

```

화면 본문이 900px 을 요구해서 자리가 모자라집니다

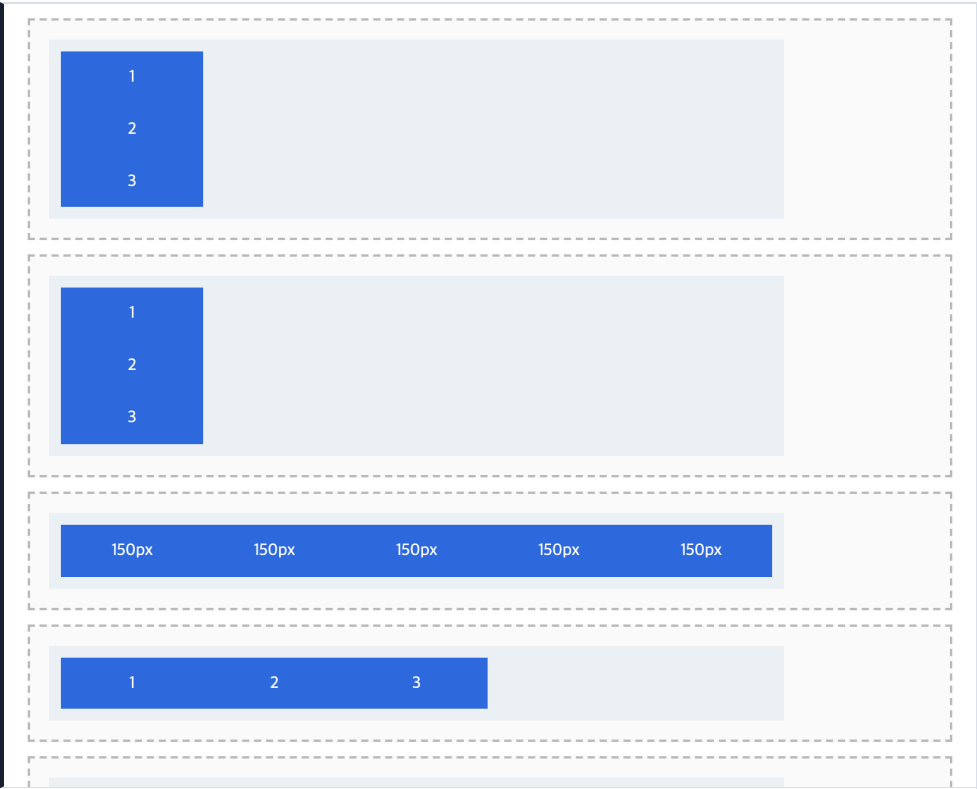
```

.bad-side .side-col {
  flex: 0 1 220px;
  background: #fdf4f3;
  border-color: #e0b4b4;
}

```

화면 사이드바가 220px 을 못 지키고 164px 까지 줄어듭니다

연습문제



실행하면 나오는 화면 — 연습문제

이 파일의 <style> 안에 문제와 빈 자리가 있습니다. VS Code 로 열어서 위에서부터 읽으며 채우세요. 한 문제를 채울 때마다 저장하고 브라우저에서 새로고침(F5) 해서 아래 상자를 확인하세요. 1~10 은 기본, 11~12 는 응용, 13~14 는 도전, 15 는 확인 문제입니다. 15번만 <style> 안의 코드를 옮기는 문제이고 나머지는 채우기만 하면 됩니다.

문제 1

1

2

3

문제 2

1	
2	
3	

문제 3

150px	
150px	
150px	
150px	
150px	

문제 4

1	
2	
3	

문제 5

왼쪽	
오른쪽	

문제 6

1	
2	
3	

문제 7

1	
2	

문제 8

정중앙

문제 9

늘어남
그대로
그대로

문제 10

짧다
조금 더 긴 글자
아주 아주 많이 긴 글자

문제 11

행복 서점

새 책
중고
문의

문제 12

그대로
아래로
그대로

문제 13

이번 주 새 책

목요일마다 새 책이 들어옵니다. 서가 맨 앞줄에 일주일 동안 놓아 두니
지나는 길에 한 번 훑어보세요. 표지가 예쁜 책은 금방 나갑니다.

여는 시간
평일 10:00 ~ 21:00

문제 14

소설
가볍게 읽기 좋습니다.
12,000원

역사

사진 자료가 많아 두껍습니다. 처음 읽는 분께는 뒤쪽 연표부터 보시길 권합니다.

21,000원

요리
사진이 큼직합니다.
17,500원

그림책
아이와 같이 보세요.
9,800원

문제 15

1

2

3

12단원 연습문제 - Flexbox

푸는 방법

- 1) 아래 "문제 N" 자리의 TODO 를 지우고 CSS 를 씁니다.
- 2) 저장(Ctrl+S) 하고 브라우저에서 새로고침(F5) 합니다.
- 3) 각 문제의 '기대 화면' 과 눈으로 맞춰 봅니다.

1~10 은 기본, 11~12 는 응용, 13~14 는 도전, 15 는 확인 문제입니다.
HTML(body) 은 고치지 않아도 풀 수 있습니다. 15번만 예외입니다.

아래 네 줄은 문제를 담아 두는 틀입니다. 고치지 마세요

```
.stage {  
  width: 600px;  
  padding: 10px;  
  background: #eef2f7;  
}  
.tile {  
  background: #2d6cdf;  
  color: #fff;  
  padding: 10px 0;  
  width: 120px;  
  box-sizing: border-box;  
  text-align: center;  
  font-size: 14px;  
}  
.tile-wide {  
  width: 150px;  
}  
.tile-big {  
  width: 200px;  
}
```

문제 1 (개념01 섹션2)

파란 상자 세 개를 가로로 나란히 놓으세요. 컨테이너에 한 줄만 쓰면 됩니다.

기대 화면: 상자 셋이 왼쪽부터 딱 붙어 한 줄로 놓입니다.

셋이 무대 왼쪽 안쪽 면에 딱 붙고 사이에 틈이 전혀 없습니다.

상자 하나가 120px 이므로 셋을 합친 360px 자리까지 찹니다.

이렇게 되면 틀린 것: 상자가 그대로 위에서 아래로 쌓여 있으면

display: flex 를 안 썼거나 상자(.tile) 쪽에 쓴 것입니다.

```
.p1-stage {
```

여기에 코드를 쓰세요

```
}
```

문제 2 (개념02 섹션3)

상자를 오른쪽 끝부터 채워서 화면에 3 2 1 순서로 보이게 하세요.

HTML 은 1, 2, 3 순서 그대로 두고 CSS 두 줄로 해결합니다.

기대 화면: 오른쪽 끝에 "1", 그 왼쪽에 "2", 그 왼쪽에 "3" 이 보입니다.

"1" 의 오른쪽 끝이 무대 오른쪽 안쪽 면에 딱 붙습니다.

이렇게 되면 틀린 것: 왼쪽부터 1 2 3 이면 -reverse 를 안 붙인 것입니다.

```
.p2-stage {
```

여기에 코드를 쓰세요

```
}
```

문제 3 (개념02 섹션5)

폭 150px 짜리 상자 다섯 개가 찌그러지지 않게 하세요.
지금은 한 줄에 억지로 들어가느라 120px 로 줄어 있습니다.

기대 화면: 윗줄에 네 개, 아랫줄에 한 개가 놓이고 상자 폭이 150px
그대로입니다.

이렇게 되면 틀린 것: 다섯 개가 계속 한 줄에 있으면 줄바꿈 속성을 안 준 것입니다.

```
.p3-stage {  
  display: flex;
```

여기에 코드를 쓰세요

```
}
```

문제 4 (개념03 섹션2)

상자 세 개를 무대의 가로 가운데로 모으세요.

기대 화면: 상자 뭉치가 가운데에 오고 좌우에 120px 씩 빈 칸이 생깁니다.
(무대 600px - 상자 360px = 240px 이 좌우로 반씩 나뉩니다)

이렇게 되면 틀린 것: 상자가 왼쪽에 붙어 있으면 값 이름이 틀렸거나

아이템 쪽에 쓴 것입니다.

```
.p4-stage {  
  display: flex;
```

여기에 코드를 쓰세요

}

문제 5 (개념03 섹션2)

상자 두 개를 무대의 양 끝에 하나씩 붙이세요.

기대 화면: 왼쪽 끝에 "왼쪽", 오른쪽 끝에 "오른쪽" 이 붙고
두 상자 사이가 360px 벌어집니다.

이렇게 되면 틀린 것: 가운데에 모여 있으면 center 를 쓴 것입니다.

```
.p5-stage {
  display: flex;
```

여기에 코드를 쓰세요

}

문제 6 (개념03 섹션4)

상자 사이를 정확히 20px 씩 띄우세요. margin 은 쓰지 마세요.

기대 화면: 상자 셋이 20px 씩 벌어지고 전체는 왼쪽에 붙어 있습니다.
상자와 상자 사이가 정확히 20px 씩 벌어집니다.

이렇게 되면 틀린 것: 마지막 상자 오른쪽에도 여백이 생겼다면 margin 을 쓴 것입니다.

```
.p6-stage {
  display: flex;
```

여기에 코드를 쓰세요

```
}
```

문제 7 (개념04 섹션2)

높이 140px 인 무대에서 상자를 위아래 가운데에 두세요.

기대 화면: 상자가 무대 한가운데 높이에 놓고 상자 키는 44px 입니다.

이렇게 되면 틀린 것: 상자 키가 140px 로 길어져 있으면 아직 기본값 stretch 입니다.

```
.p7-stage {  
  display: flex;  
  height: 140px;
```

여기에 코드를 쓰세요

```
}
```

문제 8 (개념04 섹션5)

높이 140px 인 무대에서 상자 하나를 가로세로 정중앙에 두세요. 두 줄이 필요합니다.

기대 화면: 폭 200px 상자가 무대 한가운데에 놓입니다.

상자 왼쪽에 남는 자리와 오른쪽에 남는 자리가 200px 로 같습니다.

이렇게 되면 틀린 것: 좌우만 맞고 위에 붙어 있으면 교차축 쪽을 빠뜨린 것입니다.

```
.p8-stage {  
  display: flex;  
  height: 140px;
```

여기에 코드를 쓰세요

```
}
```

문제 9 (개념05 섹션1)

첫 번째 상자만 남는 공간을 전부 가져가게 하세요. 나머지 둘은 120px 그대로입니다.

기대 화면: 첫 상자가 360px 로 넓어지고 나머지 둘은 120px 입니다.

이렇게 되면 틀린 것: 셋 다 넓어졌다면 .tile 전체에 쓴 것입니다.

첫 상자에만 쓰세요.

```
.p9-stage {
  display: flex;
}
.p9-first {
```

여기에 코드를 쓰세요

```
}
```

문제 10 (개념05 섹션4)

글자 길이가 제각각인 상자 셋을 정확히 같은 폭으로 만드세요.
단축 속성 한 줄이면 됩니다.

기대 화면: 상자 셋이 모두 200px 이 되어 무대를 꽉 채웁니다.

이렇게 되면 틀린 것: 글자가 긴 상자가 더 넓다면 flex-grow 만 쓴 것입니다.

그때는 flex-basis 가 auto 라서 원래 폭이 남아 있습니다.

```
.p10-stage {
  display: flex;
}
.p10-stage .tile {
```

여기에 코드를 쓰세요

```
}
```

문제 11 [응용] (개념06 섹션1)

내비게이션 바를 만드세요. 가게 이름은 왼쪽 끝, 메뉴는 오른쪽 끝,
글자 크기가 다른 둘을 위아래 가운데로 맞춥니다. 세 줄이 필요합니다.

기대 화면: 탐색 띠 왼쪽 끝에 "행복 서점", 오른쪽 끝에 메뉴 세 개가 놓이고
두 덩어리의 글자 가운데 선이 맞습니다.

메뉴 덩어리의 오른쪽 끝이 탐색 띠 안쪽 오른쪽 면에 딱 붙습니다.

이렇게 되면 틀린 것: 메뉴가 이름 바로 옆에 붙어 있으면 주축 정렬을 안 준
것이고,

메뉴 글자가 띠 위쪽에 붙어 있으면 교차축 정렬을 안 준 것입니다.

```
.p11-bar {
  width: 600px;
  padding: 12px 20px;
  box-sizing: border-box;
  background: #2d3a4a;
```

여기에 코드를 쓰세요

```
}
.p11-brand {
  margin: 0;
```



```
font-size: 18px;
  font-weight: bold;
}
.p11-menu {
  display: flex;
  gap: 16px;
}
.p11-menu a {
  color: #cfd9e6;
  font-size: 14px;
  text-decoration: none;
}
```

문제 12 [응용] (개념04 섹션3)

컨테이너는 이미 `align-items: flex-start` 라 상자 셋이 모두 위에 붙어 있습니다.
가운데 상자 하나만 아래쪽으로 내려보내세요. 아이템에 쓰는 속성입니다.

기대 화면: 첫째와 셋째는 위에, 가운데 상자만 무대 바닥에 붙습니다.
셋 다 키는 44px 그대로입니다.

이렇게 되면 틀린 것: 셋 다 내려갔다면 컨테이너 쪽 속성을 고친 것입니다.

```
.p12-stage {
  display: flex;
  height: 140px;
  align-items: flex-start;
}
.p12-middle {
```

여기에 코드를 쓰세요

```
}
```

문제 13 [도전] (개념06 섹션4)

본문과 사이드바 2단을 만드세요. 조건은 셋입니다.

- 두 칸 사이 간격 20px
- 사이드바는 항상 폭 200px (늘지도 줄지도 않습니다)
- 본문은 남은 폭을 전부 차지합니다

두 칸의 높이는 각자 내용만큼만 차지하게 하세요.

기대 화면: 왼쪽 본문 380px, 오른쪽 사이드바 200px 이 20px 간격으로 놓입니다.

사이드바 상자의 키가 본문보다 짧습니다.

이렇게 되면 틀린 것: 사이드바가 200px 보다 좁아졌다면 가운데 숫자(shrink)가 1로 남아 있는 것입니다. 사이드바가 본문과 같은 키로 길어졌다면

교차축 정렬을 안 바꾼 것입니다.

```
.p13-stage {  
  display: flex;
```

여기에 코드를 쓰세요 (간격과 교차축 정렬)

```
}  
.p13-main {  
  background: #fff;  
  border: 1px solid #ccd5e0;  
  padding: 12px 14px;  
  box-sizing: border-box;
```

여기에 코드를 쓰세요 (남는 폭 전부)

```
}
.p13-side {
  background: #f4f7fb;
  border: 1px solid #ccd5e0;
  padding: 12px 14px;
  box-sizing: border-box;
}
```

여기에 코드를 쓰세요 (폭 200px 고정)

```
}
```

문제 14 [도전] (개념06 섹션2)

카드 목록을 만드세요. 조건은 넷입니다.

- 카드를 가로로 놓고, 자리가 모자라면 아랫줄로 내려보냅니다
- 카드 사이 간격은 가로세로 모두 16px
- 카드 폭은 260px 에서 시작해 남는 만큼 함께 넓어집니다
- 글 길이가 달라도 가격 줄은 카드 맨 아래에 붙습니다

마지막 조건은 카드 자신을 세로 컨테이너로 만들어야 풀립니다.

기대 화면: 카드가 한 줄에 두 개씩 두 줄로 놓고 폭은 292px 입니다.

한 줄 안의 카드 키가 서로 같고, 가격이 모두 카드 바닥에 붙습니다.

이렇게 되면 틀린 것: 카드 네 개가 한 줄에 좁게 들어갔다면 줄바꿈을 안 준 것이고,

가격이 글 바로 아래에 붙어 있다면 카드를 세로 컨테이너로 안 만든 것입니다.

```
.p14-list {
```

여기에 코드를 쓰세요 (가로 배치 · 줄바꿈 · 간격)

```

}
.p14-card {
  background: #fff;
  border: 1px solid #ccd5e0;
  padding: 12px 14px;
  box-sizing: border-box;

```

여기에 코드를 쓰세요 (폭 나눠 갖기 · 세로 컨테이너)

```

}
.p14-title {
  margin: 0 0 6px;
  font-size: 15px;
  font-weight: bold;
}
.p14-text {
  margin: 0;
  font-size: 13px;
  color: #555;
}
.p14-price {
  padding-top: 8px;
  font-size: 13px;
  color: #2d6cdf;
  font-weight: bold;

```

여기에 코드를 쓰세요 (카드 바닥에 붙이기)

```

}

```

문제 15 [확인] (개념01 섹션5)

아래 CSS 는 "상자 세 개를 무대 가운데로 모으기" 를 하려고 쓴 것입니다.
그런데 새로고침해도 아무 일도 일어나지 않습니다. 에러도 안 납니다.

무엇이 잘못됐는지 찾아서 고치세요. 지우지 말고 자리를 옮기면 됩니다.

기대 화면: 상자 셋이 무대 가운데로 모이고 좌우에 120px 씩 빈 칸이 생깁니다.
문제 4 와 똑같은 화면이 되면 맞은 것입니다. 위로 올려 비교해

보세요.

이렇게 되면 틀린 것: 상자가 왼쪽에 그대로 있으면 아직 자리를 안 옮긴
것입니다.

힌트: 개념01 섹션 5 의 표를 보세요.

이 두 속성은 둘 다 "컨테이너에 쓰는 것" 칸에 있습니다.

```
.p15-stage {
}
.p15-stage .tile {
  display: flex;
  justify-content: center;
}
```

12단원 마무리 Flexbox

자주 넘어지는 자리

- 1 flex-direction: column 으로 바꾸면 justify-content 와 align-items 가 뒤바뀌는 것

Grid와 반응형

OPEN 13_Grid와_반응형

2fr – 넓은 쪽

200px 고정

1fr – 남는 만큼

자주 묻는 질문

01 grid 시작하기

02 칸 합치기와 영역

03 화면 크기와 미디어쿼리

04 반응형 단위와 이미지

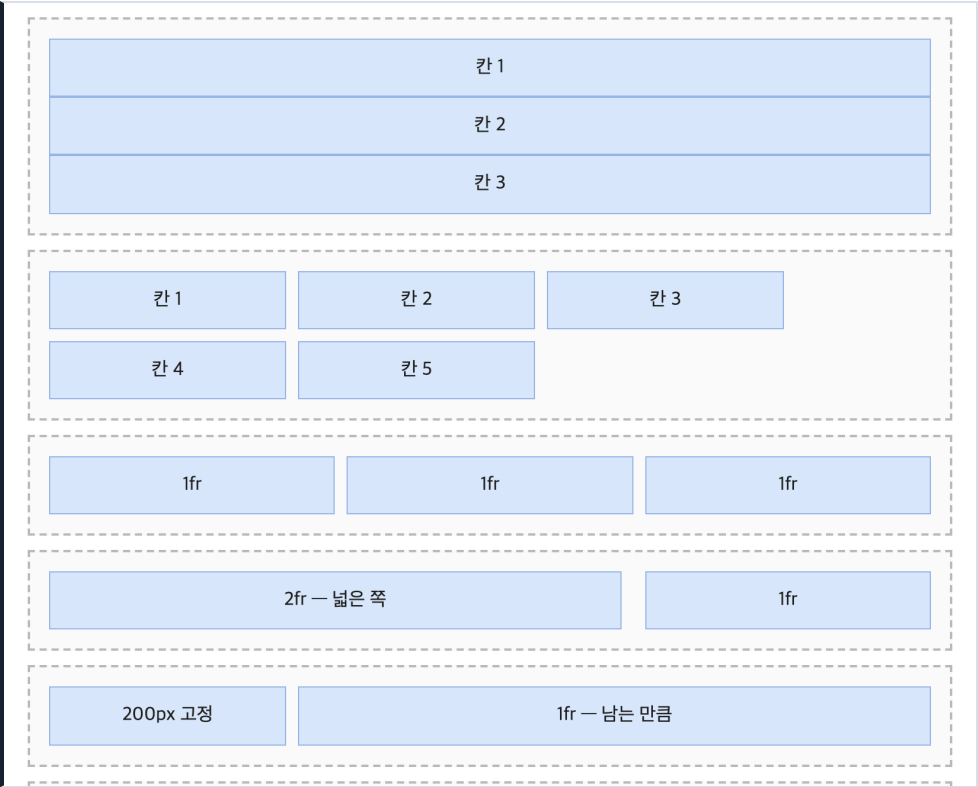
05 모바일 우선으로 만들기

– 연습문제

grid 시작하기

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.



실행하면 나오는 화면 — grid 시작하기

12단원에서 flex 를 배웠습니다. flex 는 상자들을 한 줄로 세우는 도구였습니다. 가로로 한 줄, 또는 세로로 한 줄. 어느 쪽이든 방향은 하나였습니다.

13단원에서 배우는 grid 는 다릅니다. grid 는 화면에 모눈종이 같은 격자(grid) 를 미리 그려 놓고, 그 칸에 상자를 하나씩 넣는 방식입니다.

flex (12단원) - 한 방향으로 줄을 세웁니다

1	2	3
---	---	---

가로 한 줄

grid (13단원) - 가로와 세로를 동시에 정합니다

1	2	3
4	5	6

가로 3칸 × 세로 2줄

용어를 먼저 정해 둡니다. 이 단원 내내 씁니다.

격자	가로 칸과 세로 줄로 나눈 판 전체
칸(cell)	격자에서 네모 하나
열(column)	세로로 선 칸 묶음. 왼쪽에서 오른쪽으로 셉니다
행(row)	가로로 누운 칸 묶음. 위에서 아래로 셉니다
격자 부모	display: grid 를 쓴 요소. 격자를 그리는 쪽입니다
격자 자식	그 요소의 바로 아래 자식들. 칸에 들어가는 쪽입니다

한 가지만 미리 기억하세요. flex 도 grid 도 부모에 쓰는 속성입니다. 자식에 쓰는 게 아닙니다. 부모가 “나는 이제 격자다” 라고 선언하면, 자식들이 저절로 칸에 들어갑니다.

이 파일에서는 격자를 만드는 법을 배웁니다. display: grid 로 격자를 켜고, grid-template-columns 로 칸을 나누고, fr 로 비율을 정하고, gap 으로 사이를 띄웁니다. 마지막 섹션에서 flex 와 grid 중 무엇을 쓸지 판단하는 기준을 정리합니다.

격자를 켭니다

섹션 1

먼저 부모에게 격자가 되라고 말합니다. 그런데 이것만으로는 화면이 거의 안 바뀝니다. 왜 그런지가 이 섹션의 내용입니다.

아래 상자 안에는 div 세 개가 들어 있고, 그 부모에 이렇게 썼습니다.

```
.step1-grid {
  display: grid;
}
```

display 는 11단원에서 배운 속성입니다. 요소를 어떤 방식으로 배치할지 정합니다. block, inline, flex 를 이미 봤습니다. 여기에 grid 가 하나 더 늘어난 것입니다.

결과를 보면 칸 세 개가 위에서 아래로 쌓여 있습니다. display: block 일 때와 거의 같아 보입니다. 실패한 걸까요? 아닙니다.

grid 는 “칸을 몇 개로 나눌지” 를 아직 못 들었습니다. 그래서 열을 하나만 만들고, 자식을 위에서부터 한 줄에 하나씩 넣었습니다.

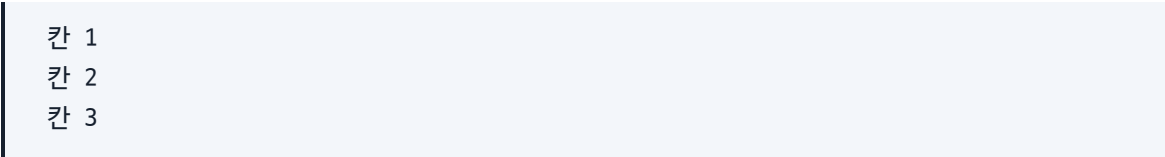


즉 지금 이 격자는 “1열 3행” 입니다. 격자는 맞습니다. 칸이 세로로만 있을 뿐입니다. 칸을 가로로 나누려면 다음 섹션의 grid-template-columns 가 필요합니다.

참고로 display: block 과 완전히 같지는 않습니다. 블록 요소였다면 문단 사이 여백(margin) 이 서로 겹쳐지지만, 격자 칸이 되면 그런 겹침이 일어나지 않습니다. 지금은 넘어가도 됩니다.

화면 파란 칸 세 개가 위에서 아래로 붙어서 쌓입니다.

각 칸의 폭은 상자 안쪽을 꽉 채운 824px 입니다



display: grid 는 부모에 씁니다. 칸이 될 자식에게 쓰는 것이 아닙니다. 이것을 바꿔 쓰는 실수가 가장 많습니다.

직접 해보기
1

위 <style> 의 .step1-grid 에서 display: grid; 를 잠시 지우고 저장(Ctrl+S) 한 뒤 새로고침(F5) 해 보세요. 화면이 거의 그대로인 것을 확인하고 다시 되돌려 놓으세요.

세로 칸 만들기

섹션 2

몇 칸으로 나눌지, 각 칸을 얼마나 넓게 할지를 한 줄로 정합니다. 값을 쓴 개수가 곧 칸의 개수입니다.

속성 이름이 길니다. 뜻으로 끊어 읽으면 외우지 않아도 됩니다.

```
grid template columns
격자   틀       열
"격자의 열을 이런 틀로 만들어라"
```

아래 상자에는 이렇게 썼습니다.

```
.step2-grid {
  display: grid;
  grid-template-columns: 200px 200px 200px;
  gap: 10px;
}
```

값이 200px 200px 200px 세 개입니다. 그래서 열이 세 개 생깁니다. 띄어쓰기로 구분합니다. 쉼표가 아닙니다.

여기서 초보자가 가장 놀라는 점이 하나 있습니다. 상자 안에는 칸이 다섯 개인데 열은 세 개뿐입니다. 나머지 두 개는 어디로 갈까요?

1	2	3	← 열이 셋이므로 여기까지가 첫 줄
4	5		← 넘친 칸은 자동으로 아랫줄로 내려갑니다

칸이 넘쳐도 옆으로 빠져나가지 않습니다. 저절로 다음 줄이 하나 더 생깁니다. 이렇게 자동으로 생긴 줄을 암시적 행이라고 부릅니다. “열 개수만 정하면 행은 알아서 늘어난다” 로 기억하면 충분합니다.

칸 세 개가 200px 씩 = 600px, 사이 간격 10px 두 개 = 20px. 합쳐서 620px 입니다. 상자 안쪽은 824px 이므로 오른쪽에 204px 가 남습니다. 그 자리는 비어 있습니다. px 로 못을 박아 두면 남는 자리를 쓰지 못합니다. 이것을 해결하는 것이 다음 섹션입니다.

화면 200px 칸 세 개가 윗줄에, 나머지 두 개가 아랫줄에 놓입니다.

오른쪽 204px 는 비어 있고, 아랫줄 세 번째 자리도 비어 있습니다

칸 1
칸 2
칸 3
칸 4
칸 5

이 상자만 .px-frame 이라는 껍데기로 한 번 더 감싸고 overflow-x: auto 를 걸어 두었습니다(10단원). 칸을 px 로 못 박아 두면 창을 좁혔을 때 620px 이 그대로 남아 밖으로 빠져나가는데, 그때 페이지 전체가 아니라 이 상자 안에서만 가로로 스크롤되게 하려는 것입니다. 왜 이런 일이 생기는지는 개념03 에서 봅니다.

▹ 직접 해보기

2

.step2-grid 의 grid-template-columns 값을 200px 200px 두 개로 줄여 보세요. 칸이 몇 줄이 되는지 세어 보고, 확인했으면 되돌려 주세요.

fr 은 남은 공간을 몇 조각으로 나눠 가질지를 뜻하는 grid 전용 단위입니다. px 처럼 고정된 길이가 아닙니다.

fr 은 fraction(조각, 비율) 의 줄임말입니다. 계산 순서가 정해져 있습니다.

1 전체 폭에서 gap 과 px 로 못 박은 칸을 먼저 뺍니다 → 이것이 '남는 공간'

2 남는 공간을 fr 숫자의 합으로 나눕니다 → 이것이 '한 조각'

3 각 칸은 자기 fr 숫자만큼 조각을 가져갑니다

예시 1 · 1fr 1fr 1fr, gap 10px, 상자 안쪽 824px

남는 공간 = $824 - (10 \times 2) = 804$
 조각 합계 = $1 + 1 + 1 = 3$
 한 조각 = $804 \div 3 = 268$
 → 세 칸이 모두 268px

예시 2 · 2fr 1fr, gap 20px, 상자 안쪽 824px

남는 공간 = $824 - 20 = 804$
 조각 합계 = $2 + 1 = 3$
 한 조각 = $804 \div 3 = 268$
 → 왼쪽 = $268 \times 2 = 536\text{px}$, 오른쪽 = $268 \times 1 = 268\text{px}$

예시 3 · 200px 1fr, gap 10px, 상자 안쪽 824px

남는 공간 = $824 - 200 - 10 = 614$
 조각 합계 = 1
 → 왼쪽은 200px 로 고정, 오른쪽은 614px

예시 3 이 실제 화면에서 가장 자주 쓰는 모양입니다. “옆 메뉴는 200px 로 고정하고 본문은 남는 만큼” 이 그대로 이 한 줄입니다.

그리고 fr 에는 아주 중요한 성질이 있습니다. 브라우저 창을 좁히면 824 라는 숫자 자체가 줄어들기 때문에, fr 로 만든 칸은 창 크기를 따라 저절로 줄어듭니다. px 로 만든 칸은 창을 아무리 좁혀도 그대로입니다.

실제로 재 보면 이렇습니다. 창 폭을 480px 으로 줄이면 상자 안쪽이 404px 이 됩니다.

1fr 1fr 1fr 268px 씩 → 128px 씩으로 줄어듭니다
 200px 1fr 왼쪽은 200px 그대로, 오른쪽만 614px → 194px 로 줄어듭니다
 섹션 2 의 200px 200px 200px 셋 다 200px 그대로. 상자 밖으로 나갑니다

px 로 못 박은 칸만 화면을 무시하고 버팁니다. 이 차이가 13단원 전체의 주제인 반응형의 출발점입니다.

화면 세 칸이 268px 씩 똑같은 폭으로 상자를 꽉 채웁니다

1fr
 1fr
 1fr

화면 왼쪽 칸이 536px, 오른쪽 칸이 268px 입니다. 왼쪽이 정확히 두 배입니다

2fr - 넓은 쪽
 1fr

화면 왼쪽은 200px 그대로, 오른쪽은 남은 614px 를 전부 차지합니다

200px 고정
 1fr - 남은 만큼

fr 은 grid 의 칸을 정할 때만 쓸 수 있습니다. width: 1fr 처럼 다른 자리에 쓰면 그 줄은 조용히 무시됩니다.

▣ 직접 해보기

3

.step3-ratio 의 값을 3fr 1fr 로 바꾸고, 왼쪽 칸이 오른쪽의 몇 배가 되는지 눈으로 확인해 보세요. 위 계산 순서대로 직접 숫자를 구해 본 뒤 화면과 맞춰 보면 더 좋습니다.

gap

섹션 4

gap 은 칸과 칸 사이에만 간격을 넣습니다. 격자 바깥쪽에는 넣지 않습니다. 이것이 margin 과 결정적으로 다른 점입니다.

12단원 flex 에서 이미 gap 을 썼습니다. grid 에서도 이름과 뜻이 똑같습니다.

gap: 10px; 가로 사이도 10px, 세로 사이도 10px
 gap: 30px 10px; 앞이 행 사이(위아래), 뒤가 열 사이(좌우)

순서가 헛갈리면 이렇게 외우지 말고 이렇게 읽으세요. CSS 에서 두 값을 쓰는 속성은 거의 항상 “세로 먼저, 가로 나중” 입니다. 10단원 개념04 의 margin: 10px 120px 도 위아래 10px, 좌우 120px 였습니다. 같은 규칙입니다.

gap 과 margin 의 차이도 그대로입니다. 12단원 개념03 섹션5 에서 본 그 이야기입니다. 그림으로 다시 보면 이렇습니다.

gap 을 쓰면

1

2

3

바깥쪽에는 여백이 없습니다

자식에 margin 을 쓰면

여백

1

2

 여백 여백

바깥쪽에도 여백이 생깁니다

격자 안쪽만 띄우고 싶을 때는 gap 이 훨씬 편합니다. 칸 개수가 몇 개든 바깥 여백이 생기지 않기 때문입니다.

아래 세 상자는 gap 값만 다릅니다. 열은 셋 다 1fr 1fr 1fr 로 같습니다. gap 이 커질수록 칸이 좁아진다는 점을 확인하세요. 전체 폭은 그대로인데 사이 간격이 커졌으니 칸이 그만큼 양보한 것입니다.

화면

칸 사이가 4px 만 벌어져 거의 붙어 보입니다. 칸 하나는 272px 입니다

1

2

3

화면

칸 사이가 28px 벌어지고 칸은 256px 로 좁아집니다.

격자 맨 왼쪽과 맨 오른쪽에는 여백이 없습니다

1

2

3

화면

좌우 사이는 좁고(10px) 위아래 두 줄 사이는 넓게(30px) 벌어집니다

1

2

3

4

5

6

▢ 직접 해보기 4

.step4-two 의 gap: 30px 10px; 를 gap: 10px 30px; 로 앞뒤만 바꿔 보세요. 어느 쪽 간격이 넓어지는지 확인하면 순서를 다시는 헷갈리지 않습니다.

행의 높이

섹션 5

열을 정하는 속성이 grid-template-columns 였다면, 행을 정하는 속성은 grid-template-rows 입니다. 쓰는 법은 똑같습니다.

아래 상자에는 이렇게 썼습니다.

```
.step5-rows {
  display: grid;
  grid-template-columns: 1fr 1fr;   열 2개
  grid-template-rows: 60px 100px;   행 2개, 높이는 60px 과 100px
  gap: 10px;
}
```

칸 네 개가 이렇게 들어갑니다.

1	2	높이 60px
3	4	높이 100px

칸 안의 글자는 한 줄뿐인데도 칸 높이가 60px, 100px 로 맞춰집니다. 내용이 아니라 격자가 높이를 정하기 때문입니다.

행 높이는 대부분 안 정하고 그냥 둡니다. 안 정하면 그 줄에서 가장 키가 큰 칸에 맞춰 높이가 저절로 결정됩니다. 섹션 1~4 의 데모들이 전부 그런 상태였습니다.

높이를 정할 때 주의할 점이 하나 있습니다. 내용이 정한 높이보다 길면 칸 밖으로 빠져나옵니다. 높이는 꼭 필요할 때만 정하고, 되도록 내용에 맡기는 편이 안전합니다.

화면 윗줄 두 칸은 높이 60px, 아랫줄 두 칸은 높이 100px 입니다.

칸 폭은 네 칸 모두 407px 입니다

1
2
3
4

▢ 직접 해보기 5

.step5-rows 의 grid-template-rows 값을 100px 60px 으로 앞뒤를 바꿔 보세요. 어느 줄이 두꺼워지는지 확인하세요.

한 줄로 정리하면 이렇습니다. flex 는 한 방향(줄), grid 는 두 방향(격자) 입니다.

아래 두 상자에는 글자 길이가 서로 다른 칸 세 개가 똑같이 들어 있습니다. 부모의 display 값만 다릅니다.

```
.cmp-flex { display: flex; gap: 10px; }
.cmp-grid { display: grid; grid-template-columns: 1fr 1fr 1fr; gap: 10px; }
```

결과가 확실히 같립니다.

flex → 칸의 폭이 글자 길이만큼 제각각입니다 (38px / 86px / 129px)
grid → 글자 길이와 상관없이 세 칸 모두 268px 로 같습니다

이유는 두 도구가 보는 기준이 다르기 때문입니다.

flex 는 내용을 먼저 봅니다. "이 글자를 담으려면 이만큼 필요하겠군"
grid 는 칸을 먼저 봅니다. "칸이 268px 이니 글자는 여기에 맞춰"

그래서 판단 기준은 이렇게 잡으면 거의 맞습니다.

메뉴 한 줄, 버튼 두세 개, 아이콘 옆에 글자	→ flex
줄이 하나뿐이고, 내용 길이에 맞춰 늘었다 줄었다	→ flex
카드 목록, 사진 목록, 표처럼 칸이 딱 맞아야 하는 것	→ grid
머리말·본문·옆칸·바닥글 같은 페이지 전체 뼈대	→ grid
가로와 세로를 동시에 맞춰야 하는 것	→ grid

둘 중 하나만 골라야 하는 것도 아닙니다. 페이지 전체 뼈대는 grid 로 잡고, 그 안의 메뉴 한 줄은 flex 로 하는 식으로 섞어 쓰는 것이 보통입니다. 개념05 에서 실제로 그렇게 만들어 봅니다.

화면 세 칸의 폭이 글자 길이만큼 다릅니다. 왼쪽부터 38px, 86px, 129px 이고

오른쪽에는 빈 자리가 남습니다

홈
공지사항
자주 묻는 질문

화면 글자 길이가 달라도 세 칸이 모두 268px 로 똑같고 상자를 꽉 채웁니다

홈
공지사항
자주 묻는 질문

화면 네 줄짜리 비교 표가 점선 상자 안에 그려집니다

◀ 직접 해보기 6

.cmp-grid 의 값을 grid-template-columns: 1fr 2fr 1fr; 로 바꿔 보세요. 가운데 칸만 넓어집니다. flex 로는 이렇게 “가운데를 정확히 2배” 로 만들기가 번거롭습니다.

자주 하는 실수

섹션 7

다시 강조합니다. CSS 는 틀려도 에러가 나지 않습니다. 틀린 줄은 그냥 없는 셈 치고 넘어갑니다. 그래서 “아무 일도 안 일어나는” 것이 가장 흔한 증상입니다.

이런 실수를 합니다

display: grid 를 자식에 씀

이런 실수를 합니다

실수 1 — display: grid 를 자식에 씀

```
.cell {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
}
```

무슨 일이 일어나나요: 칸들이 여전히 위아래로 쌓입니다. 자식 하나하나가 각자 “속이 3칸인 격자” 가 됐을 뿐, 자식들끼리는 아무 관계가 없습니다. 격자로 만들 대상은 자식들을 담고 있는 부모입니다.

이렇게 쓰세요

```
.step2-grid {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
}
```

이런 실수를 합니다

칸 개수만 씀

이런 실수를 합니다

실수 2 — 칸 개수만 씀

```
grid-template-columns: 3;
```

무슨 일이 일어나나요: 아무 일도 안 일어납니다. 칸이 나뉘지 않고 한 줄에 하나씩 그대로 쌓입니다. 이 속성은 칸의 개수가 아니라 각 칸의 크기를 하나씩 적는 자리입니다. 3칸으로 나누고 싶으면 1fr 1fr 1fr 처럼 값을 세 개 씁니다.

이런 실수를 합니다

값을 쉼표로 구분

이런 실수를 합니다

실수 3 — 값을 쉼표로 구분함

```
grid-template-columns: 1fr, 1fr, 1fr;
```

무슨 일이 일어나나요: 역시 아무 일도 안 일어납니다. 칸이 하나도 안 나뉩니다. CSS 에서 여러 값을 나열할 때는 띄어쓰기로 구분합니다. 쉼표를 쓰는 속성은 font-family 처럼 따로 정해져 있습니다.

이런 실수를 합니다

fr 을 아무 데나 씀

이런 실수를 합니다

실수 4 — fr 을 width 에 씀

```
.cell {  
  width: 1fr;  
}
```

무슨 일이 일어나나요: 그 줄이 통째로 무시되고 폭이 안 바뀝니다. fr 은 grid-template-columns 와 grid-template-rows 안에서만 쓰는 단위입니다. 일반 폭을 비율로 주고 싶으면 % 를 씁니다(개념04).

이런 실수를 합니다

칸이 넘치면 잘린다고 생각함

이런 실수를 합니다

실수 5 — 칸이 열 개수보다 많으면 잘린다고 생각함

```
grid-template-columns: 200px 200px 200px;
(자식은 7개)
```

무슨 일이 일어나나요: 잘리지 않습니다. 아랫줄이 저절로 생겨서 7개가 3 + 3 + 1 로 놓입니다. “왜 갑자기 두 줄이 됐지?” 하고 당황하는 경우가 많은데, 격자가 정상 동작한 것입니다. 한 줄에 다 넣고 싶으면 열 개수를 자식 수에 맞춰 늘려야 합니다.

이런 실수를 합니다

gap 대신 margin

이런 실수를 합니다

실수 6 — 간격을 자식의 margin 으로 냄

```
.cell {
margin: 5px;
}
```

무슨 일이 일어나나요: 칸 사이는 벌어지지만 격자 바깥쪽에도 5px 씩 여백이 생깁니다. 옆 요소와 줄이 안 맞아서 “왜 여기만 살짝 밀렸지?” 하게 됩니다. 게다가 칸 안쪽 폭이 margin 만큼 줄어들어 fr 계산도 어긋납니다. 격자 안쪽 간격은 gap 으로 내세요.

정리

◁ 직접 해보기 7

정답

1) .step1-grid 에서 display: grid; 를 지우면

```
.step1-grid {
}
```

→ 화면이 거의 그대로입니다. 칸 세 개가 여전히 위아래로 쌓입니다.

div 는 원래 블록 요소라서 한 줄씩 쌓이기 때문입니다.
display: grid 만으로는 눈에 보이는 변화가 거의 없다는 뜻입니다.
확인했으면 display: grid; 를 다시 넣으세요.

2) .step2-grid 를 이렇게 바꿉니다.

```
grid-template-columns: 200px 200px;
```

→ 열이 두 개가 되어 칸 다섯 개가 $2 + 2 + 1$ 로 세 줄이 됩니다.

열 개수를 줄이면 행이 그만큼 늘어납니다.
확인했으면 200px 200px 200px 로 되돌리세요.

3) .step3-ratio 를 이렇게 바꿉니다.

```
grid-template-columns: 3fr 1fr;
```

→ 왼쪽이 오른쪽의 3배가 됩니다.

계산: 남는 공간 $824 - 20(\text{gap}) = 804$, 조각 합계 $3 + 1 = 4$,
한 조각 $804 \div 4 = 201$. 왼쪽 603px, 오른쪽 201px 입니다.

4) .step4-two 를 이렇게 바꿉니다.

```
gap: 10px 30px;
```

→ 위아래 줄 사이가 좁아지고(10px) 좌우 칸 사이가 넓어집니다(30px).

앞의 값이 행 사이, 뒤의 값이 열 사이입니다.

5) .step5-rows 를 이렇게 바꿉니다.

```
grid-template-rows: 100px 60px;
```

→ 윗줄이 100px 로 두꺼워지고 아랫줄이 60px 로 얇아집니다.

적은 순서대로 첫째 줄, 둘째 줄에 그대로 들어갑니다.

6) .cmp-grid 를 이렇게 바꿉니다.

```
grid-template-columns: 1fr 2fr 1fr;
```

→ 조각 합계가 4가 되어 한 조각은 $(824 - 20) \div 4 = 201\text{px}$ 입니다.

양옆이 201px, 가운데가 402px 이 됩니다.
글자 길이와 상관없이 가운데만 정확히 두 배가 된다는 점이 grid 의 힘입니다.

이 파일에서 '가르치는' CSS 만 여기에 씁니다.

모든 데모의 칸에 공통으로 쓸 모양입니다. grid 와는 상관없는, 그냥 눈에 보이라고 넣은 색입니다.

```
.cell {
  background: #dbe7fb;
  border: 1px solid #9ab8e8;
  padding: 10px;
  text-align: center;
}
```

display: grid 를 켜기만 한 상태

섹션 1

```
.step1-grid {
  display: grid;
}
```

화면 칸 세 개가 위에서 아래로 한 줄씩 쌓입니다 (폭은 각각 824px)

grid-template-columns 로 세로 칸 만들기

섹션 2

```
.px-frame {
  overflow-x: auto;
}
```

화면 폭 1280 에서는 아무 변화가 없습니다.

창을 폭 695px 아래로 좁히면 그때부터 이 상자 안에 가로 스크롤 막대가 생깁니다

```
.step2-grid {
  display: grid;
  grid-template-columns: 200px 200px 200px;
  gap: 10px;
}
```

화면 200px 짜리 칸 세 개가 가로로 놓고, 네 번째부터는 아랫줄로 내려갑니다

fr 단위

섹션 3

```
.step3-even {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 10px;
}
```

화면 세 칸이 268px 씩 똑같이 나뉘어 상자를 꽉 채웁니다

```
.step3-ratio {
  display: grid;
  grid-template-columns: 2fr 1fr;
  gap: 20px;
}
```

화면 왼쪽 칸 536px, 오른쪽 칸 268px. 왼쪽이 정확히 두 배입니다

```
.step3-mix {
  display: grid;
  grid-template-columns: 200px 1fr;
  gap: 10px;
}
```

화면 왼쪽은 200px 로 고정, 오른쪽은 남은 614px 를 전부 차지합니다

gap

섹션 4

```
.step4-small {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 4px;
}
```

화면 칸 사이가 4px 만 벌어져 거의 붙어 보입니다 (칸 폭 272px)

```
.step4-big {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 28px;
}
```

화면 칸 사이가 28px 벌어지고, 그만큼 칸이 좁아집니다 (칸 폭 256px)

```
.step4-two {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 30px 10px;
}
```

화면 위아래 줄 사이는 30px, 좌우 칸 사이는 10px 벌어집니다

grid-template-rows

섹션 5

```
.step5-rows {
  display: grid;
  grid-template-columns: 1fr 1fr;
  grid-template-rows: 60px 100px;
  gap: 10px;
}
```

화면 윗줄 두 칸은 높이 60px, 아랫줄 두 칸은 높이 100px 로 그려집니다

flex 와 grid 비교

섹션 6

```
.cmp-flex {
  display: flex;
  gap: 10px;
}
```

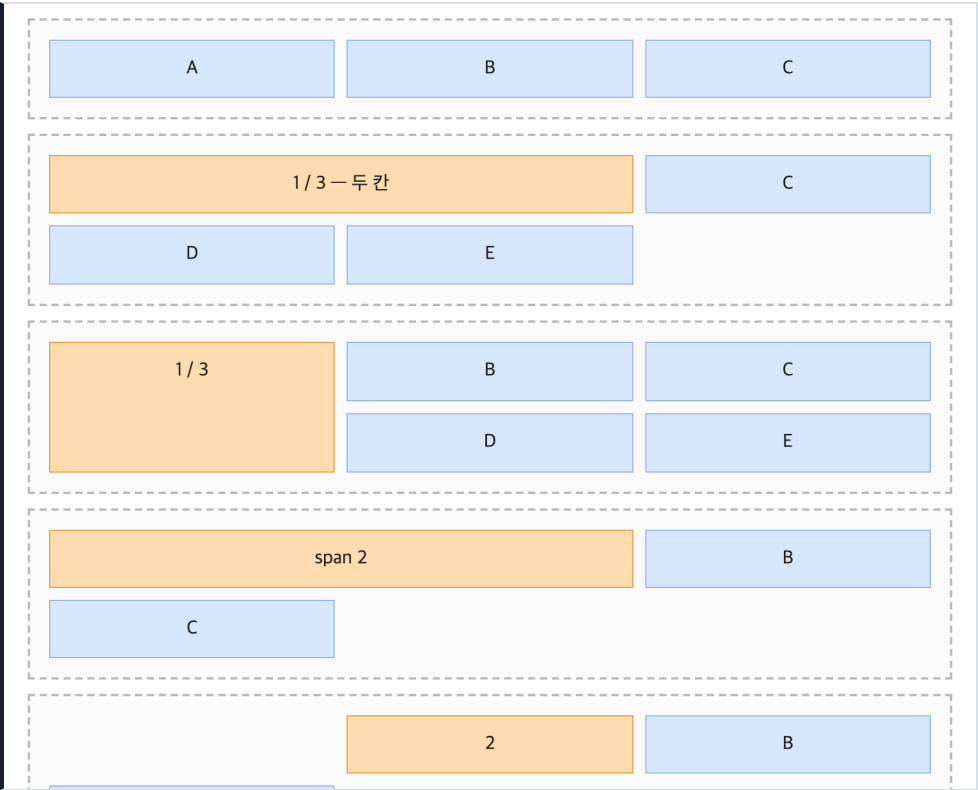
화면 세 칸의 폭이 글자 길이만큼씩 제각각입니다 (38px / 86px / 129px)

```
.cmp-grid {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 10px;
}
```

화면 글자 길이와 상관없이 세 칸이 모두 268px 로 똑같습니다

칸 합치기와 영역

RUN 이 파일을 더블클릭해서 브라우저로 여세요.



실행하면 나오는 화면 — 칸 합치기와 영역

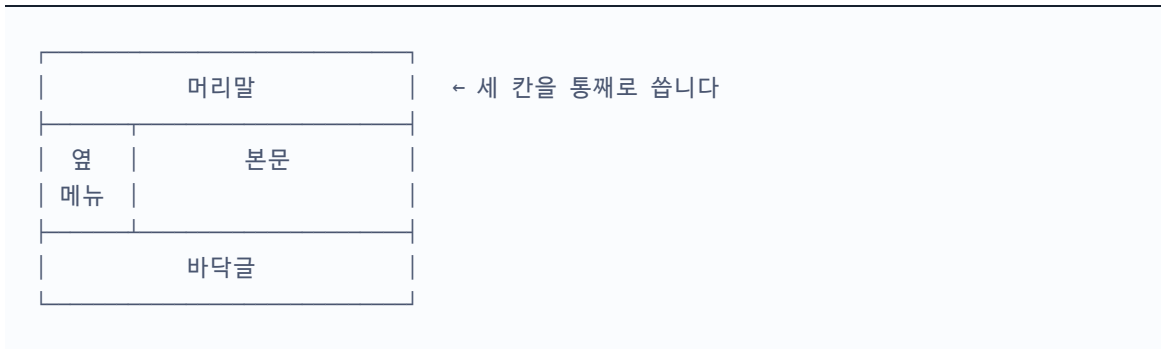
먼저 03단원과 헷갈리지 않게 선을 그어 둡니다.

03단원에서 표의 칸을 colspan 으로 합쳐 봤습니다. 격자에도 같은 일이 있습니다. 다만 쓰는 것이 완전히 다릅니다.

표	칸이 td 태그로 이미 있고, HTML 속성(colspan)으로 합칩니다
격자	칸이 CSS 로 생기고, CSS 속성(grid-column)으로 합칩니다

그러니 격자에 colspan 을 쓰면 아무 일도 일어나지 않습니다. 이름만 비슷할 뿐 다른 도구입니다.

개념01 에서 격자를 만들었습니다. 칸이 나뉘었고, 자식들이 한 칸씩 들어갔습니다. 그런데 실제 화면은 그렇게 단순하지 않습니다. 머리말은 가로로 쪽 뻗어야 하고, 옆 메뉴는 세로로 길어야 합니다.



이 파일에서는 칸을 합치는 두 가지 방법을 배웁니다.

1 선 번호로 합치기 `grid-column: 1 / 4`

2 이름으로 배치하기 `grid-template-areas`

두 번째 방법이 처음 배우는 사람에게 훨씬 쉽습니다. 글자 그림을 그리듯 배치가 눈에 보이기 때문입니다. 그래도 첫 번째 방법을 먼저 배웁니다. 선 번호를 알아야 남의 코드를 읽을 수 있고, 무엇보다 여기서 초보자가 가장 많이 걸리는 함정이 나오기 때문입니다.

파일 끝에서는 `minmax()` 와 `auto-fit` 을 배웁니다. 이 둘을 합치면 `@media` 를 한 줄도 안 쓰고도 칸 수가 저절로 바뀌는 격자를 만들 수 있습니다. 개념03 으로 넘어가는 다리입니다.

이 파일에서는 한 상자가 여러 칸을 차지하게 만드는 법을 배웁니다. 먼저 격자 선 번호를 세는 법(여기에 함정이 있습니다), 다음에 `span`, 그다음에 이름으로 배치하는 `grid-template-areas` 입니다. 마지막으로 `repeat()` · `minmax()` · `auto-fit` 을 배웁니다.

선 번호

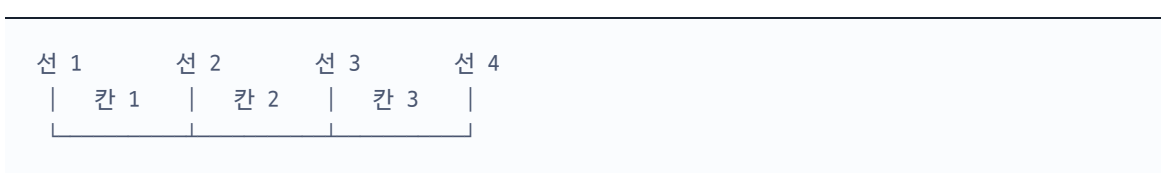
섹션 1

`grid` 는 칸이 아니라 선에 번호를 붙입니다. 번호는 1부터 시작하고, 칸이 3개면 선은 4개입니다. 이 한 문장이 이 파일에서 가장 중요합니다.

칸을 세는 것과 선을 세는 것은 다릅니다. 젓가락과 젓가락 사이를 세는 것과 같습니다.

칸을 세면	1 2 3	(3개)
선을 세면	1 2 3 4	(4개)

그림으로 보면 이렇습니다. 세로 막대가 선입니다.



즉 칸 1 은 “1번 선부터 2번 선까지” 입니다. 칸 2 는 “2번 선부터 3번 선까지” 입니다.

왜 이렇게 만들었을까요? 칸을 합치기 편하기 때문입니다. “1번 선부터 3번 선까지” 라고 하면 칸 1 과 칸 2 를 합친 것이 됩니다. 칸 번호로는 “1번 칸과 2번 칸” 처럼 두 개를 적어야 하지만, 선 번호로는 시작과 끝 두 숫자면 몇 칸이든 표현할 수 있습니다.

행(가로 줄)도 똑같습니다. 위에서부터 1, 2, 3 ... 으로 선 번호가 붙습니다. 아래 상자의 격자에서 각 선이 실제로 어디에 있는지 눈으로 확인해 두세요.

1 2 3 4 ← 선 번호

-----+-----+-----+-----+

| A | B | C |

-----+-----+-----+-----+

칸 A = 1번 선 ~ 2번 선

칸 B = 2번 선 ~ 3번 선

칸 C = 3번 선 ~ 4번 선

A의 왼쪽 끝이 1번 선, C의 오른쪽 끝이 4번 선입니다.

- A
- B
- C

칸이 3개면 선은 4개, 칸이 4개면 선은 5개입니다. 선의 개수는 항상 칸 개수보다 하나 많습니다.

➤ 직접 해보기 1

.line-grid 의 grid-template-columns 를 1fr 1fr 1fr 1fr 로 바꾸면 선은 몇 개가 될까요? 먼저 종이에 답을 적고 나서 값을 바꿔 확인하세요. (칸은 4개로 늘어나지만 상자는 3개뿐이라 마지막 칸은 비어 있습니다)

grid-column / grid-row

섹션 2

시작 선 / 끝 선을 빗금으로 이어서 씁니다. 이 두 속성은 격자 부모가 아니라 칸이 되는 자식에 씁니다.

지금까지 배운 grid 속성은 전부 부모에 쓰는 것이었습니다. grid-column 과 grid-row 는 반대입니다. 자식에 씁니다.

부모에 쓰는 것	display, grid-template-columns, grid-template-rows, gap
자식에 쓰는 것	grid-column, grid-row

“격자 판을 까는 일”은 부모가 하고, “나는 이 칸부터 저 칸까지 쓰겠다”는 자식이 스스로 정하는 것입니다. 그래서 나뉘어 있습니다.

아래 첫 상자에는 이렇게 썼습니다.

```
.colspan-grid .wide {
  grid-column: 1 / 3;
}
```

읽는 법: "1번 선에서 시작해서 3번 선에서 끝난다." 1번 선부터 3번 선까지 사이에는 칸이 두 개 있습니다. 그래서 두 칸을 차지합니다.



폭을 계산해 보면 $268 + 10(\text{gap}) + 268 = 546\text{px}$ 입니다. 두 칸을 합치면 그 사이의 gap 도 같이 먹는다는 점을 기억하세요.

행도 똑같습니다. 두 번째 상자에는 이렇게 썼습니다.

```
.rowspan-grid .tall {
  grid-row: 1 / 3;
}
```

행 높이가 50px 씩이고 gap 이 10px 이므로 $50 + 10 + 50 = 110\text{px}$ 이 됩니다.

화면 주황 칸이 546px 로 두 칸을 차지하고, 그 오른쪽에 C 가 옵니다.

D 와 E 는 아랫줄로 내려갑니다

```
1 / 3 - 두 칸
C
D
E
```

화면 왼쪽 주황 칸이 두 줄에 걸쳐 높이 110px 로 서 있고,

오른쪽에 B, C 가 윗줄, D, E 가 아랫줄에 놓입니다

```
1 / 3
B
C
D
E
```

.colspan-grid .wide 를 grid-column: 1 / 4; 로 바꿔 보세요. 주황 칸이 상자 전체를 가로지릅니다. 왜 4 인지 섹션 1 의 그림으로 설명해 보세요.

span 과 함정

섹션 3

선 번호가 헛갈릴 때는 span 을 씁니다. 몇 칸을 차지할지 개수로 쓰는 방법입니다.

아래 두 줄은 결과가 완전히 같습니다.

```
grid-column: 1 / 3;      1번 선에서 3번 선까지
grid-column: span 2;     여기서부터 두 칸
```

span 쪽은 시작 위치를 말하지 않습니다. "지금 놓이는 자리부터 두 칸" 이라는 뜻입니다. 그래서 앞에 다른 칸이 몇 개 있든 상관없이 항상 두 칸을 차지합니다. 위치가 바뀌어도 따라 움직이므로 span 쪽이 더 안전할 때가 많습니다.

★ 여기가 이 파일 최대의 함정입니다.

```
grid-column: 2;          ← 이건 "두 칸" 이 아닙니다
grid-column: span 2;     ← 이게 "두 칸" 입니다
```

숫자만 쓰면 그것은 개수가 아니라 시작 선 번호입니다. grid-column: 2 는 "2번 선에서 시작하는 한 칸" 이라는 뜻입니다. 칸이 넓어지기를 기대했는데 폭은 그대로이고 자리만 오른쪽으로 밀립니다. 예러는 나지 않습니다. 그냥 원하는 모양이 안 나올 뿐입니다.

```
grid-column: 2 를 쓰면
선1      선2      선3      선4
|  비었음  |  해당 칸  |   B   |
└────────┴────────┴────────┘
                                C 만 아랫줄로 내려갑니다
```

뒤에 오는 B 는 그 오른쪽 빈자리(3번 칸)에 그대로 들어갑니다. 자리가 다 찬 다음부터 아랫줄이 생기므로 C 만 내려갑니다. 비는 것은 맨 왼쪽 한 칸뿐입니다.

아래 두 상자를 나란히 보고 차이를 눈에 익혀 두세요.

화면 주황 칸이 546px 로 두 칸을 차지하고 그 오른쪽에 B,

아랫줄 왼쪽에 C 가 놓입니다

```
span 2
B
C
```

화면 주황 칸은 268px 그대로이고 두 번째 자리로 밀려 있습니다.

B 는 그 오른쪽 세 번째 자리에 그대로 들어가고, C 만 아랫줄로 내려갑니다. 첫 줄에서 비어 있는 것은 맨 왼쪽 한 칸뿐입니다

```
2
B
C
```

숫자 하나 = 시작 선 번호. 두 칸을 원하면 span 2. 이 문장을 소리 내어 한 번 읽어 두세요. 나중에 반드시 한 번은 걸립니다.

▸ 직접 해보기 3

.trap-grid .just2 를 grid-column: span 2; 로 고쳐 보세요. 칸의 폭과 자리가 어떻게 달라지는지 두 상자를 번갈아 보며 확인하세요.

repeat()

섹션 4

1fr 1fr 1fr 1fr 처럼 같은 값을 여러 번 쓸 때 repeat(개수, 값) 으로 짧게 씁니다. 결과는 완전히 같습니다. 칸이 12개인 격자를 만든다고 해 봅시다.

```
grid-template-columns: 1fr 1fr 1fr 1fr 1fr 1fr 1fr 1fr 1fr 1fr 1fr 1fr;
```

세다가 한 개를 빠뜨리면 11칸이 되는데, 에러가 안 나므로 알아채기 어렵습니다. repeat() 를 쓰면 이렇게 됩니다.

```
grid-template-columns: repeat(12, 1fr);
```

읽는 법: "1fr 을 12번 반복해라." 괄호 안은 개수가 먼저, 값이 나중이고 사이는 쉼표입니다. grid-template-columns 의 값들은 띄어쓰기로 구분했는데 repeat() 괄호 안만 쉼표라는 점이 헛갈립니다. 괄호 안은 함수라서 그렇습니다.

섞어서 쓸 수도 있습니다.

```
grid-template-columns: 120px repeat(2, 1fr);
→ 120px 짜리 한 칸 + 1fr 짜리 두 칸 = 모두 세 칸
```

계산해 보면 남은 공간은 $824 - 120 - 10 - 10 = 684$ 이고, 조각이 2개이므로 342px 씩입니다.

화면 200px 짜리 칸 네 개가 8px 간격으로 한 줄에 놓입니다

1
2
3
4

화면 왼쪽 칸만 120px 로 좁고, 오른쪽 두 칸이 342px 씩 넓게 나뉩니다

120px
1fr
1fr

▫ 직접 해보기 4

.rep-even 을 repeat(2, 1fr) 로 바꿔 보세요. 칸 네 개가 어떻게 배치되는지 보고, 1fr 1fr 이라고 직접 쓴 것과 결과가 같은지도 확인하세요.

grid-template-areas

섹션 5

칸마다 이름을 붙이고, 그 이름을 글자 그림처럼 늘어놓아 배치합니다. 선 번호를 한 번도 세지 않아도 됩니다.

부모에 이렇게 씁니다.

```
.area-page {  
  display: grid;  
  grid-template-columns: 200px 1fr;  
  grid-template-rows: 50px 120px 40px;  
  grid-template-areas:  
    "head head"  
    "side main"  
    "foot foot";  
  gap: 10px;  
}
```

따옴표 한 줄이 격자의 한 행입니다. 그 안의 이름 하나가 칸 하나입니다. 위 그림을 그대로 읽으면 이렇습니다.

1행: head head	← 같은 이름이 두 번이므로 두 칸을 합칩니다
2행: side main	← 왼쪽은 side, 오른쪽은 main
3행: foot foot	← 두 칸을 합칩니다

그리고 자식에는 자기 이름을 적어 줍니다.

```
.area-head { grid-area: head; }
.area-side { grid-area: side; }
.area-main { grid-area: main; }
.area-foot { grid-area: foot; }
```

이 방법의 좋은 점은 CSS 만 봐도 화면 모양이 보인다는 것입니다. 선 번호를 셀 필요도, 몇 칸인지 계산할 필요도 없습니다. 나중에 배치를 바꿀 때도 따옴표 안 글자만 바꾸면 됩니다.

지켜야 할 규칙이 세 가지 있습니다.

- 1 줄마다 이름 개수가 같아야 합니다. 하나라도 다르면 통째로 무시됩니다
- 2 이름은 따옴표 안에 씁니다. 따옴표를 빼면 역시 무시됩니다
- 3 합칠 칸은 반드시 붙어 있어야 하고 네모 모양이어야 합니다

비워 둘 칸은 마침표 하나(.) 를 씁니다. "head ." 처럼 씁니다. 이름이 아니라 "여기는 빈 칸" 이라는 표시입니다.

화면 파란 머리말이 824px 로 가로 전체, 그 아래 주황 옆칸 200px 와

초록 본문 614px, 맨 아래 회색 바닥글이 824px 로 놓입니다

```
머리말 (head)
옆 메뉴 (side)
본문 (main)
바닥글 (foot)
```

여기서 쓴 header, aside, main, footer 는 06단원에서 배운 뜻이 있는 태그입니다. 격자와는 상관없지만, 이름이 맞으면 코드를 읽기가 훨씬 쉬워집니다.

▣ 직접 해보기 5

.area-page 의 grid-template-areas 를 아래처럼 바꿔서 옆 메뉴를 오른쪽으로 옮겨 보세요. "head head" "main side" "foot foot" (따옴표 세 덩어리를 각각 다른 줄에 쓰면 읽기 좋습니다)

minmax 와 auto-fit

섹션 6

repeat(auto-fit, minmax(180px, 1fr)) 한 줄이면 화면 폭에 따라 칸 수가 저절로 바뀝니다. 미디어쿼리 없이도 됩니다.

두 가지를 나눠서 봅니다.

[1] minmax(최솟값, 최댓값)

칸의 크기에 "최소 이만큼, 최대 이만큼" 이라는 범위를 줍니다.

```
minmax(180px, 1fr)
```

→ 최소 180px 은 보장하고, 자리가 남으면 1fr 만큼 늘어나라

순서는 항상 작은 쪽이 먼저입니다. 반대로 쓰면 무시됩니다.

[2] repeat(auto-fit, ...)

repeat 의 첫 번째 자리에 숫자 대신 auto-fit 을 씁니다.
"몇 칸인지는 네가 알아서 정해라" 라는 뜻입니다.
브라우저는 minmax 의 최솟값을 보고 몇 칸이 들어가는지 계산합니다.

상자 안쪽이 824px, gap 이 12px, 최솟값이 180px 이라면

4칸 → $180 \times 4 + 12 \times 3 = 756 \leq 824$ 들어갑니다
5칸 → $180 \times 5 + 12 \times 4 = 948 > 824$ 안 들어갑니다

그래서 4칸이 되고, 남는 공간은 1fr 이 나눠 가져 197px 씩이 됩니다.
창을 좁히면 계산이 다시 돌아 3칸, 2칸, 1칸으로 줄어듭니다.

[3] auto-fit 과 auto-fill 의 차이

둘 다 "들어갈 수 있는 만큼 칸을 만든다" 는 점은 같습니다.
차이는 만들어 놓은 칸이 남았을 때입니다.

auto-fill 빈 칸을 그대로 둡니다 → 상자들이 왼쪽에 몰립니다
auto-fit 빈 칸을 접어 없앱니다 → 상자들이 늘어나 폭을 채웁니다

아래 두 상자에는 똑같이 칸이 2개씩만 들어 있습니다.
자리는 4개가 만들어지는데 상자가 2개뿐이니 차이가 눈에 보입니다.

auto-fill → 197px 짜리 두 칸이 왼쪽에 붙고 오른쪽 절반이 비입니다
auto-fit → 406px 짜리 두 칸이 상자를 꽉 채웁니다

어느 쪽을 쓸지는 간단합니다.
남는 자리를 채우고 싶으면 auto-fit, 칸 폭을 지키고 싶으면 auto-fill 입니다.
보통은 auto-fit 을 씁니다.

이 격자는 브라우저 창을 좁히면 스스로 줄 수를 바꿉니다. 그런데 "폰에서는 글자를 키우고 메뉴는 세로로" 같은 일까지는 못 합니다. 칸 수 말고 다른 것도 바꾸려면 @media 가 필요합니다. 개념03 에서 배웁니다.

화면 197px 짜리 칸 4개가 윗줄, 2개가 아랫줄에 놓입니다.

브라우저 창을 좁히면 3칸 → 2칸 → 1칸으로 저절로 줄어듭니다

카드 1
카드 2
카드 3
카드 4
카드 5
카드 6

화면 197px 짜리 칸 두 개가 왼쪽에 붙어 있고 오른쪽 절반이 비어 있습니다

A
B

화면 406px 짜리 칸 두 개가 상자를 좌우로 꽉 채웁니다

A
B

◀ 직접 해보기

6

브라우저 창의 오른쪽 끝을 마우스로 잡고 천천히 좁혀 보세요. 위 "카드 6개" 상자의 줄 수가 4칸 → 3칸 → 2칸 → 1칸으로 바뀝니다. 그다음 .auto-cards 의 180px 을 300px 로 바꾸고 같은 실험을 해 보세요. 칸 수가 더 빨리 줄어듭니다.

자주 하는 실수

섹션 7

아래 실수들은 전부 에러가 나지 않습니다. 그 줄이 조용히 무시되거나, 엉뚱한 자리에 칸이 놓일 뿐입니다.

이런 실수를 합니다

선 번호를 칸 번호로 셈

이런 실수를 합니다

실수 1 — 선 번호를 칸 번호로 셈

```
/* 칸 1 과 칸 2 를 합치고 싶어서 */
grid-column: 1 / 2;
```

무슨 일이 일어나나요: 칸이 안 넓어집니다. 그대로 한 칸입니다. 1번 선에서 2번 선까지는 칸 하나이기 때문입니다. 두 칸을 원하면 1 / 3 입니다. 선을 센다는 것을 잊으면 항상 하나씩 모자랍니다.

이렇게 쓰세요

```
grid-column: 1 / 3; /* 또는 */ grid-column: span 2;
```

이런 실수를 합니다

grid-column: 2 를 “두 칸” 으로 씀

이런 실수를 합니다

실수 2 — grid-column: 2 를 “두 칸” 으로 씀

```
grid-column: 2;
```

무슨 일이 일어나나요: 폭은 그대로인데 칸이 오른쪽으로 한 칸 밀립니다. 그리고 왼쪽 첫 칸이 텅 비어서 “왜 여기가 비었지?” 하게 됩니다. 숫자 하나는 시작 선 번호입니다. 개수가 아닙니다. 섹션 3 의 두 상자를 다시 비교해 보세요.

이런 실수를 합니다

areas 의 줄마다 개수가 다름

이런 실수를 합니다

실수 3 — grid-template-areas 의 줄마다 이름 개수가 다름

```
grid-template-areas:  
  "head head"  
  "side main main"  
  "foot foot";
```

무슨 일이 일어나나요: 가운데 줄만 이름이 세 개라 격자가 네모가 아니게 되고, grid-template-areas 줄이 통째로 버려집니다. 그런데 자식들은 여전히 grid-area 로 이름을 부르고 있어서 갈 곳을 못 찾습니다. 그래서 네 덩어리가 전부 격자 오른쪽 아래 구석의 좁은 자리에 겹쳐 쌓입니다. 화면이 크게 무너지니 알아채기는 쉽습니다. 줄마다 개수를 세로로 줄 맞춰 쓰면 눈으로 바로 보입니다.

이런 실수를 합니다

이름 오타

이런 실수를 합니다

실수 4 — 이름을 서로 다르게 씀

```
grid-template-areas: "head head" "side main" "foot foot";  
  
.area-side { grid-area: aside; } /* side 라고 써야 합니다 */
```

무슨 일이 일어나나요: 그 상자만 정해진 자리를 못 찾아 격자 오른쪽 아래에 새 줄과 새 칸이 생기고 거기에 혼자 놓입니다. 게다가 새 칸이 생긴 만큼 나머지 칸들의 폭도 같이 줄어듭니다. 이름이 하나만 어긋나도 화면 전체가 흔들립니다. 자리가 이상하면 먼저 이름 철자부터 맞춰 보세요.

이런 실수를 합니다

repeat에 쉼표를 안 씀

이런 실수를 합니다

실수 5 — `repeat()` 안에 쉼표를 안 씀

```
grid-template-columns: repeat(3 1fr);
```

무슨 일이 일어나나요: 아무 일도 안 일어납니다. 칸이 하나도 안 나뉘고 한 줄에 하나씩 쌓입니다. `repeat()` 괄호 안에서만 쉼표를 씁니다. 괄호 밖에서는 띄어쓰기라는 점이 헷갈리는 원인입니다.

이런 실수를 합니다

minmax 순서 반대

이런 실수를 합니다

실수 6 — `minmax()`의 순서를 반대로 씀

```
grid-template-columns: repeat(auto-fit, minmax(1fr, 180px));
```

무슨 일이 일어나나요: 역시 아무 일도 안 일어납니다. 칸이 안 나뉩니다. `minmax`는 최솟값이 먼저이고, 최솟값 자리에는 `fr`을 쓸 수 없습니다. "최소 180px, 최대 1fr" 이므로 `minmax(180px, 1fr)`입니다.

정리

▢ 직접 해보기

7

정답

1) 선은 5개가 됩니다. 칸이 4개면 선은 항상 하나 더 많습니다.

```
grid-template-columns: 1fr 1fr 1fr 1fr;
```

→ 칸 폭이 268px에서 $(824 - 30) \div 4 = 198.5\text{px}$ 로 줄고,

상자가 3개뿐이라 오른쪽 네 번째 칸은 비어 있습니다.

2) `.colspan-grid.wide`를 이렇게 바꿉니다.

```
grid-column: 1 / 4;
```

→ 주황 칸이 상자 전체(824px)를 가로지르고 C, D, E가 아랫줄로 내려갑니다.

칸이 3개인 격자의 선은 1, 2, 3, 4 네 개이고 맨 오른쪽 끝 선이 4번이므로 전체를 쓰려면 1 / 4 라고 써야 합니다.

3) .trap-grid .just2 를 이렇게 바꿉니다.

```
grid-column: span 2;
```

→ 268px 이던 주황 칸이 546px 로 넓어지고, 자리도 맨 왼쪽으로 돌아옵니다.

비어 있던 첫 칸이 사라집니다.

4) .rep-even 을 이렇게 바꿉니다.

```
grid-template-columns: repeat(2, 1fr);
```

→ 칸 네 개가 2 + 2 로 두 줄이 됩니다. 칸 폭은 $(824 - 8) \div 2 = 408\text{px}$ 입니다.

1fr 1fr 이라고 직접 쓴 것과 결과가 완전히 같습니다.
repeat() 는 짧게 쓰는 방법일 뿐, 새로운 기능이 아닙니다.

5) .area-page 를 이렇게 바꿉니다.

```
grid-template-areas:  
  "head head"  
  "main side"  
  "foot foot";
```

→ 초록 본문이 왼쪽으로, 주황 옆 메뉴가 오른쪽으로 자리를 바꿉니다.

다만 열 크기는 200px 1fr 그대로이므로 왼쪽이 200px 입니다.
본문을 넓게 두고 싶다면 grid-template-columns 도 1fr 200px 으로 바꾸세요.
HTML 은 한 글자도 안 고쳤다는 점이 중요합니다. 배치만 CSS 에서 바뀝니다.

6) 창을 좁히면 4칸 → 3칸 → 2칸 → 1칸으로 줄어듭니다. 180px 을 300px 로 바꾸면

```
grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
```

→ 폭이 824px 일 때 2칸이 됩니다. $300 \times 2 + 12 = 612$ 는 들어가지만

$300 \times 3 + 24 = 924$ 는 안 들어가기 때문입니다.
최솟값을 키울수록 칸 수가 줄고 칸 하나가 넓어집니다.

데모 칸의 공통 모양입니다. grid 와는 상관없는 색입니다.

```
.cell {
  background: #dbe7fb;
  border: 1px solid #9ab8e8;
  padding: 10px;
  text-align: center;
}
.cell-hi {
  background: #ffe0b3;
  border-color: #e8a33d;
}
```

선 번호를 눈으로 확인하는 격자

섹션 1

```
.line-grid {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 10px;
}
```

화면 268px 짜리 칸 세 개. 왼쪽 끝이 1번 선, 오른쪽 끝이 4번 선입니다

grid-column / grid-row 로 칸 합치기

섹션 2

```
.colspan-grid {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 10px;
}
.colspan-grid .wide {
  grid-column: 1 / 3;
}
```

화면 첫 칸이 1번 선에서 3번 선까지, 즉 두 칸 폭인 546px 가 됩니다

```
.rowspan-grid {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  grid-template-rows: 50px 50px;
  gap: 10px;
}
.rowspan-grid .tall {
  grid-row: 1 / 3;
}
```

화면 첫 칸이 두 줄을 차지해 높이 110px(50 + 10 + 50) 가 됩니다

span 과, 숫자 하나만 쓸 때의 함정

섹션 3

```
.span-grid {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 10px;
}
.span-grid .two {
  grid-column: span 2;
}
```

화면 1 / 3 과 똑같이 두 칸을 차지해 546px 가 됩니다

```
.trap-grid {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 10px;
}
.trap-grid .just2 {
  grid-column: 2;
}
```

화면 두 칸이 되지 않고, 2번 선에서 시작하는 한 칸(268px)이 됩니다.

맨 왼쪽 한 칸만 비고 뒤의 B 는 그 오른쪽에 그대로 들어갑니다

repeat()

섹션 4

```
.rep-even {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 8px;
}
```

화면 200px 짜리 칸 네 개가 한 줄로 놓입니다

```
.rep-mix {
  display: grid;
  grid-template-columns: 120px repeat(2, 1fr);
  gap: 10px;
}
```

화면 왼쪽 120px 고정, 나머지 두 칸이 342px 씩 나눠 가집니다

grid-template-areas

섹션 5

```
.area-page {
  display: grid;
  grid-template-columns: 200px 1fr;
  grid-template-rows: 50px 120px 40px;
  grid-template-areas:
    "head head"
    "side main"
    "foot foot";
  gap: 10px;
}

.area-head {
  grid-area: head;
  background: #cfe0ff;
  border: 1px solid #9ab8e8;
  padding: 10px;
}

.area-side {
  grid-area: side;
  background: #ffe3c2;
  border: 1px solid #e8a33d;
  padding: 10px;
}

.area-main {
  grid-area: main;
  background: #d9f0d9;
  border: 1px solid #8cc48c;
  padding: 10px;
}

.area-foot {
  grid-area: foot;
  background: #eeeeee;
  border: 1px solid #bbbbbb;
  padding: 10px;
}
```

화면 머리말 824px, 옆칸 200px, 본문 614px, 바닥글 824px 로 배치됩니다

```
.auto-cards {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
  gap: 12px;
}
```

화면 넓은 화면에서는 197px 짜리 4칸, 좁아지면 칸 수가 저절로 줄어듭니다

```
.fill-two {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(180px, 1fr));
  gap: 12px;
}
```

화면 칸이 2개뿐인데 자리는 4개라서, 두 칸이 197px 로 왼쪽에 몰립니다

```
.fit-two {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
  gap: 12px;
}
```

화면 빈 자리가 겹쳐서 두 칸이 406px 씩 상자를 꽉 채웁니다

화면 크기와 미디어쿼리

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

★ 이 파일은 창 크기를 바꿔 가며 봐야 합니다.

브라우저 창의 오른쪽 끝을 잡고 좁혔다 넓혔다 해 보세요.

섹션 3 에서 더 정확한 방법(F12 기기 툴바) 을 배웁니다.



실행하면 나오는 화면 — 화면 크기와 미디어쿼리

지금까지 만든 화면은 전부 “지금 이 창 크기” 에서만 예뻐했습니다. 그런데 같은 페이지를 보는 화면의 크기는 사람마다 다릅니다.

휴대폰	가로 약 360 ~ 430px
태블릿	가로 약 768 ~ 1024px
노트북	가로 약 1280 ~ 1600px
큰 모니터	가로 1920px 이상

가로 1200px 을 기준으로 만든 3단 화면을 폭 360px 짜리 휴대폰에서 열면 칸 하나가 100px 도 안 됩니다. 글자가 한 글자씩 세로로 쌓입니다. 반대로 휴대폰에 맞춰 1단으로 만들면 큰 모니터에서는 가운데만 쓰고 좌우가 텅 빈니다.

그래서 필요한 것이 반응형(responsive) 입니다. “화면 폭에 반응해서 배치를 바꾸는 것” 이라는 뜻입니다.

화면이 좁으면

1

2

3

화면이 넓으면

1

2

3

개념02 의 auto-fit 도 반응형의 한 방법이었습니다. 다만 칸 수만 바뀝니다. 글자 크기, 메뉴 방향, 숨길 요소까지 바꾸려면 @media 가 필요합니다.

먼저 용어 하나를 다시 짚습니다. 11단원 개념04 에서 fixed 의 기준을 이야기하며 이름만 나왔던 말입니다.

뷰포트(viewport)

브라우저 창에서 웹페이지가 실제로 그려지는 네모난 영역입니다.

주소창·즐거찾기 막대·창 테두리는 빼고, 페이지가 차지하는 부분만입니다.

"화면 폭" 이라고 하면 보통 이 뷰포트의 가로 길이를 말합니다.

이 파일은 창 크기를 바꿔 가며 봐야 합니다. 브라우저 창의 오른쪽 끝을 마우스로 잡고 좁혔다 넓혔다 해 보세요. 아래 상자들의 색과 칸 수가 바뀝니다. 섹션 3 에서 폭을 숫자로 정확히 맞추는 방법(F12 기기 툴바)을 단계별로 배웁니다.

고정 폭이 만드는 문제

섹션 1

폭을 px 로 못 박아 두면 창이 좁아져도 그대로입니다. 좁은 화면에서는 화면 밖으로 빠져나갑니다.

아래 상자에는 두 개의 카드가 들어 있습니다.

.fixed-card { width: 700px; }

폭을 700px 로 못 박았습니다

.fluid-card { }

폭을 아예 안 정했습니다

div 는 블록 요소이므로 폭을 안 정하면 부모 폭을 그대로 채웁니다(10단원). 부모가 좁아지면 같이 좁아진다는 뜻입니다. 이것이 “유연하다” 는 것입니다.

지금 이 창(폭 1280) 에서는 두 카드 다 멀쩡해 보입니다. 빨간 카드가 700px, 초록 카드가 824px 입니다. 문제가 안 보입니다.

창을 폭 480px 정도로 좁혀 보세요. 그때 차이가 드러납니다.

빨간 카드

700px 그대로 → 상자 밖으로 나가서 가로 스크롤이 생깁니다

초록 카드

404px 로 줄어듦 → 그대로 다 보입니다

빨간 카드는 일부러 점선 상자(scroll-frame) 안에 넣어 두었습니다. 그 상자에 overflow-x: auto 를 줬기 때문에(10단원) 넘치는 부분이 페이지 전체가 아니라 그 상자 안에서만 스크롤됩니다. 진짜 웹페이지에서 이런 일이 생기면 페이지 전체에 가로 스크롤 막대가 생깁니다. 휴대폰에서 이것만큼 짜증나는 것이 없습니다.

반응형의 첫걸음은 사실 아주 단순합니다. "꼭 필요한 곳이 아니면 폭을 px 로 못 박지 않는다." 이것 하나입니다.

화면 폭 1280 에서 빨간 카드 700px, 초록 카드 824px 입니다.

폭 480 으로 좁히면 빨간 카드는 700px 그대로라 점선 상자 안에 가로 스크롤 막대가 생기고, 초록 카드는 404px 로 줄어듭니다

width: 700px – 창을 좁혀도 안 줄어듭니다

폭을 안 정함 – 창을 좁히면 같이 줄어듭니다

⇒ 직접 해보기 1

브라우저 창의 오른쪽 끝을 마우스로 잡고 화면 절반 정도까지 좁혀 보세요. 두 카드 중 어느 쪽이 줄어드는지 확인하고, 다시 넓혀 놓으세요.

viewport meta

섹션 2

반응형을 만들기 전에 반드시 먼저 넣어야 하는 한 줄이 있습니다. 이것을 빠뜨리면 아래에서 배울 내용이 전부 헛일이 됩니다.

이 한 줄입니다. 이 파일의 head 안에도 이미 들어 있습니다.

```
<meta name="viewport" content="width=device-width, initial-scale=1" />
```

01단원에서 meta 태그를 배웠습니다. 브라우저에게 건네는 "쪽지" 라고 했습니다. charset 이 "이 파일은 UTF-8 로 저장했으니 UTF-8 로 읽어 주세요" 라는 쪽지였다면, 이 줄은 "이 페이지의 뷰포트를 이렇게 잡아 주세요" 라는 쪽지입니다.

name="viewport"	뷰포트에 대한 안내라는 뜻
width=device-width	뷰포트 폭을 기기의 실제 화면 폭으로 잡아라
initial-scale=1	처음 확대 배율을 1배(원래 크기)로 해라

★ 이 줄이 없으면 휴대폰에서 무슨 일이 일어나나요

휴대폰 브라우저는 옛날 PC 전용 페이지도 어떻게든 보여 줘야 했습니다. 그래서 이 줄이 없으면 이렇게 행동합니다.

- 1 실제 폭이 390px 인데도 "폭 980px 짜리 가상 화면" 이 있다고 칩니다
- 2 거기에 페이지를 다 그립니다
- 3 그림 전체를 $390 \div 980 = \text{약 } 0.4\text{배}$ 로 축소해서 화면에 붙입니다

결과는 이렇습니다.

글자가 개미만큼 작아집니다. 손가락으로 확대해야 읽힙니다
버튼이 너무 작아서 누르기 힘들습니다
@media (max-width: 600px) 이 절대 안 먹습니다
→ 브라우저는 지금 폭이 980px 이라고 믿고 있기 때문입니다

세 번째가 특히 무섭습니다. 미디어쿼리를 아무리 정성껏 써도 조건이 한 번도 맞지 않아서 "아무 일도 안 일어납니다." 코드는 완벽한데 화면만 안 바뀌는, 원인 찾기가 가장 어려운 상황입니다.

★ 왜 PC 에서는 확인이 안 되나요

PC 브라우저에는 저 가상 화면 개념이 없습니다. 그래서 이 줄을 지우고 PC 에서 창을 좁혀도 화면이 똑같습니다. "지웠는데 멀쩡한데요?" 라는 말이 여기서 나옵니다. 차이를 보려면 진짜 휴대폰이나, 섹션 3 의 기기 툴바가 필요합니다.

규칙은 간단합니다. 새 HTML 파일을 만들면 charset 바로 아래에 이 줄을 넣으세요. 고민할 것 없이 항상 같은 내용입니다.

```
<head>
<meta charset="UTF-8" />
<meta name="viewport" content="width=device-width, initial-scale=1" />
<title>내 페이지</title>
</head>
```

화면 검은 코드 상자 안에 head 예시 네 줄이 보입니다

이 줄은 휴대폰에서만 효과가 있습니다. PC 브라우저에서는 넣으나 빼나 화면이 같습니다. 그래서 더 잘 빠뜨립니다. 새 파일을 만들 때마다 charset 바로 아래에 같이 쓰는 습관을 들이세요.

▫ 직접 해보기 2

이 파일 맨 위 <head> 안에서 viewport 줄을 찾아보세요. 찾았으면 그 줄 앞뒤를 소리 내어 읽어 보고, 여러분이 앞 단원에서 만든 파일 중 하나를 열어 그 줄이 있는지 확인해 보세요. 없다면 지금 넣어 두세요.

기기 툴바

섹션 3

창을 손으로 끄는 것으로는 지금 폭이 몇 px 인지 알 수 없습니다. 크롬에 폭을 숫자로 정해 놓고 보는 기능이 있습니다. 이 단원은 이것 없이는 못 배웁니다.

기기 툴바(Device Toolbar) 는 크롬 개발자 도구의 기능입니다. 휴대폰이나 태블릿 화면인 척하고 페이지를 보여 줍니다. 폭을 숫자로 입력할 수 있어서, 미디어쿼리가 정확히 언제 바뀌는지 확인할 수 있습니다.

중요한 점: 기기 툴바를 켜면 viewport meta 도 진짜 휴대폰처럼 동작합니다. 그래서 섹션 2 에서 배운 내용을 여기서 눈으로 확인할 수 있습니다.

아래 순서대로 한 번만 해 보면 그다음부터는 손에 익습니다.

화면 여섯 단계짜리 번호 목록이 점선 상자 안에 보입니다

폭 숫자가 이상하게 나온다면 브라우저 확대 배율을 확인하세요. Ctrl + 0 을 누르면 100% 로 돌아갑니다. 확대가 걸려 있으면 미디어쿼리가 판단하는 폭도 같이 달라집니다.

◀ 직접 해보기 3

위 순서대로 기기 툴바를 켜고 폭을 375 로 맞춰 보세요. 그 상태에서 아래 섹션 4 의 띠가 무슨 색인지 확인하고, 폭을 800 으로 바꿔 다시 확인하세요.

@media (max-width)

섹션 4

@media 는 “이 조건일 때만 이 CSS 를 적용해라” 라는 뜻입니다. 조건이 안 맞으면 그 안의 규칙은 아예 없는 것이 됩니다.

문법은 이렇게 생겼습니다. 중괄호가 두 겹이라는 점이 핵심입니다.

```
@media (max-width: 600px) {
  .narrow-only {
    display: block;
  }
}
```

└ 바깥 중괄호: 조건이 맞을 때 적용할 규칙들을 담는 자루
안쪽 중괄호: 원래 쓰던 그냥 CSS 규칙 (선택자 + 선언)

읽는 법: “뷰포트 폭이 최대 600px 일 때(= 600px 이하일 때) 아래 규칙을 적용해라.”

max-width 는 “최댓값” 입니다. 조건이 걸리는 폭이 좁은 화면입니다. 처음 배울 때 이것이 정말 헷갈립니다. 이렇게 읽으면 덜 헷갈립니다.

max-width: 600px	폭이 600 을 넘지 않을 때 → 좁을 때
min-width: 600px	폭이 600 이상일 때 → 넓을 때

경계값은 포함입니다. max-width: 600px 은 폭이 정확히 600px 일 때도 맞습니다. 그래서 min-width: 600px 과 겹칩니다. 겹치면 나중에 쓴 규칙이 이깁니다(08단원). 겹치지 않게 하려면 601px 부터 잡거나, 아예 한쪽만 씁니다.

아래 상자에는 띠 두 개가 들어 있습니다. 기본 상태는 이렇습니다.

```
.wide-only { display: block; }   보임
.narrow-only { display: none; }  안 보임
```

그리고 폭이 600px 이하가 되면 이 둘이 서로 뒤바뀝니다. display 는 11단원에서 배웠습니다. none 이면 자리까지 통째로 사라집니다.

지금 화면에는 둘 중 하나만 보입니다. 창을 좁히면 다른 하나로 바뀝니다.

화면 폭 1280 에서는 파란 "넓은 화면입니다" 띠 하나만 보입니다.

폭 480 으로 좁히면 그 자리에 주황 "좁은 화면입니다" 띠가 대신 보입니다

```
넓은 화면입니다 (601px 이상)
좁은 화면입니다 (600px 이하)
```

▣ 직접 해보기 4

@media (max-width: 600px) 의 600 을 1000 으로 바꿔 보세요. 창을 좁히지 않아도 주황 띠로 바뀝니다. 왜 그런지 설명해 보고, 확인했으면 600 으로 되돌리세요.

@media (min-width) 와 중단점

섹션 5

min-width 는 반대입니다. "이 폭 이상일 때" 적용합니다. 배치가 바뀌는 기준 폭을 중단점(breakpoint) 이라고 부릅니다.

아래 상자에는 이렇게 썼습니다.

```
.mq-min { background: #eeeeee; }           기본은 회색

@media (min-width: 700px) {
  .mq-min { background: #d9f0d9; }         700px 이상이면 초록
}
```

기본값을 먼저 쓰고, 조건이 맞을 때 덮어쓰는 구조입니다. 조건이 안 맞으면 회색인 채로 남습니다. "없는 셈" 이 되기 때문입니다.

★ 중단점은 어디에 잡나요

정해진 답은 없습니다. 다만 기기 크기가 몰려 있는 자리가 있습니다.

360px	작은 휴대폰
390px	요즘 흔한 휴대폰
480px	큰 휴대폰 / 작은 화면의 경계로 많이 씁니다
768px	태블릿 세로. 가장 널리 쓰이는 중단점입니다
1024px	태블릿 가로, 작은 노트북
1280px	보통 노트북과 데스크톱

실무에서는 보통 두세 개면 충분합니다. 예를 들어 600px 과 900px 두 개만 잡아도 휴대폰·태블릿·데스크톱 세 가지를 다룰 수 있습니다.

중단점을 기기 이름으로 정하려 하지 마세요. 기기는 매년 새로 나옵니다. “이 폭부터 화면이 못생겨진다” 는 자리를 직접 눈으로 찾아 거기에 잡는 것이 훨씬 좋은 방법입니다. 창을 천천히 좁히면서 어색해지는 순간을 보면 됩니다.

화면 폭 1280 에서 연초록 상자입니다. 폭 480 으로 좁히면 회색으로 바뀝니다

min-width: 700px 조건이 맞으면 초록, 아니면 회색입니다

화면 네 줄짜리 중단점 표가 점선 상자 안에 그려집니다

▸ 직접 해보기

5

.mq-min 의 조건을 @media (max-width: 700px) 로 바꿔 보세요. 색이 언제 초록이 되는지가 정확히 반대로 뒤집힙니다. 확인했으면 되돌리세요.

미디어쿼리 + grid

섹션 6

13단원의 두 축이 여기서 만납니다. grid 로 칸을 만들고, @media 로 칸 수를 바꿉니다.

아래 상자에는 이렇게 썼습니다. 위에서부터 순서대로 읽으면 됩니다.

```
.mq-cards {
  display: grid;
  grid-template-columns: 1fr;      기본은 1칸
  gap: 10px;
}

@media (min-width: 600px) {
  .mq-cards { grid-template-columns: 1fr 1fr; }      600 이상이면 2칸
}

@media (min-width: 900px) {
  .mq-cards { grid-template-columns: 1fr 1fr 1fr; }  900 이상이면 3칸
}
```

조건이 겹치는 구간이 있습니다. 폭이 1000px 이면 600 조건도 맞고 900 조건도 맞습니다. 이럴 때는 나중에 쓴 규칙이 이깁니다(08단원에서 배운 순서 규칙). 그래서 900px 짜리를 아래에 써야 3칸이 됩니다. 순서를 바꾸면 영영 2칸입니다.

꼭마다 실제 칸 크기를 재 보면 이렇습니다.

폭 1280	상자 안쪽 824px	→ 268px	짜리 3칸
폭 768	상자 안쪽 692px	→ 341px	짜리 2칸
폭 480	상자 안쪽 404px	→ 404px	짜리 1칸

★ 개념02 의 auto-fit 과 무엇이 다른가요

auto-fit	칸 수만 자동으로 바뀝니다. 짧고 편합니다
@media	무엇이든 바꿀 수 있습니다. 글자 크기, 메뉴 방향, 숨기기까지

카드 목록처럼 칸 수만 바뀌면 되는 곳은 auto-fit 이 훨씬 짧습니다. “좁을 때는 옆 메뉴를 숨긴다” 같은 일은 @media 로만 됩니다. 둘은 경쟁 관계가 아니라 같이 씁니다.

화면 폭 1280 에서 268px 짜리 3칸이 두 줄로 놓입니다.

폭 768 에서는 341px 짜리 2칸, 폭 480 에서는 404px 짜리 1칸이 됩니다

카드 1
카드 2
카드 3
카드 4
카드 5
카드 6

◁ 직접 해보기 6

기기 톨바에서 폭을 599 로 맞췄다가 600 으로 한 칸 올려 보세요. 1px 차이로 배치가 바뀝니다. 이 순간이 바로 중단점입니다.

자주 하는 실수 섹션 7

미디어쿼리의 실수는 대부분 “아무 일도 안 일어난다” 로 나타납니다. 조건이 한 번도 맞지 않으면 그 안의 CSS 는 없는 것과 같기 때문입니다.

이런 실수를 합니다

viewport meta 를 빠뜨림

이런 실수를 합니다

실수 1 — viewport meta 를 빠뜨림

```
<head>
<meta charset="UTF-8" />
<title>내 페이지</title>
</head>
```

무슨 일이 일어나나요: PC 에서는 멀쩡합니다. 그런데 휴대폰에서 열면 글자가 개미만 하게 축소되어 보이고, 정성껏 쓴 미디어쿼리가 하나도 안 먹습니다. 브라우저가 화면 폭을 980px 이라고 믿고 있기 때문입니다. 이 단원에서 가장 비싼 실수입니다.

이렇게 쓰세요

```
<meta name="viewport" content="width=device-width, initial-scale=1" />
```

이런 실수를 합니다

min-width 와 max-width 를 반대로

이런 실수를 합니다

실수 2 — min-width 와 max-width 를 반대로 씀

```
/* 휴대폰에서 1칸으로 만들려고 */
@media (min-width: 600px) {
  .cards { grid-template-columns: 1fr; }
}
```

무슨 일이 일어나나요: 정확히 반대로 동작합니다. 넓은 화면에서 1칸이 되고 휴대폰에서는 그대로입니다. min-width 는 “이 값 이상”, max-width 는 “이 값 이하” 입니다. 이름의 min/max 가 화면 크기가 아니라 조건의 경계값을 가리킨다고 생각하면 덜 헛갈립니다.

이런 실수를 합니다

선택자를 안 감쌌

이런 실수를 합니다

실수 3 — @media 안에 선택자를 안 씀

```
@media (max-width: 600px) {
  grid-template-columns: 1fr;
}
```

무슨 일이 일어나나요: 아무 일도 안 일어납니다. @media 안에 규칙 전체가 들어가야 합니다. 즉 선택자와 중괄호를 한 번 더 써야 합니다. 선언만 덜렁 쓰면 무시됩니다. “중괄호가 두 겹” 이라는 것을 기억하세요.

이렇게 쓰세요

```
@media (max-width: 600px) {  
  .cards {  
    grid-template-columns: 1fr;  
  }  
}
```

이런 실수를 합니다

단위 누락

이런 실수를 합니다

실수 4 — 조건에 단위를 안 씀

```
@media (max-width: 600) {  
  .cards { grid-template-columns: 1fr; }  
}
```

무슨 일이 일어나나요: 조건이 절대 맞지 않아 그 안의 CSS 가 한 번도 적용되지 않습니다. 600 이 아니라 600px 이어야 합니다. CSS 는 0 말고는 단위를 생략할 수 없습니다.

이런 실수를 합니다

순서를 거꾸로

이런 실수를 합니다

실수 5 — @media 를 기본 규칙보다 위에 씀

```
@media (min-width: 900px) {  
  .cards { grid-template-columns: 1fr 1fr 1fr; }  
}  
  
.cards { grid-template-columns: 1fr; } /* 나중에 쓴 이쪽이 이깁니다 */
```

무슨 일이 일어나나요: 넓은 화면에서도 계속 1칸입니다. 명시도가 같으면 나중에 쓴 규칙이 이깁니다 (08단원). @media 는 명시도를 올려 주지 않으므로 순서에서 밀리면 그냥 집니다. 기본 규칙을 먼저, 미디어쿼리를 아래에 쓰세요.

이런 실수를 합니다

명시도에서 짐

이런 실수를 합니다

실수 6 — 미디어쿼리 안의 규칙이 명시도에서 짐

```
#cardList { grid-template-columns: 1fr 1fr 1fr; } /* 1-0-0 */

@media (max-width: 600px) {
  .cards { grid-template-columns: 1fr; } /* 0-1-0 */
}
```

무슨 일이 일어나나요: 좁은 화면에서도 3칸 그대로입니다. @media 안에 있다고 해서 더 세지지 않습니다. id 선택자가 클래스 선택자를 이기는 것은 그대로입니다(08단원). 안에서든 밖에서든 같은 선택자를 쓰는 것이 가장 안전합니다.

정리

▢ 직접 해보기 7

정답

1) 창을 절반 정도(약 640px) 로 좁히면

→ 초록 카드만 줄어들어 약 564px 이 되고, 빨간 카드는 700px 그대로입니다.

점선 상자보다 빨간 카드가 넓어졌으므로 그 상자 안쪽에 가로 스크롤 막대가 생깁니다. 폭을 안 정한 쪽만 화면을 따라간다는 것을 눈으로 확인하는 것이 목적입니다.

2) 이 파일 head 의 두 번째 줄에 있습니다.

```
<meta name="viewport" content="width=device-width, initial-scale=1" />
```

→ 앞 단원에서 만든 파일에 이 줄이 없다면 charset 줄 바로 아래에 넣으세요.

PC 화면은 넣기 전과 똑같습니다. 휴대폰에서만 달라집니다.

3) 기기 톨바에서 폭을 375 로 맞추면

→ 섹션 4 의 띠가 주황색("좁은 화면입니다") 으로 바뀝니다.

375 는 600 이하이므로 max-width: 600px 조건이 맞기 때문입니다. 폭을 800 으로 바꾸면 다시 파란 띠("넓은 화면입니다") 가 됩니다.

4) style 안의 조건을 이렇게 바꿉니다.

```
@media (max-width: 1000px) {
```

→ 지금 창 폭이 1000px 보다 좁다면 창을 좁히지 않아도 바로 주황 띠가 됩니다.

조건외 기준선을 옮겼으니 지금 폭이 그 안에 들어온 것입니다.
미디어쿼리는 "화면이 작아졌는지" 가 아니라 "지금 폭이 조건에 맞는지" 만 봅니다.
확인했으면 600 으로 되돌리세요.

5) .mq-min 의 조건을 이렇게 바꿉니다.

```
@media (max-width: 700px) {
```

→ 폭 1280 인 지금은 회색이 되고, 창을 700px 이하로 좁혀야 초록이 됩니다.

색이 바뀌는 방향이 정확히 반대로 뒤집힙니다.

6) 기기 톨바에서 폭을 599 로 맞추면 섹션 6 의 카드가 1칸이고, 600 으로 올리는 순간 2칸이 됩니다.

→ min-width: 600px 은 600 을 포함하기 때문입니다.

1px 차이로 배치가 갈리는 이 지점이 중단점입니다.
같은 방법으로 899 와 900 도 확인해 보세요. 2칸에서 3칸으로 바뀝니다.

고정 폭과 유연한 폭

섹션 1

```
.scroll-frame {  
  overflow-x: auto;  
  border: 1px dashed #bbb;  
  padding: 6px;  
}
```

화면 안의 내용이 넘치면 이 상자 안에만 가로 스크롤 막대가 생깁니다

```
.fixed-card {  
  width: 700px;  
  box-sizing: border-box;  
  background: #ffe0e0;  
  border: 1px solid #e0a0a0;  
  padding: 10px;  
}
```

화면 창을 아무리 좁혀도 폭이 700px 에서 안 변합니다

```
.fluid-card {  
  background: #e0f2e0;  
  border: 1px solid #8cc48c;  
  padding: 10px;  
}
```

화면 폭을 안 정했으므로 부모 폭을 그대로 채웁니다 (1280 에서 824px)

@media (max-width)

섹션 4

```
.size-badge {
  border: 2px solid #333;
  padding: 10px;
  text-align: center;
  font-weight: bold;
}
.wide-only {
  display: block;
  background: #cfe0ff;
}
.narrow-only {
  display: none;
  background: #ffd9a8;
}
@media (max-width: 600px) {
  .wide-only {
    display: none;
  }
  .narrow-only {
    display: block;
  }
}
```

화면 창이 601px 이상이면 파란 띠, 600px 이하면 주황 띠 하나만 보입니다

@media (min-width)

섹션 5

```
.mq-min {
  background: #eeeeee;
  border: 1px solid #bbbbbb;
  padding: 10px;
}
@media (min-width: 700px) {
  .mq-min {
    background: #d9f0d9;
    border-color: #8cc48c;
  }
}
```

화면 창이 700px 이상이면 초록, 그 미만이면 회색입니다

```
.mq-cards {
  display: grid;
  grid-template-columns: 1fr;
  gap: 10px;
}
.mq-cards div {
  background: #dbe7fb;
  border: 1px solid #9ab8e8;
  padding: 10px;
  text-align: center;
}
@media (min-width: 600px) {
  .mq-cards {
    grid-template-columns: 1fr 1fr;
  }
}
@media (min-width: 900px) {
  .mq-cards {
    grid-template-columns: 1fr 1fr 1fr;
  }
}
```

화면 1280 에서 268px 3칸, 768 에서 341px 2칸, 480 에서 404px 1칸입니다

반응형 단위와 이미지

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

F12 → Ctrl+Shift+M 으로 폭을 480, 768, 1280 으로 바꿔 가며 보세요.

(기기 툴바 쓰는 법은 개념03 섹션 3 에 있습니다)



실행하면 나오는 화면 — 반응형 단위와 이미지

개념03 에서 @media 로 배치를 바꾸는 법을 배웠습니다. 그런데 미디어쿼리만으로 반응형을 만들려고 하면 금방 지칩니다. 중단점을 아무리 많이 잡아도 그 사이 폭에서는 여전히 어색하기 때문입니다.

px 로만 만든 화면 + 미디어쿼리

360 600 900

중단점마다 계단처럼 툭툭 끊깁니다

유연한 단위로 만든 화면

모든 폭에서 부드럽게 늘고 줄어듭니다

그래서 순서가 이렇습니다.

1 먼저 유연한 값(% , fr, vw, max-width) 으로 만듭니다

2 그래도 못생겨지는 자리에만 @media 를 씁니다

이 파일에서는 1번에 쓰는 값들을 배웁니다.

```
%          부모를 기준으로 한 비율
vw / vh    화면(뷰포트) 을 기준으로 한 비율
max-width  "이보다 커지지는 마라" 는 상한선
min max clamp  여러 값 중에 골라 쓰는 함수
object-fit 사진을 액자에 어떻게 맞추지
```

마음가짐 하나만 먼저 정하고 시작합니다. "폭은 되도록 정하지 않는다. 정해야 한다면 상한선만 정한다."

이 파일은 고정된 px 대신 쓸 값들을 모아 놓은 곳입니다. %, vw, max-width: 100%, clamp(), object-fit 을 차례로 봅니다. 폭을 바꿔 가며 봐야 하므로 F12 → Ctrl+Shift+M 을 켜 두세요.

% 는 부모 기준

섹션 1

width: 50% 는 화면의 절반이 아닙니다. 부모 요소 안쪽 폭의 절반입니다. 이 차이가 나중에 큰 혼란을 만듭니다.

아래 회색 상자가 부모입니다. 안쪽 폭이 400px 입니다. 그 안에 자식 둘을 넣었습니다.

```
.pct-half    { width: 50%; }    → 400 × 0.5 = 200px
.pct-quarter { width: 25%; }    → 400 × 0.25 = 100px
```

기준이 되는 400 은 화면 폭도 아니고 상자 바깥 폭도 아닙니다. 부모의 콘텐츠 영역(테두리와 안쪽 여백을 뺀 부분) 폭입니다. 10단원 박스 모델입니다.

부모가 바뀌면 자식도 같이 바뀝니다. 그래서 % 는 저절로 반응형이 됩니다. 창을 좁혀 부모가 300px 이 되면 절반짜리 자식은 150px 이 됩니다. 아무 계산도 미디어쿼리도 필요 없습니다.

★ 여기서 회색 부모에는 width 가 아니라 max-width 를 썼습니다.

```
.pct-parent { max-width: 400px; }
```

"400px 을 넘지는 마라" 는 뜻입니다. 자리가 400px 보다 좁으면 그만큼만 씁니다. width: 400px 이었다면 폭 360px 짜리 휴대폰에서 화면 밖으로 나갔을 것입니다. max-width 는 10단원에서 배운 상한선 속성입니다. 섹션 3 에서 이것을 이미지에 쓰는 법을 봅니다.

★ % 가 안 먹는 경우

부모의 폭이 정해져 있지 않으면 % 계산이 안 됩니다.
 폭은 대부분 저절로 정해지니 별 문제가 없지만, 높이는 다릅니다.
 height: 100% 를 썼는데 아무 일도 안 일어난다면 거의 항상 이 경우입니다.
 "부모 높이가 얼마인지 모르는데 그 100% 를 어떻게 계산하나" 인 것입니다.

화면 회색 상자 안에 파란 띠(200px) 와 그보다 짧은 주황 띠(100px) 가

위아래로 놓입니다

width: 50% - 200px
 25%

▫ 직접 해보기 1

.pct-parent 의 max-width 를 200px 으로 바꿔 보세요. 자식 둘의 폭이 각각 몇 px 이 될지 먼저 계산해 보고 나서 확인하세요. 확인했으면 400px 으로 되돌리세요.

vw 와 vh

섹션 2

vw 는 뷰포트 폭의 100분의 1, vh 는 뷰포트 높이의 100분의 1 입니다. 부모가 무엇이든 상관하지 않습니다.
 이름부터 봅시다.

vw = viewport width 1vw = 화면 가로 1%
 vh = viewport height 1vh = 화면 세로 1%

그러니 이렇게 됩니다.

폭 1280 인 창에서 50vw = 640px, 25vw = 320px
 폭 480 인 창에서 50vw = 240px, 25vw = 120px
 높이 900 인 창에서 20vh = 180px

% 와 vw 는 둘 다 "비율" 이지만 기준이 완전히 다릅니다.

%	부모 기준	부모가 좁으면 같이 좁아집니다
vw	화면 기준	부모가 아무리 좁아도 화면 크기만 봅니다

아래 두 번째 상자에서 이 차이를 눈으로 볼 수 있습니다. 회색 부모(안쪽 400px) 안에 50% 와 50vw 를 나란히 넣었습니다.

파란 띠 (50%)	200px	부모 안에 얹전히 들어갑니다
빨간 띠 (50vw)	640px	부모를 뚫고 오른쪽으로 빠져나갑니다

vw 가 잘못된 단위라는 뜻이 아닙니다. 기준이 다를 뿐입니다. 화면 전체를 덮는 큰 제목이나 배너처럼 진짜로 화면 크기를 따라야 하는 곳에서는 vw 가 정답입니다. 다만 상자 안의 폭을 정할 때 vw 를 쓰면 대개

사고가 납니다.

★ 100vw 를 조심하세요

100vw 는 “화면 가로 전체” 입니다. 딱 맞을 것 같지만 거의 항상 넘칩니다. 이유가 두 가지입니다.

1) 그 상자가 화면 맨 왼쪽에서 시작하지 않습니다.

- 부모의 여백만큼 오른쪽에서 시작해 놓고 화면 전체 폭을 차지하니 그 여백만큼 오른쪽으로 넘칩니다.
- 2) 세로 스크롤 막대가 있으면 100vw 는 그 막대 폭까지 포함합니다. 실제로 쓸 수 있는 폭보다 15px 쯤 넓습니다.

가로 스크롤 막대가 생기는 원인 1위가 바로 이 100vw 입니다. 가로로 꽉 채우고 싶으면 그냥 폭을 안정하거나 width: 100% 를 쓰세요.

화면 폭 1280 · 높이 900 인 창에서 초록 띠가 640px, 그 아래가 320px,

보라 상자의 높이가 180px 입니다

```
width: 50vw
25vw
height: 20vh
```

화면 회색 부모 안에서 파란 띠는 부모 안에 들어가고,

빨간 띠는 부모 오른쪽 테두리를 뚫고 나가 있습니다

50% – 부모의 절반 (200px)
50vw – 화면의 절반 (640px)

vh 는 브라우저 창 높이를 기준으로 하므로 창 크기를 세로로 바꿔도 값이 달라집니다. 위 보라 상자의 180px 은 창 높이가 900px 일 때의 값입니다.

◦ 직접 해보기 2

.vw-half 를 width: 100vw; 로 바꾸고 저장한 뒤, 페이지 맨 아래를 보세요. 가로 스크롤 막대가 생깁니다. 왜 화면 폭인데도 넘치는지 위 설명에서 찾아 읽고, 확인했으면 50vw 로 되돌리세요.

이미지가 넘치지 않게

섹션 3

이미지는 원래 크기대로 그려집니다. 부모보다 커도 봐주지 않습니다. 반응형에서 가로 스크롤이 생기는 원인 대부분이 사진입니다.

아래 두 액자는 똑같이 안쪽 폭이 320px 입니다. 그 안에 가로 400px 짜리 같은 그림을 넣었습니다.

위 · CSS 를 아무것도 안 준 img

→ 400px 그대로 그려져서 액자 오른쪽으로 80px 빠져나갑니다

아래 · `.photo-good { max-width: 100%; }`

→ 320px 으로 줄어들어 액자 안에 딱 들어갑니다

`max-width` 는 “최대 이만큼” 이라는 상한선입니다. 10단원에서 배웠습니다. 100% 는 부모 안쪽 폭입니다. 둘을 합치면 이런 뜻이 됩니다.

“부모보다 커지지는 마라. 부모보다 작으면 원래 크기 그대로 있어라.”

`width: 100%` 와 헷갈리기 쉽습니다. 차이가 있습니다.

<code>width: 100%</code>	부모보다 작은 그림도 억지로 늘립니다 → 흐릿해집니다
<code>max-width: 100%</code>	작은 그림은 그대로 두고, 큰 것만 줄입니다 → 안전합니다

그래서 사진에는 거의 항상 `max-width: 100%` 를 씁니다.

★ 높이는 건드리지 마세요

위 액자의 그림은 320×160 이 되었습니다. 원래 400×200 이었으니 가로세로 비율이 그대로 유지되었습니다. `height` 를 아예 안 정했기 때문입니다. 브라우저가 가로에 맞춰 세로를 알아서 계산해 준 것입니다.

<code>max-width: 100%;</code>	→ 비율 유지. 안전합니다
<code>max-width: 100%; height: 200px;</code>	→ 가로만 줄고 세로는 200 고정 → 찌그러집니다

비율을 유지한 채로 액자를 꽉 채우고 싶다면 섹션 5 의 `object-fit` 을 씁니다.

화면 위 액자에서는 그림이 400×200 그대로라 오른쪽 테두리를 뚫고 나갑니다.

아래 액자에서는 320×160 으로 줄어 액자 안에 들어갑니다

위쪽 액자만 `.spill-frame` 이라는 껍데기로 한 번 더 감싸고 `overflow-x: auto` 를 걸어 두었습니다(10단원). 그렇게 하지 않으면 창을 폭 480px 쯤까지 좁혔을 때 이 자료 페이지 전체에 가로 스크롤이 생기기 때문입니다. 여러분이 만드는 페이지에서 이 일이 벌어지면 감싸서 가리지 말고 아래처럼 사진 자체를 고치세요.

새 프로젝트를 시작할 때 CSS 맨 위에 `img { max-width: 100%; }` 한 줄을 미리 넣어 두는 사람이 많습니다. 사진 때문에 화면이 깨지는 일을 통째로 막아 주기 때문입니다.

▹ 직접 해보기

3

.photo-good 에 `height: 240px;` 를 한 줄 더 넣어 보세요. 그림이 어떻게 되는지 확인하고 지우세요.
“높이를 건드리면 안 된다” 를 눈으로 확인하는 것이 목적입니다.

min max clamp

섹션 4

값을 하나만 쓰는 대신 여러 값 중에서 고르게 할 수 있습니다. 이름은 무섭지만 하는 일은 아주 단순합니다.

세 함수의 뜻은 이름 그대로입니다.

<code>min(a, b)</code>	a 와 b 중 작은 쪽을 씁니다
<code>max(a, b)</code>	a 와 b 중 큰 쪽을 씁니다
<code>clamp(최소, 기준, 최대)</code>	기준을 쓰되 최소와 최대를 벗어나지 않게 합니다

★ `min()` 이 “최소” 가 아니라는 점을 조심하세요

`min(500px, 100%)` 는 “최소 500px” 이 아닙니다. “둘 중 작은 쪽” 입니다.

폭이 넓을 때	<code>100% = 824px, 500px</code>	→ 작은 쪽인 <code>500px</code> 을 씁니다
폭이 좁을 때	<code>100% = 404px, 500px</code>	→ 작은 쪽인 <code>404px</code> 을 씁니다

결과적으로 “최대 500px 까지만 커지고 좁아지면 따라 줄어드는” 상자가 됩니다. `max-width: 500px` 과 사실상 같은 일을 한 줄로 한 것입니다.

★ `max()` 는 바닥을 깔아 줍니다

`max(200px, 30%)` 는 “둘 중 큰 쪽” 이므로 아무리 좁아져도 200px 아래로는 안 내려갑니다. 너무 작아져서 글자가 세로로 쌓이는 것을 막을 때 씁니다.

폭 1280	→ 30% 는 247px, 200px 보다 크므로 247px
폭 480	→ 30% 는 121px, 200px 보다 작으므로 200px

★ `clamp()` 는 위 둘을 합친 것입니다

```
clamp(200px, 50%, 380px)
```

기본은 50% 를 쓰되, 200px 보다 작아지지 말고 380px 보다 커지지도 마라.

폭 1280 → 50% 는 640px → 상한 380px 에 걸립니다

폭 768 → 50% 는 384px → 그대로 384px

폭 480 → 50% 는 240px → 그대로 240px

순서는 항상 작은 것부터입니다. 최소, 기준, 최대. 헷갈리면 이렇게 읽으세요. “적어도 이만큼, 보통은 이만큼, 많아야 이만큼.”

★ 글자 크기에 쓰면 특히 좋습니다

```
font-size: clamp(16px, 2.5vw, 28px);
```

화면이 넓어지면 제목이 커지고 좁아지면 작아지는데, 16px 아래로는 안 내려가서 읽을 수 있고 28px 위로는 안 올라가서 안 깨집니다. 미디어쿼리를 세 개 쓸 일을 한 줄로 끝냅니다.

화면 폭 1280 에서 파란 띠 500px, 주황 띠 247px, 초록 띠 380px 이고

맨 아래 글자는 28px 입니다. 폭 480 에서는 404px, 200px, 202px 이 되고 글자는 16px 이 됩니다

```
width: min(500px, 100%)
width: max(200px, 30%)
width: clamp(200px, 50%, 380px)
font-size: clamp(16px, 2.5vw, 28px)
```

▸ 직접 해보기

4

기기 톨바로 폭을 1280 → 768 → 480 으로 바꾸면서 네 줄이 각각 어떻게 변하는지 보세요. 그다음 .fn-clamp 의 최댓값 380px 을 600px 으로 바꾸면 폭 1280 에서 몇 px 이 될지 계산하고 확인하세요.

object-fit

섹션 5

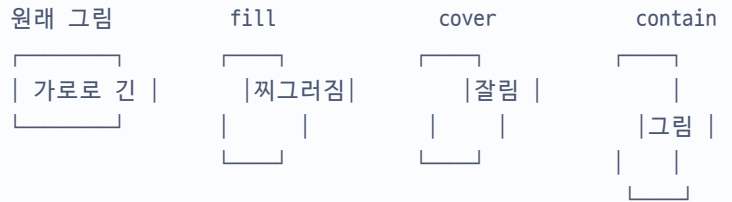
크기가 제각각인 사진들을 같은 크기 액자에 예쁘게 넣는 방법입니다. 카드 목록을 만들 때 반드시 필요합니다.

카드 목록을 만들면 사진 크기가 제각각이라 줄이 안 맞습니다. 그래서 액자 크기를 못 박습니다.

```
.thumb { width: 200px; height: 200px; }
```

여기 400 × 200 짜리 가로로 긴 그림을 넣으면 어떻게 될까요? 아무것도 안 주면 그림이 액자 모양에 맞춰 억지로 늘어납니다. 찌그러집니다. object-fit 은 “찌그러뜨리지 말고 이렇게 맞춰라” 를 정하는 속성입니다.

fill (기본값) 액자 모양대로 잡아 늘립니다 → 찌그러집니다
 cover 비율을 지킨 채 액자를 꽉 채웁니다 → 넘치는 부분이 잘립니다
 contain 비율을 지킨 채 전부 보이게 넣습니다 → 남은 자리에 여백이 생깁니다



어느 것을 쓸까요.

사진 목록, 카드 썸네일 → cover. 줄이 딱 맞아 보기 좋습니다
 로고, 도표, 잘리면 안 되는 것 → contain. 다 보여야 하니까요
 fill 은 거의 안 씁니다. 사람 얼굴이 찌그러집니다

아래 세 액자는 크기가 200 × 200 으로 모두 같고, 같은 그림이 들어 있습니다. 차이는 object-fit 값뿐입니다.

화면 세 액자 모두 200 × 200 정사각형입니다.

왼쪽은 그림이 세로로 늘어나 해가 동그라미가 아니라 긴 타원이 되었고, 가운데는 그림이 원래 비율을 지킨 채 높이에 맞춰지고 넘치는 좌우가 잘려 해가 아예 안 보이고, 오른쪽은 그림 전체(200 × 100) 가 다 보이고 위아래에 흰 여백이 있습니다

object-fit 은 액자 크기가 정해져 있을 때만 뜻이 있습니다. width 와 height 를 둘 다 정하지 않으면 맞춤 액자가 없어서 아무 일도 안 일어납니다.

▸ 직접 해보기 5

.thumb 의 height 를 120px 으로 바꿔 보세요. 세 액자가 가로로 납작해집니다. 그 상태에서 cover 와 contain 이 어떻게 달라지는지 다시 비교해 보세요.

자주 하는 실수

섹션 6

여기 실수들은 대부분 넓은 화면에서는 안 보입니다. 폭을 좁혀 봐야 드러납니다. 그래서 기기 톨바를 켜 놓는 습관이 중요합니다.

이런 실수를 합니다

100vw

이런 실수를 합니다

실수 1 — 가로로 꽉 채우려고 100vw 를 씀

```
.banner {
  width: 100vw;
}
```

무슨 일이 일어나나요: 페이지 아래에 가로 스크롤 막대가 생깁니다. 좌우로 흔들리는 페이지가 됩니다. 그 상자가 화면 맨 왼쪽에서 시작하지 않는데 화면 전체 폭을 차지하기 때문이고, 세로 스크롤 막대가 있으면 그 폭(약 15px)까지 더해지기 때문입니다. 가로로 꽉 채우고 싶으면 폭을 아예 안 정하거나 width: 100% 를 쓰세요.

이런 실수를 합니다

이미지에 max-width 를 안 줌

이런 실수를 합니다

실수 2 — 사진에 max-width: 100% 를 안 줌

```

/* CSS 없음 */
```

무슨 일이 일어나나요: PC 에서는 멀쩡한데 휴대폰에서 열면 사진 한 장 때문에 페이지 전체가 옆으로 밀립니다. 요즘 사진은 가로가 3000px 이 넘습니다. 폭 360px 화면에서 그대로 그려집니다. `img { max-width: 100%; }` 한 줄로 막습니다.

이런 실수를 합니다

height 를 같이 못 박음

이런 실수를 합니다

실수 3 — max-width 와 함께 height 를 px 로 못 박음

```
img {
  max-width: 100%;
  height: 180px;
}
```

무슨 일이 일어나나요: 창을 좁힐수록 사진이 가로로만 눌러서 찌그러집니다. 가로는 따라 줄어드는데 세로는 180px 에 묶여 있기 때문입니다. 비율을 지키려면 height 를 아예 안 쓰거나 object-fit: cover 를 같이 쓰세요.

이런 실수를 합니다

% 를 화면 기준으로 착각

이런 실수를 합니다

실수 4 — % 를 화면 기준으로 착각함

```
/* 화면의 절반으로 만들고 싶어서 */  
.side {  
  width: 50%;  
}
```

무슨 일이 일어나나요: 생각보다 훨씬 작게 나옵니다. % 는 부모 기준이라, 부모가 이미 400px 이면 50% 는 200px 입니다. 화면이 1280px 이어도 640px 이 되지 않습니다. 진짜로 화면 기준이 필요하다면 vw 를 씁니다.

이런 실수를 합니다

height: 100% 가 안 먹음

이런 실수를 합니다

실수 5 — height: 100% 를 썼는데 아무 일도 안 일어남

```
.box {  
  height: 100%;  
}
```

무슨 일이 일어나나요: 높이가 안 바뀝니다. 부모의 높이가 정해져 있지 않으면 “그 100%” 를 계산할 수가 없어서 브라우저가 이 줄을 무시합니다. 폭은 저절로 정해지지만 높이는 아닙니다. 화면 높이를 채우고 싶다면 height: 100vh 를 쓰세요.

이런 실수를 합니다

clamp 순서

이런 실수를 합니다

실수 6 — clamp() 의 순서를 헷갈림

```
font-size: clamp(28px, 2.5vw, 16px);
```

무슨 일이 일어나나요: 글자가 항상 28px 로만 나옵니다. 최소가 28px 이고 최대가 16px 이라 말이 안 되는 조건이 되어, 브라우저가 최솟값을 그대로 씁니다. 순서는 최소, 기준, 최대 입니다. “적어도 이만큼, 보통은 이만큼, 많아야 이만큼” 으로 읽으세요.

정리

▫ 직접 해 보기 6

정답

1) .pct-parent 를 이렇게 바꿉니다.

```
max-width: 200px;
```

→ 부모 안쪽이 200px 이 되므로 파란 띠는 100px, 주황 띠는 50px 이 됩니다.

% 는 부모 안쪽 폭을 기준으로 하니 부모가 절반이 되면 자식도 절반이 됩니다.
자식의 CSS 는 한 글자도 안 고쳤다는 점이 중요합니다.

2) .vw-half 를 이렇게 바꿉니다.

```
width: 100vw;
```

→ 초록 띠가 화면 폭만큼 넓어지면서 페이지 맨 아래에 가로 스크롤 막대가 생깁니다.

이 띠는 왼쪽에서 228px 떨어진 자리에서 시작하는데 폭은 화면 전체(1280px)
이므로, 오른쪽으로 228px 만큼 넘칩니다.
여기에 세로 스크롤 막대 폭까지 더해집니다.
확인했으면 50vw 로 되돌리세요.

3) .photo-good 에 이렇게 한 줄을 더합니다.

```
height: 240px;
```

→ 가로는 320px 인데 세로가 240px 이 되어 그림이 세로로 늘어납니다.

원래 400 × 200 이던 그림이 320 × 240 이 되었으니 비율이 깨진 것입니다.
산이 위로 길쭉하고 뾰족해지고, 해도 동그라미가 아니라
세로로 긴 타원이 됩니다. 확인했으면 지우세요.

4) 폭을 바꾸면 이렇게 변합니다.

폭 1280	500px / 247px / 380px / 글자 28px
폭 768	500px / 208px / 346px / 글자 19.2px
폭 480	404px / 200px / 202px / 글자 16px

.fn-clamp 의 최댓값을 600px 으로 바꾸면

```
width: clamp(200px, 50%, 600px);
```

→ 폭 1280 에서 50% 는 640px 입니다. 최댓값 600px 에 안 걸리므로

412px 이 그대로 쓰입니다. 380px 이던 띠가 412px 로 넓어집니다.

5) .thumb 를 이렇게 바꿉니다.

```
height: 120px;
```

→ 액자가 200 × 120 으로 납작해집니다.

왼쪽(fill) 은 400 × 200 을 200 × 120 에 맞추므로 가로는 0.5배,
세로는 0.6배가 됩니다. 세로로 20% 늘어난 정도라 아까보다 훨씬 덜 찌그러집니다.
가운데(cover) 는 240 × 120 으로 그린 뒤 좌우를 20px 씩만 자릅니다.
오른쪽(contain) 은 200 × 100 이 되어 위아래 흰 여백이 10px 씩으로 얇아집니다.
액자 비율이 그림 비율에 가까울수록 세 값의 차이가 줄어든다는 뜻입니다.

% 는 부모 기준

섹션 1

```
.pct-parent {  
  max-width: 400px;  
  background: #f4f4f4;  
  border: 1px solid #bbbbbb;  
  padding: 10px;  
}
```

화면 폭 1280 에서 안쪽이 400px 인 회색 상자입니다

```
.pct-half {  
  width: 50%;  
  background: #dbe7fb;  
  border: 1px solid #9ab8e8;  
  padding: 4px;  
  box-sizing: border-box;  
}
```

화면 부모 안쪽 400px 의 절반인 200px 입니다

```
.pct-quarter {  
  width: 25%;  
  background: #ffe0b3;  
  border: 1px solid #e8a33d;  
  padding: 4px;  
  box-sizing: border-box;  
}
```

화면 부모 안쪽 400px 의 4분의 1인 100px 입니다

vw 와 vh

섹션 2

```
.vw-half {
  width: 50vw;
  background: #d9f0d9;
  border: 1px solid #8cc48c;
  padding: 6px;
  box-sizing: border-box;
}
```

화면 폭 1280 에서 640px, 폭 480 에서 240px 입니다

```
.vw-quarter {
  width: 25vw;
  background: #d9f0d9;
  border: 1px solid #8cc48c;
  padding: 6px;
  box-sizing: border-box;
}
```

화면 폭 1280 에서 320px, 폭 480 에서 120px 입니다

```
.vh-box {
  height: 20vh;
  background: #f0e0f0;
  border: 1px solid #c8a0c8;
  padding: 6px;
  box-sizing: border-box;
}
```

화면 창 높이가 900px 일 때 180px 입니다. 창 높이를 바꾸면 같이 바뀝니다

```
.ref-pct {
  width: 50%;
  background: #dbe7fb;
  border: 1px solid #9ab8e8;
  padding: 4px;
  box-sizing: border-box;
}
```

화면 부모(400px) 의 절반인 200px 입니다

```
.ref-vw {
  width: 50vw;
  background: #ffd9d9;
  border: 1px solid #e0a0a0;
  padding: 4px;
  box-sizing: border-box;
}
```

화면 부모와 상관없이 화면의 절반인 640px 이라서 부모 밖으로 나갑니다

이미지가 넘치지 않게

섹션 3

```
.spill-frame {
  overflow-x: auto;
}
```

화면 폭 1280 에서는 아무 변화가 없습니다.

창을 폭 480px 쯤까지 좁히면 그때부터 이 상자 안에 가로 스크롤 막대가 생깁니다

```
.photo-frame {
  max-width: 320px;
  border: 2px solid #bbbbbb;
  padding: 8px;
  margin-bottom: 10px;
}
```

화면 안쪽이 320px 인 액자 상자입니다

```
.photo-good {
  max-width: 100%;
}
```

화면 400px 짜리 그림이 액자에 맞춰 320px 로 줄어듭니다 (높이도 160px 로 같이)

min() max() clamp()

섹션 4

```
.fn-min {
  width: min(500px, 100%);
  background: #dbe7fb;
  border: 1px solid #9ab8e8;
  padding: 6px;
  box-sizing: border-box;
}
```

화면 폭 1280 에서 500px, 폭 480 에서 404px 입니다

```
.fn-max {
  width: max(200px, 30%);
  background: #ffe0b3;
  border: 1px solid #e8a33d;
  padding: 6px;
  box-sizing: border-box;
}
```

화면 폭 1280 에서 247px, 폭 480 에서 200px 아래로는 안 내려갑니다

```
.fn-clamp {
  width: clamp(200px, 50%, 380px);
  background: #d9f0d9;
  border: 1px solid #8cc48c;
  padding: 6px;
  box-sizing: border-box;
}
```

화면 폭 1280 에서 380px, 폭 768 에서 346px, 폭 480 에서 202px 입니다

```
.fn-fs {
  font-size: clamp(16px, 2.5vw, 28px);
  margin: 0;
}
```

화면 폭 1280 에서 28px, 폭 768 에서 19.2px, 폭 480 에서 16px 입니다

object-fit

섹션 5

```
.thumb-row {
  display: flex;
  flex-wrap: wrap;
  gap: 12px;
}
.thumb {
  width: 200px;
  height: 200px;
  border: 1px solid #bbbbbb;
  box-sizing: border-box;
  background: #ffffff;
}
```

화면 200 × 200 정사각형 액자 세 개가 가로로 놓입니다

```
.fit-cover {  
  object-fit: cover;  
}
```

화면 그림이 원래 비율 그대로 액자 높이에 맞고, 넘치는 좌우가 잘려 해가 안 보입니다

```
.fit-contain {  
  object-fit: contain;  
}
```

화면 그림 전체(200 × 100) 가 다 보이고 위아래에 흰 여백이 생깁니다

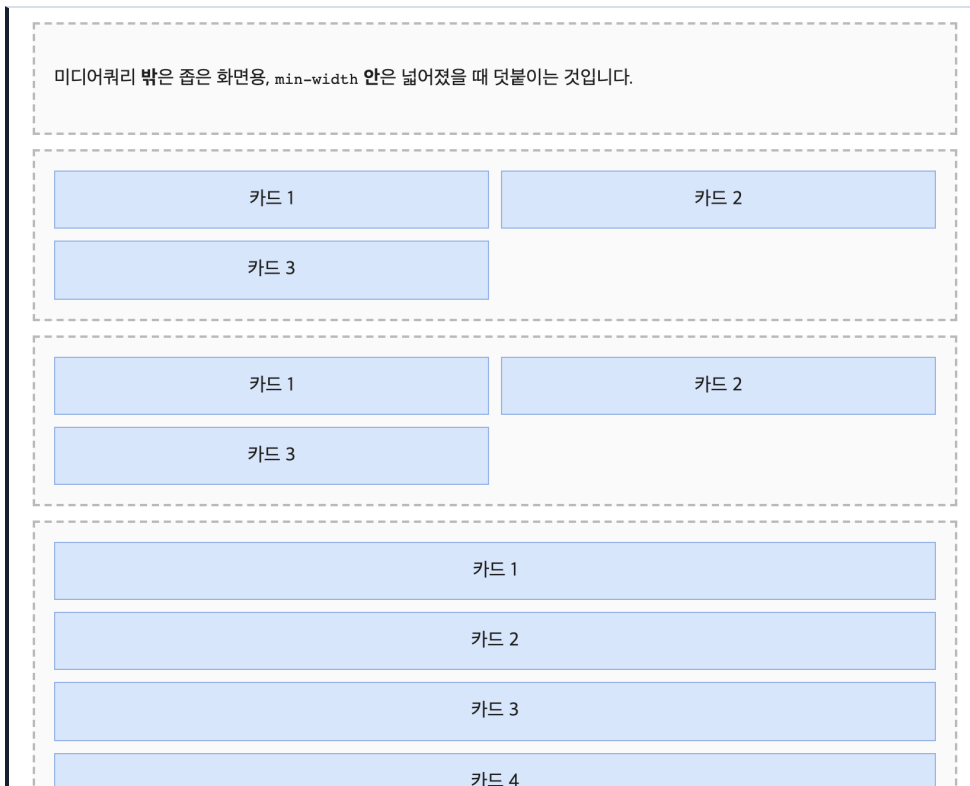
13단원 · 개념 05

모바일 우선으로 만들기

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

F12 → Ctrl+Shift+M 을 켜고 폭을 360, 480, 768, 1280 으로

바꿔 가며 보세요. 이 파일은 그렇게 봐야 뜻이 통합니다.



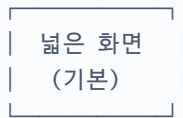
실행하면 나오는 화면 — 모바일 우선으로 만들기

13단원의 마지막 파일입니다. 지금까지 배운 것을 한 페이지로 합칩니다.

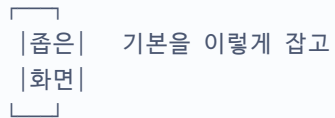
12단원	flex	한 줄로 세우기
개념01	grid	격자 만들기
개념02	areas	이름으로 배치하기
개념03	@media	폭에 따라 규칙 바꾸기
개념04	% vw clamp	유연한 값 쓰기
개념05	← 여기서	전부 합칩니다

여기서 배우는 것은 새 문법이 아니라 순서입니다. “무엇을 기본으로 삼고 무엇을 덧붙일 것인가” 하는 순서입니다.

모바일 우선



좁히면서
하나씩 되돌림



넓히면서
하나씩 덧붙임

요즘은 오른쪽, 즉 모바일 우선(mobile-first)으로 만듭니다. 이유는 섹션 2에서 코드를 나란히 놓고 봅니다.

13단원 마무리입니다. 모바일 우선이라는 만드는 순서를 배우고, flex + grid + 미디어쿼리를 합쳐 카드 목록이 1단 → 2단 → 3단으로 바뀌는 페이지를 처음부터 끝까지 만듭니다. F12 → Ctrl+Shift+M 을 켜 두고 폭을 바꿔 가며 읽으세요.

모바일 우선이란

섹션 1

미디어쿼리 밖에 쓴 CSS 를 좁은 화면용으로 삼는 것입니다. 그리고 min-width 로 “넓어지면 이렇게 더 해라” 를 덧붙입니다.

미디어쿼리 밖에 쓴 규칙은 조건이 없습니다. 그래서 모든 쪽에서 적용됩니다. 이것을 보통 “기본” 이라고 부릅니다.

```
.cards { grid-template-columns: 1fr; }      ← 기본. 항상 적용됩니다

@media (min-width: 600px) {
  .cards { grid-template-columns: repeat(2, 1fr); } ← 600 이상에서만 덧붙임
}
```

폭 400px 짜리 휴대폰에서는 조건이 안 맞으므로 기본만 남습니다. 1단입니다. 폭 1000px 짜리 노트북에서는 조건이 맞으므로 기본 위에 덮어써서 2단이 됩니다.

그러니 "기본을 무엇으로 잡을 것인가" 가 곧 "무엇을 우선할 것인가" 입니다.

기본을 1단으로 → 모바일 우선
기본을 3단으로 → 데스크톱 우선

모바일 우선으로 만들면 사고방식이 이렇게 바뀝니다.

(전) 넓은 화면을 먼저 그리고, 좁아지면 무엇을 뺄지 고민한다
(후) 꼭 필요한 것만 세로로 쌓아 놓고, 자리가 남으면 무엇을 더할지 고민한다

후자가 훨씬 편합니다. 좁은 화면에서 할 수 있는 배치는 사실 하나뿐입니다. 위에서 아래로 쌓는 것입니다. 고를 것이 없으니 고민도 없습니다. 그렇게 만들어 둔 다음, 자리가 넓어지면 옆으로 펼치면 됩니다.

화면 점선 상자 안에 위 한 문장이 보입니다

미디어쿼리 밖은 좁은 화면용, min-width
안은 넓어졌을 때 덧붙이는 것입니다.

◁ 직접 해보기

1

개념03 파일을 열어 .mq-cards 규칙을 찾아보세요. 기본이 1fr 이고 min-width 로 칸을 늘렸습니다. 그 파일도 이미 모바일 우선으로 쓰여 있었다는 것을 확인하세요.

왜 min-width 인가

섹션 2

max-width 로 만들어도 결과는 똑같습니다. 그런데도 min-width 를 권하는 이유가 네 가지 있습니다.

아래 두 상자는 화면에서 완전히 같게 보입니다. 폭을 어떻게 바뀌도 같습니다. 쓰인 CSS 만 다릅니다.

모바일 우선 · [데스크톱 우선]

```
.mf-first {
  grid-template-columns: 1fr;
}
@media (min-width: 600px) {
  .mf-first { repeat(2, 1fr) }
}
@media (min-width: 900px) {
  .mf-first { repeat(3, 1fr) }
}

.dt-first {
  grid-template-columns: repeat(3, 1fr);
}
@media (max-width: 899px) {
  .dt-first { repeat(2, 1fr) }
}
@media (max-width: 599px) {
  .dt-first { 1fr }
}
```

결과가 같은데 왜 왼쪽을 쓸까요.

- 1 기본이 가장 단순합니다. 1fr 한 줄이 기본입니다. 읽는 사람이 출발점을 바로 이해합니다. 오른쪽은 기본이 3단이라, 이 페이지가 좁은 화면에서 어떻게 되는지 미디어쿼리를 전부 읽어야 알 수 있습니다.
- 2 휴대폰이 일을 덜 합니다. 휴대폰에서는 조건이 하나도 안 맞아 기본만 적용됩니다. 오른쪽은 반대로 휴대폰에서 조건 두 개가 다 맞아 세 번 계산합니다.
- 3 숫자가 자연스럽습니다. 왼쪽은 600, 900 처럼 딱 떨어지는 값을 씁니다. 오른쪽은 겹치지 않게 하려고 899, 599 처럼 1을 뺀 값을 써야 합니다. 이 1 을 빼먹으면 폭이 정확히 600px 일 때 두 조건이 동시에 맞아 예측하기 어려운 결과가 나옵니다.
- 4 무언가를 더하는 편이 빠른 것보다 안전합니다. 좁은 화면에서 잘 보이던 것이 넓어졌다고 깨지는 일은 드뭅니다. 반대로 넓은 화면 기준으로 만든 것을 좁히면 거의 항상 깨집니다.

정리하면 이렇습니다. 결과가 아니라 만드는 사람의 순서 문제입니다. 둘 중 아무거나 골라도 화면은 같지만, 모바일 우선 쪽이 덜 틀립니다.

화면 폭 1280 에서 268px 짜리 3칸입니다.

폭 768 에서 341px 2칸, 폭 480 에서 404px 1칸이 됩니다

카드 1
카드 2
카드 3

화면 위 상자와 폭마다 완전히 같은 모양이 됩니다.

폭 1280 에서 268px 3칸, 768 에서 341px 2칸, 480 에서 404px 1칸

카드 1
카드 2
카드 3

두 방식 중 하나가 틀린 것이 아닙니다. 이미 만들어진 오래된 사이트를 고칠 때는 데스크톱 우선으로 쓰인 CSS 를 자주 만납니다. 새로 만들 때 모바일 우선을 고르라는 이야기입니다.

▣ 직접 해보기 2

기기 톨바로 폭을 600 에 맞추면, 위 두 상자가 정말 같은 모양인지 확인하세요. 그다음 .dt-first 의 max-width: 899px 를 900px 으로 바꾸고 폭 900 에서 무슨 일이 생기는지 보세요.

1단계

섹션 3

아무 조건 없이 세로로 한 줄씩 쌓는 것부터 만듭니다. 미디어쿼리는 아직 한 줄도 안 씁니다.

이렇게만 씁니다.

```
.step-one {  
  display: grid;  
  grid-template-columns: 1fr;  
  gap: 10px;  
}
```

칸이 하나뿐인 격자입니다. 그래서 카드가 위에서 아래로 쌓입니다.

"그럴 거면 grid 를 왜 쓰나요? 그냥 두면 div 는 원래 쌓이는데요." 좋은 질문입니다. 실제로 지금 단계에서는 display: grid 가 없어도 모양이 같습니다. 그래도 처음부터 격자로 만들어 두는 이유가 두 가지 있습니다.

1 gap 을 쓸 수 있습니다. 카드마다 margin 을 주지 않아도 됩니다

2 다음 단계에서 grid-template-columns 한 줄만 바꾸면 2단이 됩니다

즉 "지금은 1칸짜리 격자" 라고 선언해 두는 것입니다. 준비 작업입니다.

이 상자는 폭을 아무리 바꿔도 항상 1단입니다. 미디어쿼리가 없기 때문입니다. 다음 섹션에서 조건을 하나 붙여 봅시다.

화면 폭 1280 에서 824px 짜리 카드 네 개가 세로로 쌓입니다.

폭 480 에서는 404px 이 되지만 여전히 1단입니다

카드 1
카드 2
카드 3
카드 4

▹ 직접 해보기 **3**

.step-one 의 gap: 10px 을 gap: 0 으로 바꿔 보세요. 카드가 서로 붙습니다. 확인했으면 되돌리세요.

2단계

섹션 4

조건을 하나 붙입니다. 바꾸는 줄은 딱 한 줄입니다.

1단계 CSS 아래에 이것만 더합니다.

```
@media (min-width: 600px) {
  .step-two {
    grid-template-columns: repeat(2, 1fr);
  }
}
```

display: grid 도 gap 도 다시 안 씁니다. 기본에 이미 있기 때문입니다. 미디어쿼리 안에는 바꿀 것만 씁니다. 나머지는 그대로 물려받습니다.

이것이 모바일 우선의 실제 이득입니다. 덧붙이는 코드가 짧습니다.

폭마다 재 보면 이렇습니다.

폭 1280 상자 안쪽 824px → $(824 - 10) \div 2 = 407\text{px}$ 짜리 2칸
 폭 768 상자 안쪽 692px → $(692 - 10) \div 2 = 341\text{px}$ 짜리 2칸
 폭 480 상자 안쪽 404px → 조건이 안 맞아 404px 짜리 1칸

600이라는 숫자는 어디서 왔을까요. 정해진 답은 없습니다. 카드 하나가 너무 좁아지지 않는 폭을 눈으로 찾아 정한 것입니다. 카드 안에 사진과 제목이 들어간다면 한 칸이 280px 쯤은 되어야 보기 좋습니다. 두 칸이면 560px + 여백이니 600px 쯤이 자연스러운 자리입니다.

화면 폭 1280 에서 407px 짜리 2칸이 두 줄로 놓입니다.

폭 480에서는 404px 짜리 1칸이 되어 네 줄로 쌓입니다

카드 1
 카드 2
 카드 3
 카드 4

▫ 직접 해보기 4

기기 톨바로 폭을 599 와 600 으로 번갈아 맞춰 보세요. 1px 차이로 1단과 2단이 갈립니다.

3단계

섹션 5

조건을 하나 더 아래에 붙입니다. 순서를 지키는 것이 여기서 중요해집니다.

2단계 CSS 아래에 이것만 더합니다.

```
@media (min-width: 900px) {
  .step-three {
    grid-template-columns: repeat(3, 1fr);
  }
}
```

폭이 1000px 이면 min-width: 600px 조건도 맞고 min-width: 900px 조건도 맞습니다. 두 규칙이 같은 속성을 서로 다르게 정하고 있습니다. 어느 쪽이 이길까요.

08단원에서 배운 대로 명시도가 같으면 나중에 쓴 것이 이깁니다. 그래서 900px 짜리를 아래에 써야 3단이 됩니다. 위아래를 바꾸면 넓은 화면에서도 영영 2단입니다. 에러는 안 납니다.

작은 값부터 큰 값 순서로 위에서 아래로.
 이것이 모바일 우선에서 미디어쿼리를 쓰는 순서입니다.

완성된 CSS 는 이렇게 생겼습니다. 위에서 아래로 읽으면 이야기가 됩니다.

```
.step-three { grid-template-columns: 1fr; }
@media (min-width: 600px) { ... repeat(2, 1fr) }
@media (min-width: 900px) { ... repeat(3, 1fr) }
```

기본은 1단
좀 넓으면 2단
더 넓으면 3단

꼭마다 재 보면 이렇습니다.

꼭 1280 $(824 - 20) \div 3 = 268\text{px}$ 짜리 3칸
 꼭 768 $(692 - 10) \div 2 = 341\text{px}$ 짜리 2칸
 꼭 480 404px 짜리 1칸

화면 꼭 1280 에서 268px 짜리 3칸이 두 줄, 꼭 768 에서 341px 짜리 2칸이 세 줄,

꼭 480 에서 404px 짜리 1칸이 여섯 줄이 됩니다

카드 1
 카드 2
 카드 3
 카드 4
 카드 5
 카드 6

▹ 직접 해보기 5

.step-three 의 두 미디어쿼리 순서를 서로 바꿔 보세요(900 을 위로, 600 을 아래로). 꼭 1280 에서 몇 칸이 되는지 확인하고 이유를 설명한 뒤 되돌리세요.

머리말과 메뉴

섹션 6

칸 수만 바꾸면 반쪽입니다. 메뉴 방향과 글자 크기도 화면에 맞춰야 합니다. 여기서 12단원 flex 가 다시 나옵니다.

메뉴는 격자보다 flex 가 어울립니다. 한 줄이고, 내용 길이가 제각각이기 때문입니다. 모바일 우선이므로 기본은 세로로 쌓습니다.

```
.mf-nav {
  display: flex;
  flex-direction: column;   기본은 세로
  gap: 6px;
}

@media (min-width: 600px) {
  .mf-nav {
    flex-direction: row;      넓어지면 가로 한 줄
    justify-content: center;   가운데로 모으기
  }
}
```

좁은 화면에서 메뉴를 가로로 두면 글자가 잘리거나 두 줄로 접힙니다. 세로로 쌓으면 손가락으로 누르기도 훨씬 편합니다.

머리말 글자는 개념04의 `clamp()`를 씁니다.

```
font-size: clamp(18px, 3vw, 30px);
```

폭 1280 → 3vw 는 38.4px → 최댓값 30px 에 걸립니다
 폭 768 → 3vw 는 23.04px → 그대로 23.04px
 폭 480 → 3vw 는 14.4px → 최솟값 18px 로 올라갑니다

미디어쿼리를 하나도 안 쓰고 글자 크기가 부드럽게 따라옵니다. 칸 수처럼 툭툭 끊겨야 하는 것은 `@media`로, 글자 크기처럼 부드럽게 변해도 되는 것은 `clamp()`로. 이렇게 나눠 쓰면 코드가 짧아집니다.

화면 폭 1280 에서 파란 머리말 글자가 30px 이고 메뉴 세 개가 가로 한 줄로

가운데 모입니다. 폭 480에서는 글자가 18px 이 되고 메뉴가 세로로 쌓입니다

우리 동네 도서관

공지사항
 도서 검색
 이용 안내

▸ 직접 해보기 6

`.mf-nav`의 미디어쿼리 안에서 `justify-content: center;`를 `space-between;`으로 바꿔 보세요 (12단원). 메뉴 세 개가 좌우 끝까지 벌어집니다.

완성 페이지

섹션 7

지금까지 만든 조각을 한 페이지로 합칩니다. 13단원에서 배운 것이 전부 들어 있습니다.

페이지 뼈대는 개념02의 grid-template-areas로 만듭니다. 모바일 우선이므로 기본은 한 줄로 짚 쌓는 모양입니다.

```
.page {
  display: grid;
  grid-template-columns: 1fr;
  grid-template-areas:
    "head"
    "menu"
    "body"
    "side"
    "foot";
  gap: 10px;
}
```

칸이 하나뿐이니 다섯 덩어리가 위에서 아래로 놓입니다. 휴대폰에서 보는 모양이 바로 이것입니다.

폭이 900px 이상이 되면 옆칸을 오른쪽으로 옮깁니다.

```
@media (min-width: 900px) {
  .page {
    grid-template-columns: 1fr 220px;
    grid-template-areas:
      "head head"
      "menu menu"
      "body side"
      "foot foot";
  }
}
```

★ HTML은 한 글자도 안 고쳤습니다.

이것이 grid-template-areas의 가장 큰 장점입니다. HTML에서 side가 body보다 뒤에 있어도, 배치는 CSS가 정합니다. 따옴표 안 글자만 바꾸면 옆칸이 왼쪽으로도 오른쪽으로도 갑니다.

그 안의 카드 목록은 섹션 4와 같습니다.

```
.page-cards { grid-template-columns: 1fr; }
@media (min-width: 600px) { .page-cards { repeat(2, 1fr) } }
```

카드 안 사진에는 개념04의 max-width: 100%를 걸어 둡니다. 안 그러면 300px 짜리 사진이 좁은 칸을 뚫고 나갑니다.

```
.page-card img { max-width: 100%; display: block; }
```

꼭마다 실제로 재 보면 이렇습니다.

폭 1280	본문 594px + 옆칸 220px,	카드 한 칸 279px (2칸)
폭 768	본문 692px 한 줄,	카드 한 칸 328px (2칸)
폭 480	전부 404px 한 줄,	카드 한 칸 378px (1칸)

기기 톨바로 폭을 천천히 줄이면서 무엇이 언제 바뀌는지 보세요. 600 에서 카드가 2칸에서 1칸으로, 메뉴가 가로에서 세로로 바뀝니다. 900 에서 옆칸이 오른쪽에서 아래로 내려옵니다.

화면 폭 1280 에서 파란 머리말, 가운데 정렬된 메뉴, 그 아래 왼쪽 본문(594px)

안에 카드가 2칸, 오른쪽에 노란 옆칸(220px), 맨 아래 검은 바닥글입니다. 폭 480 에서는 머리말·메뉴·본문·옆칸·바닥글이 전부 세로로 한 줄씩 쌓이고 메뉴도 세로가 되며 카드도 1칸이 됩니다

우리 동네 도서관

공지사항
도서 검색
이용 안내

이달의 추천 도서

바다의 지도

겨울의 정원

느린 산책

작은 부엌

오늘의 안내
월요일은 쉽니다.

우리 동네 도서관 · 연락처 02-000-0000

▹ 직접 해보기 7

@media (min-width: 900px) 안의 grid-template-areas 를 "side body" 로 바꿔 옆칸을 왼쪽으로 옮겨 보세요. 열 크기 1fr 220px 도 같이 바뀌어야 옆칸이 좁게 나옵니다. HTML 은 건드리지 마세요.

자주 하는 실수

섹션 8

모바일 우선의 실수는 대부분 방향과 순서에서 납니다. 문법은 맞는데 결과만 반대로 나옵니다.

이런 실수를 합니다

방향이 반대

이런 실수를 합니다

실수 1 — 기본을 3단으로 잡고 min-width 로 1단을 만들려 함

```
.cards { grid-template-columns: repeat(3, 1fr); }

@media (min-width: 600px) {
  .cards { grid-template-columns: 1fr; }
}
```

무슨 일이 일어나나요: 정확히 반대로 나옵니다. 휴대폰에서 3단이 되어 카드가 100px 도 안 되게 좁아지고, 넓은 화면에서 1단이 되어 카드가 화면 전체로 늘어납니다. 기본은 항상 좁은 화면용이어야 합니다.

이렇게 쓰세요

```
.cards { grid-template-columns: 1fr; }

@media (min-width: 600px) {
  .cards { grid-template-columns: repeat(3, 1fr); }
}
```

이런 실수를 합니다

순서를 거꾸로

이런 실수를 합니다

실수 2 — 큰 중단점을 위에 씀

```
@media (min-width: 900px) { .cards { repeat(3, 1fr) } }  
@media (min-width: 600px) { .cards { repeat(2, 1fr) } }
```

무슨 일이 일어나나요: 넓은 화면에서도 계속 2단입니다. 폭 1200px 이면 두 조건이 다 맞는데, 명시도가 같으므로 나중에 쓴 600px 짜리가 이깁니다(08단원). 모바일 우선에서는 작은 값부터 큰 값 순서로 위에서 아래로 씁니다.

이런 실수를 합니다

중단점이 겹침

이런 실수를 합니다

실수 3 — 중단점이 1px 겹침

```
@media (max-width: 600px) { .cards { 1fr } }  
@media (min-width: 600px) { .cards { repeat(2, 1fr) } }
```

무슨 일이 일어나나요: 폭이 정확히 600px 일 때 두 조건이 동시에 맞습니다. 나중에 쓴 쪽이 이기므로 2단이 되는데, 코드를 읽는 사람은 1단을 기대합니다. 나중에 순서를 바꾸면 결과가 조용히 달라집니다. 한쪽을 599px 로 하거나, 아예 min-width 만 쓰세요.

이런 실수를 합니다

기본에 고정 px

이런 실수를 합니다

실수 4 — 기본에 고정 px 폭을 씀

```
.wrap { width: 960px; }  
  
@media (max-width: 600px) {  
  .wrap { width: auto; }  
}
```

무슨 일이 일어나나요: 폭이 601px ~ 959px 인 화면에서 가로 스크롤이 생깁니다. 중단점 사이의 구간을 잊은 것입니다. 기본은 폭을 아예 정하지 않거나 max-width: 960px 처럼 상한선으로 주세요(개념04). 그러면 중단점이 하나도 필요 없습니다.

이런 실수를 합니다

viewport meta 누락

이런 실수를 합니다

실수 5 — viewport meta 를 안 넣고 모바일 우선 CSS 를 씀

```
<head>
<meta charset="UTF-8" />
</head>
```

무슨 일이 일어나나요: 휴대폰에서 3단짜리 데스크톱 화면이 그대로 축소되어 나옵니다. 브라우저가 폭을 980px 이라고 믿어 min-width 조건이 전부 맞아 버리기 때문입니다. 모바일 우선으로 아무리 잘 써도 이 한 줄이 없으면 헛일입니다(개념03 섹션 2).

이런 실수를 합니다

메뉴 방향을 안 바꿈

이런 실수를 합니다

실수 6 — 카드만 바꾸고 메뉴는 그대로 둠

```
.nav { display: flex; } /* 방향을 안 바꿈 */
```

무슨 일이 일어나나요: 카드는 예쁘게 1단이 되는데 메뉴만 옆으로 빠져나갑니다. 글자가 잘리거나 줄이 접혀서 겹칩니다. 좁은 화면에서는 flex-direction: column 이 기본이어야 합니다. 폭을 좁혀 놓고 페이지를 위에서 아래로 한 번 훑어보는 습관을 들이세요.

정리

▢ 직접 해보기 8

정답

1) 개념03 파일의 style 안에 이렇게 되어 있습니다.

```
.mq-cards { grid-template-columns: 1fr; }
@media (min-width: 600px) { .mq-cards { 1fr 1fr } }
@media (min-width: 900px) { .mq-cards { 1fr 1fr 1fr } }
```

→ 기본이 1칸이고 min-width 로 칸을 늘렸으니 모바일 우선입니다.

작은 값(600)이 위, 큰 값(900)이 아래인 순서도 지켜져 있습니다.

2) 폭 600 에서 두 상자는 똑같이 2칸입니다. .dt-first 의 조건을 max-width: 900px 으로 바꾸면

```
@media (max-width: 900px) { .dt-first { repeat(2, 1fr) } }
```

→ 폭이 정확히 900px 일 때 두 상자의 모양이 달라집니다.

.mf-first 는 min-width: 900px 이 맞아서 3칸,
.dt-first 는 max-width: 900px 이 맞아서 2칸이 됩니다.
경계값이 겹치면 이런 어긋남이 생깁니다. 확인했으면 899px 로 되돌리세요.

3) .step-one 을 이렇게 바꿉니다.

```
gap: 0;
```

→ 카드 네 개가 위아래로 딱 붙어서 테두리가 두 줄로 겹쳐 보입니다.

gap 은 격자 칸 사이 간격이므로 0 으로 하면 사이가 사라집니다.

4) 폭 599 에서는 카드가 1단, 폭 600 에서는 2단입니다.

→ min-width: 600px 은 600 을 포함하기 때문입니다.

599 에서는 조건이 안 맞아 기본(1fr)만 남습니다.

5) .step-three 의 두 미디어쿼리 순서를 바꾸면

```
@media (min-width: 900px) { ... repeat(3, 1fr) }  
@media (min-width: 600px) { ... repeat(2, 1fr) }
```

→ 폭 1280 에서 3칸이 아니라 2칸이 됩니다.

폭 1280 은 900 조건도 600 조건도 다 맞습니다.
명시도가 같으므로 나중에 쓴 600px 짜리가 이깁니다.
에러는 나지 않고 조용히 2칸이 됩니다. 확인했으면 되돌리세요.

6) .mf-nav 의 미디어쿼리 안을 이렇게 바꿉니다.

```
justify-content: space-between;
```

→ 메뉴 세 개가 왼쪽 끝, 가운데, 오른쪽 끝으로 벌어집니다.

12단원에서 배운 flex 정렬이 미디어쿼리 안에서도 그대로 동작합니다.

7) 미디어쿼리 안을 이렇게 바꿉니다.

```
@media (min-width: 900px) {
  .page {
    grid-template-columns: 220px 1fr;
    grid-template-areas:
      "head head"
      "menu menu"
      "side body"
      "foot foot";
  }
}
```

→ 노란 옆칸이 왼쪽으로, 본문이 오른쪽으로 자리를 바꿉니다.

열 크기를 1fr 220px 그대로 두면 옆칸이 594px 로 넓어져 버리므로
220px 1fr 로 같이 바뀌어야 합니다.
HTML 은 한 글자도 안 고쳤다는 점을 꼭 확인하세요.

카드 하나의 모양입니다. 배치와는 상관없는 색과 여백입니다.

```
.card {
  background: #dbe7fb;
  border: 1px solid #9ab8e8;
  padding: 10px;
  text-align: center;
}
```

같은 결과를 두 가지 방법으로

섹션 2

모바일 우선 · 기본은 좁은 화면용, 넓어질 때 덧붙입니다

```
.mf-first {
  display: grid;
  grid-template-columns: 1fr;
  gap: 10px;
}
@media (min-width: 600px) {
  .mf-first {
    grid-template-columns: repeat(2, 1fr);
  }
}
@media (min-width: 900px) {
  .mf-first {
    grid-template-columns: repeat(3, 1fr);
  }
}
```

화면 폭 1280 에서 268px 3칸, 768 에서 341px 2칸, 480 에서 404px 1칸

데스크톱 우선 · 기본은 넓은 화면용, 좁아질 때 되돌립니다

```
.dt-first {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 10px;
}
@media (max-width: 899px) {
  .dt-first {
    grid-template-columns: repeat(2, 1fr);
  }
}
@media (max-width: 599px) {
  .dt-first {
    grid-template-columns: 1fr;
  }
}
```

화면 위와 완전히 같은 결과입니다. 코드만 한 줄 더 길니다

1단계 — 기본(1단)

섹션 3

```
.step-one {
  display: grid;
  grid-template-columns: 1fr;
  gap: 10px;
}
```

화면 어떤 폭에서도 카드가 세로로 한 줄씩 쌓입니다

2단계 — 600px 이상이면 2단

섹션 4

```
.step-two {
  display: grid;
  grid-template-columns: 1fr;
  gap: 10px;
}
@media (min-width: 600px) {
  .step-two {
    grid-template-columns: repeat(2, 1fr);
  }
}
```

화면 폭 1280 에서 407px 2칸, 폭 480 에서 404px 1칸

3단계 — 900px 이상이면 3칸

섹션 5

```
.step-three {
  display: grid;
  grid-template-columns: 1fr;
  gap: 10px;
}
@media (min-width: 600px) {
  .step-three {
    grid-template-columns: repeat(2, 1fr);
  }
}
@media (min-width: 900px) {
  .step-three {
    grid-template-columns: repeat(3, 1fr);
  }
}
```

화면 폭 1280 에서 268px 3칸, 768 에서 341px 2칸, 480 에서 404px 1칸

머리말과 메뉴

섹션 6

```
.mf-head {
  background: #2d6cdf;
  color: #ffffff;
  padding: 12px;
  text-align: center;
  font-size: clamp(18px, 3vw, 30px);
  font-weight: bold;
}
```

화면 파란 띠. 글자는 폭 1280 에서 30px, 폭 480 에서 18px 입니다

```
.mf-nav {
  display: flex;
  flex-direction: column;
  gap: 6px;
  margin-top: 8px;
}
.mf-nav span {
  background: #eef2f7;
```

```
padding: 8px;
  text-align: center;
}
@media (min-width: 600px) {
  .mf-nav {
    flex-direction: row;
    justify-content: center;
  }
}
```

화면 폭 480 에서 메뉴가 세로로 쌓이고, 폭 600 이상에서 가로 한 줄이 됩니다


```
.page {
  display: grid;
  grid-template-columns: 1fr;
  grid-template-areas:
    "head"
    "menu"
    "body"
    "side"
    "foot";
  gap: 10px;
}

.page-head {
  grid-area: head;
  background: #2d6cdf;
  color: #ffffff;
  padding: 12px;
  font-size: clamp(18px, 3vw, 30px);
  font-weight: bold;
}

.page-menu {
  grid-area: menu;
  display: flex;
  flex-direction: column;
  gap: 6px;
}

.page-menu span {
  background: #eef2f7;
  border: 1px solid #9ab8e8;
  padding: 8px;
  text-align: center;
}

.page-body {
  grid-area: body;
  border: 1px solid #cccccc;
  padding: 12px;
  box-sizing: border-box;
}
```

```

}
.page-side {
  grid-area: side;
  background: #fff3d9;
  border: 1px solid #e8c67d;
  padding: 12px;
  box-sizing: border-box;
}
.page-foot {
  grid-area: foot;
  background: #333333;
  color: #ffffff;
  padding: 10px;
  text-align: center;
  font-size: 14px;
}
}
.page-cards {
  display: grid;
  grid-template-columns: 1fr;
  gap: 10px;
}
.page-card {
  border: 1px solid #cccccc;
  padding: 8px;
}
}
.page-card img {
  max-width: 100%;
  display: block;
}
}
@media (min-width: 600px) {
  .page-menu {
    flex-direction: row;
    justify-content: center;
  }
  .page-cards {
    grid-template-columns: repeat(2, 1fr);
  }
}

```

```
    }  
  }  
  @media (min-width: 900px) {  
    .page {  
      grid-template-columns: 1fr 220px;  
      grid-template-areas:  
        "head head"  
        "menu menu"  
        "body side"  
        "foot foot";  
    }  
  }  
}
```

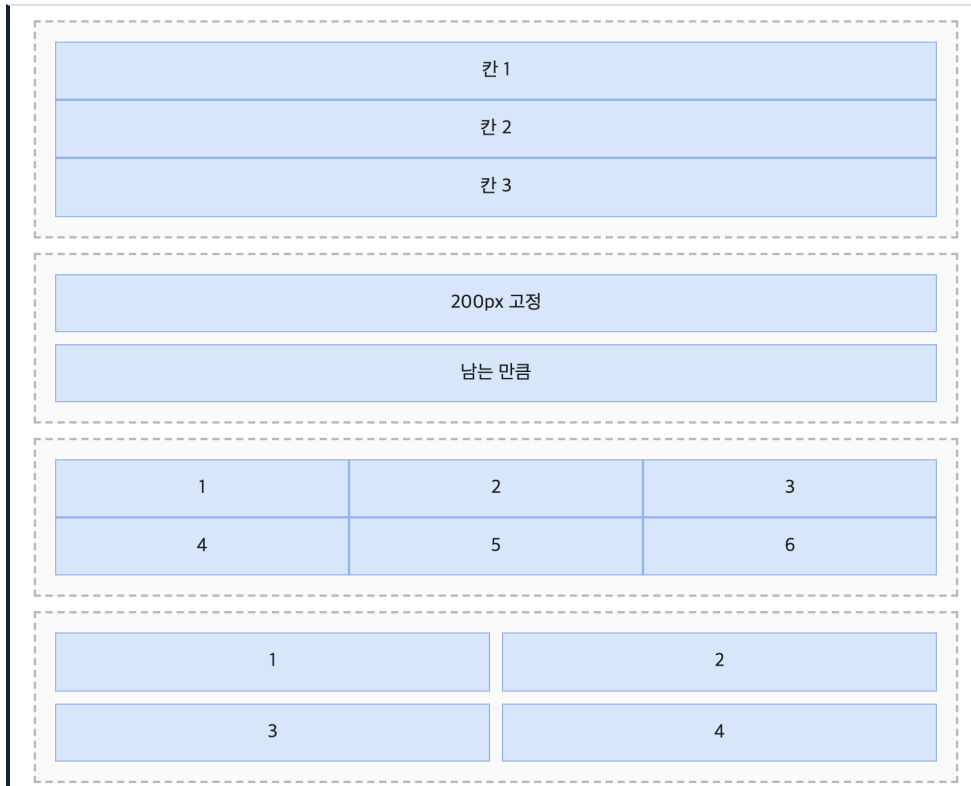
화면 폭 1280 에서 본문 594px + 옆칸 220px 로 나뉘고 카드가 2칸,

폭 480 에서 전부 세로로 한 줄씩 쌓입니다

Grid 와 반응형

RUN 이 파일을 더블클릭해서 브라우저로 열고,

VS Code 로도 열어 위쪽 style 안의 TODO 자리를 채우세요.



실행하면 나오는 화면 — Grid 와 반응형

이 파일 위쪽 <style> 안에 빈 자리가 준비되어 있습니다. 각 문제 설명도 그 자리 바로 위에 적혀 있으니 VS Code 로 함께 열어 보세요. 코드를 쓰고 저장(Ctrl+S) 한 뒤 브라우저에서 새로고침(F5) 하면 아래 상자가 바뀝니다. 폭을 바꿔 봐야 하는 문제가 많습니다. F12 → Ctrl+Shift+M 으로 기기 톨바를 켜 두세요. 1~12 는 기본, 13~14 는 응용, 15 는 도전, 16 은 고장 난 코드를 고치는 문제입니다.

문제 1

문제 1 — 격자를 만들고 세 칸으로 나누기

(개념01 섹션 1:2) .q1 을 격자로 만들고 칸을 세 개로 나누세요. 칸 크기는 셋 다 같게, 칸 사이는 10px 입니다.

칸 1
칸 2
칸 3

문제 2

문제 2 — 고정 칸과 남는 칸

(개념01 섹션 3) .q2 의 왼쪽 칸은 200px 로 고정하고, 오른쪽 칸은 남는 자리를 전부 쓰게 하세요.

200px 고정
남는 만큼

문제 3

문제 3 — 행 간격과 열 간격을 다르게

(개념01 섹션 4) .q3 의 위아래 줄 사이는 24px, 좌우 칸 사이는 10px 로 띄우세요. 값 두 개를 한 줄로 씁니다.

1
2
3
4
5
6

문제 4

문제 4 — 줄 높이 정하기

(개념01 섹션 5) .q4 의 첫째 줄 높이를 50px, 둘째 줄 높이를 90px 로 정하세요.

1
2
3
4

문제 5

문제 5 — 선 번호로 두 칸 합치기

(개념02 섹션 2) .q5-wide 가 1번 선에서 3번 선까지, 즉 두 칸을 차지하게 하세요.

두 칸
C
D

문제 6

문제 6 — span 으로 세 칸 차지하기

(개념02 섹션 3) .q6-wide 가 세 칸을 통째로 차지하게 하세요. 이번에는 선 번호 대신 span 을 쓰세요.

세 칸을 통째로
B
C
D

문제 7

문제 7 — repeat() 으로 네 칸 만들기

(개념02 섹션 4) .q7 을 네 칸으로 나누세요. 크기는 모두 같습니다. 1fr 을 네 번 쓰지 말고 repeat() 를 쓰세요.

```
1
2
3
4
```

문제 8

문제 8 — 이름으로 배치하기

(개념02 섹션 5) .q8 에서 top 이 윗줄 두 칸을 차지하고, 아랫줄은 왼쪽이 left, 오른쪽이 right 가 되게 하세요.

```
top
left
right
```

문제 9

문제 9 — 미디어쿼리 없이 반응형 격자

(개념02 섹션 6) .q9 의 칸 하나는 최소 250px 이고, 자리가 남으면 남는 만큼 늘어나야 합니다. 빈 자리는 접혀야 합니다.

```
카드 1
카드 2
카드 3
카드 4
카드 5
카드 6
```

문제 10

문제 10 — 미디어쿼리로 2칸 만들기

(개념03 섹션 4·6) .q10 은 기본이 1칸입니다. 폭이 700px 이상일 때만 2칸이 되게 하세요.

카드 1
카드 2
카드 3
카드 4

문제 11

문제 11 — 사진이 액자 밖으로 안 나가게

(개념04 섹션 3) .q11-photo 가 액자 밖으로 나가지 않게 하세요. 가로세로 비율은 그대로 유지되어야 합니다.

문제 12

문제 12 — 썸네일을 꽉 채우기

(개념04 섹션 5) .q12-thumb 는 180 × 180 짜리 액자입니다. 지금은 그림이 찌그러져 있습니다(해가 세로로 긴 타원입니다). 비율을 지킨 채 액자를 꽉 채우고, 넘치는 부분은 잘리게 하세요.

문제 13

문제 13 [응용] — 글자 크기를 부드럽게

(개념04 섹션 4) .q13-title 의 글자 크기를 화면 폭의 2.5% 로 하되, 최소 16px, 최대 28px 을 벗어나지 않게 하세요. 미디어쿼리를 쓰지 말고 한 줄로 하세요.

우리 동네 도서관

문제 14

문제 14 [응용] — 모바일 우선 1단 → 2단 → 3단

(개념05 섹션 3·4·5) .q14 를 기본 1칸, 600px 이상 2칸, 900px 이상 3칸 으로 만드세요. 기본 규칙은 이미 되어 있습니다. 순서에 주의하세요.

```
카드 1
카드 2
카드 3
카드 4
카드 5
카드 6
```

문제 15

문제 15 [도전] — 반응형 페이지 뼈대

(개념02 섹션 5 + 개념05 섹션 7) .q15 를 좁은 화면에서는 네 덩어리가 세로로 쌓이게, 800px 이상에서는 head 아래에 왼쪽 main · 오른쪽 side(200px), 맨 아래 foot 이 오게 만드세요.

```
head
main
side
foot
```

문제 16

문제 16 [확인] — 왜 반대로 동작할까요

아래 .q16 은 “넓은 화면에서 3칸, 좁은 화면에서 1칸” 을 만들려고 쓴 것인데 정확히 반대로 동작합니다. 에러 메시지는 어디에도 안 뜹니다. <style> 맨 아래에서 잘못된 단어 한 개를 찾아 고치세요.

```
카드 1
카드 2
카드 3
```

13단원 연습문제 - Grid 와 반응형

푸는 방법

- 1) 이 파일을 VS Code 로 열고, 동시에 브라우저로도 엽니다
- 2) 아래 TODO 자리에 CSS 를 씁니다
- 3) 저장(Ctrl+S) 하고 브라우저에서 새로고침(F5) 합니다
- 4) 상자 모양이 '기대 화면' 과 같은지 봅니다

폭을 바꿔 봐야 하는 문제가 많습니다.

F12 → Ctrl+Shift+M 으로 기기 툴바를 켜 두세요(개념03 섹션 3).

★ 적혀 있는 px 값은 모두 '브라우저 창을 넓게(950px 이상) 열었을 때' 의 값입니다.

이 페이지의 본문은 최대 900px 까지만 넓어지므로, 그보다 넓은 창에서는 아무리 더 넓혀도 숫자가 안 변합니다.

반대로 창을 좁히면 본문이 같이 줄어들어 숫자가 전부 달라집니다.

1~8 번은 기기 툴바를 끄고 넓은 창에서 푸세요.

9 번부터는 문제마다 어느 폭에서 보라고 따로 적어 두었습니다.

1~12 는 기본, 13~14 는 응용, 15 는 도전, 16 은 확인 문제입니다.

아래 여섯 개는 이미 되어 있습니다. 고치지 마세요

.q-scroll 은 아직 안 푼 문제(8번, 11번) 때문에 상자가 옆으로 빠져나가더라도

자료 페이지 전체가 밀리지 않게 막아 주는 껍데기입니다(10단원 overflow).

```
.q-scroll {
  overflow-x: auto;
}
.q-cell {
  background: #dbe7fb;
  border: 1px solid #9ab8e8;
  padding: 10px;
  text-align: center;
}
.q-hi {
  background: #ffe0b3;
  border-color: #e8a33d;
}
.q-frame {
  max-width: 300px;
```

```
padding: 6px;
}
.q-band {
  background: #f4f4f4;
  border: 1px solid #cccccc;
  padding: 10px;
  text-align: center;
}
.q-dark {
  background: #333333;
  color: #ffffff;
  padding: 8px;
  text-align: center;
}
```

문제 1 (개념01 섹션 1·2)

.q1 을 격자로 만들고 칸을 세 개로 나누세요.
칸 크기는 셋 다 같아야 하고, 칸 사이는 10px 입니다.
세 줄이 필요합니다.

기대 화면: 파란 상자 세 개가 가로로 나란히 놓이고 폭이 각각 268px 입니다.

이렇게 되면 틀린 것: 상자가 세로로 쌓이면 display: grid 가 빠진 것입니다.

```
.q1 {
```

여기에 코드를 쓰세요

```
}
```

문제 2 (개념01 섹션 3)

.q2 의 왼쪽 칸은 200px 로 고정하고, 오른쪽 칸은 남은 자리를 전부 쓰게 하세요.

display 와 gap 은 이미 되어 있습니다. 한 줄만 쓰면 됩니다.

기대 화면: 왼쪽 칸이 200px, 오른쪽 칸이 614px 입니다.

이렇게 되면 틀린 것: 두 칸이 똑같은 폭이면 오른쪽에 1fr 대신 200px 을 쓴 것입니다.

```
.q2 {  
  display: grid;  
  gap: 10px;
```

여기에 코드를 쓰세요

```
}
```

문제 3 (개념01 섹션 4)

.q3 의 위아래 줄 사이는 24px, 좌우 칸 사이는 10px 로 띄우세요.
값 두 개를 한 줄로 씁니다.

기대 화면: 좌우보다 위아래가 눈에 띄게 넓게 벌어집니다. 칸 폭은 268px
입니다.

이렇게 되면 틀린 것: 좌우가 더 넓게 벌어졌다면 두 값의 순서가 바뀐 것입니다.

```
.q3 {  
  display: grid;  
  grid-template-columns: 1fr 1fr 1fr;
```

여기에 코드를 쓰세요

```
}
```

문제 4 (개념01 섹션 5)

.q4 의 첫째 줄 높이를 50px, 둘째 줄 높이를 90px 로 정하세요.
한 줄만 쓰면 됩니다.

기대 화면: 윗줄 두 칸이 얇고 아랫줄 두 칸이 두껍습니다. 칸 폭은 407px
입니다.

이렇게 되면 틀린 것: 네 칸의 높이가 다 같으면 grid-template-rows 가 아니라

grid-template-columns 를 고친 것입니다.

```
.q4 {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 10px;
```

여기에 코드를 쓰세요

```
}
```

문제 5 (개념02 섹션 2)

.q5-wide 가 1번 선에서 3번 선까지, 즉 두 칸을 차지하게 하세요.
부모(.q5) 는 이미 3칸으로 나뉘어 있습니다.

기대 화면: 주황 칸이 546px 로 넓어져 두 칸을 차지합니다.

이렇게 되면 틀린 것: 주황 칸이 268px 그대로면 선 번호를 하나 적게 센 것입니다.

```
.q5 {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 10px;
}
.q5-wide {
```

여기에 코드를 쓰세요

```
}
```

문제 6 (개념02 섹션 3)

.q6-wide 가 세 칸을 통째로 차지하게 하세요.
이번에는 선 번호 대신 span 을 쓰세요.

기대 화면: 주황 칸이 824px 로 상자 전체를 가로지르고, 나머지 칸은 아랫줄로
깎입니다.

이렇게 되면 틀린 것: 주황 칸이 268px 이고 자리만 옮겨졌다면

span 없이 숫자만 쓴 것입니다.

```
.q6 {  
  display: grid;  
  grid-template-columns: 1fr 1fr 1fr;  
  gap: 10px;  
}  
.q6-wide {
```

여기에 코드를 쓰세요

```
}
```

문제 7 (개념02 섹션 4)

.q7 을 네 칸으로 나누세요. 네 칸의 크기는 모두 같습니다.
1fr 을 네 번 쓰지 말고 repeat() 를 쓰세요.

기대 화면: 칸 네 개가 200px 씩 한 줄에 놓입니다.

이렇게 되면 틀린 것: 칸이 하나도 안 나뉘면 `repeat()` 괄호 안 쉼표를 빼뜨린 것입니다.

```
.q7 {
  display: grid;
  gap: 8px;
```

여기에 코드를 쓰세요

```
}
```

문제 8 (개념02 섹션 5)

.q8 을 이렇게 배치하세요.

윗줄 : top 이 두 칸을 통째로 차지

아랫줄: 왼쪽이 left, 오른쪽이 right

자식의 grid-area 이름은 이미 붙어 있습니다. 이름을 늘어놓는 줄만 쓰면 됩니다.

기대 화면: 위에 파란 띠가 824px 로 놓이고, 그 아래 왼쪽 200px · 오른쪽 614px 입니다.

이렇게 되면 틀린 것: 세 덩어리가 세로로 쌓이면 줄마다 이름 개수가 안 맞는 것입니다.

```
.q8 {
  display: grid;
  grid-template-columns: 200px 1fr;
  gap: 10px;
```

여기에 코드를 쓰세요

```

}
.q8-top {
  grid-area: top;
}
.q8-left {
  grid-area: left;
}
.q8-right {
  grid-area: right;
}

```

문제 9 (개념02 섹션 6)

.q9 를 미디어쿼리 없이 반응형으로 만드세요.
칸 하나는 최소 250px 이고, 자리가 남으면 남는 만큼 늘어나야 합니다.
빈 자리는 접혀야 합니다.

기대 화면: 폭 1280 에서 268px 짜리 3칸, 폭 768 에서 341px 2칸,
폭 480 에서 404px 1칸이 됩니다.

이렇게 되면 틀린 것: 칸이 하나도 안 나뉘면 minmax 의 순서가 반대인 것입니다.

```

.q9 {
  display: grid;
  gap: 10px;

```

여기에 코드를 쓰세요

```

}

```

문제 10 (개념03 섹션 4·6)

.q10 은 기본이 1칸입니다. 폭이 700px 이상일 때만 2칸이 되게 하세요.
아래에 @media 블록을 통째로 쓰세요. 중괄호가 두 겹이라는 것을 기억하세요.

기대 화면: 폭 1280 에서 407px 짜리 2칸, 폭 480 에서 404px 짜리 1칸입니다.

이렇게 되면 틀린 것: 좁을 때 2칸이 되면 min-width 와 max-width 를 바꿔 쓴 것입니다.

```
.q10 {
  display: grid;
  grid-template-columns: 1fr;
  gap: 10px;
}
```

여기에 @media 블록을 쓰세요

문제 11 (개념04 섹션 3)

.q11-photo 가 액자 밖으로 나가지 않게 하세요.
그림의 가로세로 비율은 그대로 유지되어야 합니다. 한 줄이면 됩니다.
기대 화면: 그림이 300 × 150 으로 줄어들어 회색 액자 안에 들어갑니다.

이렇게 되면 틀린 것: 그림이 세로로 눌러 보이면 height 를 같이 정한 것입니다.

```
.q11-photo {
```

여기에 코드를 쓰세요

```
}
```

문제 12 (개념04 섹션 5)

.q12-thumb 는 180 × 180 짜리 액자입니다. 지금은 그림이 찌그러져 있습니다. 비율을 지킨 채 액자를 꽉 채우고, 넘치는 부분은 잘리게 하세요. 한 줄이면 됩니다.

기대 화면: 산이 찌그러지지 않고 원래 비율로 보입니다. 액자 크기는 그대로 180 × 180 이고,

그림 좌우가 잘려서 오른쪽에 있던 해는 아예 안 보이게 됩니다.

이렇게 되면 틀린 것: 위아래에 흰 여백이 생기면 cover 가 아니라 contain 을 쓴 것입니다.

해가 세로로 긴 타원으로 보이면 아직 기본값 fill 인 것입니다.

```
.q12-thumb {  
  width: 180px;  
  height: 180px;  
  border: 1px solid #bbbbbb;  
  box-sizing: border-box;  
}
```

여기에 코드를 쓰세요

```
}
```

문제 13 [응용] (개념04 섹션 4)

.q13-title 의 글자 크기를 화면 폭에 따라 부드럽게 바꾸세요.
기준은 화면 폭의 2.5%, 최소 16px, 최대 28px 입니다.
미디어쿼리를 쓰지 말고 한 줄로 하세요.

기대 화면: 폭 1280 에서 28px, 폭 768 에서 19.2px, 폭 480 에서 16px 입니다.

이렇게 되면 틀린 것: 폭을 바꿔도 글자 크기가 그대로면 vw 대신 px 을 쓴 것입니다.

```
.q13-title {  
  font-weight: bold;  
  margin: 0;  
}
```

여기에 코드를 쓰세요

```
}
```

문제 14 [응용] (개념05 섹션 3·4·5)

.q14 를 모바일 우선으로 만드세요.

기본 1칸

600px 이상 2칸

900px 이상 3칸

기본 규칙은 이미 되어 있습니다. @media 블록 두 개를 아래에 쓰세요.

순서에 주의하세요.

기대 화면: 폭 1280 에서 268px 3칸, 폭 768 에서 341px 2칸, 폭 480 에서 404px 1칸.

이렇게 되면 틀린 것: 폭 1280 에서 3칸이 아니라 2칸이면 두 블록의 순서가 뒤바뀐 것입니다.

```
.q14 {
  display: grid;
  grid-template-columns: 1fr;
  gap: 10px;
}
```

여기에 @media 블록 두 개를 쓰세요

문제 15 [도전] (개념02 섹션 5 + 개념05 섹션 7)

.q15 를 반응형 페이지 뼈대로 만드세요.

기본(좁은 화면) head, main, side, foot 이 위에서 아래로 한 줄씩
800px 이상 head 가 위, 그 아래 왼쪽 main · 오른쪽 side, 맨 아래

foot

오른쪽 side 의 폭은 200px 로 고정

자식의 grid-area 이름은 이미 붙어 있습니다.

기본 규칙과 @media 블록을 모두 직접 쓰세요.

기대 화면: 폭 1280 에서 main 이 614px, side 가 200px 로 나란히 놓입니다.

폭 480 에서는 네 덩어리가 404px 폭으로 세로로 쌓입니다.

이렇게 되면 틀린 것: 폭 480 에서도 두 칸으로 나뉘면 기본에 열을 두 개 쓴 것입니다.

```
.q15 {  
  display: grid;  
  gap: 10px;
```

여기에 기본 규칙(열과 areas)을 쓰세요

```
}  
.q15-head {  
  grid-area: head;  
}  
.q15-main {  
  grid-area: main;  
}  
.q15-side {  
  grid-area: side;  
}  
.q15-foot {  
  grid-area: foot;  
}
```

여기에 800px 이상일 때의 @media 블록을 쓰세요

문제 16 [확인] (개념03 섹션 5·7 + 개념05 섹션 8)

아래 .q16 코드는 "넓은 화면에서 3칸, 좁은 화면에서 1칸" 을 만들려고 쓴 것입니다.

그런데 정확히 반대로 동작합니다. 예러 메시지는 어디에도 안 뜹니다.

지금 실제로 벌어지는 일

폭 1280 에서 칸이 안 나뉘고 카드가 세로로 한 줄씩 쌓입니다

폭 480 으로 좁히면 오히려 3칸이 되어 카드가 128px 로 좁아집니다

잘못된 곳은 한 군데입니다. 값도 선택자도 아니고, 단어 하나입니다.

무엇이 잘못됐는지 말로 설명해 본 뒤에 고치세요.

기대 화면: 폭 1280 에서 268px 짜리 3칸, 폭 480 에서 404px 짜리 1칸입니다.

이렇게 되면 틀린 것: 폭 1280 에서 여전히 한 줄에 하나씩 쌓이면

아직 안 고쳤거나 값을 잘못 건드린 것입니다.

```
.q16 {
  display: grid;
  grid-template-columns: 1fr;
  gap: 10px;
}
@media (max-width: 700px) {
  .q16 {
    grid-template-columns: 1fr 1fr 1fr;
  }
}
```

자주 넘어지는 자리

- 1 `Ctrl+Shift+M` 으로 화면 폭을 줄여 카드가 접히는 것
- 2 `viewport meta` 를 지우면 휴대폰에서 어떻게 되는지

종합 연습

OPEN 14_종합_연습

종합 01 - 프로필 카드

단계 3 부터는 아래 각 규칙의 자리에 코드를 씁니다.
단계마다 화면이 어떻게 바뀌어야 하는지는
body 안 각 단계의 주석에 적어 두었습니다.

01 프로필 카드

02 음식점 메뉴판

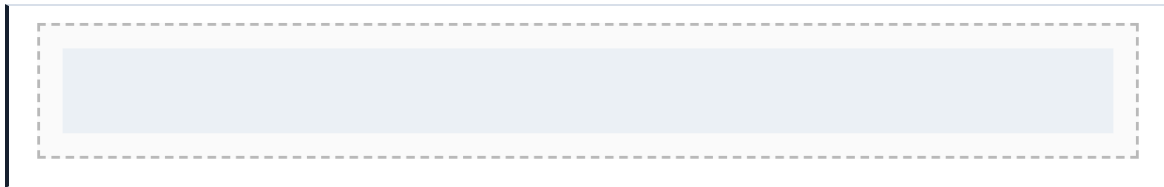
03 회원가입 폼

04 반응형 랜딩페이지

프로필 카드

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.



실행하면 나오는 화면 — 프로필 카드

자기소개 카드 한 장을 처음부터 끝까지 만듭니다. 01단원부터 10단원까지 배운 것만 씁니다.

여기서 쓰지 않는 것이 하나 있습니다. 바로 배치입니다. display, position, flex, grid 는 11단원부터 나옵니다. 지금은 쓰지 마세요. 그것들 없이도 카드 한 장은 충분히 그럴듯하게 나옵니다.

왜냐하면 블록 요소는 아무것도 시키지 않아도 한 줄을 통째로 차지하면서 위에서 아래로 착착 쌓이기 때문입니다. 카드는 원래 그렇게 쌓이는 모양입니다. 흐름을 거스를 일이 없습니다.

```
article (카드 전체)
  p      사진
  h2     이름
  p      직함
  p      한 줄 소개
  ul     정보 목록
  p      연락 링크
```

여러분이 할 일은 이 순서대로 쌓인 상자들에 색 · 여백 · 테두리 · 글자 모양을 입히는 것뿐입니다.

단계 1 과 2 는 HTML 을 직접 씁니다. 단계 3 부터는 위쪽 style 안을 채웁니다.

단계를 순서대로 하나씩 채우세요. 한 단계마다 저장하고 새로고침해서 어디가 달라졌는지 눈으로 확인하는 것이 이 연습의 핵심입니다. 각 단계의 자세한 설명과 확인 방법은 이 파일을 VS Code 로 열면 주석에 있습니다. 막히면 종합01_프로필_카드_정답.html 을 여세요.

정리 —————

단계 1 —————

아래 회색 무대 상자 안에 카드의 뼈대를 직접 쓰세요.

1) 카드 한 장 전체를 `<article class="profile-card">` 로 감쌉니다.

카드는 그 자체로 뜻이 통하는 덩어리라서 div 보다 article 이 어울립니다.

2) 그 안에 이름을 h2, 직함을 p 로 넣습니다.

클래스 이름은 profile-name, profile-job 으로 정확히 붙이세요.
위쪽 style 이 이 이름을 찾아 쓰기 때문입니다.

3) 이름과 직함의 내용은 여러분 마음대로 정해도 됩니다.

기대 출력 회색 무대 안에 이름이 크고 굵게 한 줄, 그 아래 직함이 한 줄 보입니다.
아직 아무 색도 여백도 없습니다. 글자 두 줄이 전부입니다.
이렇게 되면 틀린 것: 이름과 직함이 한 줄에 나란히 붙었다면 태그를 span
으로 쓴 것입니다.
h2 와 p 는 블록 요소라 각자 한 줄을 차지합니다.

단계 2

카드 안을 마저 채웁니다. 순서가 중요합니다. 쓴 순서대로 위에서 아래로 쌓입니다.

1) 이름보다 위에 사진을 넣습니다.

`<p class="profile-photo-box">` 로 감싸고 그 안에 `img` 를 넣으세요.
그림 파일은 이미지/프로필.svg 입니다. alt 도 반드시 씁니다.
`img` 에는 `class="profile-photo"` 를 붙입니다.
왜 p 로 한 번 감싸는지는 단계 6 에서 알게 됩니다.

2 직함 아래에 한 줄 소개를 `<p class="profile-intro">` 로 넣습니다.

3 그 아래에 정보 목록을 `<ul class="profile-info">` 로 넣고 li 를 세 개 씁니다.

각 li 안의 항목 이름은 `<strong>` 으로 감쌉니다.
예: `<li><strong>사는 곳</strong> 서울 마포구</li>`

4) 맨 아래에 연락 링크를 넣습니다.

`<p class="profile-contact">` 안에
`<a class="profile-link" href="mailto:주소">메일 보내기</a>` 를 씁니다.

기대 출력 위에서부터 네모난 사진 → 이름 → 직함 → 소개 → 글머리 기호가 붙은 목록 세 줄
→ 파란 밑줄 링크 순서로 쌓입니다. 사진은 왼쪽에 붙어 있습니다.
이렇게 되면 틀린 것: 사진 자리에 깨진 그림 표시와 alt 글자가 보이면
경로가 틀린 것입니다.
이미지 폴더는 이 파일과 같은 자리에 있으므로 앞에 ../ 를 붙이지 않습니다.

단계 1 과 단계 2 의 HTML 을 이 자리에 쓰세요

단계 3

style 안 .profile-card 규칙의 단계 3 자리에 카드의 배경색을 흰색(#ffffff), 글자색을 진회색(#333333)으로 정하세요.

기대 출력 회색 무대 위에 흰 띠가 가로로 쭉 깔립니다. 그것이 카드입니다.
아직 폭을 안 줬기 때문에 카드가 무대의 가로를 통째로 차지합니다.
이렇게 되면 틀린 것: 아무 변화가 없으면 색 값에 # 을 빠뜨렸거나
세미콜론이 없는 것입니다.
CSS 는 값이 조금이라도 틀리면 그 줄을 통째로 버립니다.

단계 4

같은 .profile-card 규칙의 단계 4 자리에 카드의 가로를 320px 로 정하고 무대의 좌우 한가운데로 보내세요. 가운데로 보내는 것은 margin 의 일입니다.

기대 출력 흰 띠가 가로 320px 짜리 세로로 긴 상자로 줄어들고 무대 한가운데에 놓입니다.
카드 왼쪽에 남는 회색 자리와 오른쪽에 남는 회색 자리의 폭이 똑같습니다.
이렇게 되면 틀린 것: 카드가 왼쪽에 붙어 있다면 좌우 값을 auto 로 안 준 것입니다.
글자만 가운데로 갔다면 text-align 을 쓴 것입니다. 그것은 다른 일입니다.

단계 5

같은 .profile-card 규칙의 단계 5 자리에 세 가지를 주세요.

- 1 사방 24px 짜리 안쪽 여백
- 2 1px 두께의 연한 회색(#d8dee8) 실선 테두리
- 3 14px 만큼 둥근 모서리

기대 출력 글자가 카드 벽에서 24px 떨어지고, 카드 둘레에 얇은 회색 실선이 그어지고, 네 귀퉁이가 부드럽게 둥글어집니다.
그리고 카드가 좌우로 조금 넓어집니다. 바깥 폭이 320px 에서 370px 이 됩니다.
이렇게 되면 틀린 것: 폭이 320px 그대로라면 어디선가 box-sizing: border-box 가 켜진 것입니다.
테두리가 아예 안 보이면 모양(solid) 을 빠뜨린 것입니다.

단계 6

사진을 동그랗게 만들고 카드 가운데로 보냅니다.

1) .profile-photo 에 가로 120px, 세로 120px 을 주고

모서리를 50% 깎고, 4px 두께의 파란(#2d6cdf) 실선 테두리를 두르세요.

2) .profile-photo-box 에는 안쪽 글자를 가운데로 보내는 속성과

바깥 여백 margin: 0 0 14px 을 주세요.

기대 출력 네모였던 사진이 완전한 동그라미가 되고 파란 테를 두른 채 카드 한가운데 놓입니다.
테두리까지 합쳐 사진이 차지하는 폭은 128px 입니다.
이렇게 되면 틀린 것: 사진이 타원이면 가로와 세로 중 하나를 빠뜨린 것입니다.
사진이 여전히 왼쪽에 붙어 있다면 img 에 margin: 0 auto 를 준 것입니다.
img 는 인라인이라 그 방법이 통하지 않습니다.

단계 7

이름과 직함을 다듬습니다.

1) .profile-name — 바깥 여백 margin: 0 0 4px, 글자 크기 1.75rem,

글자를 가운데로, 글자 사이를 1px 벌리기

2) .profile-job — 바깥 여백 margin: 0 0 16px, 글자 크기 0.875rem,

글자색 #6b7280, 글자를 가운데로

기대 출력 이름이 28px 로 커지면서 가운데로 오고, 직함이 그 바로 아래(4px 아래) 에 14px 회색 글자로 붙습니다. 둘이 한 덩어리로 보입니다.
이렇게 되면 틀린 것: 이름과 직함 사이가 멀찍이 떨어져 있다면 margin 을 0 으로 안 누른 것입니다.
h2 와 p 에는 브라우저가 미리 위아래 여백을 넣어 줍니다.

단계 8

한 줄 소개에 왼쪽 강조 띠를 붙입니다. .profile-intro 에 이렇게 주세요.

· 바깥 여백 margin: 0 0 18px · 안쪽 여백 padding: 8px 0 8px 12px · 왼쪽에만 4px 두께의 주황 (#f0a830) 실선 · 배경색 #fff8e8 · 글자 크기 0.9375rem, 줄 높이 1.6

기대 출력 소개 문단의 왼쪽 끝에 주황색 세로 막대가 서고 문단 전체에 연노랑 배경이 깔립니다.
글자는 막대에서 12px 떨어져 시작합니다.
이렇게 되면 틀린 것: 네 방향에 다 주황 선이 그어졌다면 border 단축을 쓴 것입니다.
글자가 주황 막대에 딱 붙어 있다면 padding 의 왼쪽 값이 0 인 것입니다.

단계 9

정보 목록을 정리합니다. 규칙 세 개를 채웁니다.

· `.profile-info` — `margin: 0 0 18px`, 왼쪽 안쪽 여백 `20px`, 글자 크기 `0.9375rem` · `.profile-info li` — 위아래 `6px` 안쪽 여백, 아래쪽에만 `1px` 연회색(`#e0e0e0`) 점선 · `.profile-info strong` — 글자색 `#2d6cdf`

점선은 짧은 막대가 이어진 모양입니다. 동그란 점이 아닙니다.

기대 출력 목록이 왼쪽으로 당겨져 카드 안에 자리를 잘 잡고, 세 줄 각각의 아래에 연회색 점선이 한 줄씩 그어집니다. 항목 이름 세 개만 파란색이 됩니다. 이렇게 되면 틀린 것: 목록 전체가 파래졌다면 선택자에서 `strong` 을 빼먹은 것입니다. 글머리 기호만 카드 왼쪽 벽에 바짝 붙어 위 문단들과 어긋나 보인다면 왼쪽 안쪽 여백을 `0` 으로 준 것입니다. 글머리 기호는 `ul` 의 안쪽 여백 자리에 그려지기 때문입니다.

단계 10

마지막입니다. 연락 링크를 눈에 띄는 띠로 만듭니다.

· `.profile-contact` — `margin: 0`, 위아래 `12px` 안쪽 여백, 글자를 가운데로,

배경색 `#2d6cdf`, 모서리 `8px` 둥글게

· `.profile-link` — 글자색 흰색, 밑줄 없애기, 굵게, 글자 사이 `1px` · `.profile-link:hover` — 밑줄 되살리기

배경을 `a` 가 아니라 `a` 를 감싼 `p` 에 주는 것이 요령입니다. 왜 그런지 생각해 보세요. 힌트는 “`a` 는 인라인이다” 입니다.

기대 출력 카드 맨 아래에 파란 띠가 가로로 깔리고 그 위에 흰 글자로 “메일 보내기” 가 가운데 놓입니다. 밑줄은 없습니다. 글자 위에 마우스를 올리면 밑줄이 나타납니다. 이렇게 되면 틀린 것: 파란 배경이 글자 뒤에만 좁게 칠해졌다면 배경색을 `a` 에 준 것입니다. 마우스를 올려도 아무 변화가 없다면 `:hover` 를 띄어 썼는지 확인하세요. `.profile-link:hover` 처럼 반드시 붙여 씁니다.

종합 01 - 프로필 카드

단계 3 부터는 아래 각 규칙의 자리에 코드를 씁니다.

한 단계를 채울 때마다 저장(`Ctrl+S`) 하고 브라우저에서 `F5` 를 누르세요.

단계마다 화면이 어떻게 바뀌어야 하는지는 `body` 안 각 단계의 주석에 적어 두었습니다.

카드가 놓이는 무대입니다. 미리 넣어 두었으니 고치지 마세요.

```
.card-stage {
  background-color: #eef2f7;
  padding: 30px 0;
}
```

단계 3 · 4 · 5 는 이 규칙 하나를 함께 채웁니다

```
.profile-card {
```

TODO 단계 3: 배경색과 글자색

TODO 단계 4: 폭과 가운데 정렬

TODO 단계 5: 안쪽 여백 · 테두리 · 둥근 모서리

```
}
```

단계 6

```
.profile-photo-box {
```

여기에 코드를 쓰세요

```
}
.profile-photo {
```

여기에 코드를 쓰세요

```
}
```

단계 7

```
.profile-name {
```

여기에 코드를 쓰세요

```
}  
.profile-job {
```

여기에 코드를 쓰세요

```
}
```

단계 8

```
.profile-intro {
```

여기에 코드를 쓰세요

```
}
```

단계 9

```
.profile-info {
```

여기에 코드를 쓰세요

```
}
.profile-info li {
```

여기에 코드를 쓰세요

```
}
.profile-info strong {
```

여기에 코드를 쓰세요

```
}
```

단계 10

```
.profile-contact {
```

여기에 코드를 쓰세요

```
}
.profile-link {
```

여기에 코드를 쓰세요

```
}  
.profile-link:hover {
```

여기에 코드를 쓰세요

```
}
```


음식점 메뉴판

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.



실행하면 나오는 화면 — 음식점 메뉴판

작은 식당의 메뉴판 한 장을 처음부터 끝까지 만듭니다. 01단원부터 12단원까지 배운 것을 씁니다.

종합 01 의 프로필 카드와 결정적으로 다른 점이 하나 있습니다. 카드가 여러 장이고, 그 여러 장이 가로로 나란히 놓여야 한다는 것입니다.

종합 01	카드 한 장	→ 위에서 아래로 쌓이는 기본 흐름으로 충분
종합 02	카드 여섯 장	→ 가로로 배치할 도구가 필요함

그 도구가 12단원의 flex 입니다. 이 연습의 절반은 flex 를 다시 꺼내 쓰는 일입니다.

.menu-list	카드 여섯 장을 가로로 늘어놓고 줄바꿈시키는 컨테이너
.menu-card	바깥에서는 아이템, 안에서는 또 다른 세로 컨테이너
.menu-price-row	이름과 가격을 양끝으로 밀어내는 가장 작은 컨테이너

컨테이너가 세 겹으로 포개져 있습니다. 겹먹을 것 없습니다. 각 컨테이너는 자기 바로 아래 자식만 봅니다. 한 겹씩 따로 생각하면 됩니다.

쓰지 않는 것이 두 가지 있습니다. grid 와 @media 입니다. 둘 다 13단원 내용입니다. 여기서는 flex 만으로 만듭니다. 그런데도 브라우저 폭을 줄이면 카드가 저절로 접힙니다. 단계 7 에서 확인하세요.

단계 1 · 2 · 3 은 HTML 을 직접 씁니다. 단계 4 부터는 위쪽 style 안을 채웁니다.

단계를 순서대로 하나씩 채우세요. 한 단계마다 저장하고 새로고침해서 어디가 달라졌는지 눈으로 확인하는 것이 이 연습의 핵심입니다. 각 단계의 자세한 설명과 확인 방법은 이 파일을 VS Code 로 열면 주석에 있습니다. 막히면 종합02_음식점_메뉴판_정답.html 을 여세요.

정리 —————

단계 1

아래 살구색 무대 상자 안에 메뉴판의 머리글을 직접 쓰세요.

1 머리글 전체를 `<header class="shop-head">` 로 감쌉니다.

2 그 안에 가게 이름을 `h2`, 한 줄 소개를 `p` 로 넣습니다.

클래스 이름은 `shop-name`, `shop-tagline` 으로 정확히 붙이세요.
위쪽 `style` 이 이 이름을 찾아 쓰기 때문입니다.

3) 가게 이름과 소개 내용은 여러분 마음대로 정해도 됩니다.

이 페이지 맨 위에는 이미 자료 제목용 `h1` 이 있습니다. 그래서 가게 이름은 `h1` 이 아니라 `h2` 로 씁니다.

기대 출력 살구색 무대 안에 가게 이름이 크고 굵게 한 줄, 그 아래 소개가 한 줄
보입니다.
아직 아무 색도 없습니다. 검은 글자 두 줄이 전부입니다.
이렇게 되면 틀린 것: 두 줄이 한 줄에 나란히 붙었다면 `span` 을 쓴 것입니다.
`h2` 와 `p` 는 블록 요소라 각자 한 줄을 차지합니다.

단계 2

머리글 아래에 카테고리 띠와 메뉴 카드를 씁니다.

1) 카테고리 띠는 `<p class="menu-tabs">` 안에 `<span class="menu-tab">` 네 개입니다.

내용은 밥 / 면 / 분식 / 후식 입니다.

2 카드 여섯 장을 감싸는 `<div class="menu-list">` 를 만듭니다.

3 그 안에 `<article class="menu-card">` 를 여섯 개 씁니다. 한 장은 이 순서입니다.

```
<span class="menu-badge">인기</span>      ← 첫 카드에만 넣습니다

<h3 class="menu-name">메뉴 이름</h3>
<p class="menu-desc">메뉴 설명</p>
<p class="menu-price-row">
  <span class="menu-label">한 그릇</span>
  <span class="menu-price">9,000원</span>
</p>
```

4) 그림 파일은 이미지 폴더에 여섯 개가 들어 있습니다.

이미지/메뉴_비빔밥.svg · 메뉴_잔치국수.svg · 메뉴_김치만두.svg ·
메뉴_떡볶이.svg · 메뉴_수정과.svg · 메뉴_팔빙수.svg
alt 도 반드시 씁니다.

5) 설명 길이는 카드마다 일부러 다르게 쓰세요. 단계 8 에서 그 차이가 쓰입니다.

기대 출력 밥 면 분식 후식 네 글자가 한 줄에 이어 붙고,
그 아래로 카드 여섯 장이 세로로 길게 쌓입니다.
카드마다 작은 그림(가로 160px) 이 왼쪽에 놓고 이름 · 설명 ·
"한 그릇 9,000원" 이 이어집니다. 첫 카드 맨 위에는 "인기" 가 보입니다.
이렇게 되면 틀린 것: 그림 자리에 깨진 표시와 alt 글자가 보이면 경로가
틀린 것입니다.
이미지 폴더는 이 파일과 같은 자리에 있으므로 앞에 ../ 를 붙이지 않습니다.
카드가 여섯 장 다 안 보이면 article 을 닫지 않은 것입니다.

단계 3

카드 아래에 세트 표 · 안내 · 바닥글을 씁니다.

1) <h3 class="set-title">세트 메뉴</h3> 를 쓰고

그 아래 <table class="set-table"> 을 만듭니다.
thead 안에 머리카 세 개(세트 이름 · 구성 · 가격) 를 th 로,
tbody 안에 세트 세 줄을 td 로 씁니다.

2 <ul class="shop-notice"> 에 안내 세 줄을 li 로 씁니다.

3 맨 아래 <footer class="shop-foot"> 안에 p 두 개를 씁니다.

하나는 영업시간, 하나는 전화번호입니다.
클래스는 shop-hours, shop-tel 로 붙이세요.

기대 출력 "세트 메뉴" 라는 제목 아래에 선 없는 표가 세 줄 나옵니다.
표 폭은 글자 길이만큼만(약 350px) 이고 머리카 세 개만 굵게 가운데
놓입니다.
그 아래 글머리 기호가 붙은 안내 세 줄, 맨 아래 영업시간과 전화번호가
각각 한 줄씩 세로로 놓입니다.
이렇게 되면 틀린 것: 표에 격자선이 이미 보인다면 br 이나 hr 을 섞어 쓴
것입니다.
지금은 선이 없는 것이 정상입니다. 선은 단계 10 에서 긋습니다.

단계 1 · 2 · 3 의 HTML 을 이 자리에 쓰세요

단계 4

style 안 단계 4 자리 세 곳을 채웁니다.

· .shop-head — 배경색 #3a2b21, 글자색 #ffffff,

안쪽 여백 padding: 20px 16px, 12px 둥근 모서리,
글자를 가운데로, 바깥 여백 margin: 0 0 18px

· .shop-name — margin: 0 0 6px, 글자 크기 1.75rem, 글자 사이 2px · .shop-tagline — margin: 0,
글자 크기 0.875rem, 글자색 #e6d3bd

기대 출력 살구색 무대 위쪽에 짙은 갈색 띠(높이 117px) 가 깔리고, 그 안에
가게 이름이 28px 흰 글자로, 소개가 14px 연한 베이지 글자로 가운데
놓입니다.
이렇게 되면 틀린 것: 갈색 띠가 두툼하고 이름과 소개가 멀찍이 떨어져
있다면
margin 을 0 으로 안 누른 것입니다. h2 와 p 에는 브라우저가
위아래 여백을 미리 넣어 둡니다.
소개 글자까지 흰색이면 .shop-tagline 의 color 를 빼뜨린 것입니다.

단계 5

카테고리 띠를 가로 한 줄로 만듭니다. 이 연습에서 처음 쓰는 flex 입니다.

· .menu-tabs — display 를 flex 로, 주축 정렬을 가운데로,

줄바꿈을 켜고, 아이템 사이 간격 10px,
바깥 여백 margin: 0 0 18px

· .menu-tab — 배경색 #f3e3cd, 1px 실선 테두리 #e2d5c3,

999px 둥근 모서리, 안쪽 여백 padding: 6px 18px,
글자 크기 0.875rem, 글자색 #5a4326

display: flex 는 아이템이 아니라 컨테이너에 씁니다. 여기서 컨테이너는 span 이 아니라 span 을 감싼 p
입니다.

기대 출력 밥 면 분식 후식 네 글자가 알약 모양 딱지가 되어
10px 간격으로 무대 한가운데 한 줄로 놓입니다. 딱지 높이는 38px 입니다.
이렇게 되면 틀린 것: 딱지가 위아래로 납작하게 눌러 있다면 display: flex
를
p 가 아니라 span 에 쓴 것입니다. span 은 원래 인라인이라
위아래 padding 을 줘도 줄 높이가 늘어나지 않습니다.
딱지가 왼쪽에 몰려 있다면 justify-content 를 빼뜨린 것입니다.

단계 6

카드 한 장의 모양을 만듭니다. 아직 배치는 건드리지 않습니다.

· .menu-card — box-sizing 을 border-box 로, 배경색 #ffffff,

1px 실선 테두리 #e2d5c3, 12px 둥근 모서리, 안쪽 여백 14px
(단계 6 자리에만 씁니다. 아래 TODO 세 개는 아직 비워 두세요)

· .menu-photo — 가로를 부모의 100% 로, 세로는 auto,

8px 둥근 모서리, 바깥 여백 margin: 0 0 10px

· .menu-name — margin: 0 0 6px, 글자 크기 1.0625rem · .menu-desc — margin: 0 0 12px, 글자 크기 0.875rem,

줄 높이 1.5, 글자색 #6b5a48

기대 출력 카드 여섯 장이 흰 상자가 되어 여전히 세로로 쌓입니다.
카드 폭은 무대를 꽉 채운 776px 이고, 그림이 카드 안쪽 폭에 맞춰
746px 까지 커집니다. 화면이 아주 길어집니다. 지금은 그게 정상입니다.
이렇게 되면 틀린 것: 그림이 옆으로만 늘어나 납작해 보인다면 height 에
숫자를 쓴 것입니다.
height 는 auto 여야 가로세로 비율이 유지됩니다.

단계 7

이제 카드를 가로로 늘어놓습니다.

· .menu-list — display 를 flex 로, 줄바꿈을 켜고, 아이템 사이 간격 16px,

바깥 여백 margin: 0 0 22px

· .menu-card — 단계 7 자리에 flex 단축을 한 줄 씩입니다.

"220px 에서 시작해서, 남으면 같이 늘고, 모자라면 같이 준다" 입니다.
값 세 개를 순서대로 쓰면 됩니다.

기대 출력 카드 여섯 장이 한 줄에 세 장씩 두 줄로 놓입니다.
카드 하나의 폭은 248px, 그림은 218px 이 됩니다.
한 줄 안에서는 카드 키가 똑같아집니다. 윗줄 세 장이 368px,
아랫줄 세 장이 341px 입니다. 키가 맞춰지는 범위는 '한 줄 안' 이라서
줄이 다르면 높이가 달라도 됩니다(12단원 개념06 섹션2 에서 본
그대로입니다).
그런데 설명이 짧은 카드는 가격 줄 아래에 흰 빈자리가 남습니다.
그것을 단계 8 에서 없앱니다.
이렇게 되면 틀린 것: 카드 여섯 장이 한 줄에 억지로 들어가 잔뜩
찌그러졌다면
flex-wrap 을 빠뜨린 것입니다. 기본값이 nowrap 이기 때문입니다.
카드가 한 줄에 두 장만 들어간다면 단계 6 의 box-sizing 이 빠진 것입니다.

여기서 한 번 해 보세요. 브라우저 창의 오른쪽 끝을 잡고 폭을 줄여 보세요. 한 줄에 세 장이던 카드가 두 장, 한 장으로 저절로 접힙니다. @media 를 한 줄도 안 썼는데 그렇게 됩니다.

단계 8

카드 안쪽을 정리해서 가격 줄을 카드 바닥에 붙입니다.

· .menu-card — 단계 8 자리에 display 를 flex 로, 방향을 세로로 · .menu-price-row — 단계 8 자리에 margin: 0 을 쓰고,

그 아래 줄에 위쪽 바깥 여백만 auto 로 다시 주고,
위쪽 안쪽 여백 10px, 위쪽에만 1px 점선 #e2d5c3

카드는 두 가지 역할을 동시에 합니다. 바깥에서 보면 .menu-list 의 아이템이고, 안에서 보면 그림과 글자를 담는 컨테이너입니다.

기대 출력 카드 키가 조금 줄어 윗줄이 352px, 아랫줄이 324px 이 되고,
설명 길이가 달라도 여섯 장의 가격 줄이 전부 카드 맨 아래에 붙습니다.
같은 줄 안에서는 가격 줄의 높이가 서로 딱 맞습니다.
가격 줄 위에 회색 점선이 한 줄 그어집니다.
그리고 첫 카드의 "인기" 글자가 카드 폭을 꽉 채우는 한 줄로 바뀝니다.
이상해 보여도 정상입니다. 단계 9 에서 제자리를 찾아 줍니다.
이렇게 되면 틀린 것: 가격 줄이 그대로 설명 바로 아래에 붙어 있다면
margin-top: auto 를 쓰기 전에 margin: 0 을 나중에 써서
덮어서 버린 것입니다. 단축을 먼저, 개별을 나중에 씁니다.

단계 9

이름과 가격을 카드의 양끝으로 밀어내고, 인기 배지를 카드 모서리에 겁니다.

· .menu-price-row — 단계 9 자리에 display 를 flex 로,

주축 정렬을 space-between 으로,
교차축 정렬을 baseline 으로

· .menu-label — 글자 크기 0.8125rem, 글자색 #8a7761 · .menu-price — 글자 크기 1.0625rem,
굵게, 글자색 #c0392b · .menu-card — 단계 9 자리에 배지의 기준점이 되도록 한 줄 · .menu-badge
— 흐름에서 꺼내서 top: -10px, right: -6px,

배경색 #c0392b, 글자색 #ffffff, 글자 크기 0.75rem,
안쪽 여백 padding: 3px 10px, 999px 둥근 모서리

배지를 카드 밖으로 빠져나가게 하려면 흐름에서 꺼내야 합니다. 꺼낸 요소는 기준이 될 조상을 찾습니다. 그 기준을 카드로 정해 주는 한 줄이 필요합니다.

기대 출력 카드 높이가 326px 이 되고, 가격 줄 왼쪽 끝에 회색 작은 글자, 오른쪽 끝에 빨간 굵은 값이 붙습니다. 글자 크기가 달라도 두 글자가 같은 밑줄 위에 나란히 섭니다.
 "인기" 는 첫 카드 오른쪽 위 모서리에 걸친 빨간 알약이 됩니다.
 이렇게 되면 틀린 것: 배지가 메뉴판 맨 위나 화면 구석으로 날아갔다면 .menu-card 에 기준점 한 줄을 안 쓴 것입니다.
 작은 글자가 살짝 떠 보인다면 baseline 대신 center 를 쓴 것입니다.

단계 10

마지막입니다. 세트 표와 안내, 바닥글을 다듬습니다.

· .set-title — margin: 0 0 8px, 글자 크기 1.125rem, 글자색 #3a2b21 · .set-table — 가로로 100% 로, border-collapse 를 collapse 로,

margin: 0 0 18px, 배경색 #ffffff, 글자 크기 0.875rem

· .set-table th, td — 1px 실선 테두리 #e2d5c3, 안쪽 여백 8px 10px, 왼쪽 정렬 · .set-table th — 배경색 #f3e3cd, 글자색 #5a4326 · .shop-notice — margin: 0 0 18px, 왼쪽 안쪽 여백 20px,

글자 크기 0.875rem, 글자색 #6b5a48

· .shop-foot — display 를 flex 로, 줄바꿈을 켜고,

주축 정렬을 space-between, 교차축 정렬을 center,
 간격 8px, 배경색 #3a2b21, 글자색 #e6d3bd,
 10px 둥근 모서리, 안쪽 여백 12px 16px,
 글자 크기 0.8125rem

· .shop-foot p — margin: 0

border-collapse 는 표에만 쓰는 속성이라 이 자료에서 따로 가르치지 않았습니다. 맞닿은 두 칸의 테두리를 하나로 합치는 한 줄이라고만 알아 두세요. 이것을 빼면 칸마다 자기 선을 그어서 격자선이 두 겹으로 보입니다.

기대 출력 표가 무대 폭을 꽉 채워 776px 이 되고 격자선이 한 겹으로 깔끔하게 그어집니다.
 머리카 세 개만 베이지색이 됩니다.
 맨 아래에 높이 46px 짜리 갈색 띠가 생기고, 영업시간이 왼쪽 끝, 전화번호가 오른쪽 끝에 놓입니다.
 이렇게 되면 틀린 것: 바닥 띠가 두 줄로 나뉘어 있다면 .shop-foot p 의 margin: 0 이 빠진 것입니다. p 의 기본 여백이 띠를 밀어 올립니다.
 격자선이 두 겹으로 보이면 border-collapse 를 빠뜨린 것입니다.

다 채웠으면 브라우저 폭을 375px 까지 줄여 보세요. 카드가 한 줄에 한 장이 되고, 카테고리 띠지와 바닥 띠도 두 줄로 접힙니다. 가로 스크롤바는 끝까지 생기지 않아야 합니다.

종합 02 - 음식점 메뉴판

단계 4 부터는 아래 각 규칙의 자리에 코드를 씁니다.

한 단계를 채울 때마다 저장(Ctrl+S) 하고 브라우저에서 F5 를 누르세요.

단계마다 화면이 어떻게 바뀌어야 하는지는

body 안 각 단계의 주석에 적어 두었습니다.

메뉴판이 놓이는 무대입니다. 미리 넣어 두었으니 고치지 마세요.

```
.menu-stage {  
  background-color: #fdf6ec;  
  padding: 24px;  
}
```

단계 4

```
.shop-head {
```

여기에 코드를 쓰세요

```
}  
.shop-name {
```

여기에 코드를 쓰세요

```
}  
.shop-tagline {
```

여기에 코드를 쓰세요


```
}
```

단계 5

```
.menu-tabs {
```

여기에 코드를 쓰세요

```
}  
.menu-tab {
```

여기에 코드를 쓰세요

```
}
```

단계 6

```
.menu-card {
```

TODO 단계 6: 상자 계산 방식 · 배경색 · 테두리 · 둥근 모서리 · 안쪽 여백

TODO 단계 7: 카드가 몇 px 에서 시작해 어떻게 늘고 줄지

TODO 단계 8: 카드 안을 세로 컨테이너로

TODO 단계 9: 배지의 기준점 표시

```
}  
.menu-photo {
```

여기에 코드를 쓰세요

```
}  
.menu-name {
```

여기에 코드를 쓰세요

```
}  
.menu-desc {
```

여기에 코드를 쓰세요

```
}
```

단계 7

```
.menu-list {
```

여기에 코드를 쓰세요

```
}
```

단계 8 · 9 는 이 규칙 하나를 함께 채웁니다

```
.menu-price-row {  
  
  }  
}
```

TODO 단계 8: 바깥 여백 · 위쪽 안쪽 여백 · 위쪽 점선

TODO 단계 9: 이름과 가격을 양끝으로

단계 9

```
.menu-label {  
  
  }  
  .menu-price {  
  
  }  
  .menu-badge {  
  
  }  
}
```

여기에 코드를 쓰세요

여기에 코드를 쓰세요

여기에 코드를 쓰세요

단계 10

```
.set-title {
```

여기에 코드를 쓰세요

```
}  
.set-table {
```

여기에 코드를 쓰세요

```
}  
.set-table th,  
.set-table td {
```

여기에 코드를 쓰세요

```
}  
.set-table th {
```

여기에 코드를 쓰세요

```
}  
.shop-notice {
```

여기에 코드를 쓰세요

```
}  
.shop-foot {
```

여기에 코드를 쓰세요

```
}  
.shop-foot p {
```

여기에 코드를 쓰세요

```
}
```

회원가입 폼

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.



실행하면 나오는 화면 — 회원가입 폼

회원가입 폼 한 장을 처음부터 끝까지 만듭니다. HTML 은 05단원의 폼, CSS 는 07단원부터 11단원까지 배운 것을 씁니다.

쓰지 않는 것: flex 와 grid 입니다. 12·13단원의 내용입니다. 그래도 아쉬울 것이 없습니다. 폼은 원래 위에서 아래로 한 줄씩 쌓이는 모양이고, 그 모양을 만드는 데 필요한 것은 display: block 한 줄뿐이기 때문입니다.

이 연습에서 회수하는 것은 크게 넷입니다.

- 05단원 label 과 input 을 for 와 id 로 연결하는 일
- 08단원 :focus 로 "지금 입력 중인 칸" 만 골라 꾸미는 일
- 10단원 width 100% 와 box-sizing 으로 칸 폭을 가지런히 맞추는 일
- 11단원 display: block 으로 이름표를 한 줄 차지하게 만들고,
relative 와 absolute 한 쌍으로 배지를 모서리에 붙이는 일

단계 1 과 2 는 HTML 을 직접 씁니다. 단계 3 부터는 위쪽 style 안을 채웁니다. 단계 10 만 HTML 을 한 줄 더 씁니다.

단계를 순서대로 하나씩 채우세요. 한 단계마다 저장하고 새로고침해서 어디가 달라졌는지 눈으로 확인하는 것이 이 연습의 핵심입니다. 각 단계의 자세한 설명과 확인 방법은 이 파일을 VS Code 로 열면 주석에 있습니다. 막히면 종합03_회원가입_폼_정답.html 을 여세요.

정리

단계 1

아래 회색 무대 상자 안에 폼의 뼈대를 직접 쓰세요.

1) <form class="signup-form" action="/join" method="post"> 로 전체를 감쌉니다.

보낼 서버가 아직 없으니 action 은 그냥 /join 이라고 적어 둡니다.

2) 그 안에 <fieldset class="form-group"> 을 두 개 만듭니다.

첫째 묶음의 제목은 "계정 정보", 둘째는 "추가 정보" 입니다.
제목은 fieldset 의 첫 자식으로 넣는 <legend> 에 씁니다.

3) 두 묶음 다음에 제출 버튼을 넣습니다.

```
<button class="form-submit" type="submit">가입하기</button>
```

기대 출력 회색 무대 안에 모서리가 둥근 상자 두 개가 위아래로 놓이고,
각 상자의 위쪽 테두리에 파란 제목 글자가 얹혀 있습니다.
그 아래에 작은 회색 [가입하기] 버튼이 하나 있습니다.
(fieldset 과 legend 의 모양은 style 에 미리 넣어 두었습니다)
이렇게 되면 틀린 것: 제목이 상자 테두리에 얹히지 않고 상자 안쪽 첫 줄에
들어와 있다면
legend 가 fieldset 의 첫 자식이 아닌 것입니다.
버튼을 눌렀을 때 아무 일도 없다면 버튼이 form 밖에 있는 것입니다.

단계 2

묶음 안을 채웁니다. "계정 정보" 묶음에는 아래 세 벌을 넣습니다. 한 벌은 이름표 · 입력칸 · 도움말 세 줄입니다.

```
<label for="signupId">아이디 <span class="required">*</span></label>
```

```
<input class="form-input" type="text" id="signupId" name="userId"
```

```
placeholder="hello2026" minlength="4" required />
```

```
<p class="form-help">영문 소문자와 숫자를 섞어 4자 이상으로 씁니다.</p>
```

· 아이디 type="text", id="signupId", minlength="4" · 비밀번호 type="password", id="signupPw",
minlength="8" · 메일 주소 type="email", id="signupMail"

셋 다 required 를 붙이고, 이름표 끝에 * 를 겁니다. label 의 for 와
input 의 id 를 반드시 같은 값으로 맞추세요.

"추가 정보" 묶음에는 셋을 넣습니다.

· 사는 지역 <select class="form-input" id="signupCity" name="city"> 와 option 네 개 · 한 줄 소개
<textarea class="form-input" id="signupIntro" name="intro" rows="3"> · 동의 label 로
체크박스를 감싸는 방법을 씁니다

```
<label class="form-agree"> <input type="checkbox" name="agree" value="yes" required
/>
  이용약관에 동의합니다 <span class="required">*</span>
</label>
```

기대 출력 각 묶음 안에 이름표와 입력칸이 한 줄에 나란히 붙어 보이고,
그 아래 줄에 도움말이 본문 크기 그대로 보입니다. 별표는 아직
검은색입니다.
"사는 지역" 과 "한 줄 소개" 는 한 줄에 뒤엎겨 보입니다. 지금은
정상입니다.
이렇게 되면 틀린 것: 이름표 글자를 눌렀는데 입력칸에 커서가 안 생기면
for 와 id 의 값이 서로 다른 것입니다. 이것이 연결을 확인하는 방법입니다.
긴 글 칸에 공백이 미리 들어가 있다면 textarea 의 여는 태그와 닫는 태그
사이에 줄바꿈을 넣은 것입니다. 딱 붙여 쓰세요.

단계 1 과 단계 2 의 HTML 을 이 자리에 쓰세요

단계 3

style 안 .signup-form 규칙의 단계 3 자리에 여섯 가지를 주세요.

· 가로 420px · 무대의 좌우 한가운데로 · 사방 24px 짜리 안쪽 여백 · 배경색 흰색(#ffffff) · 1px 두께의
연회색(#d8dee8) 실선 테두리 · 12px 만큼 둥근 모서리

기대 출력 무대 가로를 다 차지하던 폼이 흰 상자로 바뀌어 한가운데에 놓입니다.
상자의 바깥 폭은 470px 입니다. 420 에 좌우 여백 48 과 테두리 2 를 더한
값입니다.
이렇게 되면 틀린 것: 상자가 왼쪽에 붙어 있다면 좌우 margin 을 auto 로 안
준 것입니다.
폭이 420px 이라면 어디선가 box-sizing: border-box 가 켜진 것입니다.

단계 4

지금은 이름표가 입력칸 왼쪽에 딱 붙어 있습니다. label 이 인라인이기 때문입니다. .signup-form label
규칙에 네 가지를 주세요.

· 한 줄을 통째로 차지하게 만드는 display 값 · 바깥 여백 margin: 14px 0 4px · 글자 크기 0.9375rem ·
굵게

기대 출력 이름표가 입력칸 왼쪽에서 위로 올라가 각자 한 줄을 차지하고,
입력칸은 그 아래 줄로 내려갑니다.
한 줄에 뒤엎겨 있던 "사는 지역" 과 "한 줄 소개" 도 각자 줄을 갖습니다.
이렇게 되면 틀린 것: 이름표는 줄을 차지하는데 위아래 여백이 안 생겼다면
margin 은 썼지만 display 를 안 바꾼 것입니다.
인라인 요소에는 위아래 margin 이 먹지 않습니다.

단계 5

필수 항목의 별표를 눈에 띄게 만듭니다. `.required` 규칙에 두 가지를 주세요.

· 글자색 `#c0392b` · 굵게

기대 출력 이름표 옆의 별표 네 개만 빨갛고 굵어집니다. 이름표 글자는 그대로 검은색입니다.
이렇게 되면 틀린 것: 이름표 전체가 빨개졌다면 선택자를 `label` 로 쓴 것입니다.
아무 변화가 없다면 `HTML` 에서 별표를 `span` 으로 감싸지 않은 것입니다.

단계 6

입력칸들의 폭이 제각각입니다. 한 줄에 딱 차게 맞춥니다. `.form-input` 규칙에 주세요.

· 가로 100% · 테두리까지 폭에 포함시키는 `box-sizing` 값 · 안쪽 여백 `8px 10px · 1px` 두께의 회색 (`#c7ced9`) 실선 테두리 · `6px` 만큼 둥근 모서리 · 글자 크기 `1rem` · 부모의 글꼴을 그대로 받아 오는 `font-family` 값

기대 출력 글자칸 · 목록 상자 · 긴 글 칸의 폭이 모두 `390px` 로 똑같아지고, 왼쪽 끝과 오른쪽 끝이 자로 잰 듯 맞습니다. 폼 안 글꼴도 하나로 통일됩니다.
이렇게 되면 틀린 것: 칸이 `412px` 이 되어 `fieldset` 테두리를 뚫고 `7px` 빠져나갔다면 `box-sizing` 을 빠뜨린 것입니다. `100%` 에 여백과 테두리가 더 붙은 것입니다.
동의 체크박스까지 가로로 길게 늘어났다면 체크박스에도 `form-input` 클래스를 붙인 것입니다. 체크박스에는 붙이지 않습니다.

단계 7

도움말이 본문만큼 커서 눈에 먼저 들어옵니다. 결다리로 물러나게 만듭니다. `.form-help` 규칙에 주세요.

· 바깥 여백 `margin: 4px 0 0` · 글자 크기 `0.8125rem` · 글자색 `#6b7280`

기대 출력 도움말이 `13px` 회색으로 작아지고, 자기가 설명하는 칸 바로 아래 `4px` 자리에 붙습니다. 도움말과 다음 이름표 사이가 도움말과 칸 사이보다 넓어집니다.
이렇게 되면 틀린 것: 도움말이 아래쪽 이름표에 더 가까워 보인다면 위아래 여백을 그대로 둔 것입니다. `p` 에는 브라우저가 위아래 `16px` 을 미리 넣어 둡니다.

단계 8

지금 어느 칸에 글을 쓰고 있는지 한눈에 보이게 만듭니다. `.form-input:focus` 규칙에 주세요.

· 테두리 색 `#2d6cdf` · 배경색 `#f2f7ff` · `2px` 두께의 연파랑(`#cddffb`) 실선 `outline`

기대 출력 칸을 하나 누르면 그 칸만 테두리가 파래지고 배경이 연파랑으로 바뀌며, 테두리 바깥에 연한 파란 테가 한 겹 더 생깁니다. Tab 키로 다음 칸에 가면 강조도 따라 옮겨 갑니다. 이렇게 되면 틀린 것: 모든 칸이 처음부터 파랄다면 :focus 를 빠뜨린 것입니다. 아무 변화가 없다면 .form-input :focus 처럼 띄어 쓴 것입니다. 가상 클래스는 반드시 붙여 씁니다. 칸을 누를 때마다 옆 글자들이 덜컥 밀린다면 outline 대신 border 두께를 키운 것입니다. outline 은 상자 크기를 바꾸지 않습니다.

단계 9

제출 버튼이 아직 브라우저 기본 모양입니다. .form-submit 규칙에 주세요.

· 가로 100%, 위쪽 바깥 여백 20px · 위아래 12px 짜리 안쪽 여백 (좌우는 0) · 배경색 #2d6cdf, 글자색 흰색 · 테두리 없애기 · 8px 만큼 둥근 모서리 · 부모 글꼴 받아 오기, 글자 크기 1rem, 굵게, 글자 사이 2px

그리고 .form-submit:hover 에는 더 진한 파랑(#1b4ea8) 배경색을 주세요.

기대 출력 작던 회색 버튼이 폼 안쪽 폭 420px 을 다 채우는 파란 버튼이 됩니다. 마우스를 올리면 배경이 더 진해집니다. 이렇게 되면 틀린 것: 파란 바탕 둘레에 회색 입체 테가 남아 있다면 테두리를 안 없앴 것입니다. 버튼 폭이 390px 이라면 버튼을 fieldset 안에 넣은 것입니다. 100% 는 언제나 부모의 안쪽 폭 기준입니다.

단계 10

마지막입니다. 폼 상자의 오른쪽 위 모서리에 작은 배지를 걸칩니다. 이 단계만 HTML 을 한 줄 더 씁니다.

1) HTML — form 의 첫 자식으로 넣습니다.

```
<p class="form-badge">가입 무료</p>
```

지금 넣으면 폼 맨 위에 그냥 한 줄로 보입니다. 아직 정상입니다.

2) CSS — .form-badge 규칙에 주세요.

- 흐름에서 빼내어 기준 상자에 붙이는 position 값
- top: -14px, right: 18px
- margin: 0, 안쪽 여백 4px 12px
- 배경색 #f0a830, 글자색 흰색
- 글자 크기 0.8125rem, 굵게
- border-radius: 999px

3) CSS — .signup-form 규칙의 단계 10 자리에 한 줄을 더합니다.

무엇을 적어야 배지가 페이지가 아니라 폼을 기준으로 붙을지 생각해 보세요.

기대 출력 주황색 알약 모양 배지가 폼 상자의 오른쪽 위 모서리에 걸쳐 붙습니다.
 배지의 절반쯤이 폼 테두리 위로 올라와 있습니다.
 배지가 흐름에서 빠졌으므로 폼 안 내용은 한 칸도 밀리지 않습니다.
 이렇게 되면 틀린 것: 배지가 페이지 오른쪽 맨 위로 날아갔다면 3) 을 안 한
 것입니다.
 absolute 는 static 이 아닌 가장 가까운 조상을 기준으로 삼는데,
 그런 조상이 없으면 페이지 전체가 기준이 됩니다.
 배지가 네모라면 border-radius 값이 배지 높이보다 작은 것입니다.

종합 03 - 회원가입 폼

단계 3 부터는 아래 각 규칙의 자리에 코드를 씁니다.
 한 단계를 채울 때마다 저장(Ctrl+S) 하고 브라우저에서 F5 를 누르세요.
 단계마다 화면이 어떻게 바뀌어야 하는지는
 body 안 각 단계의 주석에 적어 두었습니다.

폼이 놓이는 무대입니다. 미리 넣어 두었으니 고치지 마세요.

```
.form-stage {
  background-color: #eef2f7;
  padding: 34px 0;
}
```

fieldset 과 legend 의 모양입니다. 미리 넣어 두었으니 고치지 마세요.

```
.form-group {
  margin: 0 0 18px;
  padding: 4px 14px 14px;
  border: 1px solid #dde3ec;
  border-radius: 8px;
}
.form-group legend {
  padding: 0 6px;
  color: #2d6cdf;
  font-size: 0.9375rem;
  font-weight: bold;
}
```

단계 3

```
.signup-form {
```

TODO 단계 3: 폭 · 가운데 정렬 · 안쪽 여백 · 배경색 · 테두리 · 둥근 모서리

TODO 단계 10: 배지의 기준점이 되는 한 줄

```
}
```

단계 4

```
.signup-form label {
```

여기에 코드를 쓰세요

```
}
```

단계 5

```
.required {
```

여기에 코드를 쓰세요

```
}
```

단계 6

```
.form-input {
```

여기에 코드를 쓰세요

```
}
```

단계 7

```
.form-help {
```

여기에 코드를 쓰세요

```
}
```

단계 8

```
.form-input:focus {
```

여기에 코드를 쓰세요

```
}
```

단계 9

```
.form-submit {
```

여기에 코드를 쓰세요

```
}
```

```
.form-submit:hover {
```

여기에 코드를 쓰세요

```
}
```

단계 10

```
.form-badge {
```

여기에 코드를 쓰세요

```
}
```

반응형 랜딩페이지

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

동시에 VS Code 로도 열어 두고, 코드와 화면을 나란히 보세요.

그리고 이 연습에서는 브라우저 창의 폭을 자주 줄였다 늘렸다 하세요.



실행하면 나오는 화면 — 반응형 랜딩페이지

이 자료의 마지막 결과물입니다. 01단원부터 13단원까지 전부 씁니다. 회사 소개 페이지나 제품 소개 페이지에서 흔히 보는 한 장짜리 페이지를 만듭니다.

| | |
|-----------|-----------------------------|
| 머리글 + 메뉴 | header nav |
| 큰 소개 영역 | section (제목 · 설명 · 버튼 · 그림) |
| 특징 카드 세 장 | section article |
| 바닥글 세 칸 | footer |

목표는 모양이 아니라 이것입니다.

브라우저 폭을 줄이면 카드가 3단 → 2단 → 1단으로 접히고
메뉴가 가로에서 세로로 바뀌며, 가로 스크롤바는 끝까지 생기지 않는다.

만드는 순서가 이 연습의 절반입니다. 좁은 화면을 먼저 만들고 넓은 화면을 덧붙입니다.

| | |
|------------------|----------------------------------|
| 단계 5 · 6 · 7 · 8 | 미디어쿼리 밖 = 좁은 화면용 (휴대폰) |
| 단계 9 | @media (min-width: 600px) 로 덧붙이기 |
| 단계 10 | @media (min-width: 900px) 로 덧붙이기 |

이것을 모바일 우선이라고 합니다. 반대로 만들면 코드가 길어지고, 무엇보다 휴대폰에서 먼저 깨집니다. 그런데 노트북에서는 멀쩡해 보여서 깨진 줄도 모릅니다.

확인하는 법: F12 를 누르고 Ctrl + Shift + M 을 누르면 화면 폭을 숫자로 정해 놓고 볼 수 있습니다. 375 · 768 · 1280 세 값으로 확인하면 충분합니다.

단계 1 · 2 · 3 은 HTML 을 직접 씁니다. 단계 4 부터는 위쪽 style 안을 채웁니다.

단계를 순서대로 하나씩 채우세요. 한 단계마다 저장하고 새로고침한 뒤 브라우저 폭을 줄였다 늘렸다 하면서 확인하는 것이 이 연습의 핵심입니다. 각 단계의 자세한 설명과 확인 방법은 이 파일을 VS Code 로 열면 주석에 있습니다. 막히면 종합04_반응형_랜딩페이지_정답.html 을 여세요.

정리

단계 1

아래 흰 무대 상자 안에 페이지의 큰 뼈대를 씁니다. 세 덩어리입니다.

1) `<header class="site-head">` 안에

로고를 `<p class="site-logo">` 로 한 줄 쓰고,
그 아래 `<nav class="site-nav">` 안에 `<a>` 를 세 개 씁니다.
내용은 소개 / 특징 / 문의 이고, href 는 각각 `#hero` · `#features` · `#contact` 입니다.

2 그 아래에 `<main>` 을 씁니다. 안은 단계 2 와 3 에서 채웁니다.

3 맨 아래에 `<footer class="site-foot" id="contact">` 를 씁니다.

href 앞에 우물 정(#) 을 붙이면 같은 페이지 안에서 그 id 자리로 넘어갑니다. 그래서 아래 요소에 id 를 같이 붙여 두는 것입니다.

기대 출력 흰 상자 안에 로고 한 줄, 그 아래 "소개 특징 문의" 세 글자가
파란 밑줄 링크로 한 줄에 이어 붙습니다. 아직 색도 여백도 없습니다.
이렇게 되면 틀린 것: 세 링크가 각자 한 줄씩 차지하면 a 가 아니라 p 로
감싼 것입니다.
a 는 인라인이라 옆으로 이어 붙는 것이 정상입니다.

단계 2

main 안에 소개 영역을 씁니다.

```
<section class="hero" id="hero"> <div class="hero-text"> <h2 class="hero-title">...  
페이지의 가장 큰 한 줄 ...</h2> <p class="hero-lead">... 두세 줄 설명 ...  
</p> <p class="hero-cta"> <a class="hero-button" href="#features">특징 보러 가기  
</a> </p> </div> <p class="hero-figure">  </p>  
</section>
```

제목 · 설명 · 버튼 세 개를 div 로 한 번 더 묶는 것을 빠뜨리지 마세요. 단계 10 에서 이 덩어리가 통째로 왼쪽
칸에 들어가야 합니다.

기대 출력 큰 제목 한 줄, 설명 두세 줄, 파란 밑줄 링크 "특징 보러 가기",
그 아래에 가로 480px 짜리 그림이 왼쪽에 붙어 나옵니다.
이렇게 되면 틀린 것: 그림 자리에 깨진 표시와 alt 글자가 보이면 경로가
틀린 것입니다.
이미지 폴더는 이 파일과 같은 자리에 있으므로 앞에 `../` 를 붙이지 않습니다.

단계 3

main 안 소개 영역 아래에 특징 영역을 쓰고, footer 안을 채웁니다.

1) <section class="features" id="features"> 안에

<h2 class="features-title"> 로 제목을 한 줄 쓰고,
<div class="feature-list"> 안에 <article class="feature-card"> 를 세 개 씩 씁니다.
카드 한 장은 이 순서입니다.

```
 <h3 class="feature-name">카드 제목</h3> <p class="feature-text">두세 줄 설명</p>
```

아이콘 파일은 세 개입니다.

이미지/랜딩_아이콘_연필.svg · 랜딩_아이콘_팔레트.svg · 랜딩_아이콘_화면.svg

2) <footer class="site-foot" id="contact"> 안에 <div class="foot-col"> 을 세 개 씁니다.

한 칸 안에는 <p class="foot-title"> 한 줄과 <p class="foot-text"> 한 줄입니다.

기대 출력 제목 한 줄 아래로 카드 세 장이 세로로 길게 쌓입니다.
카드마다 96px 짜리 아이콘이 왼쪽에 붙고 이름과 설명이 이어집니다.
맨 아래 바닥글 세 칸도 세로로 쌓여 여섯 줄이 보입니다.
이렇게 되면 틀린 것: 아이콘이 세 개 다 같은 그림이면 src 를 복사만 하고
안 고친 것입니다.
카드가 세 장 다 안 보이면 article 을 닫지 않은 것입니다.

단계 1·2·3 의 HTML 을 이 자리에 쓰세요

단계 4

반응형 페이지의 안전벨트 두 개를 먼저 채웁니다.

1) 이 파일 맨 위 head 안, charset 줄 바로 아래에 뷰포트 meta 를 한 줄 넣으세요.

name 은 viewport, content 는 width=device-width, initial-scale=1 입니다.

2) style 안 .landing img 에 두 줄을 쓰세요.

가로와 상한선을 부모의 100% 로 정하고, 세로는 auto 로 둡니다.

상한선을 거는 것과 폭을 정하는 것은 다릅니다. max-width 는 “이보다 커지지 마라”, width 는 “무조건 이만큼이 되어라” 입니다. width 를 쓰면 작은 아이콘까지 억지로 늘어나 흐릿해집니다.

기대 출력 넓은 화면에서는 아무것도 안 바뀝니다. 정말로 아무 변화가 없습니다. 확인은 폭을 줄여서 합니다. F12 → Ctrl + Shift + M 으로 폭을 375 로 놓으세요.

쓰기 전에는 대표 그림이 480px 을 고집해 오른쪽이 뭉텅 잘려 있습니다. 쓰고 나면 그림이 297px 로 줄어들어 전체가 다 보입니다.

(단계 6 에서 소개 영역에 안쪽 여백을 주고 나면 261px 이 됩니다)

이렇게 되면 틀린 것: 폭 375 에서도 그림 오른쪽이 계속 잘려 있다면 max-width 가 아니라 min-width 를 썼거나 단위를 빠뜨린 것입니다.

그림이 위아래로 찌그러졌다면 height 에 숫자를 쓴 것입니다.

그림이 잘리기만 하고 가로 스크롤바는 안 생기는 이유가 궁금할 수 있습니다. 무대(.landing) 에 overflow: hidden 을 미리 걸어 두었기 때문입니다. 그 overflow 마저 없었다면 페이지 전체에 가로 스크롤바가 생겼을 것입니다. 어느 쪽이든 고장입니다.

단계 5

머리글과 메뉴를 만듭니다. 여기서부터 단계 8 까지는 전부 좁은 화면 기준입니다. 좁은 화면에서는 가로로 늘어놓을 자리가 없으니 세로로 쌓는 것이 기본입니다.

· .site-head — display 를 flex 로, 방향을 세로로,

교차축 정렬을 center, 간격 10px,
배경색 #14243d, 안쪽 여백 16px

· .site-logo — margin: 0, 글자색 #ffffff, 글자 크기 20px,

굵게, 글자 사이 2px

· .site-nav — display 를 flex 로, 방향을 세로로, 간격 6px, 가로를 100% 로 · .site-nav a — 배경색 #22375a, 글자색 #d8e4f7, 안쪽 여백 8px 14px,

6px 둥근 모서리, 글자를 가운데로, 밑줄 없애기, 글자 크기 14px

· .site-nav a:hover — 배경색 #2d6cdf, 글자색 #ffffff

기대 출력 높이가 207px 짜리 남색 머리글이 생기고, 로고가 가운데 한 줄, 그 아래에 메뉴 단추 세 개가 폭 790px 로 세로로 쌓입니다.

단추에 마우스를 올리면 밝은 파란색으로 바뀝니다.

이렇게 되면 틀린 것: 메뉴 글자가 가로로 이어 붙어 있다면 .site-nav 에 flex 를 안 준 것입니다.

단추가 위아래로 납작하면 .site-nav 가 flex 컨테이너가 아닌 것입니다.

a 는 인라인이라 그냥은 위아래 padding 이 안 먹습니다.

flex 아이템이 되어야 블록처럼 취급되어 제대로 먹습니다.

단계 6

· 11단원 개념02 display)

소개 영역을 만듭니다.

· .hero — display 를 grid 로, 칸을 1fr 한 개로,

간격 18px, align-items 를 center 로,
배경색 #eef3fb, 안쪽 여백 26px 18px

align-items 는 12단원 개념04 에서 flex 에 쓰던 속성입니다. grid 에서도 이름과 뜻이 똑같습니다. 칸 안에서 내용을 위아래 가운데에 둡니다. 13단원에서는 안 나왔지만 새로 외울 것이 없어서 여기서 바로 씁니다. 단계 10 에서 소개 영역이 두 칸이 되면 글과 그림의 키가 달라지는데, 그때 이 한 줄이 둘을 같은 높이 가운데에 맞춰 줍니다. · .hero-title — margin: 0 0 10px, 줄 높이 1.3, 글자색 #14243d,

글자 크기는 clamp 로 최소 22px · 기준 4vw · 최대 36px

· .hero-lead — margin: 0 0 18px, 글자 크기 15px, 글자색 #44506b · .hero-cta — margin: 0 · .hero-button — display 를 inline-block 으로, 배경색 #2d6cdf,

글자색 #ffffff, 안쪽 여백 12px 26px,
8px 둥근 모서리, 밑줄 없애기, 굵게

· .hero-figure — margin: 0, 글자를 가운데로

칸을 하나만 만드는 것이 이상해 보일 수 있습니다. 일부러 그렇게 합니다. 좁은 화면이 기본이기 때문입니다. 단계 10 에서 이 한 줄만 고쳐 두 칸으로 만듭니다.

기대 출력 연한 파란 칸 안에 큰 제목(넓은 화면에서 36px), 설명, 파란 버튼이 놓이고, 그 아래 대표 그림이 가운데로 옵니다. 버튼은 가로 159px · 세로 51px 입니다.
이렇게 되면 틀린 것: 파란 버튼이 화면 폭을 통째로 차지하면 display 를 block 으로 준 것입니다.
버튼이 얇은 줄로만 보이면 inline 인 채로 둔 것입니다.
인라인은 위아래 padding 이 줄 높이를 늘리지 못합니다.
제목 크기가 안 변하면 clamp 안의 값 사이 쉼표를 빠뜨린 것입니다.

단계 7

특징 카드를 만듭니다. 여기도 칸 하나짜리 격자로 시작합니다.

· .features — 안쪽 여백 26px 18px · .features-title — margin: 0 0 16px, 글자 크기 22px,

글자를 가운데로, 글자색 #14243d

· .feature-list — display 를 grid 로, 칸을 1fr 한 개로, 간격 14px · .feature-card — 배경색 #ffffff, 1px 실선 테두리 #d6dfee,

10px 둥근 모서리, 안쪽 여백 18px, 글자를 가운데로

· .feature-icon — 가로 64px · .feature-name — margin: 10px 0 6px, 글자 크기 17px, 글자색 #14243d · .feature-text — margin: 0, 글자 크기 14px, 줄 높이 1.6, 글자색 #55617a

기대 출력 카드 세 장이 테두리를 두른 흰 상자가 되어 세로로 쌓입니다.
카드 폭은 786px, 아이콘은 96px 에서 64px 로 줄고 가운데로 옵니다.
카드 사이가 14px 씩 벌어지고, 마지막 카드 아래에는 그 간격이 안 붙습니다.
이렇게 되면 틀린 것: 카드 사이가 안 벌어졌다면 gap 을 .feature-card 에 쓴
것입니다.
gap 은 컨테이너에 쓰는 속성입니다. 아이템에 쓰면 조용히 무시됩니다.

단계 8

바닥글을 만듭니다. 이번에는 미디어쿼리 없이 저절로 접히는 격자를 씁니다.

· .site-foot — display 를 grid 로,

칸을 repeat(auto-fit, minmax(180px, 1fr)) 로,
간격 16px, 배경색 #14243d, 글자색 #c9d6ea,
안쪽 여백 22px 18px

· .foot-title — margin: 0 0 6px, 글자색 #ffffff, 글자 크기 15px, 굵게 · .foot-text — margin: 0, 글자 크기 13px, 줄 높이 1.6

읽는 법은 이렇습니다.

```
repeat( auto-fit , minmax(180px, 1fr) )  
      ↳ 칸 수      ↳ 칸 하나의 크기
```

minmax(180px, 1fr) 은 “최소 180px, 남으면 1fr 만큼 더” 입니다. auto-fit 은 “180px 짜리 칸을 들어갈 수 있는 만큼 만들어라” 입니다.

기대 출력 넓은 화면에서 남색 바닥 띠가 생기고 그 안이 세 칸(칸 하나 251px) 으로 나뉩니다. 띠 높이는 117px 입니다.
폭을 375 로 줄이면 세 칸이 한 칸씩 세로로 접힙니다. @media 는 한 줄도 없습니다.
이렇게 되면 틀린 것: 세 칸이 안 생기고 세로로 쌓인 채라면 minmax 의 첫 값이 너무 큰 것입니다.
칸이 네 개로 보인다면 auto-fit 이 아니라 auto-fill 을 쓴 것입니다.
auto-fill 은 빈 칸도 그대로 남겨 둡니다.

단계 9

여기서부터가 진짜 반응형입니다. style 맨 아래 @media (min-width: 600px) 블록 안 세 자리를 채우세요.

· .site-head — 방향을 가로로, 주축 정렬을 space-between · .site-nav — 방향을 가로로, 가로를 auto 로 되돌리기 · .feature-list — 칸을 repeat(2, 1fr) 로

min-width: 600px 은 “폭이 600px 이상일 때” 입니다. 경계값 600 도 포함입니다. 바뀌야 하는 줄만 씁니다. 배경색과 여백은 좁을 때와 같으므로 다시 쓰지 않습니다.

기대 출력 넓은 화면에서 머리글 높이가 207px 에서 72px 로 줄고, 로고가 왼쪽 끝, 메뉴 세 개가 오른쪽 끝에 가로 한 줄로 붙습니다. 특징 카드는 한 줄에 두 장(카드 하나 386px) 이 됩니다. 폭을 599 이하로 줄이면 즉시 단계 5 의 세로 모양으로 돌아갑니다. 이렇게 되면 틀린 것: 메뉴가 오른쪽 끝까지 안 가고 로고를 왼쪽으로 밀어붙이면 .site-nav 의 가로를 auto 로 되돌리는 줄을 빠뜨린 것입니다. 좁은 화면용으로 준 100% 가 그대로 남아 있는 것입니다. 아무 일도 안 일어난다면 @media 안에 선택자를 안 쓰고 속성만 쓴 것입니다. 중괄호는 두 겹이어야 합니다.

단계 10

마지막입니다. 이번에는 @media 블록을 통째로 직접 쓰세요. style 맨 아래, 단계 10 이라고 적힌 주석 바로 아래가 그 자리입니다.

@media (min-width: 900px) 안에 두 줄만 바꿉니다.

· .hero — 칸을 1fr 1fr 로 · .feature-list — 칸을 repeat(3, 1fr) 로

순서를 지키세요. 600 짜리보다 아래에 써야 합니다. 폭 1280 에서는 600 짜리와 900 짜리가 둘 다 참이 됩니다. 이때 나중에 쓴 것이 이깁니다. @media 는 명시도를 올려 주지 않기 때문입니다. 위아래를 바꿔 쓰면 넓은 화면에서 2단이 나옵니다. 에러는 안 납니다. 그냥 조용히 틀립니다.

기대 출력 폭 1280 에서 소개 영역이 좌우 두 칸으로 나뉘어 글이 왼쪽(384px), 그림이 오른쪽(384px) 에 놓입니다. 특징 카드는 한 줄에 세 장(카드 하나 253px) 이 됩니다. 폭을 899 로 줄이면 카드가 2단, 소개는 1단으로 돌아갑니다. 폭을 599 로 줄이면 카드가 1단, 메뉴가 세로로 돌아갑니다. 이렇게 되면 틀린 것: 소개 영역의 그림만 오른쪽으로 가고 제목 · 설명 · 버튼이 세 칸으로 흩어졌다면 단계 2 에서 글 세 줄을 div 로 안 묶은 것입니다. 두 칸짜리 격자에 네 개를 넣으면 두 줄로 흘러넘칩니다.

다 되면 마지막 점검입니다. 폭을 375 · 768 · 1280 세 값으로 놓고 가로 스크롤바가 생기는지 보세요. 한 번도 생기지 않아야 합니다. 완성된 모습과 폭별 결과표는 종합04_반응형_랜딩페이지_정답.html 에 있습니다.

종합 04 - 반응형 랜딩페이지

단계 4 부터는 아래 각 규칙의 자리에 코드를 씁니다.
한 단계를 채울 때마다 저장(Ctrl+S) 하고 브라우저에서 F5 를 누르세요.

쓰는 순서가 중요합니다.
위쪽에는 좁은 화면용 규칙만 쓰고, 맨 아래에 @media 두 개를
작은 값(600) → 큰 값(900) 순서로 붙입니다. 이것이 모바일 우선입니다.

랜딩 페이지가 놓이는 무대입니다. 미리 넣어 두었으니 고치지 마세요.

```
.landing {  
  background-color: #ffffff;  
  border: 1px solid #cfd8e6;  
  border-radius: 12px;  
  overflow: hidden;  
}
```

단계 4

```
.landing img {
```

여기에 코드를 쓰세요

```
}
```

단계 5

```
.site-head {
```

여기에 코드를 쓰세요

```
}
.site-logo {
```

여기에 코드를 쓰세요

```
}
.site-nav {
```

여기에 코드를 쓰세요

```
}
.site-nav a {
```

여기에 코드를 쓰세요

```
}
.site-nav a:hover {
```

여기에 코드를 쓰세요

```
}
```

단계 6

```
.hero {
```

여기에 코드를 쓰세요

```
}  
.hero-title {
```

여기에 코드를 쓰세요

```
}  
.hero-lead {
```

여기에 코드를 쓰세요

```
}  
.hero-cta {
```


여기에 코드를 쓰세요

```
}
.hero-button {
```

여기에 코드를 쓰세요

```
}
.hero-figure {
```

여기에 코드를 쓰세요

```
}
```

단계 7

```
.features {
```

여기에 코드를 쓰세요

```
}
.features-title {
```

여기에 코드를 쓰세요

```
}  
.feature-list {
```

여기에 코드를 쓰세요

```
}  
.feature-card {
```

여기에 코드를 쓰세요

```
}  
.feature-icon {
```

여기에 코드를 쓰세요

```
}  
.feature-name {
```

여기에 코드를 쓰세요

```
}
.feature-text {
```

여기에 코드를 쓰세요

```
}
```

단계 8

```
.site-foot {
```

여기에 코드를 쓰세요

```
}
.foot-title {
```

여기에 코드를 쓰세요

```
}
.foot-text {
```

여기에 코드를 쓰세요

```
}
```

단계 9

```
@media (min-width: 600px) {  
  .site-head {
```

여기에 코드를 쓰세요

```
}  
  .site-nav {
```

여기에 코드를 쓰세요

```
}  
  .feature-list {
```

여기에 코드를 쓰세요

```
}  
}
```

단계 10

이번에는 @media 블록을 통째로 직접 쓰세요. 이 주석 바로 아래에 씁니다.

고칠 것은 .hero 와 .feature-list 두 줄뿐입니다.

부 록 가

연습문제·종합연습 정답

먼저 스스로 풀어 본 다음에 보세요. 정답과 다르게 썼더라도 기대 출력이 같으면 맞은 것입니다.
다만 “왜 그렇게 되는지” 설명은 꼭 읽으세요.

11단원 연습문제 정답 배치 display와 position

읽는 법

- 1. 이 파일을 브라우저로 열어 화면을 봅니다.
 - 2. VS Code 로도 열어 style 안의 정답과 해설을 읽습니다.
 - 3. 여러분이 쓴 답과 모양이 같아도 방법이 다를 수 있습니다.
- 해설을 읽고 “왜 그렇게 되는가” 를 확인하세요.



실행하면 나오는 화면 — 배치 (display와 position)

각 문제의 정답은 이 파일 위쪽 <style> 안에 들어 있습니다. 정답 바로 아래에 왜 그렇게 되는지를 두세 줄로 적어 두었습니다. 화면만 보지 말고 그 해설을 꼭 읽으세요.

문제 1 (개념02 섹션 1)

아래 두 조각은 span 이라 한 줄에 나란히 붙어 있습니다. 첫 번째 조각(#q1) 만 한 줄을 통째로 차지하게 만드세요.

정답: #q1 { display: block; }

화면 첫 조각이 상자 왼쪽 끝부터 오른쪽 끝까지 늘어나고

두 번째 조각이 그 아래 다음 줄로 내려갑니다

저는 첫 번째 조각입니다

저는 두 번째 조각입니다

문제 2 (개념02 섹션 2·4)

아래 조각(#q2) 을 가로 180px, 세로 60px 짜리 상자로 만드세요.

정답: #q2 { display: inline-block; width: 180px; height: 60px; }

화면 넓고 두꺼운 파란 상자가 되고 "옆에 붙어 있는 글자" 는

상자 오른쪽에, 상자 안 글자와 같은 줄에 맞춰 붙어 있습니다

크기를 가질 조각

옆에 붙어 있는 글자

문제 3 (개념02 섹션 5)

가운데 줄(#q3) 을 자리까지 통째로 없애 보세요.

정답: #q3 { display: none; }

화면 A 줄과 C 줄만 보이고 두 줄이 서로 바로 붙어 있습니다

A 줄입니다

B 줄입니다

C 줄입니다

문제 4 (개념02 섹션 5)

가운데 줄(#q4) 을 자리는 남긴 채 안 보이게 하세요.

정답: #q4 { visibility: hidden; }

화면 A 줄과 C 줄 사이에 한 줄만큼 빈자리가 뽕 뚫려 있습니다

- A 줄입니다
- B 줄입니다
- C 줄입니다

문제 5 (개념03 섹션 2·3)

상자(#q5) 를 오른쪽으로 40px, 아래로 20px 옮기세요.

정답: #q5 { position: relative; left: 40px; top: 20px; }

화면 파란 상자가 회색 무대 왼쪽 위 구석에서 오른쪽 아래로

비스듬히 내려가 있습니다

옮길 상자

문제 6 (개념03 섹션 3·4)

두 번째 상자(#q6) 만 원래 자리보다 12px 위로 올리세요.

정답: #q6 { position: relative; top: -12px; }

화면 오른쪽 상자만 12px 위로 떠 있고 왼쪽 상자는 제자리입니다.

두 상자의 가로 위치는 서로 그대로입니다

그대로 있을 상자
위로 올릴 상자

문제 7 (개념04 섹션 2)

카드(#q7Card) 오른쪽 위 구석에 배지(#q7Badge) 를 8px 씩 떨어뜨려 붙이세요.

정답: #q7Card { position: relative; }

#q7Badge { position: absolute; top: 8px; right: 8px; }

화면 빨간 NEW 조각이 카드 오른쪽 위 구석에 붙어 있고

글자 두 줄은 카드 왼쪽에 그대로 있습니다

NEW

오늘의 메뉴

크림 파스타와 마늘빵

문제 8 (개념04 섹션 1.2)

상자(#q8Box) 를 무대 오른쪽 아래 구석에서 10px 씩 떨어진 곳에 놓으세요.

정답: #q8Box { position: absolute; right: 10px; bottom: 10px; }

화면 파란 상자가 회색 무대 오른쪽 아래 구석에 붙어 있고,

글자가 무대 맨 위 왼쪽으로 올라와 있습니다

구석으로 갈 상자
이 글자는 무대 안에 그대로 남습니다.

문제 9 (개념05 섹션 2.3)

초록 상자(#q9Front) 가 분홍 상자 위로 올라오게 만드세요.

정답: #q9Front 에 position: relative; 와 z-index: 3; 을 더합니다.

화면 초록 상자가 분홍 상자의 오른쪽 아래 부분을 덮고 있습니다

뒤에 있는 상자
앞으로 올 상자

문제 10 (개념05 섹션 5)

파란 네모(#q10Thumb) 를 왼쪽에 붙이고 글이 그 옆으로 흐르게 하세요.

정답: #q10Thumb { float: left; }

화면 파란 네모가 왼쪽에 붙고 처음 네 줄은 네모 오른쪽에서 시작합니다.

다섯째 줄부터는 글자가 상자 왼쪽 끝에서 시작합니다

그림

이 글은 왼쪽 네모를 피해 흐르게 될 문장입니다. 문장이 충분히 길어야 네모 옆을 지나 아래까지 이어지는 모습을 볼 수 있습니다. 신문 기사에서 사진 옆에 글이 붙어 있는 모양을 떠올리면 됩니다. 이 방식은 지금도 글 옆에 그림을 놓을 때 쓰입니다. 화면을 좌우로 나누는 일에는 쓰지 않습니다. 그림과 글이 서로 다른 상자에 들어 있는데도 한 덩어리처럼 보이는 것이 이 방식의 좋은 점입니다. 이쯤 오면 글줄이 네모 아래로 내려와 상자 왼쪽 끝에서 시작하는 것이 보입니다.

문제 11 [응용] (개념02 섹션 4)

응용 · 카드 세 장을 한 줄에 나란히 놓고, 세 장 전체를 무대 가운데로 모으세요.

정답: .q11-card 에 display: inline-block; width: 150px; height: 70px; 을 더하고,

부모 #q11Row 에 text-align: center; 를 씁니다.

화면 같은 크기 카드 세 장이 한 줄에 나란히 놓이고,

세 장 뭉치가 점선 상자의 가로 가운데에 옵니다. 카드 사이에 6px 째 되는 얇은 틈이 보입니다

카드 1
카드 2
카드 3

문제 12 [응용] (개념04 섹션 4)

응용 · 파란 머리줄(#q12Head) 이 스크롤해도 상자 맨 위에 붙어 있게 만드세요.

정답: #q12Head 에 position: sticky; 와 top: 0; 을 더합니다.

화면 상자 안에서 마우스 휠을 굴리면 파란 머리줄만 상자 맨 위에 붙어 남고

숫자 줄들이 그 밑으로 지나갑니다

```

머리줄입니다
1번 줄입니다
2번 줄입니다
3번 줄입니다
4번 줄입니다
5번 줄입니다
6번 줄입니다
7번 줄입니다
8번 줄입니다
9번 줄입니다
10번 줄입니다

```

문제 13 [도전] (개념04 섹션 2 · 개념05 섹션 3)

도전 · 빨간 라벨(#q13Label) 을 무대 오른쪽 아래 구석에서 20px 씩 떨어뜨려 놓고

노란 덮개보다 위에 보이게 만드세요.

정답: #q13Label 에 position: absolute; right: 20px; bottom: 20px; z-index: 2; 를 더합니다.

화면 노란 덮개가 무대 오른쪽을 덮고 있고,

그 위에 빨간 “추천” 라벨이 오른쪽 아래 구석에 또렷하게 올라와 있습니다

```

추천
노란 덮개

```

문제 14 [도전] (개념02 섹션 4 · 개념04 섹션 2)

도전 · 메뉴 칸 세 개를 한 줄에 놓고, 가운데 칸 오른쪽 위 구석에 배지를 붙이세요.

정답: .q14-item 에 display: inline-block; width: 150px; height: 60px;

```

#q14Item2 에 position: relative;
#q14New 에 position: absolute; top: 4px; right: 4px;

```

화면 메뉴 칸 세 개가 한 줄에 놓이고,

가운데 “파스타” 칸의 오른쪽 위 구석에만 작은 빨간 배지가 붙어 있습니다

국수

파스타

NEW

덮밥

문제 15 [확인] (개념03 섹션 2 · 개념05 섹션 3)

확인 · top, left, z-index 가 다 적혀 있는데 셋 다 아무 일도 안 합니다.

한 줄만 더해서 살려 내세요.

정답: #q15 에 position: relative; 한 줄을 더합니다.

세 속성 모두 "position 이 static 이 아닐 것" 을 요구하기 때문입니다.

화면 노란 상자가 분홍 상자를 덮고 있고,

문제 파일보다 오른쪽으로 20px, 아래로 20px 더 내려가 있습니다

분홍 상자

노란 상자

정리 —————

== —————

여기부터 "문제 1" 이 나오기 전까지는 문제용 꺾德基입니다.
문제 파일과 똑같습니다.

```
.q-stage {
  position: relative;
  height: 120px;
  background: #f7f7f7;
  border: 1px solid #ccc;
  padding: 10px;
}
.q-box {
  background: #dbe7fb;
  border: 1px solid #2d6cdf;
  padding: 8px 10px;
}
.q-tile {
  display: inline-block;
  width: 160px;
  height: 50px;
  background: #dbe7fb;
  border: 1px solid #2d6cdf;
  text-align: center;
  line-height: 50px;
}
.q-slab {
  width: 260px;
  height: 60px;
  background: #dbe7fb;
  border: 1px solid #2d6cdf;
  text-align: center;
  line-height: 60px;
}
.q-row {
  background: #f4f4f4;
  border: 1px solid #ddd;
  padding: 4px 8px;
}
.q-card {
  background: #f4f8ff;
```

```

    border: 2px solid #2d6cdf;
    padding: 14px;
}
.q-badge {
    background: #c0392b;
    color: #fff;
    font-size: 13px;
    padding: 3px 8px;
}
.q-txt {
    margin: 0;
}
.q-float {
    overflow: hidden;
    background: #fafafa;
    border: 1px solid #ddd;
    padding: 10px;
}
.q-thumb {
    width: 110px;
    height: 80px;
    margin: 0 14px 8px 0;
    background: #dbe7fb;
    border: 1px solid #2d6cdf;
    text-align: center;
    line-height: 80px;
}
.q-scroll {
    height: 160px;
    overflow: auto;
    background: #fff;
    border: 2px solid #999;
}
.q-item {
    padding: 6px 12px;
    border-bottom: 1px solid #eee;

```



```

}
#q9Back,
#q15Back {
  position: relative;
  z-index: 2;
  background: #ffd8d8;
}
#q13Cover {
  position: absolute;
  top: 10px;
  right: 10px;
  width: 300px;
  height: 110px;
  background: #ffe08a;
  border: 1px solid #d9a100;
  text-align: center;
  line-height: 110px;
}

```

여기부터 정답입니다.

문제 1 (개념02 섹션 1)

```

#q1 {
  display: block;
}

```

화면 첫 조각이 상자 폭을 꽉 채우고 두 번째 조각이 다음 줄로 내려갑니다.

block 은 한 줄을 통째로 차지하므로 뒤 요소가 아래로 밀립니다.

span 이라는 태그 이름은 그대로지만 화면 모양은 display 가 정합니다.

문제 2 (개념02 섹션 2·4)

```
#q2 {  
  display: inline-block;  
  width: 180px;  
  height: 60px;  
}
```

화면 가로 202px, 세로 78px 짜리 상자가 되고 오른쪽 글자는 그 옆에 남습니다.

inline 이면 width·height 가 무시되므로 display 를 먼저 바꿔야 합니다.

180 과 60 에 안쪽 여백과 테두리가 더해져 화면 크기는 202 × 78 이 됩니다.

문제 3 (개념02 섹션 5)

```
#q3 {  
  display: none;  
}
```

화면 B 줄이 사라지고 A 줄과 C 줄이 서로 바로 붙습니다.

display: none 은 자리까지 없애기 때문에 아래 줄이 위로 올라옵니다.

문제 4 (개념02 섹션 5)

```
#q4 {  
  visibility: hidden;  
}
```

화면 A 줄과 C 줄 사이에 한 줄만큼 빈자리가 남습니다.

visibility: hidden 은 투명하게만 만들 뿐 자리는 그대로 차지합니다.

문제 3 과 문제 4 의 차이가 바로 이 빈자리 하나입니다.

문제 5 (개념03 섹션 2·3)

```
#q5 {
  position: relative;
  left: 40px;
  top: 20px;
}
```

화면 파란 상자가 무대 왼쪽 위에서 오른쪽 아래로 비스듬히 내려갑니다.

position 을 static 이 아닌 값으로 바꿔야 top·left 가 살아납니다.

relative 의 기준은 화면이 아니라 자기 자신의 원래 자리입니다.

문제 6 (개념03 섹션 3·4)

```
#q6 {
  position: relative;
  top: -12px;
}
```

화면 두 번째 상자만 12px 떠 보이고 첫 번째 상자는 그대로입니다.

top 은 "위에서 떨어진 거리" 라서 음수를 주면 위로 올라갑니다.

옮겨도 원래 자리는 그대로 예약되어 있어 옆 상자가 안 움직입니다.

문제 7 (개념04 섹션 2)

```
#q7Card {
  position: relative;
}
#q7Badge {
  position: absolute;
  top: 8px;
  right: 8px;
}
```

화면 빨간 NEW 조각이 카드 오른쪽 위 구석에 붙습니다.

카드의 relative 는 "내가 기준이다" 라는 표시일 뿐 카드를 움직이지 않습니다.

이 한 줄이 없으면 배지는 페이지 맨 위 구석으로 날아갑니다.

문제 8 (개념04 섹션 1·2)

```
#q8Box {  
  position: absolute;  
  right: 10px;  
  bottom: 10px;  
}
```

화면 파란 상자가 무대 오른쪽 아래 구석에 붙고, 무대 안 글자가 위로 올라옵니다.

absolute 는 흐름에서 빠지므로 상자가 있던 자리가 통째로 없어집니다.

오른쪽·아래에 붙일 때는 left·top 대신 right·bottom 을 쓰면 계산이 필요 없습니다.

문제 9 (개념05 섹션 2·3)

```
#q9Front {  
  margin-top: -30px;  
  margin-left: 50px;  
  background: #d8f0d8;  
  position: relative;  
  z-index: 3;  
}
```

화면 초록 상자가 분홍 상자를 덮습니다.

z-index 는 position 이 static 이 아닐 때만 먹으므로 두 줄이 한 쌍입니다.

3 이 2 보다 크므로 초록 상자가 앞으로 나옵니다.

문제 10 (개념05 섹션 5)

```
#q10Thumb {  
  float: left;  
}
```

화면 파란 네모가 왼쪽에 붙고 글자가 그 오른쪽에서 시작합니다.

네모가 끝나는 줄부터는 글자가 상자 왼쪽 끝에서 시작합니다.

이렇게 글이 그림을 감싸며 흐르는 것이 float 의 원래 용도입니다.

문제 11 [응용] (개념02 섹션 4)

```
.q11-card {
  background: #eef2f7;
  border: 2px solid #2d6cdf;
  padding: 8px;
  text-align: center;
  display: inline-block;
  width: 150px;
  height: 70px;
}
#q11Row {
  text-align: center;
}
```

화면 카드 세 장이 한 줄에 나란히 놓이고, 세 장 뭉치가 가로 가운데에 옵니다.

사이에는 6px 쯤 되는 얇은 틈이 보입니다.
div 는 원래 block 이라 세로로 쌓이므로 display 를 바꿔야 옆에 붙습니다.
가운데로 모으는 것이 이 문제의 핵심입니다.
inline-block 은 '인라인' 이라서 부모가 글자처럼 다룹니다.
그래서 09단원에서 배운 text-align: center 가 그대로 통합니다.
margin: 0 auto 는 여기서 안 통합니다. 그건 블록 상자 하나를 옮기는 방법이고,
지금 옮기려는 것은 한 줄 안에 늘어선 인라인 덩어리이기 때문입니다.

카드 사이 틈은 코드에서 카드 사이에 친 줄바꿈이 공백이 된 것입니다.

문제 12 [응용] (개념04 섹션 4)

```
#q12Head {
  background: #2d6cdf;
  color: #fff;
  padding: 8px 12px;
  font-weight: bold;
  position: sticky;
  top: 0;
}
```

화면 상자 안에서 스크롤하면 파란 줄만 맨 위에 붙어 남습니다.

sticky 는 top 이나 bottom 으로 붙을 지점을 알려 줘야 동작합니다.

top: 0 은 “스크롤 영역 위쪽 0px 지점에 닿으면 붙어라” 라는 뜻입니다.

문제 13 [도전] (개념04 섹션 2 · 개념05 섹션 3)

```
#q13Label {
  background: #c0392b;
  color: #fff;
  font-size: 13px;
  padding: 4px 10px;
  position: absolute;
  right: 20px;
  bottom: 20px;
  z-index: 2;
}
```

화면 빨간 라벨이 무대 오른쪽 아래 구석에서 노란 덮개 위에 올라와 보입니다.

덮개에는 z-index 가 없으므로 라벨에 2 를 주면 라벨이 앞으로 나옵니다.

position: absolute 가 없으면 z-index 는 그냥 무시됩니다.

문제 14 [도전] (개념02 섹션 4 · 개념04 섹션 2)

```
.q14-item {
  background: #eef2f7;
  border: 1px solid #7a9bd4;
  text-align: center;
  line-height: 60px;
  display: inline-block;
  width: 150px;
  height: 60px;
}
#q14Item2 {
  position: relative;
}
#q14New {
  background: #c0392b;
  color: #fff;
  font-size: 12px;
  line-height: 1.4;
  padding: 2px 6px;
  position: absolute;
  top: 4px;
```

```
right: 4px;
}
```

화면 메뉴 칸 세 개가 한 줄에 놓이고 가운데 칸 오른쪽 위에 빨간 배지가 붙습니다.

기준이 될 조상은 가운데 칸이므로 그 칸에만 relative 를 겁니다.

첫째·셋째 칸에 relative 를 걸어도 화면은 안 바뀌지만 뜻이 어긋납니다.

문제 15 [확인] (개념03 섹션 2 · 개념05 섹션 3)

```
#q15 {
  margin-top: -30px;
  margin-left: 50px;
  background: #ffe08a;
  top: 20px;
  left: 20px;
  z-index: 3;
  position: relative;
}
```

화면 노란 상자가 분홍 상자를 덮고, 동시에 오른쪽·아래로 20px 씩 더 움직입니다.

top, left, z-index 세 속성은 모두 "position 이 static 이 아닐 것" 을 요구합니다.

그래서 position 한 줄이 빠지면 세 줄이 한꺼번에 무시됩니다.

12단원 연습문제 정답 Flexbox



실행하면 나오는 화면 — 정답

답은 이 파일의 <style> 안에 들어 있습니다. 각 답 아래에 왜 그렇게 되는지와 다른 방법을 적어 두었습니다. 먼저 스스로 풀어 본 뒤에 여세요. 답이 달라도 화면이 같으면 맞은 것입니다. 특히 11번과 13번은 여러 방법이 있습니다.

문제 1

화면 상자 셋이 딱 붙어 한 줄로 놓입니다.

무대 안쪽 왼쪽 면에서 0 / 120 / 240px 자리에 하나씩 있습니다

- 1
- 2
- 3

문제 2

화면 오른쪽 끝에 1, 그 왼쪽에 2, 그 왼쪽에 3 이 보입니다

1
2
3

문제 3

화면 윗줄에 네 개, 아랫줄에 한 개. 폭은 모두 150px 입니다

150px
150px
150px
150px
150px

문제 4

화면 상자 뭉치가 가운데에 오고 좌우에 120px 씩 빈 칸이 생깁니다

1
2
3

문제 5

화면 양 끝에 하나씩 붙고 사이가 360px 벌어집니다

왼쪽
오른쪽

문제 6

화면 상자 사이가 20px 씩 벌어집니다

1
2
3

문제 7

화면 상자가 무대 한가운데 높이에 놓고 키는 44px 입니다

1
2

문제 8

화면 상자가 무대 한가운데에 놓입니다. 좌우에 남는 자리가 200px 로 같습니다

정중앙

문제 9

화면 첫 상자가 360px, 나머지 둘은 120px 입니다

늘어남
그대로
그대로

문제 10

화면 글자 길이와 상관없이 셋 다 200px 입니다

짧다
조금 더 긴 글자
아주 아주 많이 긴 글자

문제 11

화면 왼쪽 끝에 가게 이름, 오른쪽 끝에 메뉴가 놓입니다

(메뉴 덩어리의 오른쪽 끝이 남색 띠 안쪽 오른쪽 면에 붙습니다)

행복 서점

새 책

중고

문의

문제 12

화면 가운데 상자만 무대 바닥에 붙습니다. 셋 다 키는 44px 입니다

그대로

아래로

그대로

문제 13

화면 본문 380px, 사이드바 200px 이 20px 간격으로 놓입니다

이번 주 새 책

목요일마다 새 책이 들어옵니다. 서가 맨 앞줄에 일주일 동안 놓아 두니
지나는 길에 한 번 훑어보세요. 표지가 예쁜 책은 금방 나갑니다.

여는 시간

평일 10:00 ~ 21:00

문제 14

화면 카드가 한 줄에 두 개씩 두 줄, 폭은 292px 입니다

소설

가볍게 읽기 좋습니다.

12,000원

역사

사진 자료가 많아 두껍습니다. 처음 읽는 분께는 뒤쪽 연표부터 보시길 권합니다.

21,000원

요리

사진이 큼직합니다.

17,500원

그림책

아이와 같이 보세요.

9,800원

문제 15

화면 상자 셋이 무대 가운데로 모이고 좌우에 120px 씩 빈 칸이 생깁니다

- 1
- 2
- 3

정리

12단원 연습문제 정답 - Flexbox

문제 파일과 뼈대가 같습니다. TODO 자리에 답이 채워져 있고 각 답 아래에 '왜 그렇게 되는지' 를 붙여 두었습니다.

답이 달라도 화면이 같으면 맞은 것입니다.
다른 방법이 있는 문제에는 그 방법도 같이 적어 두었습니다.

아래 네 줄은 문제를 담아 두는 틀입니다

```
.stage {  
  width: 600px;  
  padding: 10px;
```

```

}

.tile {
  background: #2d6cdf;
  color: #fff;
  padding: 10px 0;
  width: 120px;
  box-sizing: border-box;
  text-align: center;
  font-size: 14px;
}

.tile-wide {
  width: 150px;
}

.tile-big {
  width: 200px;
}

```

문제 1 (개념01 섹션2)

파란 상자 세 개를 가로로 나란히 놓으세요.

```

.p1-stage {
  display: flex;
}

```

화면 상자 셋이 무대 왼쪽 안쪽 면에 딱 붙어 한 줄로 놓입니다. 사이에 틈이 없습니다

해설: display: flex 는 '자식들을 한 줄로 늘어놓아라' 는 명령이라
상자가 아니라 상자를 감싼 부모에 씁니다.

상자가 부모 폭 전부를 차지하던 블록 성질도 함께 사라집니다.

문제 2 (개념02 섹션3)

화면에 3 2 1 순서로 보이게 하세요.

```

.p2-stage {
  display: flex;
  flex-direction: row-reverse;
}

```

화면 오른쪽 끝부터 1, 2, 3 이 채워져 눈에는 3 2 1 로 보입니다.

"1" 의 오른쪽 끝이 무대 오른쪽 안쪽 면에 딱 붙습니다
해설: -reverse 는 주축의 시작점을 반대쪽으로 옮깁니다.

HTML 순서는 그대로이므로 코드를 읽어 주는 프로그램에는 1, 2, 3 으로 읽힙니다.

문제 3 (개념02 섹션5)

상자 다섯 개가 찌그러지지 않게 하세요.

```
.p3-stage {  
  display: flex;  
  flex-wrap: wrap;  
}
```

화면 윗줄에 네 개, 아랫줄에 한 개가 놓이고 폭은 모두 150px 그대로입니다

해설: flex-wrap 의 기본값은 nowrap 이라 자리가 모자라도 줄을 안 바꾸고
아이템을 줄여서 옥여넣습니다. wrap 을 주면 아랫줄로 내려가므로

더 이상 줄일 필요가 없어져 150px 이 지켜집니다.

문제 4 (개념03 섹션2)

상자 세 개를 가로 가운데로 모으세요.

```
.p4-stage {  
  display: flex;  
  justify-content: center;  
}
```

화면 상자 뭉치가 가운데에 오고 좌우에 120px 씩 빈 칸이 생깁니다

해설: 남은 공간 240px 을 좌우로 120px 씩 나눠 둔 결과입니다.
($600 - 120 \times 3 = 240$, $240 \div 2 = 120$ 이므로

첫 상자는 무대 안쪽 왼쪽 면에서 120px 떨어진 자리에서 시작합니다)

문제 5 (개념03 섹션2)

상자 두 개를 양 끝에 붙이세요.

```
.p5-stage {
  display: flex;
  justify-content: space-between;
}
```

화면 왼쪽 끝과 오른쪽 끝에 하나씩 붙고 사이가 360px 벌어집니다

해설: space-between 은 남은 공간을 '아이템 사이'에만 나눠 줍니다.

틈이 한 군데뿐이니 남은 360px 이 통째로 그 자리에 들어갑니다.

문제 6 (개념03 섹션4)

상자 사이를 20px 씩 띄우세요.

```
.p6-stage {
  display: flex;
  gap: 20px;
}
```

화면 상자 사이가 20px 씩 벌어지고 전체는 왼쪽에 붙어 있습니다

해설: gap 은 아이템 '사이'에만 들어가고 무대 가장자리에는 붙지 않습니다.

margin-right 로 했다면 마지막 상자 오른쪽에도 20px 이 붙습니다.

문제 7 (개념04 섹션2)

상자를 위아래 가운데에 두세요.

```
.p7-stage {
  display: flex;
  height: 140px;
  align-items: center;
}
```

화면 상자가 무대 한가운데 높이에 놓고 상자 키는 44px 입니다

해설: align-items 의 기본값은 stretch 라, 값을 주기 전에는 상자 키가 140px 로 늘어나 있었습니다. center 를 주면 늘리기를 그만두고

내용만큼(44px)으로 돌아간 뒤 가운데로 갑니다.

문제 8 (개념04 섹션5)

상자 하나를 가로세로 정중앙에 두세요.

```
.p8-stage {
  display: flex;
  height: 140px;
  justify-content: center;
  align-items: center;
}
```

화면 폭 200px 상자가 무대 한가운데에 놓입니다. 좌우에 남는 자리가 200px 로 같습니다

해설: 축이 두 개뿐이니 둘 다 center 면 무조건 정중앙입니다.

이 조합은 flex-direction 이 row 든 column 이든 결과가 같습니다.

문제 9 (개념05 섹션1)

첫 번째 상자만 남는 공간을 전부 가져가게 하세요.

```
.p9-stage {
  display: flex;
}
.p9-first {
  flex-grow: 1;
}
```

화면 첫 상자가 360px 로 넓어지고 나머지 둘은 120px 그대로입니다

해설: 남는 공간은 $600 - 120 \times 3 = 240\text{px}$ 입니다.
몫을 주장한 아이템이 하나뿐이니 240px 을 통째로 가져가
 $120 + 240 = 360\text{px}$ 이 됩니다.

나머지 둘은 기본값 flex-grow: 0 이라 한 몫도 안 받습니다.

문제 10 (개념05 섹션4)

글자 길이가 다른 상자 셋을 정확히 같은 폭으로 만드세요.


```
.p10-stage {
  display: flex;
}
.p10-stage .tile {
  flex: 1;
}
```

화면 상자 셋이 모두 200px 이 되어 무대를 꽉 채웁니다

해설: flex: 1 은 flex-grow: 1 · flex-shrink: 1 · flex-basis: 0% 입니다.
basis 가 0 이라 원래 폭(120px)과 글자 길이를 아예 안 보고
600px 을 그냥 3 등분합니다.

flex-grow: 1 만 썼다면 basis 가 auto 로 남아 폭이 제각각이 됩니다.

문제 11 [응용] (개념06 섹션1)

가게 이름은 왼쪽 끝, 메뉴는 오른쪽 끝, 둘을 위아래 가운데로.

```
.p11-bar {
  width: 600px;
  padding: 12px 20px;
  box-sizing: border-box;
  background: #2d3a4a;
  display: flex;
  justify-content: space-between;
  align-items: center;
}
```

화면 왼쪽 끝에 "행복 서점", 오른쪽 끝에 메뉴가 놓이고 글자 가운데 선이 맞습니다.

메뉴 덩어리의 오른쪽 끝이 남색 띠 안쪽 오른쪽 면에 붙습니다
해설: 아이템이 둘이므로 space-between 이면 하나는 맨 앞, 하나는 맨 뒤로 갑니다.

align-items: center 를 넣는 이유는 두 글자 크기가 18px 과 14px 로 다르기 때문입니다. 안 넣으면 기본값 stretch 로 둘 다 띠 높이만큼 늘어나 글자가 위쪽에 붙습니다.

다른 답: justify-content 대신 메뉴에 margin-left: auto 를 줘도 같습니다.
.p11-menu { margin-left: auto; }

아이템이 셋 이상일 때는 이쪽이 더 편합니다.

```
.p11-brand {
  margin: 0;
```

```

color: #fff;
    font-size: 18px;
    font-weight: bold;
}
.p11-menu {
    display: flex;
    gap: 16px;
}
.p11-menu a {
    color: #cfd9e6;
    font-size: 14px;
    text-decoration: none;
}

```

문제 12 [응용] (개념04 섹션3)

가운데 상자 하나만 아래쪽으로 내려보내세요.

```

.p12-stage {
    display: flex;
    height: 140px;
    align-items: flex-start;
}
.p12-middle {
    align-self: flex-end;
}

```

화면 첫째와 셋째는 위에 붙고 가운데 상자만 무대 바닥에 붙습니다.

셋 다 키는 44px 입니다
 해설: align-items 는 컨테이너에 쓰는 속성이라 아이템 전체에 적용됩니다.
 하나만 예외로 두려면 그 아이템에 align-self 를 씁니다.
 값의 종류는 align-items 와 같고 붙이는 자리만 다릅니다.

주축 쪽에는 이런 속성이 없습니다. justify-self 는 flex 에 없습니다.

문제 13 [도전] (개념06 섹션4)

본문 + 사이드바 2단. 간격 20px, 사이드바 200px 고정, 본문은 남는 만큼.

```

.p13-stage {
    display: flex;
    gap: 20px;
}

```

```

}

.p13-main {
  background: #fff;
  border: 1px solid #ccd5e0;
  padding: 12px 14px;
  box-sizing: border-box;
  flex: 1;
}

.p13-side {
  background: #f4f7fb;
  border: 1px solid #ccd5e0;
  padding: 12px 14px;
  box-sizing: border-box;
  flex: 0 0 200px;
}

```

화면 왼쪽 본문이 380px, 오른쪽 사이드바가 200px 로 20px 간격을 두고 놓입니다.

사이드바 상자의 키(80px)가 본문(124px)보다 짧습니다
 해설: $600 - 200 - 20 = 380$ 이 본문 뒹입니다. 본문에 flex: 1 을 주면
 폭을 직접 적지 않아도 남은 만큼을 알아서 차지합니다.
 flex: 0 0 200px 의 가운데 0 이 flex-shrink 입니다.
 여기가 1 이면 자리가 모자랄 때 사이드바가 200px 을 못 지킵니다.
 align-items: flex-start 를 안 주면 기본값 stretch 때문에

사이드바 상자가 본문과 같은 키로 길어집니다.

문제 14 [도전] (개념06 섹션2)

카드 목록. 줄바꿈, 간격 16px, 폭 260px 부터 시작, 가격은 카드 바닥에.

```

.p14-list {
  display: flex;
  flex-wrap: wrap;
  gap: 16px;
}

.p14-card {
  background: #fff;
  border: 1px solid #ccd5e0;
  padding: 12px 14px;
  box-sizing: border-box;
  flex: 1 1 260px;
  display: flex;
  flex-direction: column;
}

```

```
margin: 0 0 6px;
    font-size: 15px;
    font-weight: bold;
  }
  .p14-text {
    margin: 0;
    font-size: 13px;
    color: #555;
  }
  .p14-price {
    padding-top: 8px;
    font-size: 13px;
    color: #2d6cdf;
    font-weight: bold;
    margin-top: auto;
  }
}
```

화면 카드가 한 줄에 두 개씩 두 줄로 놓이고 폭은 292px 입니다.

윗줄 카드 둘은 키가 145px 로 같고, 가격 줄이 모두 카드 바닥에 붙습니다
 해설: 한 줄에 몇 개가 들어가는지는 flex-basis 가 정합니다.
 $260 \times 2 + 16 = 536$ 은 600 안에 들어가지만
 $260 \times 3 + 16 \times 2 = 812$ 는 안 들어가므로 두 개씩입니다.
 남은 폭을 둘이 나눠 가져 ($600 - 16$) $\div 2 = 292px$ 이 됩니다.
 카드 키가 같아지는 것은 align-items 의 기본값 stretch 덕분이라
 따로 할 일이 없습니다.
 가격을 바닥에 붙이려면 카드 자신을 세로 컨테이너로 만든 다음
 가격 줄에 margin-top: auto 를 줍니다.

남는 세로 공간이 전부 가격 줄 위로 몰려서 가격이 바닥으로 밀립니다.

문제 15 [확인] (개념01 섹션5)

두 줄을 아이템(.tile)에서 컨테이너(.p15-stage)로 옮기면 됩니다.

```
.p15-stage {
  display: flex;
  justify-content: center;
}
```

화면 상자 셋이 무대 가운데로 모이고 좌우에 120px 씩 빈 칸이 생깁니다

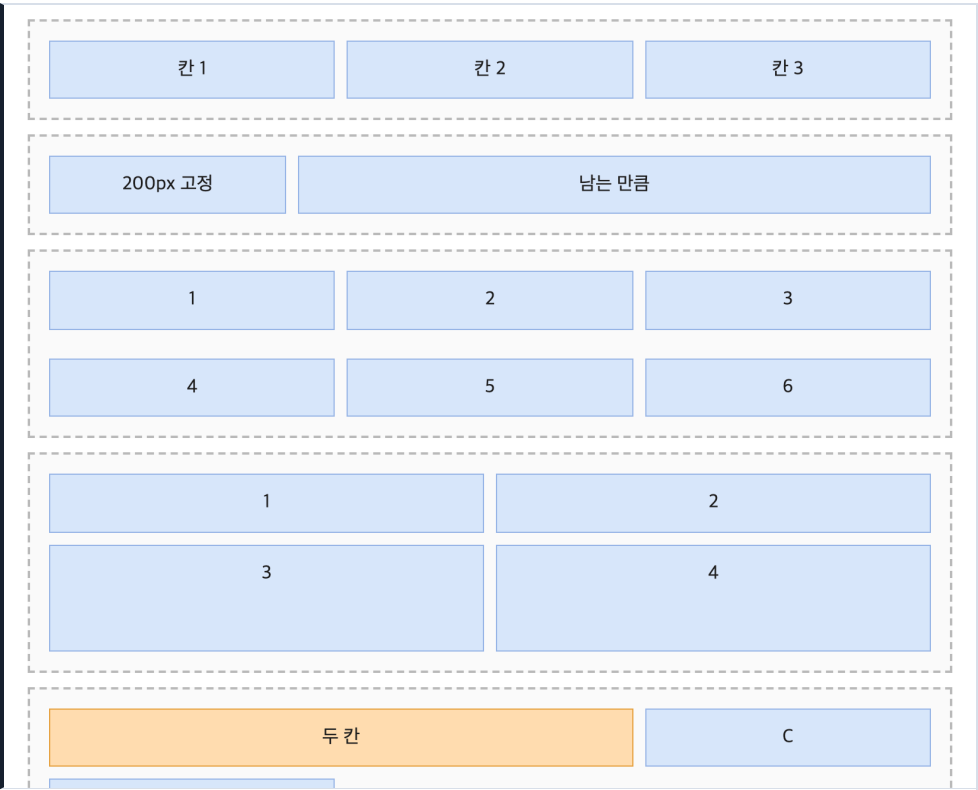
해설: 원래 코드는 두 줄을 .p15-stage .tile 에 썼습니다.
display: flex 는 '내 자식들을 늘어놓아라' 는 명령인데
상자 안에는 글자만 있고 자식 요소가 없으니 할 일이 없었습니다.
justify-content 도 '내 아이들을 어디에 모을까' 이므로
지시할 상대가 없어 그냥 무시됐습니다.
속성도 값도 맞았고 자리만 틀렸던 것입니다.
CSS 는 이럴 때 에러를 내지 않고 조용히 넘어갑니다.

그래서 '화면이 안 바뀐다' 가 유일한 신호입니다.

13단원 연습문제 정답 Grid와 반응형

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

답은 위쪽 style 안에 있습니다. 각 답 아래 해설을 같이 읽으세요.



실행하면 나오는 화면 — Grid 와 반응형

답은 이 파일 위쪽 <style> 안에 있습니다. 각 답 바로 아래에 결과와 해설을 붙여 두었으니 VS Code 로 함께 열어 읽으세요. 폭에 따라 달라지는 문제가 많습니다. F12 → Ctrl+Shift+M 으로 폭을 480, 768, 1280 으로 바꿔 가며 확인하세요.

문제 1

문제 1 — 격자를 만들고 세 칸으로 나누기

(개념01 섹션 1·2) .q1 을 격자로 만들고 칸을 세 개로 나누세요. 칸 크기는 셋 다 같게, 칸 사이는 10px 입니다.

화면 파란 칸 세 개가 268px 씩 가로로 나란히 놓입니다

칸 1
칸 2
칸 3

문제 2

문제 2 — 고정 칸과 남는 칸

(개념01 섹션 3) .q2 의 왼쪽 칸은 200px 로 고정하고, 오른쪽 칸은 남는 자리를 전부 쓰게 하세요.

화면 왼쪽 칸이 200px, 오른쪽 칸이 614px 입니다

200px 고정
남는 만큼

문제 3

문제 3 — 행 간격과 열 간격을 다르게

(개념01 섹션 4) .q3 의 위아래 줄 사이는 24px, 좌우 칸 사이는 10px 로 띄우세요. 값 두 개를 한 줄로 씁니다.

화면 좌우 사이는 10px, 위아래 두 줄 사이는 24px 로 벌어집니다.

칸 폭은 268px 입니다

1
2
3
4
5
6

문제 4

문제 4 — 줄 높이 정하기

(개념01 섹션 5) .q4 의 첫째 줄 높이를 50px, 둘째 줄 높이를 90px 로 정하세요.

화면 윗줄 두 칸의 높이가 50px, 아랫줄 두 칸이 90px 입니다.

칸 폭은 네 칸 모두 407px 입니다

1
2
3
4

문제 5

문제 5 — 선 번호로 두 칸 합치기

(개념02 섹션 2) .q5-wide 가 1번 선에서 3번 선까지, 즉 두 칸을 차지하게 하세요.

화면 주황 칸이 546px 로 두 칸을 차지하고 그 오른쪽에 C,

아랫줄 왼쪽에 D 가 놓입니다

두 칸
C
D

문제 6

문제 6 — span 으로 세 칸 차지하기

(개념02 섹션 3) .q6-wide 가 세 칸을 통째로 차지하게 하세요. 이번에는 선 번호 대신 span 을 쓰세요.

화면 주황 칸이 824px 로 윗줄 전체를 차지하고

B, C, D 가 아랫줄에 268px 씩 놓입니다


```
세 칸을 통째로
B
C
D
```

문제 7

문제 7 — repeat() 으로 네 칸 만들기

(개념02 섹션 4) .q7 을 네 칸으로 나누세요. 크기는 모두 같습니다. 1fr 을 네 번 쓰지 말고 repeat() 를 쓰세요.

화면 칸 네 개가 200px 씩 8px 간격으로 한 줄에 놓입니다

```
1
2
3
4
```

문제 8

문제 8 — 이름으로 배치하기

(개념02 섹션 5) .q8 에서 top 이 윗줄 두 칸을 차지하고, 아랫줄은 왼쪽이 left, 오른쪽이 right 가 되게 하세요.

화면 위에 파란 띠가 824px 로 놓고, 아래 왼쪽이 주황 200px,

오른쪽이 파랑 614px 입니다

```
top
left
right
```

문제 9

문제 9 — 미디어쿼리 없이 반응형 격자

(개념02 섹션 6) .q9 의 칸 하나는 최소 250px 이고, 자리가 남으면 남는 만큼 늘어나야 합니다. 빈 자리는 접혀야 합니다.

화면 폭 1280 에서 268px 짜리 3칸, 폭 768 에서 341px 2칸,

폭 480 에서 404px 1칸이 됩니다

카드 1
카드 2
카드 3
카드 4
카드 5
카드 6

문제 10

문제 10 — 미디어쿼리로 2칸 만들기

(개념03 섹션 4·6) .q10 은 기본이 1칸입니다. 폭이 700px 이상일 때만 2칸이 되게 하세요.

화면 폭 1280 에서 407px 짜리 2칸, 폭 480 에서 404px 짜리 1칸입니다

카드 1
카드 2
카드 3
카드 4

문제 11

문제 11 — 사진이 액자 밖으로 안 나가게

(개념04 섹션 3) .q11-photo 가 액자 밖으로 나가지 않게 하세요. 가로세로 비율은 그대로 유지되어야 합니다.

화면 그림이 300 × 150 으로 줄어들어 회색 액자 안에 딱 들어갑니다

문제 12

문제 12 — 썸네일을 꽉 채우기

(개념04 섹션 5) .q12-thumb 는 180 × 180 짜리 액자입니다. 비율을 지킨 채 액자를 꽉 채우고, 넘치는 부분은 잘리게 하세요. 가로로 긴 그림이라 좌우가 잘리고, 오른쪽 끝의 해는 화면에서 사라집니다.

화면 액자는 180 × 180 그대로이고 그림 좌우가 잘립니다.

산이 찌그러지지 않고 원래 비율로 보이며, 오른쪽에 있던 해는 잘려 나가 안 보입니다

문제 13

문제 13 [응용] — 글자 크기를 부드럽게

(개념04 섹션 4) .q13-title 의 글자 크기를 화면 폭의 2.5% 로 하되, 최소 16px, 최대 28px 을 벗어나지 않게 하세요.

화면 폭 1280 에서 28px, 폭 768 에서 19.2px, 폭 480 에서 16px 입니다

우리 동네 도서관

문제 14

문제 14 [응용] — 모바일 우선 1단 → 2단 → 3단

(개념05 섹션 3·4·5) .q14 를 기본 1칸, 600px 이상 2칸, 900px 이상 3칸 으로 만드세요.

화면 폭 1280 에서 268px 3칸, 폭 768 에서 341px 2칸,

폭 480 에서 404px 1칸입니다

카드 1
카드 2
카드 3
카드 4
카드 5
카드 6

문제 15

문제 15 [도전] — 반응형 페이지 뼈대

(개념02 섹션 5 + 개념05 섹션 7) .q15 를 좁은 화면에서는 네 덩어리가 세로로 쌓이게, 800px 이상에서는 head 아래에 왼쪽 main · 오른쪽 side(200px), 맨 아래 foot 이 오게 만드세요.

화면 폭 1280 에서 head 824px, 그 아래 main 614px 와 side 200px 가 나란히,

맨 아래 foot 824px 입니다. 폭 768 과 480 에서는 네 덩어리가 세로로 한 줄씩 쌓입니다

head
main
side
foot

문제 16

문제 16 [확인] — 왜 반대로 동작할까요

고친 곳은 한 군데입니다. @media (max-width: 700px) 를 @media (min-width: 700px) 로 바꿨습니다. max-width 는 “이 값 이하일 때” 라서 좁은 화면에서만 조건이 맞았습니다.

화면 폭 1280 에서 268px 짜리 3칸, 폭 480 에서 404px 짜리 1칸입니다

카드 1
카드 2
카드 3

정리

13단원 연습문제 정답 - Grid 와 반응형

문제 파일과 같은 뼈대에 답을 채운 파일입니다.

답만 보지 말고, 각 답 아래의 '화면' 과 해설을 같이 읽으세요.

꼭에 따라 달라지는 문제가 많으니 F12 → Ctrl+Shift+M 을 켜 두세요.

★ 적혀 있는 px 값은 모두 '브라우저 창을 넓게(950px 이상) 열었을 때' 의 값입니다.

이 페이지의 본문은 최대 900px 까지만 넓어지므로, 그보다 넓은 창에서는 아무리 더 넓혀도 숫자가 안 변합니다.

반대로 창을 좁히면 본문이 같이 줄어들어 숫자가 전부 달라집니다.

1~8 번은 기기 툴바를 끄고 넓은 창에서 보세요.

9 번부터는 문제마다 어느 쪽에서 보라고 따로 적어 두었습니다.

아래 여섯 개는 이미 되어 있습니다. 고치지 마세요

.q-scroll 은 아직 안 푼 문제(8번, 11번) 때문에 상자가 옆으로 빠져나가더라도 자료 페이지 전체가 밀리지 않게 막아 주는 껍데기입니다(10단원 overflow).

답을 다 채운 이 파일에서는 빠져나가는 것이 없으므로 아무 일도 안 합니다.

```
.q-scroll {
  overflow-x: auto;
}
.q-cell {
  background: #dbe7fb;
  border: 1px solid #9ab8e8;
  padding: 10px;
  text-align: center;
}
.q-hi {
  background: #ffe0b3;
  border-color: #e8a33d;
}
.q-frame {
  max-width: 300px;
  border: 2px solid #bbbbbb;
  padding: 6px;
}
.q-band {
  background: #f4f4f4;
  border: 1px solid #cccccc;
```

```
text-align: center;
}
.q-dark {
  background: #333333;
  color: #ffffff;
  padding: 8px;
  text-align: center;
}
```

문제 1 (개념01 섹션 1·2)

.q1 을 격자로 만들고 칸을 세 개로 나누세요.

칸 크기는 셋 다 같아야 하고, 칸 사이는 10px 입니다.

```
.q1 {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 10px;
}
```

화면 파란 칸 세 개가 268px 씩 가로로 나란히 놓입니다.

해설: display: grid 는 부모에 씁니다. 자식은 저절로 칸이 됩니다.
grid-template-columns 에 적은 값의 개수가 곧 칸의 개수입니다.

상자 안쪽 824px 에서 gap 두 개(20px) 를 빼면 804, 나누기 3 은 268 입니다.

문제 2 (개념01 섹션 3)

.q2 의 왼쪽 칸은 200px 로 고정하고, 오른쪽 칸은 남은 자리를 전부 쓰게 하세요.

```
.q2 {
  display: grid;
  gap: 10px;
  grid-template-columns: 200px 1fr;
}
```

화면 왼쪽 칸이 200px, 오른쪽 칸이 614px 입니다.

해설: fr 은 '남는 공간' 을 나눠 가지는 단위입니다.
824 에서 고정 200 과 gap 10 을 먼저 빼면 614 가 남고,

1fr 하나뿐이라 그 614 를 통째로 가져갑니다.

문제 3 (개념01 섹션 4)

.q3 의 위아래 줄 사이는 24px, 좌우 칸 사이는 10px 로 띄우세요.

```
.q3 {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 24px 10px;
}
```

화면 좌우는 좁게(10px), 위아래 두 줄 사이는 넓게(24px) 벌어집니다. 칸 폭은 268px.

해설: 값이 두 개면 앞이 행 사이(위아래), 뒤가 열 사이(좌우) 입니다.
margin: 10px 20px 이 위아래 먼저였던 것과 같은 순서입니다.

열 간격만 268px 계산에 들어가므로 24px 은 칸 폭에 영향을 주지 않습니다.

문제 4 (개념01 섹션 5)

.q4 의 첫째 줄 높이를 50px, 둘째 줄 높이를 90px 로 정하세요.

```
.q4 {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 10px;
  grid-template-rows: 50px 90px;
}
```

화면 윗줄 두 칸의 높이가 50px, 아랫줄 두 칸이 90px 입니다. 칸 폭은 407px.

해설: rows 는 columns 와 쓰는 법이 똑같고, 적은 순서대로 첫째 줄·둘째 줄입니다.
칸 안의 글자는 한 줄뿐인데도 높이가 맞춰집니다.

내용이 아니라 격자가 높이를 정하기 때문입니다.

문제 5 (개념02 섹션 2)

.q5-wide 가 1번 선에서 3번 선까지, 즉 두 칸을 차지하게 하세요.

```
.q5 {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 10px;
}
.q5-wide {
  grid-column: 1 / 3;
}
```

화면 주황 칸이 546px 로 넓어지고, 그 오른쪽에 C, 아랫줄에 D 가 놓입니다.

해설: 격자는 칸이 아니라 선에 번호를 붙입니다. 1부터 시작합니다.
1번 선과 3번 선 사이에는 칸이 두 개 있습니다.

폭은 $268 + 10(\text{gap}) + 268 = 546$ 입니다. 합칠 때 gap 도 같이 먹습니다.

문제 6 (개념02 섹션 3)

.q6-wide 가 세 칸을 통째로 차지하게 하세요. span 을 쓰세요.

```
.q6 {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 10px;
}
.q6-wide {
  grid-column: span 3;
}
```

화면 주황 칸이 824px 로 윗줄 전체를 차지하고 B, C, D 가 아랫줄에 놓입니다.

해설: span 3 은 '여기서부터 세 칸' 입니다. 선 번호로 쓰면 1 / 4 와 같습니다.
grid-column: 3 이라고 쓰면 세 칸이 아니라 '3번 선에서 시작하는 한 칸' 입니다.

숫자 하나는 언제나 시작 선 번호라는 점을 기억하세요.

문제 7 (개념02 섹션 4)

.q7 을 네 칸으로 나누세요. repeat() 를 쓰세요.


```
.q7 {
  display: grid;
  gap: 8px;
  grid-template-columns: repeat(4, 1fr);
}
```

화면 칸 네 개가 200px 씩 8px 간격으로 한 줄에 놓입니다.

해설: repeat(개수, 값) 은 1fr 1fr 1fr 1fr 이라고 직접 쓴 것과 완전히 같습니다.

괄호 안에서만 심표를 쓰고 괄호 밖에서는 띄어쓰기라는 점이 헛갈리는 자리입니다.

824 에서 gap 세 개(24px) 를 빼면 800, 나누기 4 는 200 입니다.

문제 8 (개념02 섹션 5)

top 이 윗줄 두 칸, 아랫줄은 왼쪽이 left, 오른쪽이 right.

```
.q8 {
  display: grid;
  grid-template-columns: 200px 1fr;
  gap: 10px;
  grid-template-areas:
    "top top"
    "left right";
}
.q8-top {
  grid-area: top;
}
.q8-left {
  grid-area: left;
}
.q8-right {
  grid-area: right;
}
```

화면 위에 파란 띠가 824px 로 놓이고, 아래 왼쪽이 주황 200px, 오른쪽이 파랑 614px 입니다.

해설: 따옴표 한 줄이 격자의 한 행이고, 그 안의 이름 하나가 칸 하나입니다. 같은 이름을 두 번 쓰면 그만큼 칸이 합쳐집니다.

줄마다 이름 개수가 다르면 배치가 통째로 무시되니 세로로 줄을 맞춰 쓰세요.

문제 9 (개념02 섹션 6)

칸 하나는 최소 250px, 남으면 늘어나고, 빈 자리는 접히게.

```
.q9 {  
  display: grid;  
  gap: 10px;  
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));  
}
```

화면 폭 1280 에서 268px 짜리 3칸, 폭 768 에서 341px 2칸, 폭 480 에서 404px 1칸입니다.

해설: auto-fit 은 '몇 칸이 들어가는지 네가 세라' 는 뜻입니다.
 $250 \times 3 + 10 \times 2 = 770$ 은 824 에 들어가지만 $250 \times 4 + 30 = 1030$ 은 안 들어가서 3칸입니다.

minmax 는 최솟값이 먼저입니다. 미디어쿼리를 한 줄도 안 썼다는 점을 보세요.

문제 10 (개념03 섹션 4·6)

기본은 1칸, 폭이 700px 이상일 때만 2칸.

```
.q10 {  
  display: grid;  
  grid-template-columns: 1fr;  
  gap: 10px;  
}  
@media (min-width: 700px) {  
  .q10 {  
    grid-template-columns: 1fr 1fr;  
  }  
}
```

화면 폭 1280 에서 407px 짜리 2칸, 폭 480 에서 404px 짜리 1칸입니다.

해설: @media 안에는 선택자까지 써야 합니다. 중괄호가 두 겹입니다.
min-width 는 '이 값 이상' 이므로 넓어질 때 조건이 맞습니다.

조건이 안 맞으면 그 안의 규칙은 아예 없는 것이 되어 기본만 남습니다.

문제 11 (개념04 섹션 3)

.q11-photo 가 액자 밖으로 안 나가게. 비율은 유지.

```
.q11-photo {
  max-width: 100%;
}
```

화면 그림이 300 × 150 으로 줄어들어 회색 액자 안에 딱 들어갑니다.

해설: 100% 는 부모(액자) 안쪽 폭인 300px 입니다.
max-width 는 상한선이라 '이보다 커지지만 마라' 는 뜻이고, 작은 그림은 안 늘립니다.

height 를 안 정했으므로 브라우저가 가로에 맞춰 세로를 150px 로 계산해 줍니다.

문제 12 (개념04 섹션 5)

180 × 180 액자를 비율 지킨 채 꽉 채우고, 넘치면 자르기.

```
.q12-thumb {
  width: 180px;
  height: 180px;
  border: 1px solid #bbbbbb;
  box-sizing: border-box;
  object-fit: cover;
}
```

화면 액자는 그대로 180 × 180 이고, 그림이 액자 높이에 맞춰지면서 좌우가 잘립니다.

산이 찌그러지지 않고 원래 비율로 보이는 대신,
오른쪽 끝에 있던 해는 잘려 나가 안 보입니다.
해설: 기본값 fill 은 액자 모양대로 잡아 늘려서 찌그러뜨립니다.
cover 는 비율을 지킨 채 꽉 채우고 넘치는 부분을 자릅니다. 썸네일의 기본입니다.

다 보여야 하는 로고나 도표라면 contain 을 씁니다.

문제 13 [응용] (개념04 섹션 4)

화면 폭의 2.5%, 최소 16px, 최대 28px.

```
.q13-title {
  font-weight: bold;
  margin: 0;
  font-size: clamp(16px, 2.5vw, 28px);
}
```

화면 폭 1280 에서 28px, 폭 768 에서 19.2px, 폭 480 에서 16px 입니다.

해설: clamp 의 순서는 최소, 기준, 최대입니다.
폭 1280 에서 2.5vw 는 32px 이지만 최대 28px 에 걸려 28px 이 됩니다.

폭 480 에서는 12px 이지만 최소 16px 로 올라가 읽을 수 있는 크기가 유지됩니다.

문제 14 [응용] (개념05 섹션 3·4·5)

기본 1칸, 600px 이상 2칸, 900px 이상 3칸.

```
.q14 {
  display: grid;
  grid-template-columns: 1fr;
  gap: 10px;
}
@media (min-width: 600px) {
  .q14 {
    grid-template-columns: repeat(2, 1fr);
  }
}
@media (min-width: 900px) {
  .q14 {
    grid-template-columns: repeat(3, 1fr);
  }
}
```

화면 폭 1280 에서 268px 3칸, 폭 768 에서 341px 2칸, 폭 480 에서 404px 1칸입니다.

해설: 기본을 좁은 화면용으로 두고 min-width 로 덧붙이는 것이 모바일 우선입니다.

폭 1280 은 600 조건도 900 조건도 다 맞습니다.

명시도가 같으면 나중에 쓴 것이 이기므로 900px 짜리를 아래에 써야 3칸이 됩니다.

문제 15 [도전] (개념02 섹션 5 + 개념05 섹션 7)

좁은 화면에서는 세로로 네 줄, 800px 이상에서는 main + side(200px).

```
.q15 {
  display: grid;
  gap: 10px;
  grid-template-columns: 1fr;
  grid-template-areas:
    "head"
    "main"
    "side"
    "foot";
}

.q15-head {
  grid-area: head;
}

.q15-main {
  grid-area: main;
}

.q15-side {
  grid-area: side;
}

.q15-foot {
  grid-area: foot;
}

@media (min-width: 800px) {
  .q15 {
    grid-template-columns: 1fr 200px;
    grid-template-areas:
      "head head"
      "main side"
      "foot foot";
  }
}
```

화면 폭 1280 에서 head 가 824px, 그 아래 main 614px 와 side 200px 가 나란히,

맨 아래 foot 이 824px 입니다.

폭 768 과 480 에서는 네 덩어리가 세로로 한 줄씩 쌓입니다.

해설: 기본에 열을 하나만 두면 이름이 한 줄에 하나씩이라 자동으로 세로로 쌓입니다.

넓어질 때만 열을 두 개로 늘리고 따옴표 안 이름을 다시 넣어놓으면 됩니다.

HTML 은 한 글자도 안 고쳤습니다. 배치는 전부 CSS 가 정합니다.

문제 16 [확인] (개념03 섹션 5·7 + 개념05 섹션 8)

고친 곳: @media (max-width: 700px) → @media (min-width: 700px)

원래 코드는 이랬습니다.

```
@media (max-width: 700px) {  
  .q16 { grid-template-columns: 1fr 1fr 1fr; }  
}
```

max-width 는 '이 값 이하일 때' 이므로 좁은 화면에서만 조건이 맞습니다.
그래서 폭 480 에서 3칸이 되고, 폭 1280 에서는 조건이 안 맞아 기본인 1칸이
그대로 남았습니다. 하고 싶었던 것과 정확히 반대입니다.

단어 하나만 min-width 로 바꾸면 조건이 뒤집혀서 넓을 때 3칸이 됩니다.

기본이 1칸이고 넓어질 때 덧붙이는 모양이니 모바일 우선이기도 합니다.

```
.q16 {  
  display: grid;  
  grid-template-columns: 1fr;  
  gap: 10px;  
}  
@media (min-width: 700px) {  
  .q16 {  
    grid-template-columns: 1fr 1fr 1fr;  
  }  
}
```

화면 폭 1280 에서 268px 짜리 3칸, 폭 480 에서 404px 짜리 1칸입니다.

해설: 값도 선택자도 멀쩡했습니다. 조건의 방향만 반대였습니다.
이런 실수는 에러가 안 나기 때문에 폭을 바꿔 보기 전에는 알 수가
없습니다.

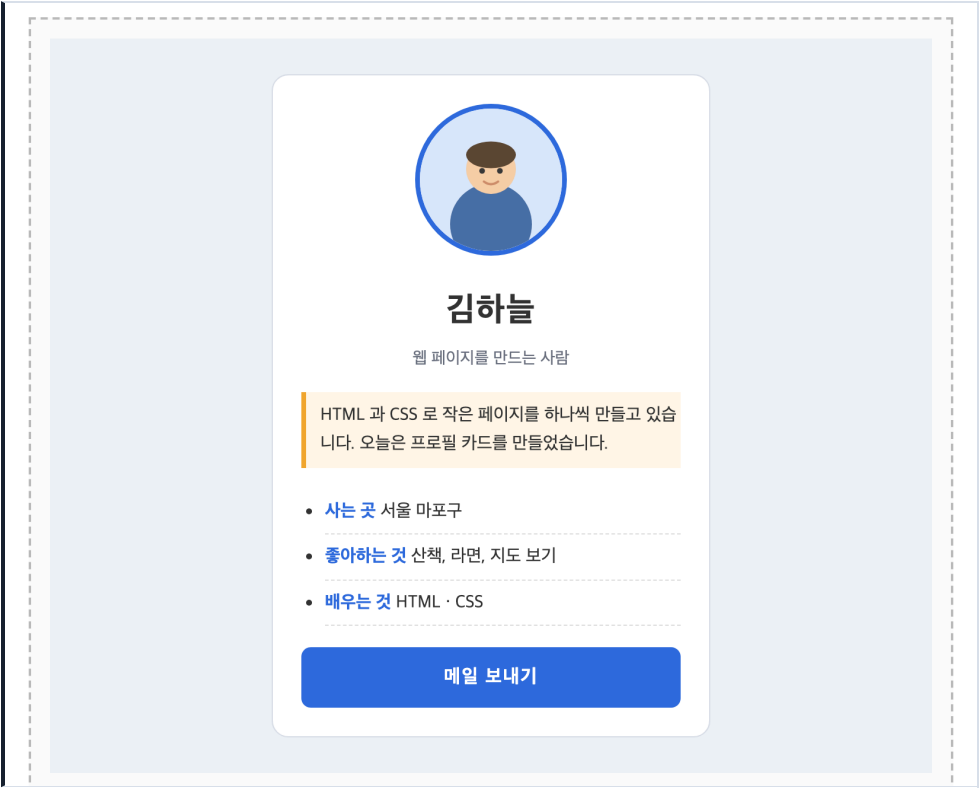
미디어쿼리를 쓸 때마다 기기 툴바로 양쪽 폭을 한 번씩 확인하는 습관을 들이세요.

14단원 ·

14단원 종합 01 정답 프로필 카드

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

여러분이 만든 종합01_프로필_카드.html 과 나란히 열어 대조하세요.



실행하면 나오는 화면 — 종합 01 정답 — 프로필 카드

답이 글자 하나까지 같을 필요는 없습니다. 화면이 같고 F12 로 켜 숫자가 같으면 맞은 것입니다. 예를 들어 단계 5 의 padding: 24px 은 padding: 24px 24px 24px 24px 라고 써도 같습니다.

이 카드에는 display 도 position 도 flex 도 grid 도 쓰이지 않았습니다. 블록 요소가 위에서 아래로 쌓이는 기본 흐름 + 박스 모델 + 색과 글자, 10단원까지 배운 것만으로 여기까지 만들 수 있습니다.

아래가 완성된 모습입니다. 각 단계의 답과 해설은 이 파일을 VS Code 로 열면 <style> 안과 각 단계 주석에 있습니다. 답이 달라도 결과가 같으면 맞은 것입니다.

완성된 카드입니다. 단계 1:2 에서 만든 HTML 이 이 모양입니다.

화면 연회색 무대 한가운데에 흰 카드가 한 장 놓입니다.

위에서부터 동그란 파란 테 사진 → 이름 → 회색 직함 → 주황 띠가 붙은 소개 → 점선으로 나뉜 정보 세 줄 → 파란 연락 띠 순서로 쌓여 있습니다

김하늘

웹 페이지를 만드는 사람

HTML 과 CSS 로 작은 페이지를 하나씩 만들고 있습니다. 오늘은 프로필 카드를 만들었습니다.

사는 곳 서울 마포구

좋아하는 것 산책, 라면, 지도 보기

배우는 것 HTML · CSS

메일 보내기

정리

단계 1

카드 한 장은 그 자체로 뜻이 통하는 덩어리입니다. 그래서 div 대신 article 로 감쌉니다. 무대 상자(card-stage) 안에 아래를 씁니다.

답:

```
<article class="profile-card"> <h2 class="profile-name">김하늘
</h2> <p class="profile-job">웹 페이지를 만드는 사람</p>
</article>
```

해설: article 도 h2 도 p 도 전부 블록 요소입니다.

블록 요소는 한 줄을 통째로 차지하고 위에서 아래로 쌓입니다. 이것이 문서 흐름입니다. 그래서 아무 CSS 를 안 줘도 이름이 먼저, 직함이 그다음 줄에 놓입니다.

단계 2

답:


```
<p class="profile-photo-box"> 
</p>
... 이름과 직함 ...
<p class="profile-intro">HTML 과 CSS 로 ...</p>
<ul class="profile-info"> <li><strong>사는 곳</strong> 서울 마포구</li> <li><strong>
좋아하는 것</strong> 산책, 라면, 지도 보기</li> <li><strong>배우는 것</strong> HTML ·
CSS</li>
</ul>
<p class="profile-contact"> <a class="profile-link" href="mailto:haneul@example.com">
메일 보내기</a>
</p>
```

해설: 사진을 p 로 한 번 감싼 이유가 단계 6 에 있습니다. img 는 인라인이라

자기 혼자서는 가운데로 못 갑니다. 감싼 블록에 부탁해야 합니다.
링크도 인라인이라 같은 이유로 p 로 감쌌습니다.
alt 는 그림이 안 보일 때 대신 읽히는 글이므로 반드시 씁니다.
mailto 뒤에는 쌍점(:) 만 옵니다. 빗금 두 개를 붙이지 않습니다.

단계 3

답: background-color: #ffffff;

color: #333333;

해설: 배경이 흰색이면 회색 무대 위에서 카드의 자리가 눈에 보이기 시작합니다.

color 는 글자색이고 자식에게 상속되므로 카드 안 글자가 모두 진회색이 됩니다.
#ffffff 는 #fff 로, #333333 은 #333 으로 줄여 써도 같습니다.

단계 4

답: width: 320px;

margin: 0 auto;

해설: margin 의 좌우를 auto 로 두면 남는 자리를 왼쪽과 오른쪽이 똑같이 나뉘어 갑니다.

그래서 상자가 가운데로 갑니다. 이때 width 가 반드시 있어야 합니다.
폭이 없으면 블록 요소는 한 줄을 다 채우므로 남는 자리가 없어 auto 가 할 일이
없습니다.
text-align: center 는 글자만 옮깁니다. 상자를 옮기는 것은 margin 의 일입니다.

단계 5

답: padding: 24px;

```
border: 1px solid #d8dee8;
border-radius: 14px;
```

해설: width 는 콘텐츠 영역만 가리킵니다. 그래서 카드의 실제 바깥 폭은

$320 + 24 + 24 + 1 + 1 = 370\text{px}$ 이 됩니다. 폭이 320px 그대로라면
어디선가 box-sizing: border-box 가 켜진 것입니다.
border 는 두께 → 모양 → 색 순서이고, 모양(solid) 을 빠뜨리면 선이 아예 안 보입니다.

단계 6

답: .profile-photo-box { text-align: center; margin: 0 0 14px; }

```
.profile-photo { width: 120px; height: 120px;
                  border-radius: 50%; border: 4px solid #2d6cdf; }
```

해설: 가로와 세로가 같아야 50% 가 완전한 동그라미가 됩니다. 세로를 빠뜨리면 타원이 됩니다.

img 는 인라인이라 margin: 0 auto 가 안 먹습니다. 대신 감싼 p 에 text-align: center 를
주면 안쪽 인라인 요소인 img 가 가운데로 갑니다.
테두리 4px 이 양옆에 붙어 사진이 차지하는 폭은 128px 이 됩니다.

단계 7

답: .profile-name { margin: 0 0 4px; font-size: 1.75rem;

```
                text-align: center; letter-spacing: 1px; }
.profile-job { margin: 0 0 16px; font-size: 0.875rem;
                color: #6b7280; text-align: center; }
```

해설: h2 와 p 는 브라우저가 미리 위아래 margin 을 넣어 둡니다. 그대로 두면 이름과 직함이

멀찍이 떨어집니다. margin 을 0 으로 눌러야 한 덩어리로 보입니다.
1rem 은 16px 이므로 1.75rem 은 28px, 0.875rem 은 14px 입니다.
글자 크기에 rem 을 쓰면 여러 겹 중첩돼도 값이 곱해지지 않습니다.

단계 8

답: `margin: 0 0 18px;`

```
padding: 8px 0 8px 12px;
border-left: 4px solid #f0a830;
background-color: #fff8e8;
font-size: 0.9375rem;
line-height: 1.6;
```

해설: 테두리는 네 방향을 따로 그을 수 있습니다. 왼쪽만 그으면 강조 띠가 됩니다.

띠와 글자가 붙지 않도록 `padding-left` 를 같이 줍니다. 띠는 `padding` 바깥에 그어지므로 `padding` 이 없으면 글자가 띠에 달라붙습니다.
`padding` 값 네 개는 위 → 오른쪽 → 아래 → 왼쪽 시계 방향입니다.

단계 9

답: `.profile-info { margin: 0 0 18px; padding-left: 20px; font-size: 0.9375rem; }`

```
.profile-info li { padding: 6px 0; border-bottom: 1px dashed #e0e0e0; }
.profile-info strong { color: #2d6cdf; }
```

해설: ul 은 브라우저가 `padding-left` 를 40px 이나 넣어 줍니다. 그대로 두면 목록만 안쪽으로

푹 들어가 보이므로 20px 로 줄입니다. 클머리 기호는 이 안쪽 여백 자리에 그려지므로 0 으로 만들면 기호가 카드 왼쪽 벽에 바짝 붙어 위 문단들과 어긋나 보입니다.
`.profile-info li` 는 "profile-info 안에 있는 li" 를 고르는 자손 선택자입니다.
 공백 하나가 "안에 있는" 이라는 뜻입니다. 붙여 쓰면 뜻이 완전히 달라집니다.

단계 10

답: `.profile-contact { margin: 0; padding: 12px 0; text-align: center;`

```
background-color: #2d6cdf; border-radius: 8px; }
.profile-link { color: #ffffff; text-decoration: none;
font-weight: bold; letter-spacing: 1px; }
.profile-link:hover { text-decoration: underline; }
```

해설: 띠를 만든 것은 a 가 아니라 a 를 감싼 p 입니다. a 는 인라인이라 글자만큼만 차지해서

위아래 padding 을 줘도 줄 높이가 늘어나지 않습니다. 그래서 배경을 블록인 p 에 줍니다. 대신 늘리는 범위는 글자뿐입니다. 띠 전체를 누를 수 있게 만드는 방법은 11단원의 display 에서 배웁니다. 밑줄을 없앴으면 무엇으로든 링크임을 남겨야 합니다. 여기서는 굵기와 색으로 남기고, 마우스를 올렸을 때 :hover 로 밑줄을 되살렸습니다.

종합 01 정답 - 프로필 카드

단계 번호는 문제 파일과 똑같습니다. 나란히 열어 놓고 비교하세요.

카드가 놓이는 무대입니다. 문제 파일에 미리 들어 있던 규칙입니다.

```
.card-stage {  
  background-color: #eef2f7;  
  padding: 30px 0;  
}
```

단계 3 · 4 · 5 는 이 규칙 하나를 함께 채웁니다

```
.profile-card {
```

background-color: #ffffff; /* 단계 3 color: #333333; /* 단계 3 width: 320px; /* 단계 4 margin: 0 auto; /* 단계 4 padding: 24px; /* 단계 5 border: 1px solid #d8dee8; /* 단계 5 border-radius: 14px; /* 단계 5

```
}
```

화면 회색 무대 한가운데에 흰 카드 한 장이 놓입니다. 바깥 폭은 370px 입니다

단계 6

```
.profile-photo-box {  
  text-align: center;  
  margin: 0 0 14px;  
}  
.profile-photo {  
  width: 120px;  
  height: 120px;  
  border-radius: 50%;  
  border: 4px solid #2d6cdf;  
}
```

화면 네모였던 사진이 동그래지고 파란 테를 두른 채 카드 가운데에 놓입니다

단계 7

```
.profile-name {
  margin: 0 0 4px;
  font-size: 1.75rem;
  text-align: center;
  letter-spacing: 1px;
}
.profile-job {
  margin: 0 0 16px;
  font-size: 0.875rem;
  color: #6b7280;
  text-align: center;
}
```

화면 이름이 28px 로 커지고 바로 아래 붙습니다. 직함은 14px 회색으로 작아집니다

단계 8

```
.profile-intro {
  margin: 0 0 18px;
  padding: 8px 0 8px 12px;
  border-left: 4px solid #f0a830;
  background-color: #fff8e8;
  font-size: 0.9375rem;
  line-height: 1.6;
}
```

화면 소개 문단 왼쪽에 주황 세로 띠가 생기고 연노랑 배경이 깔립니다

단계 9

```
.profile-info {
  margin: 0 0 18px;
  padding-left: 20px;
  font-size: 0.9375rem;
}
.profile-info li {
  padding: 6px 0;
  border-bottom: 1px dashed #e0e0e0;
}
.profile-info strong {
  color: #2d6cdf;
}
```

단계 10

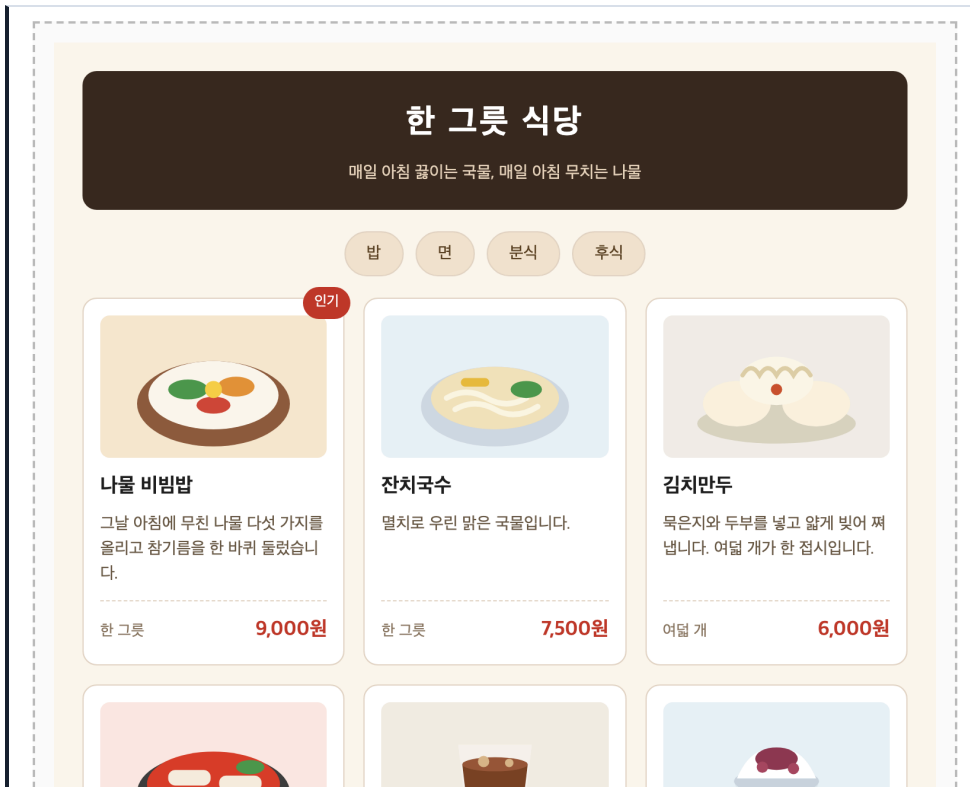
```
.profile-contact {  
  margin: 0;  
  padding: 12px 0;  
  text-align: center;  
  background-color: #2d6cdf;  
  border-radius: 8px;  
}  
.profile-link {  
  color: #ffffff;  
  text-decoration: none;  
  font-weight: bold;  
  letter-spacing: 1px;  
}  
.profile-link:hover {  
  text-decoration: underline;  
}
```

14단원 ·

14단원 종합 02 정답 음식점 메뉴판

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

여러분이 만든 종합02_음식점_메뉴판.html 과 나란히 열어 대조하세요.



실행하면 나오는 화면 — 종합 02 정답 — 음식점 메뉴판

답이 글자 하나까지 같을 필요는 없습니다. 화면이 같고 F12 로 쏜 숫자가 같으면 맞은 것입니다.

이 메뉴판에는 grid 도 @media 도 쓰이지 않았습니다. 13단원 내용이기 때문입니다. 가로 배치는 전부 flex 하나로 했습니다.

정답 style 을 보면 같은 선택자(.menu-card) 가 여러 번 나옵니다. 단계별로 무엇이 더해지는지 눈에 보이라고 일부러 나눠 둔 것입니다. 실제로 쓸 때는 한 규칙으로 합쳐 쓰는 편이 보통이고, 결과는 똑같습니다.

아래가 완성된 모습입니다. 각 단계의 답과 해설은 이 파일을 VS Code 로 열면 <style> 안과 각 단계 주석에 있습니다. 답이 달라도 결과가 같으면 맞은 것입니다.

화면 살구색 무대 위에 갈색 머리글 띠, 알약 네 개, 카드 여섯 장(3장씩 두 줄),

세트 표, 안내 목록, 갈색 바닥 띠가 차례로 쌓입니다

한 그릇 식당
매일 아침 끓이는 국물, 매일 아침 무치는 나물

밥
면
분식
후식

인기

나물 비빔밥

그날 아침에 무친 나물 다섯 가지를 올리고 참기름을 한 바퀴 돌렸습니다.

한 그릇
9,000원

잔치국수
멸치로 우린 맑은 국물입니다.

한 그릇
7,500원

김치만두

묵은지와 두부를 넣고 얇게 빳어 찌넵니다. 여덟 개가 한 접시입니다.

여덟 개
6,000원

옛날 떡볶이

고추장에 조청을 넣어 맵기보다 달큰합니다.

한 접시
6,500원

수정과

계피와 생강을 오래 달여 차게 식혔습니다.

한 잔
3,000원

팔빙수

직접 삶은 단팥을 올립니다. 여름에만 냅니다. 두 사람이 먹기 좋은 양입니다.

한 그릇
8,000원

세트 메뉴

세트 이름
구성
가격

점심 세트
밥 한 가지 + 수정과
11,000원

든든 세트
밥 한 가지 + 만두 + 수정과
14,000원

후식 세트
면 한 가지 + 팥빙수
13,500원

모든 반찬은 더 달라고 하시면 더 드립니다.
포장하실 때는 용기값 500원이 붙습니다.
매주 월요일은 쉽니다.

영업시간 11:00 부터 21:00 까지
전화 02-000-0000

정리

단계 1

답:

```
<header class="shop-head"> <h2 class="shop-name">한 그릇 식당</h2> <p class="shop-tagline">매일 아침 끓이는 국물, 매일 아침 무치는 나물</p>
</header>
```

해설: header 는 “이 덩어리가 머리글이다” 라는 뜻만 가진 태그입니다.

모양은 하나도 안 바뀝니다. div 로 써도 화면은 같습니다.
그래도 header 를 쓰는 이유는 나중에 이 파일을 읽는 사람과
화면을 읽어 주는 프로그램이 구조를 알아볼 수 있기 때문입니다.
이 페이지에는 이미 자료 제목용 h1 이 맨 위에 있으므로 가게 이름은 h2 로 씁니다.
제목 태그는 크기를 고르는 도구가 아니라 순위를 매기는 도구입니다.

단계 2

답: 카테고리 따는 p 하나에 span 네 개입니다.

```
<p class="menu-tabs"> <span class="menu-tab">밥</span> <span class="menu-tab">면</span> <span class="menu-tab">분식</span> <span class="menu-tab">후식</span>
</p>
```

카드 여섯 장은 div 하나로 감쌉니다. 카드 한 장은 이 모양입니다.

```
<div class="menu-list"> <article class="menu-card"> <span class="menu-badge">인기</span>  <h3 class="menu-name">나물 비빔밥</h3> <p class="menu-desc">그날 아침에 무친 나물 다섯 가지를 ...</p> <p class="menu-price-row"> <span class="menu-label">한 그릇</span> <span class="menu-price">9,000원</span> </p> </article>
... 나머지 다섯 장 ...
</div>
```

해설: 메뉴 하나하나는 그 자체로 뜻이 통하는 덩어리라서 div 보다 article 이 어울립니다.

반대로 카드 여섯 장을 묶는 바깥 상자는 뜻이 아니라 배치를 위한 것이라 div 를 씁니다.
배치를 article 안 맨 위에 두는 이유는 단계 9 에서 밝혀집니다.
지금은 "인기" 라는 글자가 그냥 카드 맨 위에 한 줄 나올 뿐입니다.
가격 줄을 p 하나 안에 span 두 개로 나눠 둔 것이 단계 9 의 준비물입니다.
span 은 뜻이 없는 인라인 상자입니다. "여기부터 여기까지" 를 표시할 때 씁니다.

단계 3

답:

```
<h3 class="set-title">세트 메뉴</h3>
<table class="set-table"> <thead> <tr><th>세트 이름</th><th>구성</th><th>가격</th>
</tr> </thead> <tbody> <tr><td>점심 세트</td><td>밥 한 가지 + 수정과</td><td>11,000원
</td></tr>
  ... 두 줄 더 ...
</tbody>
</table> <ul class="shop-notice"> <li>모든 반찬은 더 달라고 하시면 더 드립니다.</li>
  ... 두 줄 더 ...
</ul> <footer class="shop-foot"> <p class="shop-hours">영업시간 11:00 부터 21:00 까지
</p> <p class="shop-tel">전화 02-000-0000</p>
</footer>
```

해설: 표는 “가로줄과 세로줄이 둘 다 뜻을 가진 자료”에만 씁니다.

세트 표는 세로로 세트 이름 · 구성 · 가격이라는 뜻이 있으니 표가 맞습니다.
반대로 카드 여섯 장은 세로 열에 뜻이 없으므로 표로 만들면 안 됩니다.
th 는 머리칸이라 브라우저가 미리 굵게 + 가운데 정렬로 그려 줍니다.
안내 세 줄은 순서가 뜻을 갖지 않으므로 ol 이 아니라 ul 입니다.

단계 4

답: .shop-head { background-color: #3a2b21; color: #ffffff;

```
padding: 20px 16px; border-radius: 12px;
text-align: center; margin: 0 0 18px; }
.shop-name { margin: 0 0 6px; font-size: 1.75rem; letter-spacing: 2px; }
.shop-tagline { margin: 0; font-size: 0.875rem; color: #e6d3bd; }
```

해설: color 는 상속되는 속성입니다. header 에 흰색을 한 번 주면 안쪽 h2 와 p 가

따로 시키지 않아도 흰 글자가 됩니다. 소개 줄만 다른 색을 원하니 거기만 덮어썼습니다.
h2 와 p 에는 브라우저가 미리 위아래 margin 을 넣어 둡니다.
그대로 두면 두 줄이 멀찍이 떨어지고 갈색 띠도 필요 이상으로 두꺼워집니다.
그래서 margin 을 0 으로 눌러 놓고 필요한 만큼만 다시 줍니다.
1rem 이 16px 이므로 1.75rem 은 28px, 0.875rem 은 14px 입니다.

단계 5

답: .menu-tabs { display: flex; justify-content: center;

```
flex-wrap: wrap; gap: 10px; margin: 0 0 18px; }
.menu-tab { background-color: #f3e3cd; border: 1px solid #e2d5c3;
border-radius: 999px; padding: 6px 18px;
font-size: 0.875rem; color: #5a4326; }
```

해설: display: flex 는 부모인 p 에 씁니다. 그러면 안에 있는 span 네 개가

자동으로 아이템이 되어 가로로 줄을 씁니다.
 여기서 눈여겨볼 것이 하나 있습니다. span 은 원래 인라인이라
 위아래 padding 을 줘도 줄 높이가 늘어나지 않습니다(11단원 개념02).
 그런데 flex 아이템이 되는 순간 블록처럼 취급되어 위아래 padding 이 제대로 먹습니다.
 알약 모양이 납작해지지 않는 이유가 이것입니다.
 justify-content 는 주축(여기서는 가로) 에서 남는 자리를 어떻게 쓸지 정합니다.
 center 는 남는 자리를 양쪽 끝에 반씩 밀어 두어 딱지들을 가운데로 모읍니다.
 border-radius 를 반높이보다 크게 주면 그냥 반원이 되므로 999px 로 크게 줍니다.

단계 6

답: .menu-card { box-sizing: border-box; background-color: #ffffff;

```
border: 1px solid #e2d5c3; border-radius: 12px; padding: 14px; }
.menu-photo { width: 100%; height: auto; border-radius: 8px; margin: 0 0 10px; }
.menu-name { margin: 0 0 6px; font-size: 1.0625rem; }
.menu-desc { margin: 0 0 12px; font-size: 0.875rem;
line-height: 1.5; color: #6b5a48; }
```

해설: box-sizing: border-box 를 켜 두면 단계 7 에서 정할 폭 220px 안에

padding 14px 과 테두리 1px 이 포함됩니다. 이걸 안 켜면 카드가
 $220 + 14 + 14 + 1 + 1 = 250px$ 짜리로 시작해 한 줄에 두 장밖에 안 들어갑니다.
 .menu-photo 의 width: 100% 는 "부모의 안쪽 폭만큼" 이라는 뜻입니다.
 그림 파일 자체는 160px 짜리지만 카드 안쪽 폭인 218px 로 늘어납니다.
 height 는 건드리지 않아야 가로세로 비율이 유지됩니다. auto 가 그 뜻입니다.

단계 7

답: .menu-list { display: flex; flex-wrap: wrap; gap: 16px; margin: 0 0 22px; }

```
.menu-card { flex: 1 1 220px; }
```

해설: flex: 1 1 220px 는 "220px 에서 시작해서, 남으면 같이 늘고, 모자라면 같이 준다"

입니다. 첫 값이 grow, 둘째가 shrink, 셋째가 basis 입니다.
 flex-wrap: wrap 이 없으면 여섯 장이 한 줄에 억지로 밀려 들어가 다 찌그러집니다.
 wrap 을 켜야 자리가 모자랄 때 아랫줄로 내려갑니다.
 무대 안쪽 폭이 776px 이므로 220px 짜리 카드는 세 장($660 + \text{간격 } 32 = 692$) 까지 들어가고 네 장($880 + 48 = 928$) 은 못 들어갑니다. 그래서 한 줄에 세 장입니다.
 남은 84px 을 세 장이 28px 씩 나눠 가져 카드 폭이 248px 이 됩니다.
 브라우저 폭을 줄이면 두 장씩, 한 장씩으로 저절로 접힙니다.
 @media 를 한 줄도 안 썼는데 그렇게 됩니다. 이것이 flex-wrap 의 힘입니다.

단계 8

답: `.menu-card { display: flex; flex-direction: column; }`

```
.menu-price-row { margin: 0; margin-top: auto;
                  padding-top: 10px; border-top: 1px dashed #e2d5c3; }
```

해설: 카드는 두 가지 역할을 동시에 합니다. 바깥에서 보면 menu-list 의 아이템이고,

안에서 보면 그림과 글자를 담는 컨테이너입니다. 컨테이너 역할을 켜는 것이 display: flex 이고, 세로로 쌓으라는 것이 flex-direction: column 입니다.
 세로 컨테이너에서 margin-top: auto 는 "위쪽 남은 자리를 전부 나에게" 라는 뜻입니다.
 그래서 설명이 짧은 카드에서도 가격 줄이 아래로 밀려 내려갑니다.
 카드 높이가 서로 같은 이유는 따로 height 를 준 적이 없기 때문입니다.
 flex 는 같은 줄 아이템의 높이를 기본으로 늘려서 맞춥니다(align-items 의 기본값 stretch). 그래서 높이를 손으로 재서 맞추는 일이 없습니다.

단계 9

답: `.menu-price-row { display: flex; justify-content: space-between;`

```
    align-items: baseline; }
.menu-label { font-size: 0.8125rem; color: #8a7761; }
.menu-price { font-size: 1.0625rem; font-weight: bold; color: #c0392b; }
.menu-card { position: relative; }
.menu-badge { position: absolute; top: -10px; right: -6px;
              background-color: #c0392b; color: #ffffff; font-size: 0.75rem;
              padding: 3px 10px; border-radius: 999px; }
```

해설: space-between 은 남은 자리를 아이템 사이에만 넣고 양 끝에는 안 넣습니다.

아이템이 두 개뿐이니 하나는 왼쪽 끝, 하나는 오른쪽 끝에 붙입니다.
 글자 크기가 13px 과 17px 로 다르므로 align-items: baseline 을 씁니다.
 그러면 두 글자가 같은 밑줄 위에 서서 나란히 보입니다.
 center 로 하면 작은 글자가 살짝 떠 보입니다.
 배지는 카드 밖으로 빠져나가야 하므로 흐름에서 꺼냅니다.
 absolute 의 기준은 static 이 아닌 가장 가까운 조상입니다. 그래서 카드에
 position: relative 를 한 줄 써서 "네 기준은 나야" 라고 표시해 둡니다.
 이 한 줄이 없으면 배지가 페이지 전체를 기준으로 삼아 엉뚱한 곳으로 날아갑니다.

단계 10

답: `.set-title { margin: 0 0 8px; font-size: 1.125rem; color: #3a2b21; }`

```
.set-table { width: 100%; border-collapse: collapse; margin: 0 0 18px;
              background-color: #ffffff; font-size: 0.875rem; }
.set-table th,
.set-table td { border: 1px solid #e2d5c3; padding: 8px 10px; text-align: left; }
.set-table th { background-color: #f3e3cd; color: #5a4326; }
.shop-notice { margin: 0 0 18px; padding-left: 20px;
               font-size: 0.875rem; color: #6b5a48; }
.shop-foot { display: flex; flex-wrap: wrap; justify-content: space-between;
             align-items: center; gap: 8px; background-color: #3a2b21;
             color: #e6d3bd; border-radius: 10px; padding: 12px 16px;
             font-size: 0.8125rem; }
.shop-foot p { margin: 0; }
```

해설: 표의 각 칸에 border 를 주면 칸마다 자기 선을 하나씩 그어서 선이 두 줄로 겹쳐 보입니다.

border-collapse: collapse 는 맞닿은 두 선을 하나로 합칩니다.
 이 속성은 표에만 쓰는 것이라 이 자료에서 따로 가르치지 않았습니다.
 지금은 "표에서 격자선을 깔끔하게 만드는 한 줄" 로만 알아 두세요.
 .set-table th, .set-table td 처럼 쉼표로 이으면 두 선택자에 같은 규칙을 줍니다.
 바닥글은 단계 5 와 같은 flex 이고, 값만 center 에서 space-between 으로 바뀌었습니다.
 같은 손잡이 하나로 가운데 모으기와 양끝 밀어내기를 둘 다 하는 셈입니다.
 flex-wrap: wrap 을 같이 켜 두면 폭이 아주 좁아졌을 때 두 줄로 접혀서
 글자가 서로 겹치지 않습니다.

종합 02 정답 - 음식점 메뉴판

단계 번호는 문제 파일과 똑같습니다. 나란히 열어 놓고 비교하세요.

메뉴판이 놓이는 무대입니다. 문제 파일에 미리 들어 있던 규칙입니다.

```
.menu-stage {
  background-color: #fdf6ec;
  padding: 24px;
}
```

단계 4

```
.shop-head {
  background-color: #3a2b21;
  color: #ffffff;
  padding: 20px 16px;
  border-radius: 12px;
  text-align: center;
  margin: 0 0 18px;
}
.shop-name {
  margin: 0 0 6px;
  font-size: 1.75rem;
  letter-spacing: 2px;
}
.shop-tagline {
  margin: 0;
  font-size: 0.875rem;
  color: #e6d3bd;
}
```

화면 연한 살구색 무대 위쪽에 짙은 갈색 띠가 깔리고, 그 안에 가게 이름 28px 과

연한 베이지색 소개 14px 이 가운데 놓입니다

단계 5

```
.menu-tabs {
  display: flex;
  justify-content: center;
  flex-wrap: wrap;
  gap: 10px;
  margin: 0 0 18px;
}
.menu-tab {
  background-color: #f3e3cd;
  border: 1px solid #e2d5c3;
  border-radius: 999px;
  padding: 6px 18px;
```



```
color: #5a4326;
}
```

화면 알약 모양 딱지 네 개가 10px 간격으로 가운데 한 줄에 놓입니다

단계 6

```
.menu-card {
  box-sizing: border-box;
  background-color: #ffffff;
  border: 1px solid #e2d5c3;
  border-radius: 12px;
  padding: 14px;
}
.menu-photo {
  width: 100%;
  height: auto;
  border-radius: 8px;
  margin: 0 0 10px;
}
.menu-name {
  margin: 0 0 6px;
  font-size: 1.0625rem;
}
.menu-desc {
  margin: 0 0 12px;
  font-size: 0.875rem;
  line-height: 1.5;
  color: #6b5a48;
}
```

화면 흰 카드에 얇은 갈색 테두리와 둥근 모서리가 생기고 그림이 카드 폭에 맞게 줄어듭니다

단계 7

```
.menu-list {
  display: flex;
  flex-wrap: wrap;
  gap: 16px;
  margin: 0 0 22px;
}
.menu-card {
  flex: 1 1 220px;
}
```

화면 카드 여섯 장이 한 줄에 세 장씩 두 줄로 놓입니다. 카드 하나의 폭은 248px
입니다

단계 8

```
.menu-card {  
  display: flex;  
  flex-direction: column;  
}  
.menu-price-row {  
  margin: 0;  
  margin-top: auto;  
  padding-top: 10px;  
  border-top: 1px dashed #e2d5c3;  
}
```

화면 설명 길이가 달라도 가격 줄이 카드 맨 아래에 나란히 붙습니다

단계 9

```
.menu-price-row {  
  display: flex;  
  justify-content: space-between;  
  align-items: baseline;  
}  
.menu-label {  
  font-size: 0.8125rem;  
  color: #8a7761;  
}  
.menu-price {  
  font-size: 1.0625rem;  
  font-weight: bold;  
  color: #c0392b;  
}  
.menu-card {  
  position: relative;  
}  
.menu-badge {  
  position: absolute;  
  top: -10px;  
  right: -6px;  
  background-color: #c0392b;  
  color: #ffffff;  
  font-size: 0.75rem;
```

```
border-radius: 999px;
}
```

화면 가격 줄 왼쪽 끝에 회색 작은 글자, 오른쪽 끝에 빨간 굵은 값이 붙고,

첫 카드 오른쪽 위 모서리에 빨간 알약 배지가 걸립니다

단계 10

```
.set-title {
  margin: 0 0 8px;
  font-size: 1.125rem;
  color: #3a2b21;
}
.set-table {
  width: 100%;
  border-collapse: collapse;
  margin: 0 0 18px;
  background-color: #ffffff;
  font-size: 0.875rem;
}
.set-table th,
.set-table td {
  border: 1px solid #e2d5c3;
  padding: 8px 10px;
  text-align: left;
}
.set-table th {
  background-color: #f3e3cd;
  color: #5a4326;
}
.shop-notice {
  margin: 0 0 18px;
  padding-left: 20px;
  font-size: 0.875rem;
  color: #6b5a48;
}
.shop-foot {
  display: flex;
  flex-wrap: wrap;
  justify-content: space-between;
  align-items: center;
  gap: 8px;
  background-color: #3a2b21;
  color: #e6d3bd;
```

```
border-radius: 10px;
padding: 12px 16px;
font-size: 0.8125rem;
}
.shop-foot p {
margin: 0;
}
```

화면 표에 격자선이 그어지고 머리칸만 베이지색이 됩니다.

맨 아래 갈색 띠에는 영업시간이 왼쪽 끝, 전화번호가 오른쪽 끝에 놓입니다

14단원 ·

14단원 종합 03 정답

회원가입 폼

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

여러분이 만든 종합03_회원가입_폼.html 과 나란히 열어 대조하세요.

가입 무료

계정 정보

아이디 *

hello2026

영문 소문자와 숫자를 섞어 4자 이상으로 씁니다.

비밀번호 *

8자 이상이어야 합니다. 친 글자는 점으로 가려집니다.

메일 주소 *

hello@example.com

가입 확인 메일을 이 주소로 보냅니다.

추가 정보

사는 지역

고르지 않음

한 줄 소개

어떤 것을 배우고 싶은지 적어 주세요

실행하면 나오는 화면 — 종합 03 정답 — 회원가입 폼

답이 글자 하나까지 같을 필요는 없습니다. 화면이 같고 F12 로 잰 숫자가 같으면 맞은 것입니다.

이 폼에는 flex 도 grid 도 쓰이지 않았습니다. 12·13단원의 내용이기 때문입니다. 이름표를 입력칸 위에 올리는 일은 display: block 한 줄로 끝났고, 배지를 모서리에 붙이는 일은 relative 와 absolute 한 쌍으로 끝났습니다.

아래가 완성된 모습입니다. 각 단계의 답과 해설은 이 파일을 VS Code 로 열면 <style> 안과 각 단계 주석에 있습니다. 입력칸을 하나씩 눌러 보면 지금 입력 중인 칸만 파랗게 바뀝니다.

화면 회색 무대 한가운데에 흰 폼 상자가 놓입니다.

오른쪽 위 모서리에 주황 배지가 걸쳐 있고, 안쪽에는 파란 제목이 달린 둥근 상자 두 개가 위아래로 놓이며, 맨 아래에 파란 가입하기 버튼이 있습니다

가입 무료

계정 정보

아이디 *

영문 소문자와 숫자를 섞어 4자 이상으로 씁니다.

비밀번호 *

8자 이상이어야 합니다. 친 글자는 점으로 가려집니다.

메일 주소 *

가입 확인 메일을 이 주소로 보냅니다.

추가 정보

사는 지역

고르지 않음

서울

부산

제주

한 줄 소개

나중에 언제든지 고칠 수 있습니다.

이용약관에 동의합니다 *

가입하기

정리 ———

단계 1

답:

```
<form class="signup-form" action="/join" method="post"> <fieldset class="form-group"> <legend>계정 정보</legend> </fieldset> <fieldset class="form-group"> <legend>추가 정보</legend> </fieldset> <button class="form-submit" type="submit">가입하기</button>
</form>
```

해설: fieldset 은 관련 있는 칸들을 한 묶음으로 만들고, legend 는 그 묶음의 제목입니다.

legend 는 fieldset 의 첫 자식이어야 테두리 위에 얹힙니다.
button 은 type 을 안 쓰면 기본이 submit 이지만, 무엇을 하는 버튼인지 코드에 남기기 위해 type="submit" 을 적어 두는 편이 좋습니다.
제출 버튼은 반드시 form 안에 있어야 동작합니다.

단계 2

답: 계정 정보 묶음 안에 이런 짝을 세 벌 씁니다.

```
<label for="signupId">아이디 <span class="required">*</span></label>
<input class="form-input" type="text" id="signupId" name="userId" placeholder="hello2026" minlength="4" required />
<p class="form-help">영문 소문자와 숫자를 섞어 4자 이상으로 씁니다.</p>
```

비밀번호는 type="password" 와 minlength="8",
메일 주소는 type="email" 을 씁니다.

추가 정보 묶음에는 select · textarea · 체크박스를 넣습니다.

```
<label class="form-agree"> <input type="checkbox" name="agree" value="yes" required />
이용약관에 동의합니다 <span class="required">*</span>
</label>
```

해설: label 의 for 와 input 의 id 가 같은 값이어야 연결됩니다. name 과는 다른 일입니다.

id 는 label 과 짝을 짓는 이름이고, name 은 서버로 보낼 때 값에 붙는 이름표입니다.
체크박스는 label 로 감싸는 방법을 썼습니다. 감싸면 for 와 id 가 없어도 연결됩니다.
연결이 됐는지는 글자를 눌러 보면 압니다. 칸이 커지면 연결된 것입니다.
required 는 화면에 아무 표시도 만들지 않습니다. 그래서 별표를 사람이 직접 씁니다.

단계 3

```
답: width: 420px;
```

```
margin: 0 auto;
padding: 24px;
background-color: #ffffff;
border: 1px solid #d8dee8;
border-radius: 12px;
```

해설: 종합01 의 카드와 똑같은 조합입니다. width 가 있어야 margin 의 auto 가 일을 합니다.

여기서도 width 는 콘텐츠 영역만 가리키므로 폼의 바깥 폭은
 $420 + 24 + 24 + 1 + 1 = 470\text{px}$ 이 됩니다.

단계 4

```
답: display: block;
```

```
margin: 14px 0 4px;
font-size: 0.9375rem;
font-weight: bold;
```

해설: label 은 원래 인라인입니다. 인라인은 글자처럼 옆으로 이어지기 때문에

이름표와 입력칸이 한 줄에 나란히 붙습니다.
display: block 은 그 타고난 성질을 바꿔서 한 줄을 통째로 차지하게 만듭니다.
그러면 뒤따르는 입력칸이 다음 줄로 밀려나 이름표가 칸 위에 놓입니다.
인라인일 때는 위아래 margin 이 안 먹었지만, 블록이 되고 나서는 먹습니다.
그래서 margin 을 준 것도 이 단계입니다.

단계 5

```
답: color: #c0392b;
```

```
font-weight: bold;
```

해설: 별표는 span 으로 감싸 두었습니다. span 은 아무 뜻도 없는 인라인 껍데기라서

"여기 한 조각만 다르게 꾸미고 싶다" 할 때 씁니다.
클래스는 같은 이름을 여러 곳에 붙일 수 있으므로 별표 네 개가 한꺼번에 바뀝니다.
색만으로 필수를 알리면 색을 구분하기 어려운 사람이 놓칩니다.
그래서 별표라는 모양을 함께 씁니다.

단계 6

답: width: 100%;

```
box-sizing: border-box;
padding: 8px 10px;
border: 1px solid #c7ced9;
border-radius: 6px;
font-size: 1rem;
font-family: inherit;
```

해설: box-sizing: border-box 를 켜지 않으면 100% 로 잡힌 390px 에 좌우 padding 20px 과

테두리 2px 이 더해져 칸의 폭이 412px 이 됩니다. 22px 이 남아서 fieldset 의 오른쪽 테두리를 7px 뚫고 나갑니다.
border-box 를 켜면 width 가 테두리까지 포함한 전체 폭이 되어 390px 에 딱 맞습니다.
입력칸은 브라우저 기본 글꼴을 쓰기 때문에 font-family: inherit 을 주지 않으면 폼 안 글꼴만 다르게 보입니다. inherit 은 부모 값을 그대로 받아 오라는 뜻입니다.
체크박스에는 form-input 을 붙이지 않았습니니다. 붙이면 네모 하나가 폼 폭만큼 늘어나 버립니다.

단계 7

답: margin: 4px 0 0;

```
font-size: 0.8125rem;
color: #6b7280;
```

해설: p 에는 브라우저가 위아래로 16px 씩 여백을 넣어 둡니다. 그대로 두면 도움말이

자기가 설명하는 칸에서 멀어지고 오히려 다음 이름표에 가까워집니다.
위 여백을 4px 로 줄이고 아래를 0 으로 만들어 칸에 붙였습니다.
0.8125rem 은 13px 입니다. 도움말은 본문보다 작아야 겹다리로 읽힙니다.

단계 8

답: border-color: #2d6cdf;

```
background-color: #f2f7ff;
outline: 2px solid #cddffb;
```

해설: :focus 는 그 칸이 입력을 받는 중일 때만 켜지는 상태입니다.

콜론 하나를 선택자 뒤에 붙여 씁니다. 띄어 쓰면 자손 선택자가 되어 아무 일도 안 됩니다.

outline 은 border 와 달리 상자의 크기를 바꾸지 않습니다.

그래서 칸이 커질 때 옆 칸들이 밀리지 않습니다. border 로 굵기를 바꾸면 켄 때마다 화면이 덜컥거립니다.

브라우저가 원래 그려 주던 기본 테를 지우고 싶더라도 아무 표시도 없게 만들지 마세요. 키보드만 쓰는 사람은 지금 어느 칸에 있는지 알 방법이 사라집니다.

단계 9

답: `.form-submit { width: 100%; margin-top: 20px; padding: 12px 0;`

```
background-color: #2d6cdf; color: #ffffff; border: none;
border-radius: 8px; font-family: inherit; font-size: 1rem;
font-weight: bold; letter-spacing: 2px; }
.form-submit:hover { background-color: #1b4ea8; }
```

해설: button 은 브라우저가 회색 배경과 입체 테두리를 미리 그려 둡니다.

`border: none` 으로 그 테두리를 지워야 우리가 준 배경색이 제대로 보입니다.

button 은 인라인 블록이라 width 와 위아래 padding 이 그대로 먹습니다.

그래서 display 를 건드리지 않고도 폼 폭을 꽉 채울 수 있습니다.

버튼은 form 의 바로 아래 자식이므로 100% 는 폼의 안쪽 폭 420px 입니다.

fieldset 안에 있는 입력칸의 100% 와 기준이 다릅니다.

단계 10

답: HTML — form 의 첫 자식으로 넣습니다.

`<p class="form-badge">가입 무료</p>`

CSS – `.signup-form` 규칙에 한 줄을 더합니다.

```
position: relative;
```

그리고 배지 규칙을 만듭니다.

```
.form-badge { position: absolute; top: -14px; right: 18px; margin: 0;
padding: 4px 12px; background-color: #f0a830; color: #ffffff;
font-size: 0.8125rem; font-weight: bold; border-radius: 999px; }
```

해설: absolute 는 흐름에서 빠져 나와 “static 이 아닌 가장 가까운 조상” 을 기준으로 붙습니다.

폼에 `position: relative` 를 주지 않으면 기준이 폼이 아니라 페이지 전체가 되어 배지가 화면 맨 위 오른쪽으로 날아갑니다. 그래서 이 둘은 한 쌍입니다.
`relative` 는 자기 자리를 그대로 지키므로 폼의 모양은 하나도 안 바뀝니다.
 기준점 표시로만 쓴 것입니다.
`top` 에 음수를 주면 기준 상자보다 위로 올라가 모서리에 걸친 모양이 됩니다.
`border-radius` 를 높이보다 크게 주면 양 끝이 반원이 되어 알약 모양이 됩니다.

종합 03 정답 - 회원가입 폼

단계 번호는 문제 파일과 똑같습니다. 나란히 열어 놓고 비교하세요.

폼이 놓이는 무대입니다. 문제 파일에 미리 들어 있던 규칙입니다.

```
.form-stage {
  background-color: #eef2f7;
  padding: 34px 0;
}
```

`fieldset` 과 `legend` 의 모양입니다. 문제 파일에 미리 들어 있던 규칙입니다.

```
.form-group {
  margin: 0 0 18px;
  padding: 4px 14px 14px;
  border: 1px solid #dde3ec;
  border-radius: 8px;
}
.form-group legend {
  padding: 0 6px;
  color: #2d6cdf;
  font-size: 0.9375rem;
  font-weight: bold;
}
```

단계 3

```
.signup-form {
  width: 420px;
  margin: 0 auto;
  padding: 24px;
  background-color: #ffffff;
  border: 1px solid #d8dee8;
  border-radius: 12px;
```

`position: relative;` /* 단계 10 에서 한 줄 더 붙습니다

```
}
```

화면 회색 무대 한가운데에 흰 폼 상자가 놓입니다. 바깥 폭은 470px 입니다

단계 4

```
.signup-form label {
  display: block;
  margin: 14px 0 4px;
  font-size: 0.9375rem;
  font-weight: bold;
}
```

화면 옆에 붙어 있던 이름표가 입력칸 위로 올라가 한 줄을 차지합니다

단계 5

```
.required {
  color: #c0392b;
  font-weight: bold;
}
```

화면 이름표 옆의 별표 네 개만 빨강게 바뀝니다

단계 6

```
.form-input {
  width: 100%;
  box-sizing: border-box;
  padding: 8px 10px;
  border: 1px solid #c7ced9;
  border-radius: 6px;
  font-size: 1rem;
  font-family: inherit;
}
```

화면 입력칸 · 목록 상자 · 긴 글 칸이 모두 fieldset 안쪽 폭 390px 로 맞춰집니다

단계 7

```
.form-help {
  margin: 4px 0 0;
  font-size: 0.8125rem;
  color: #6b7280;
}
```

화면 도움말이 13px 회색으로 작아지고 입력칸 바로 아래에 붙습니다

단계 8

```
.form-input:focus {
  border-color: #2d6cdf;
  background-color: #f2f7ff;
  outline: 2px solid #cddffb;
}
```

화면 누른 칸만 테두리가 파래지고 배경이 연파랑으로 바뀌며 바깥에 연한 테가 한 겹 더 생깁니다

단계 9

```
.form-submit {
  width: 100%;
  margin-top: 20px;
  padding: 12px 0;
  background-color: #2d6cdf;
  color: #ffffff;
  border: none;
  border-radius: 8px;
  font-family: inherit;
  font-size: 1rem;
  font-weight: bold;
  letter-spacing: 2px;
}
.form-submit:hover {
  background-color: #1b4ea8;
}
```

화면 회색 기본 버튼이 폼 폭을 다 채우는 파란 버튼이 됩니다

단계 10

```
.form-badge {  
  position: absolute;  
  top: -14px;  
  right: 18px;  
  margin: 0;  
  padding: 4px 12px;  
  background-color: #f0a830;  
  color: #ffffff;  
  font-size: 0.8125rem;  
  font-weight: bold;  
  border-radius: 999px;  
}
```

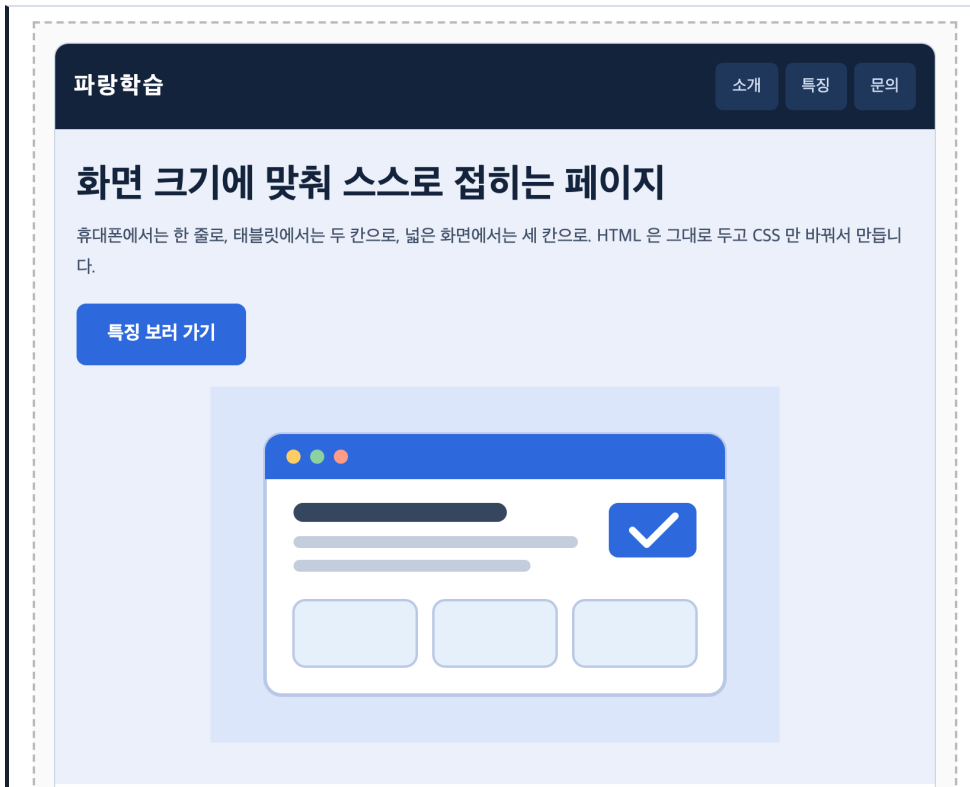
화면 주황 알약 모양 배지가 폼 상자의 오른쪽 위 모서리에 걸쳐 붙습니다

14단원 ·

14단원 종합 04 정답 반응형 랜딩페이지

RUN 이 파일을 더블클릭해서 브라우저로 여세요.

그리고 브라우저 창의 오른쪽 끝을 잡고 폭을 천천히 줄여 보세요.



실행하면 나오는 화면 — 종합 04 정답 — 반응형 랜딩페이지

답이 글자 하나까지 같을 필요는 없습니다. 화면이 같고 F12 로 잰 숫자가 같으면 맞은 것입니다.

이 파일은 이 자료의 마지막 결과물입니다. 01단원의 태그부터 13단원의 미디어쿼리까지 전부 한 장에 들어 있습니다.

| | |
|---------|---|
| 01~06단원 | header nav main section article footer a img h2 h3 p 표시 |
| 07~09단원 | 색 · 글자 크기 · 정렬 · 링크 밑줄 없애기 |
| 10단원 | padding border border-radius overflow |
| 11단원 | display |
| 12단원 | flex 로 만든 머리글과 메뉴 |
| 13단원 | grid · @media · clamp · max-width 100% · 뷰포트 meta |

읽는 순서를 지키세요. style 안이 이렇게 되어 있습니다.

- ① 좁은 화면용 규칙 (미디어쿼리 밖)
- ② @media (min-width: 600px)
- ③ @media (min-width: 900px)

좁은 쪽을 기본으로 삼고 넓어질 때 덧붙이는 것을 모바일 우선이라고 합니다. 반대로 쓰면 코드가 길어지고, 무엇보다 휴대폰에서 먼저 깨집니다.

아래가 완성된 모습입니다. 브라우저 창의 폭을 줄였다 늘렸다 해 보세요. 특징 카드가 3단 → 2단 → 1단으로 접히고 메뉴도 가로에서 세로로 바뀝니다. 각 단계의 답과 해설은 이 파일을 VS Code 로 열면 <style> 안과 각 단계 주석에 있습니다.

| | |
|----|---|
| 화면 | 폭 1280 에서 남색 머리글(로고 왼쪽 · 메뉴 오른쪽), 좌우 두 칸짜리 소개 영역, |
|----|---|

가로 세 칸짜리 특징 카드, 세 칸짜리 남색 바닥글이 차례로 쌓입니다

파라학습

소개
특징
문의

화면 크기에 맞춰 스스로 접히는 페이지

휴대폰에서는 한 줄로, 태블릿에서는 두 칸으로, 넓은 화면에서는 세 칸으로.
HTML 은 그대로 두고 CSS 만 바꿔서 만듭니다.

특징 보러 가기

이 페이지가 지키는 세 가지

뜻이 있는 태그

머리글은 header, 메뉴는 nav, 바닥글은 footer 로 씁니다.
화면 모양은 같아도 읽는 사람과 기계가 구조를 알아봅니다.

한 벌의 색과 글자

색과 글자 크기를 몇 가지로 정해 두고 그것만 씁니다.
그래야 페이지 전체가 한 사람이 만든 것처럼 보입니다.

어느 쪽에서도 안 깨짐

폭을 줄여도 가로 스크롤바가 생기지 않습니다.
그림에는 상한선을 걸고, 칸 수는 화면 크기에 맞깁니다.

파랑학습

HTML 과 CSS 를 처음부터 배우는 곳입니다.

찾아오는 길

서울 어딘가 파랑로 13길 14

문의

전화 02-000-0000 / 평일 10:00 부터 18:00 까지

정리 ———

정리 ———

단계 1 ———

답:

```
<header class="site-head"> <p class="site-logo">파랑학습</p> <nav class="site-
nav"> <a href="#hero">소개</a> <a href="#features">특징</a> <a href="#contact">문의
</a> </nav>
</header>
<main>
  ... 단계 2 와 단계 3 의 내용이 여기 들어갑니다 ...
</main>
<footer class="site-foot" id="contact">
  ... 단계 3 ...
</footer>
```

해설: header · nav · main · footer 는 모양을 하나도 안 바꿉니다.

div 로 다 써도 화면은 똑같습니다. 그래도 이 태그를 쓰는 이유는
"이 덩어리가 무엇인지" 를 코드에 남기기 위해서입니다.
main 은 한 페이지에 하나만 씁니다. 그 페이지의 본문이라는 뜻이기 때문입니다.
href 앞에 우물 정(#) 을 붙이면 같은 페이지 안의 그 id 자리로 넘어갑니다.
그래서 아래 section 과 footer 에 id 를 붙여 둔 것입니다.
메뉴 항목을 ul 과 li 로 감싸는 방식도 많이 씁니다. 여기서는 a 세 개만 넣었습니다.

단계 2

답:

```
<section class="hero" id="hero"> <div class="hero-text"> <h2 class="hero-title">화면
크기에 맞춰 스스로 접히는 페이지</h2> <p class="hero-lead">휴대폰에서는 한 줄로 ...
</p> <p class="hero-cta"> <a class="hero-button" href="#features">특징 보러 가기
</a> </p> </div> <p class="hero-figure">  </p>
</section>
```

해설: 글 덩어리를 div 로 한 번 더 묶은 이유가 있습니다.

단계 10 에서 소개 영역을 두 칸짜리 격자로 만들 텐데, 그때 왼쪽 칸에
제목 · 설명 · 버튼이 통째로 들어가야 하기 때문입니다.
묶지 않으면 세 요소가 각각 한 칸씩 차지해 칸이 모자랍니다.
버튼처럼 보이지만 button 이 아니라 a 입니다. 누르면 다른 곳으로 가는 것이
링크의 일이고, 품을 보내는 것이 button 의 일입니다. 여기서는 링크가 맞습니다.
그림을 p 로 한 번 감싼 이유는 단계 6 에서 가운데로 보내기 위해서입니다.

단계 3

답: 카드 한 장은 이 모양이고, 세 장을 div 로 감쌌습니다.

```

<section class="features" id="features"> <h2 class="features-title">이 페이지가
지키는 세 가지</h2> <div class="feature-list"> <article class="feature-
card">  <h3 class="feature-name">뜻이 있는 태그</h3> <p class="feature-text">머리글은
header, 메뉴는 nav ...</p> </article>
... 두 장 더 ...
</div>
</section>

```

바닥글은 div 세 개입니다.

```

<footer class="site-foot" id="contact"> <div class="foot-col"> <p class="foot-title">
파랑학습</p> <p class="foot-text">HTML 과 CSS 를 처음부터 배우는 곳입니다.</p> </div>
... 두 칸 더 ...
</footer>

```

해설: 제목 순위를 눈여겨보세요. 자료 제목이 h1, 구획 제목이 h2, 카드 이름이 h3 입니다.

제목 태그는 글자 크기를 고르는 도구가 아니라 순위를 매기는 도구입니다.
크기는 CSS 로 따로 정합니다. 실제로 단계 7 에서 h3 크기를 17px 로 바꿉니다.
바닥글 세 칸을 div 로 만든 이유는 그 자체로는 뜻이 없는 배치용 상자이기 때문입니다.
반대로 특징 카드 한 장은 그 자체로 뜻이 통하므로 article 을 씁니다.

단계 4

답: head 안에

```

<meta name="viewport" content="width=device-width, initial-scale=1" />

```

style 안에

```

.landing img { max-width: 100%; height: auto; }

```

해설: 이 두 가지는 반응형 페이지의 안전벨트입니다. 없어도 넓은 화면에서는

아무 문제가 없어 보입니다. 그래서 더 자주 빠뜨립니다.

뷰포트 meta 가 없으면 휴대폰 브라우저가 "이 페이지는 980px 짜리겠지" 하고 멋대로 축소해서 보여 줍니다. 그러면 @media 가 전부 넓은 화면 쪽으로 걸려서 애써 만든 반응형이 통째로 안 먹습니다. 노트북에서는 멀쩡하니 알아채기 어렵습니다.

max-width: 100% 는 "부모보다 커지지는 마라" 는 뜻입니다.
width: 100% 와 다릅니다. width 는 항상 부모만큼 늘리라는 명령이라 작은 그림도 억지로 늘어나 흐릿해집니다. max-width 는 상한선만 걸어 둡니다.
height 를 auto 로 두어야 가로가 줄 때 세로도 같은 비율로 줄어듭니다.
여기서 그림 파일은 480px 짜리인데, 폭 375 에서는 자리가 261px 뿐이라 261px 로 줄어듭니다.
폭 768 에서는 자리가 654px 이라 그림이 원래 크기 480px 그대로 나옵니다.
max-width 는 "커지지 마라" 일 뿐 "늘어나라" 가 아니기 때문입니다.

이 줄이 없으면 폭 375 에서 그림이 480px 을 고집해 무대 밖으로 빠져나갑니다.
이 페이지에서는 무대(.landing) 에 overflow: hidden 이 미리 걸려 있어서 스크롤바 대신 그림 오른쪽이 몽텅 잘려 나갑니다.
무대에 overflow: hidden 마저 없었다면 페이지 전체에 가로 스크롤바가 생겼을 것입니다.
어느 쪽이든 고장입니다. 좁은 화면에서 확인해야 보인다는 점이 무서운 것입니다.

단계 5

답: `.site-head { display: flex; flex-direction: column; align-items: center;`

```
gap: 10px; background-color: #14243d; padding: 16px; }
.site-logo { margin: 0; color: #ffffff; font-size: 20px;
font-weight: bold; letter-spacing: 2px; }
.site-nav { display: flex; flex-direction: column; gap: 6px; width: 100%; }
.site-nav a { background-color: #22375a; color: #d8e4f7; padding: 8px 14px;
border-radius: 6px; text-align: center;
text-decoration: none; font-size: 14px; }
.site-nav a:hover { background-color: #2d6cdf; color: #ffffff; }
```

해설: 여기가 모바일 우선의 출발점입니다. 미디어쿼리 밖에 쓴 것이 곧 좁은 화면용입니다.

좁은 화면에서는 가로로 늘어놓을 자리가 없으니 세로로 쌓는 것이 기본입니다.
그래서 flex-direction 을 처음부터 column 으로 둡니다.
a 는 인라인이라 원래는 위아래 padding 을 줘도 줄 높이가 안 늘어납니다.
그런데 .site-nav 가 flex 컨테이너라 그 자식인 a 가 아이템이 되면서 블록처럼 취급됩니다. 그래서 padding 8px 이 위아래로 제대로 먹습니다.
width: 100% 는 좁은 화면에서 메뉴 단추가 화면 폭을 꽉 채우게 합니다.
손가락으로 누르는 화면에서는 누를 자리가 넓을수록 좋습니다.

단계 6

· 11단원 개념02 display)

```
답: .hero { display: grid; grid-template-columns: 1fr; gap: 18px;

        align-items: center; background-color: #eef3fb; padding: 26px 18px; }
.hero-title { margin: 0 0 10px; font-size: clamp(22px, 4vw, 36px);
        line-height: 1.3; color: #14243d; }
.hero-lead { margin: 0 0 18px; font-size: 15px; color: #44506b; }
.hero-cta { margin: 0; }
.hero-button { display: inline-block; background-color: #2d6cdf;
        color: #ffffff; padding: 12px 26px; border-radius: 8px;
        text-decoration: none; font-weight: bold; }
.hero-figure { margin: 0; text-align: center; }
```

해설: align-items 는 12단원 개념04 에서 flex 에 쓰던 그 속성입니다.

grid 에서도 이름과 뜻이 똑같아서 새로 외울 것이 없습니다.
13단원에서는 안 나왔지만 여기서 바로 써도 됩니다.
단계 10 에서 소개 영역이 두 칸이 되면 글 덩어리와 그림의 키가 달라지는데,
이 한 줄이 둘을 같은 높이 가운데에 맞춰 줍니다.

grid-template-columns: 1fr 은 "칸 하나짜리 격자" 입니다.
칸이 하나뿐이니 지금은 글 덩어리와 그림이 위아래로 쌓입니다.
일부러 이렇게 시작합니다. 단계 10 에서 이 한 줄만 고쳐 두 칸으로 만들 것입니다.
HTML 은 한 글자도 안 건드리고 CSS 한 줄로 배치가 바뀝니다.

clamp(22px, 4vw, 36px) 는 "최소 22px, 기준 화면 폭의 4%, 최대 36px" 입니다.
폭 1280 에서 4vw 는 51.2px 이지만 상한선 36px 에 걸려 36px 이 됩니다.
폭 768 에서는 30.72px, 폭 375 에서는 15px 이 되어 하한선 22px 에 걸립니다.
미디어쿼리 세 개를 쓸 일을 한 줄로 대신한 셈입니다.

a 에 display: inline-block 을 준 이유는 위아래 padding 때문입니다.
버튼은 flex 아이템이 아니므로 자동으로 블록이 되지 않습니다.
그렇다고 block 으로 만들면 한 줄을 통째로 차지해 버튼이 가로로 쭉 늘어납니다.
글자만큼만 차지하면서 크기도 정할 수 있는 절충안이 inline-block 입니다.

단계 7

```
답: .features { padding: 26px 18px; }
```

```
.features-title { margin: 0 0 16px; font-size: 22px;
                  text-align: center; color: #14243d; }
.feature-list { display: grid; grid-template-columns: 1fr; gap: 14px; }
.feature-card { background-color: #ffffff; border: 1px solid #d6dfee;
               border-radius: 10px; padding: 18px; text-align: center; }
.feature-icon { width: 64px; }
.feature-name { margin: 10px 0 6px; font-size: 17px; color: #14243d; }
.feature-text { margin: 0; font-size: 14px; line-height: 1.6; color: #55617a; }
```

해설: 여기도 칸 하나짜리 격자로 시작합니다. 좁은 화면이 기본이기 때문입니다.

카드 세 장이 세로로 쌓이고 사이가 14px 씩 벌어집니다.
gap 은 칸 사이에만 들어가고 바깥쪽에는 안 붙습니다. 그래서 마지막 카드
아래에만 여백이 남는 일이 없습니다. margin 으로 하면 그 처리를 손으로 해야 합니다.

여기서 flex 대신 grid 를 쓴 이유를 생각해 보세요.
카드 목록은 한 줄이 아니라 격자입니다. 칸 수를 내가 정하고 싶습니다.
flex 는 "몇 개 들어가는지" 를 브라우저가 폭을 보고 정하지만,
grid 는 "몇 칸" 이라고 내가 못 박습니다. 그래서 단계 9·10 처럼
폭 구간마다 칸 수를 정확히 3단 · 2단 · 1단으로 바꾸려면 grid 가 편합니다.

단계 8

답: .site-foot { display: grid;

```

        grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
        gap: 16px; background-color: #14243d; color: #c9d6ea;
        padding: 22px 18px; }
.foot-title { margin: 0 0 6px; color: #ffffff;
              font-size: 15px; font-weight: bold; }
.foot-text { margin: 0; font-size: 13px; line-height: 1.6; }
```

해설: 이 한 줄은 읽는 법을 알면 어렵지 않습니다.

```
repeat( auto-fit , minmax(180px, 1fr) )
      ↳ 칸 수      ↳ 칸 하나의 크기
```

minmax(180px, 1fr) 은 "이 칸은 최소 180px, 남으면 1fr 만큼 더" 입니다.
auto-fit 은 "180px 짜리 칸을 들어갈 수 있는 만큼 만들어라" 입니다.
그래서 폭이 줄면 칸 수가 저절로 3 → 2 → 1 로 바뀝니다.
@media 를 한 줄도 안 썼는데 그렇게 됩니다.

바닥글 안쪽 폭이 폭 1280 에서 786px 이므로 180px 칸은 넉넉히 세 개가 들어갑니다.
칸 하나가 251px 이 됩니다.
폭 375 에서는 안쪽 폭이 261px 뿐이라 180 짜리 두 칸(180 + 16 + 180 = 376) 이
안 들어가서 한 칸만 남습니다. 그 한 칸이 1fr 로 늘어나 261px 을 다 차지합니다.

칸 수가 딱 떨어져야 하는 곳(특징 카드) 에는 @media 를 쓰고,
아무 수나 괜찮은 곳(바닥글) 에는 auto-fit 을 쓰면 코드가 짧아집니다.

단계 9

```
답: @media (min-width: 600px) {
```

```
.site-head { flex-direction: row; justify-content: space-between; }
.site-nav { flex-direction: row; width: auto; }
.feature-list { grid-template-columns: repeat(2, 1fr); }
}
```

해설: 중괄호가 두 겹이라는 점을 꼭 보세요. 바깥 중괄호는 @media 것이고,

안쪽 중괄호는 선택자 것입니다. 안쪽에 선택자를 빠뜨리고 속성만 쓰면
아무 일도 일어나지 않습니다. 에러도 안 납니다.

min-width: 600px 은 "폭이 600px 이상일 때" 입니다. 경계값 600 도 포함입니다.
바뀌어야 하는 줄만 씁니다. 배경색과 padding 은 좁을 때와 같으므로 다시 안 씁니다.
이것이 모바일 우선의 이점입니다. 데스크톱 우선으로 봤다면
여기서 색과 여백까지 전부 다시 써야 했을 것입니다.

.site-nav 의 width 를 auto 로 되돌리는 줄을 빠뜨리기 쉽습니다.
빠뜨리면 메뉴가 100% 를 계속 차지해서 로고를 왼쪽으로 밀어붙입니다.
좁은 화면용으로 준 값은 넓어질 때 되돌려야 한다는 것을 기억하세요.

단계 10

```
답: @media (min-width: 900px) {
```



```
.hero { grid-template-columns: 1fr 1fr; }
.feature-list { grid-template-columns: repeat(3, 1fr); }
}
```

해설: 미디어쿼리는 작은 값부터 큰 값 순서로 위에서 아래로 씁니다.

600 짜리와 900 짜리가 둘 다 참일 때(예를 들어 폭 1280) 나중에 쓴 900 짜리가 이깁니다. 명시도가 같으면 나중에 쓴 것이 이기기 때문입니다.
 @media 는 명시도를 올려 주지 않습니다. 순서가 전부입니다.
 그래서 900 짜리를 600 짜리보다 위에 쓰면 넓은 화면에서 2단이 나옵니다.
 에러는 안 납니다. 그냥 조용히 틀립니다.

.hero 에 준 1fr 1fr 은 같은 폭의 두 칸입니다.
 왼쪽 칸에 글 덩어리(hero-text), 오른쪽 칸에 그림(hero-figure) 이 들어갑니다.
 단계 2 에서 글 세 줄을 div 로 묶어 둔 것이 여기서 값을 합니다.
 묶지 않았다면 제목 · 설명 · 버튼 · 그림 네 개가 두 칸에 두 줄로 흩어집니다.

 repeat(3, 1fr) 은 1fr 1fr 1fr 과 같습니다. 괄호 안만 쉼표라는 점을 조심하세요.
 grid-template-columns 자체는 쉼표로 나누지 않고 띄어쓰기로 나눕니다.

종합 04 정답 - 반응형 랜딩페이지
 단계 번호는 문제 파일과 똑같습니다. 나란히 열어 놓고 비교하세요.

읽는 순서가 중요합니다.
 위에서부터 좁은 화면용 규칙이 먼저 나오고,
 맨 아래에 @media 두 개가 작은 값 → 큰 값 순서로 붙습니다.
 이것이 모바일 우선입니다.

아래 .width-table 은 이 정답 파일의 설명용 표입니다. 연습 내용이 아니니 그냥 넘어가세요.

```
.width-table {
  width: 100%;
  border-collapse: collapse;
  margin: 0 0 24px;
  font-size: 13px;
}
.width-table th,
.width-table td {
  border: 1px solid #ccd5e0;
  padding: 6px 8px;
  text-align: left;
}
.width-table thead th {
  background-color: #eef2f7;
}
```

랜딩 페이지가 놓이는 무대입니다. 문제 파일에 미리 들어 있던 규칙입니다.

```
.landing {
  background-color: #ffffff;
  border: 1px solid #cfd8e6;
  border-radius: 12px;
  overflow: hidden;
}
```

단계 4

```
.landing img {
  max-width: 100%;
  height: auto;
}
```

화면 폭 1280 에서는 그림이 384px, 폭 768 에서는 원래 크기인 480px 입니다.

폭 375 까지 줄이면 자리에 맞춰 261px 로 줄어듭니다.

이 두 줄이 없으면 폭 375 에서 480px 그대로 버텨서 그림 오른쪽이 잘립니다

단계 5

```
.site-head {
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 10px;
  background-color: #14243d;
  padding: 16px;
}
.site-logo {
  margin: 0;
  color: #ffffff;
  font-size: 20px;
  font-weight: bold;
  letter-spacing: 2px;
}
.site-nav {
  display: flex;
  flex-direction: column;
  gap: 6px;
  width: 100%;
}
```

```
background-color: #22375a;
color: #d8e4f7;
padding: 8px 14px;
border-radius: 6px;
text-align: center;
text-decoration: none;
font-size: 14px;
}
.site-nav a:hover {
background-color: #2d6cdf;
color: #ffffff;
}
```

화면 폭 599 이하에서 탐색 머리글 안에 로고가 한 줄, 그 아래 메뉴 세 개가

세로로 쌓입니다. 폭 600 이상은 단계 9 가 가로로 되돌립니다

단계 6

```
.hero {
display: grid;
grid-template-columns: 1fr;
gap: 18px;
align-items: center;
background-color: #eef3fb;
padding: 26px 18px;
}
.hero-title {
margin: 0 0 10px;
font-size: clamp(22px, 4vw, 36px);
line-height: 1.3;
color: #14243d;
}
.hero-lead {
margin: 0 0 18px;
font-size: 15px;
color: #44506b;
}
.hero-cta {
margin: 0;
}
.hero-button {
display: inline-block;
background-color: #2d6cdf;
color: #ffffff;
padding: 12px 26px;
```

```

text-decoration: none;
    font-weight: bold;
}
.hero-figure {
    margin: 0;
    text-align: center;
}

```

화면 연한 파란 칸에 큰 제목과 설명, 파란 버튼이 놓이고 그 아래 대표 그림이 옵니다.

폭 900 이상은 단계 10 이 좌우 두 칸으로 바꿉니다.

제목 크기는 폭 1280 에서 36px, 768 에서 30.72px, 375 에서 22px 입니다

단계 7

```

.features {
    padding: 26px 18px;
}
.features-title {
    margin: 0 0 16px;
    font-size: 22px;
    text-align: center;
    color: #14243d;
}
.feature-list {
    display: grid;
    grid-template-columns: 1fr;
    gap: 14px;
}
.feature-card {
    background-color: #ffffff;
    border: 1px solid #d6dfee;
    border-radius: 10px;
    padding: 18px;
    text-align: center;
}
.feature-icon {
    width: 64px;
}
.feature-name {
    margin: 10px 0 6px;
    font-size: 17px;
    color: #14243d;
}

```

```
margin: 0;
font-size: 14px;
line-height: 1.6;
color: #55617a;
}
```

화면 폭 599 이하에서 카드 세 장이 한 칸짜리 격자에 세로로 쌓입니다.

폭 600 이상은 단계 9 가, 폭 900 이상은 단계 10 이 칸 수를 바꿉니다

단계 8

```
.site-foot {
display: grid;
grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
gap: 16px;
background-color: #14243d;
color: #c9d6ea;
padding: 22px 18px;
}
.foot-title {
margin: 0 0 6px;
color: #ffffff;
font-size: 15px;
font-weight: bold;
}
.foot-text {
margin: 0;
font-size: 13px;
line-height: 1.6;
}
```

화면 폭 1280 과 768 에서는 세 칸이 가로로, 폭 375 에서는 한 칸씩 세로로 쌓입니다

단계 9

```
@media (min-width: 600px) {
.site-head {
flex-direction: row;
justify-content: space-between;
}
.site-nav {
flex-direction: row;
width: auto;
}
```

```
.feature-list {
  grid-template-columns: repeat(2, 1fr);
}
```

화면 폭 768 에서 로고가 왼쪽 끝, 메뉴 세 개가 오른쪽 끝에 한 줄로 붙고

특징 카드가 두 칸으로 나뉩니다

단계 10

```
@media (min-width: 900px) {
  .hero {
    grid-template-columns: 1fr 1fr;
  }
  .feature-list {
    grid-template-columns: repeat(3, 1fr);
  }
}
```

화면 폭 1280 에서 소개 영역이 글과 그림 두 칸으로 나뉘고

특징 카드가 한 줄에 세 장으로 놓입니다

부 록 나

심화문제 정답

심화문제는 선택 과제입니다. 풀지 않고 넘어가도 다음 단원에 지장이 없습니다.